scikit-learn

Knowledge-Base Reference

Version 1.9.1

The estimator API, preprocessing and pipelines, cross-validation and tuning, metrics, linear and ensemble models, common pitfalls and model persistence, plus an API quick reference.

Source: official documentation, https://scikit-learn.org/stable/
Content © 2007-2026 The scikit-learn developers (BSD-3-Clause)
Compiled October 29, 2026

Contents

About this reference **4**

Getting started **5**
- Fitting and predicting: estimator basics . . . 5
- Transformers and pre-processors . . . 6
- Pipelines: chaining pre-processors and estimators . . . 6
- Model evaluation . . . 7
- Automatic parameter searches . . . 7
- Next steps . . . 8

Common pitfalls and recommended practices **9**
- Inconsistent preprocessing . . . 9
- Data leakage . . . 10
- Controlling randomness . . . 12

Preprocessing data **18**
- Standardization, or mean removal and variance scaling . . . 18
- Non-linear transformation . . . 22
- Normalization . . . 24
- Encoding categorical features . . . 25
- Discretization . . . 31
- Imputation of missing values . . . 33
- Generating polynomial features . . . 34
- Custom transformers . . . 36

Imputation of missing values **37**
- Univariate vs. Multivariate Imputation . . . 37
- Univariate feature imputation . . . 37
- Multivariate feature imputation . . . 38
- Nearest neighbors imputation . . . 39
- Keeping the number of features constant . . . 40
- Marking imputed values . . . 40
- Estimators that handle NaN values . . . 42

Pipelines and composite estimators **43**
- Pipeline: chaining estimators . . . 43
- Transforming target in regression . . . 46
- FeatureUnion: composite feature spaces . . . 48
- ColumnTransformer for heterogeneous data . . . 48
- Visualizing Composite Estimators . . . 51

Feature selection **52**
- Removing features with low variance . . . 52
- Univariate feature selection . . . 52
- Recursive feature elimination . . . 53
- Feature selection using SelectFromModel . . . 54
- Sequential Feature Selection . . . 55
- Feature selection as part of a pipeline . . . 56

Cross-validation **57**
Computing cross-validated metrics . . . 59
Cross validation iterators . . . 62
A note on shuffling . . . 70
Cross validation and model selection . . . 70
Permutation test score . . . 71

Tuning hyper-parameters **72**
Exhaustive Grid Search . . . 72
Randomized Parameter Optimization . . . 73
Searching for optimal parameters with successive halving . . . 74
Tips for parameter search . . . 79
Alternatives to brute force parameter search . . . 81

Metrics and scoring **82**
Which scoring function should I use? . . . 82
Scoring API overview . . . 83
The `scoring` parameter: defining model evaluation rules . . . 84
Classification metrics . . . 88
Multilabel ranking metrics . . . 111
Regression metrics . . . 113
Clustering metrics . . . 123
Dummy estimators . . . 123

Validation and learning curves **126**
Validation curve . . . 126
Learning curve . . . 127

Linear models **129**
Ordinary Least Squares . . . 129
Ridge regression and classification . . . 130
Lasso . . . 131
Multi-task Lasso . . . 135
Elastic-Net . . . 136
Multi-task Elastic-Net . . . 136
Least Angle Regression . . . 137
LARS Lasso . . . 137
Orthogonal Matching Pursuit (OMP) . . . 138
Bayesian Regression . . . 138
Logistic regression . . . 140
Generalized Linear Models . . . 145
Stochastic Gradient Descent - SGD . . . 147
Robustness regression: outliers and modeling errors . . . 148
Quantile Regression . . . 151
Polynomial regression: extending linear models with basis functions . . . 152

Ensembles: gradient boosting, random forests, bagging, voting, stacking **155**
Gradient-boosted trees . . . 155
Random forests and other randomized tree ensembles . . . 166
Bagging meta-estimator . . . 171
Voting Classifier . . . 172
Voting Regressor . . . 174
Stacked generalization . . . 174
AdaBoost . . . 176

Model persistence **179**

Workflow Overview . . . 180
ONNX . . . 181
skops.io . . . 181
pickle, joblib, and cloudpickle . . . 182
Security & Maintainability Limitations . . . 182
Summarizing the key points . . . 184

API quick reference **185**
Model selection . . . 185
Pipelines and preprocessing . . . 199
Models . . . 209
Metrics . . . 222
Inspection . . . 231

About this reference

This knowledge-base reference collects core chapters of the official **scikit-learn 1.9.1** documentation (https://scikit-learn.org/stable/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the reStructuredText files the website is built from), taken from the `1.9.1` release tag of github.com/scikit-learn/scikit-learn. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 1.9.1 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Note on figures. Many scikit-learn guide pages show plots that are generated by running the example gallery when the website is built. Those generated plots are not part of the documentation source, so they are not included; the text, formulas and code are complete.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. scikit-learn and its documentation are distributed under the BSD 3-Clause License, Copyright (c) 2007-2026 The scikit-learn developers. Redistribution with this notice is permitted. Full text: https://github.com/scikit-learn/scikit-learn/blob/main/COPYING

Chapter sources

- Getting started: https://scikit-learn.org/stable/getting_started.html
- Common pitfalls and recommended practices: https://scikit-learn.org/stable/common_pitfalls.html
- Preprocessing data: https://scikit-learn.org/stable/modules/preprocessing.html
- Imputation of missing values: https://scikit-learn.org/stable/modules/impute.html
- Pipelines and composite estimators: https://scikit-learn.org/stable/modules/compose.html
- Feature selection: https://scikit-learn.org/stable/modules/feature_selection.html
- Cross-validation: https://scikit-learn.org/stable/modules/cross_validation.html
- Tuning hyper-parameters: https://scikit-learn.org/stable/modules/grid_search.html
- Metrics and scoring: https://scikit-learn.org/stable/modules/model_evaluation.html
- Validation and learning curves: https://scikit-learn.org/stable/modules/learning_curve.html
- Linear models: https://scikit-learn.org/stable/modules/linear_model.html
- Ensembles: gradient boosting, random forests, bagging, voting, stacking: https://scikit-learn.org/stable/modules/ensemble.html
- Model persistence: https://scikit-learn.org/stable/model_persistence.html

Getting started

Source: *https://scikit-learn.org/stable/getting_started.html*

`Scikit-learn` is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.

The purpose of this guide is to illustrate some of the main features of `scikit-learn`. It assumes basic working knowledge of machine learning practices (model fitting, predicting, cross-validation, etc.). Please refer to our installation instructions to install `scikit-learn`, or jump to the next_steps section for additional guidance on using `scikit-learn`.

Fitting and predicting: estimator basics

`Scikit-learn` provides dozens of built-in machine learning algorithms and models, called estimators. Each estimator can be fitted to some data using its fit method.

Here is a simple example where we fit a `RandomForestClassifier` to some very basic data:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> clf = RandomForestClassifier(random_state=0)
>>> X = [[ 1,  2,  3],  # 2 samples, 3 features
...      [11, 12, 13]]
>>> y = [0, 1]  # classes of each sample
>>> clf.fit(X, y)
RandomForestClassifier(random_state=0)
```

The fit method generally accepts 2 inputs:

- The samples matrix (or design matrix) X. The size of `X` is typically `(n_samples, n_features)`, which means that samples are represented as rows and features are represented as columns.
- The target values y which are real numbers for regression tasks, or integers for classification (or any other discrete set of values). For unsupervised learning tasks, `y` does not need to be specified. `y` is usually a 1d array where the `i` th entry corresponds to the target of the `i` th sample (row) of `X`.

Both `X` and `y` are usually expected to be numpy arrays or equivalent array-like data types, though some estimators work with other formats such as sparse matrices.

Once the estimator is fitted, it can be used for predicting target values of new data. You don't need to re-train the estimator:

```
>>> clf.predict(X)  # predict classes of the training data
array([0, 1])
>>> clf.predict([[4, 5, 6], [14, 15, 16]])  # predict classes of new data
array([0, 1])
```

You can check ml_map on how to choose the right model for your use case.

Transformers and pre-processors

Machine learning workflows are often composed of different parts. A typical pipeline consists of a pre-processing step that transforms or imputes the data, and a final predictor that predicts target values.

In `scikit-learn`, pre-processors and transformers follow the same API as the estimator objects (they actually all inherit from the same `BaseEstimator` class). The transformer objects don't have a predict method but rather a transform method that outputs a newly transformed sample matrix `X`:

```
>>> from sklearn.preprocessing import StandardScaler
>>> X = [[0, 15],
...      [1, -10]]
>>> # scale data according to computed scaling values
>>> StandardScaler().fit(X).transform(X)
array([[-1.,  1.],
       [ 1., -1.]])
```

Sometimes, you want to apply different transformations to different features: the ColumnTransformer is designed for these use-cases.

Pipelines: chaining pre-processors and estimators

Transformers and estimators (predictors) can be combined together into a single unifying object: a `Pipeline`. The pipeline offers the same API as a regular estimator: it can be fitted and used for prediction with `fit` and `predict`. As we will see later, using a pipeline will also prevent you from data leakage, i.e. disclosing some testing data in your training data.

In the following example, we load the Iris dataset, split it into train and test sets, and compute the accuracy score of a pipeline on the test data:

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
...
>>> # create a pipeline object
>>> pipe = make_pipeline(
...     StandardScaler(),
...     LogisticRegression()
... )
...
>>> # load the iris dataset and split it into train and test sets
>>> X, y = load_iris(return_X_y=True)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
...
>>> # fit the whole pipeline
>>> pipe.fit(X_train, y_train)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
>>> # we can now use it like any other estimator
>>> accuracy_score(pipe.predict(X_test), y_test)
0.97...
```

Model evaluation

Fitting a model to some data does not entail that it will predict well on unseen data. This needs to be directly evaluated. We have just seen the `train_test_split` helper that splits a dataset into train and test sets, but `scikit-learn` provides many other tools for model evaluation, in particular for `cross-validation <cross_validation>`.

We here briefly show how to perform a 5-fold cross-validation procedure, using the `cross_validate` helper. Note that it is also possible to manually iterate over the folds, use different data splitting strategies, and use custom scoring functions. Please refer to our User Guide for more details:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.model_selection import cross_validate
...
>>> X, y = make_regression(n_samples=1000, random_state=0)
>>> lr = LinearRegression()
...
>>> result = cross_validate(lr, X, y)  # defaults to 5-fold CV
>>> result['test_score']  # r_squared score is high because dataset is easy
array([1., 1., 1., 1., 1.])
```

Automatic parameter searches

All estimators have parameters (often called hyper-parameters in the literature) that can be tuned. The generalization power of an estimator often critically depends on a few parameters. For example a `RandomForestRegressor` has a `n_estimators` parameter that determines the number of trees in the forest, and a `max_depth` parameter that determines the maximum depth of each tree. Quite often, it is not clear what the exact values of these parameters should be since they depend on the data at hand.

`Scikit-learn` provides tools to automatically find the best parameter combinations (via cross-validation). In the following example, we randomly search over the parameter space of a random forest with a `RandomizedSearchCV` object. When the search is over, the `RandomizedSearchCV` behaves as a `RandomForestRegressor` that has been fitted with the best set of parameters. Read more in the `User Guide <grid_search>`:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.ensemble import RandomForestRegressor
>>> from sklearn.model_selection import RandomizedSearchCV
>>> from sklearn.model_selection import train_test_split
>>> from scipy.stats import randint
...
>>> # create a synthetic dataset
>>> X, y = make_regression(n_samples=20640,
...                        n_features=8,
...                        noise=0.1,
...                        random_state=0)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
...
>>> # define the parameter space that will be searched over
>>> param_distributions = {'n_estimators': randint(1, 5),
...                        'max_depth': randint(5, 10)}
...
>>> # now create a searchCV object and fit it to the data
>>> search = RandomizedSearchCV(estimator=RandomForestRegressor(random_state=0),
```

```
...                              n_iter=5,
...                              param_distributions=param_distributions,
...                              random_state=0)
>>> search.fit(X_train, y_train)
RandomizedSearchCV(estimator=RandomForestRegressor(random_state=0), n_iter=5,
                   param_distributions={'max_depth': ...,
                                        'n_estimators': ...},
                   random_state=0)
>>> search.best_params_
{'max_depth': 9, 'n_estimators': 4}

>>> # the search object now acts like a normal random forest estimator
>>> # with max_depth=9 and n_estimators=4
>>> search.score(X_test, y_test)
0.84...
```

Note
In practice, you almost always want to `search over a pipeline <composite_grid_search>`, instead of a single estimator. One of the main reasons is that if you apply a pre-processing step to the whole dataset without using a pipeline, and then perform any kind of cross-validation, you would be breaking the fundamental assumption of independence between training and testing data. Indeed, since you pre-processed the data using the whole dataset, some information about the test sets are available to the train sets. This will lead to over-estimating the generalization power of the estimator (you can read more in this Kaggle post).
Using a pipeline for cross-validation and searching will largely keep you from this common pitfall.

Next steps

We have briefly covered estimator fitting and predicting, pre-processing steps, pipelines, cross-validation tools and automatic hyper-parameter searches. This guide should give you an overview of some of the main features of the library, but there is much more to `scikit-learn`!

Please refer to our user_guide for details on all the tools that we provide. You can also find an exhaustive list of the public API in the api_ref.

You can also look at our numerous examples that illustrate the use of `scikit-learn` in many different contexts, or have a look at the external_resources for learning materials.

Common pitfalls and recommended practices

Source: *https://scikit-learn.org/stable/common_pitfalls.html*

The purpose of this chapter is to illustrate some common pitfalls and anti-patterns that occur when using scikit-learn. It provides examples of what **not** to do, along with a corresponding correct example.

Inconsistent preprocessing

scikit-learn provides a library of data-transforms, which may clean (see preprocessing), reduce (see data_reduction), expand (see kernel_approximation) or generate (see feature_extraction) feature representations. If these data transforms are used when training a model, they also must be used on subsequent datasets, whether it's test data or data in a production system. Otherwise, the feature space will change, and the model will not be able to perform effectively.

For the following example, let's create a synthetic dataset with a single feature:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.model_selection import train_test_split

>>> random_state = 42
>>> X, y = make_regression(random_state=random_state, n_features=1, noise=1)
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.4, random_state=random_state)
```

Wrong

The train dataset is scaled, but not the test dataset, so model performance on the test dataset is worse than expected:

```
>>> from sklearn.metrics import mean_squared_error
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.preprocessing import StandardScaler

>>> scaler = StandardScaler()
>>> X_train_transformed = scaler.fit_transform(X_train)
>>> model = LinearRegression().fit(X_train_transformed, y_train)
>>> mean_squared_error(y_test, model.predict(X_test))
62.80...
```

Right

Instead of passing the non-transformed X_test to predict, we should transform the test data, the same way we transformed the training data:

```
>>> X_test_transformed = scaler.transform(X_test)
>>> mean_squared_error(y_test, model.predict(X_test_transformed))
0.90...
```

Alternatively, we recommend using a `Pipeline <sklearn.pipeline.Pipeline>`, which makes it easier to chain transformations with estimators, and reduces the possibility of forgetting a transformation:

```
>>> from sklearn.pipeline import make_pipeline

>>> model = make_pipeline(StandardScaler(), LinearRegression())
>>> model.fit(X_train, y_train)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('linearregression', LinearRegression())])
>>> mean_squared_error(y_test, model.predict(X_test))
0.90...
```

Pipelines also help avoiding another common pitfall: leaking the test data into the training data.

Data leakage

Data leakage occurs when information that would not be available at prediction time is used when building the model. This results in overly optimistic performance estimates, for example from `cross-validation <cross_validation>`, and thus poorer performance when the model is used on actually novel data, for example during production.

A common cause is not keeping the test and train data subsets separate. Test data should never be used to make choices about the model. **The general rule is to never call** fit **on the test data**. While this may sound obvious, this is easy to miss in some cases, for example when applying certain pre-processing steps.

Although both train and test data subsets should receive the same preprocessing transformation (as described in the previous section), it is important that these transformations are only learnt from the training data. For example, if you have a normalization step where you divide by the average value, the average should be the average of the train subset, **not** the average of all the data. If the test subset is included in the average calculation, information from the test subset is influencing the model.

How to avoid data leakage

Below are some tips on avoiding data leakage:

- Always split the data into train and test subsets first, particularly before any preprocessing steps.
- Never include test data when using the fit and fit_transform methods. Using all the data, e.g., fit(X), can result in overly optimistic scores.

  Conversely, the transform method should be used on both train and test subsets as the same preprocessing should be applied to all the data. This can be achieved by using fit_transform on the train subset and transform on the test subset.
- The scikit-learn pipeline is a great way to prevent data leakage as it ensures that the appropriate method is performed on the correct data subset. The pipeline is ideal for use in cross-validation and hyper-parameter tuning functions.

An example of data leakage during preprocessing is detailed below.

Data leakage during pre-processing

> Note
> We here choose to illustrate data leakage with a feature selection step. This risk of leakage is however relevant with almost all transformations in scikit-learn, including (but not limited to)

A number of feature_selection functions are available in scikit-learn. They can help remove irrelevant, redundant and noisy features as well as improve your model build time and performance. As with any other type of preprocessing, feature selection should **only** use the training data. Including the test data in feature selection will optimistically bias your model.

To demonstrate we will create this binary classification problem with 10,000 randomly generated features:

```
>>> import numpy as np
>>> n_samples, n_features, n_classes = 200, 10000, 2
>>> rng = np.random.RandomState(42)
>>> X = rng.standard_normal((n_samples, n_features))
>>> y = rng.choice(n_classes, n_samples)
```

Wrong

Using all the data to perform feature selection results in an accuracy score much higher than chance, even though our targets are completely random. This randomness means that our X and y are independent and we thus expect the accuracy to be around 0.5. However, since the feature selection step 'sees' the test data, the model has an unfair advantage. In the incorrect example below we first use all the data for feature selection and then split the data into training and test subsets for model fitting. The result is a much higher than expected accuracy score:

```
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.feature_selection import SelectKBest
>>> from sklearn.ensemble import HistGradientBoostingClassifier
>>> from sklearn.metrics import accuracy_score

>>> # Incorrect preprocessing: the entire data is transformed
>>> X_selected = SelectKBest(k=25).fit_transform(X, y)

>>> X_train, X_test, y_train, y_test = train_test_split(
...     X_selected, y, random_state=42)
>>> gbc = HistGradientBoostingClassifier(random_state=1)
>>> gbc.fit(X_train, y_train)
HistGradientBoostingClassifier(random_state=1)

>>> y_pred = gbc.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.76
```

Right

To prevent data leakage, it is good practice to split your data into train and test subsets **first**. Feature selection can then be formed using just the train dataset. Notice that whenever we use fit or fit_transform, we only use the train dataset. The score is now what we would expect for the data, close to chance:

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, random_state=42)
>>> select = SelectKBest(k=25)
>>> X_train_selected = select.fit_transform(X_train, y_train)

>>> gbc = HistGradientBoostingClassifier(random_state=1)
>>> gbc.fit(X_train_selected, y_train)
HistGradientBoostingClassifier(random_state=1)

>>> X_test_selected = select.transform(X_test)
>>> y_pred = gbc.predict(X_test_selected)
```

```
>>> accuracy_score(y_test, y_pred)
0.5
```

Here again, we recommend using a `Pipeline` to chain together the feature selection and model estimators. The pipeline ensures that only the training data is used when performing fit and the test data is used only for calculating the accuracy score:

```
>>> from sklearn.pipeline import make_pipeline
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, random_state=42)
>>> pipeline = make_pipeline(SelectKBest(k=25),
...                          HistGradientBoostingClassifier(random_state=1))
>>> pipeline.fit(X_train, y_train)
Pipeline(steps=[('selectkbest', SelectKBest(k=25)),
                ('histgradientboostingclassifier',
                 HistGradientBoostingClassifier(random_state=1))])

>>> y_pred = pipeline.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.5
```

The pipeline can also be fed into a cross-validation function such as `cross_val_score`. Again, the pipeline ensures that the correct data subset and estimator method is used during fitting and predicting:

```
>>> from sklearn.model_selection import cross_val_score
>>> scores = cross_val_score(pipeline, X, y)
>>> print(f"Mean accuracy: {scores.mean():.2f}+/-{scores.std():.2f}")
Mean accuracy: 0.43+/-0.05
```

Controlling randomness

Some scikit-learn objects are inherently random. These are usually estimators (e.g. `RandomForestClassifier`) and cross-validation splitters (e.g. `KFold`). The randomness of these objects is controlled via their random_state parameter, as described in the Glossary. This section expands on the glossary entry, and describes good practices and common pitfalls w.r.t. this subtle parameter.

> Note
> Recommendation summary
> For an optimal robustness of cross-validation (CV) results, pass RandomState instances when creating estimators, or leave random_state to None. Passing integers to CV splitters is usually the safest option and is preferable; passing RandomState instances to splitters may sometimes be useful to achieve very specific use-cases. For both estimators and splitters, passing an integer vs passing an instance (or None) leads to subtle but significant differences, especially for CV procedures. These differences are important to understand when reporting results.
> For reproducible results across executions, remove any use of random_state=None.

Using None or RandomState instances, and repeated calls to fit and split

The random_state parameter determines whether multiple calls to fit (for estimators) or to split (for CV splitters) will produce the same results, according to these rules:

- If an integer is passed, calling fit or split multiple times always yields the same results.

- If None or a RandomState instance is passed: fit and split will yield different results each time they are called, and the succession of calls explores all sources of entropy. None is the default value for all random_state parameters.

We here illustrate these rules for both estimators and CV splitters.

Note
Since passing random_state=None is equivalent to passing the global RandomState instance from numpy (random_state=np.random.mtrand._rand), we will not explicitly mention None here. Everything that applies to instances also applies to using None.

Estimators

Passing instances means that calling fit multiple times will not yield the same results, even if the estimator is fitted on the same data and with the same hyper-parameters:

```
>>> from sklearn.linear_model import SGDClassifier
>>> from sklearn.datasets import make_classification
>>> import numpy as np

>>> rng = np.random.RandomState(0)
>>> X, y = make_classification(n_features=5, random_state=rng)
>>> sgd = SGDClassifier(random_state=rng)

>>> sgd.fit(X, y).coef_
array([[ 8.85418642,  4.79084103, -3.13077794,  8.11915045, -0.56479934]])

>>> sgd.fit(X, y).coef_
array([[ 6.70814003,  5.25291366, -7.55212743,  5.18197458,  1.37845099]])
```

We can see from the snippet above that repeatedly calling sgd.fit has produced different models, even if the data was the same. This is because the Random Number Generator (RNG) of the estimator is consumed (i.e. mutated) when fit is called, and this mutated RNG will be used in the subsequent calls to fit. In addition, the rng object is shared across all objects that use it, and as a consequence, these objects become somewhat inter-dependent. For example, two estimators that share the same RandomState instance will influence each other, as we will see later when we discuss cloning. This point is important to keep in mind when debugging.

If we had passed an integer to the random_state parameter of the `SGDClassifier`, we would have obtained the same models, and thus the same scores each time. When we pass an integer, the same RNG is used across all calls to fit. What internally happens is that even though the RNG is consumed when fit is called, it is always reset to its original state at the beginning of fit.

CV splitters

Randomized CV splitters have a similar behavior when a RandomState instance is passed; calling split multiple times yields different data splits:

```
>>> from sklearn.model_selection import KFold
>>> import numpy as np

>>> X = y = np.arange(10)
>>> rng = np.random.RandomState(0)
>>> cv = KFold(n_splits=2, shuffle=True, random_state=rng)

>>> for train, test in cv.split(X, y):
...     print(train, test)
```

```
[0 3 5 6 7] [1 2 4 8 9]
[1 2 4 8 9] [0 3 5 6 7]

>>> for train, test in cv.split(X, y):
...     print(train, test)
[0 4 6 7 8] [1 2 3 5 9]
[1 2 3 5 9] [0 4 6 7 8]
```

We can see that the splits are different from the second time split is called. This may lead to unexpected results if you compare the performance of multiple estimators by calling split many times, as we will see in the next section.

Common pitfalls and subtleties

While the rules that govern the random_state parameter are seemingly simple, they do however have some subtle implications. In some cases, this can even lead to wrong conclusions.

Estimators

Different random_state **types lead to different cross-validation procedures**

Depending on the type of the random_state parameter, estimators will behave differently, especially in cross-validation procedures. Consider the following snippet:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import cross_val_score
>>> import numpy as np

>>> X, y = make_classification(random_state=0)

>>> rf_123 = RandomForestClassifier(random_state=123)
>>> cross_val_score(rf_123, X, y)
array([0.85, 0.95, 0.95, 0.9 , 0.9 ])

>>> rf_inst = RandomForestClassifier(random_state=np.random.RandomState(0))
>>> cross_val_score(rf_inst, X, y)
array([0.9 , 0.95, 0.95, 0.9 , 0.9 ])
```

We see that the cross-validated scores of rf_123 and rf_inst are different, as should be expected since we didn't pass the same random_state parameter. However, the difference between these scores is more subtle than it looks, and **the cross-validation procedures that were performed by** `cross_val_score` **significantly differ in each case**:

- Since rf_123 was passed an integer, every call to fit uses the same RNG: this means that all random characteristics of the random forest estimator will be the same for each of the 5 folds of the CV procedure. In particular, the (randomly chosen) subset of features of the estimator will be the same across all folds.
- Since rf_inst was passed a RandomState instance, each call to fit starts from a different RNG. As a result, the random subset of features will be different for each fold.

While having a constant estimator RNG across folds isn't inherently wrong, we usually want CV results that are robust w.r.t. the estimator's randomness. As a result, passing an instance instead of an integer may be preferable, since it will allow the estimator RNG to vary for each fold.

Note
Here, `cross_val_score` will use a non-randomized CV splitter (as is the default), so both estimators will be evaluated on the same splits. This section is not about variability in the splits. Also, whether we pass an integer or an instance to illustration purpose: what matters is what we pass to the

Cloning
Another subtle side effect of passing RandomState instances is how

```
>>> from sklearn import clone
>>> from sklearn.ensemble import RandomForestClassifier
>>> import numpy as np
```

```
>>> rng = np.random.RandomState(0)
>>> a = RandomForestClassifier(random_state=rng)
>>> b = clone(a)
```

Since a RandomState instance was passed to a, a and b are not clones in the strict sense, but rather clones in the statistical sense: a and b will still be different models, even when calling fit(X, y) on the same data. Moreover, a and b will influence each other since they share the same internal RNG: calling a.fit will consume b's RNG, and calling b.fit will consume a's RNG, since they are the same. This bit is true for any estimators that share a random_state parameter; it is not specific to clones.
If an integer were passed, a and b would be exact clones and they would not influence each other.

Warning
Even though `clone` is rarely used in user code, it is called pervasively throughout scikit-learn codebase: in particular, most meta-estimators that accept non-fitted estimators call (`GridSearchCV`,

CV splitters

When passed a RandomState instance, CV splitters yield different splits each time split is called. When comparing different estimators, this can lead to overestimating the variance of the difference in performance between the estimators:

```
>>> from sklearn.naive_bayes import GaussianNB
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import KFold
>>> from sklearn.model_selection import cross_val_score
>>> import numpy as np

>>> rng = np.random.RandomState(0)
>>> X, y = make_classification(random_state=rng)
>>> cv = KFold(shuffle=True, random_state=rng)
>>> lda = LinearDiscriminantAnalysis()
>>> nb = GaussianNB()

>>> for est in (lda, nb):
...     print(cross_val_score(est, X, y, cv=cv))
[0.8  0.75 0.75 0.7  0.85]
[0.85 0.95 0.95 0.85 0.95]
```

Directly comparing the performance of the `LinearDiscriminantAnalysis` estimator vs the `GaussianNB`

estimator **on each fold** would be a mistake: **the splits on which the estimators are evaluated are different**. Indeed, `cross_val_score` will internally call cv.split on the same `KFold` instance, but the splits will be different each time. This is also true for any tool that performs model selection via cross-validation, e.g. `GridSearchCV` and `RandomizedSearchCV`: scores are not comparable fold-to-fold across different calls to search.fit, since cv.split would have been called multiple times. Within a single call to search.fit, however, fold-to-fold comparison is possible since the search estimator only calls cv.split once.

For comparable fold-to-fold results in all scenarios, one should pass an integer to the CV splitter: cv = KFold(shuffle=True, random_state=0).

Note
While fold-to-fold comparison is not advisable with RandomState instances, one can however expect that average scores allow to conclude whether one estimator is better than another, as long as enough folds and data are used.

Note
What matters in this example is what was passed to instance or an integer to `make_classification` is not relevant for our illustration purpose. Also, neither

General recommendations

Getting reproducible results across multiple executions

In order to obtain reproducible (i.e. constant) results across multiple *program executions*, we need to remove all uses of random_state=None, which is the default. The recommended way is to declare a rng variable at the top of the program, and pass it down to any object that accepts a random_state parameter:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import train_test_split
>>> import numpy as np

>>> rng = np.random.RandomState(0)
>>> X, y = make_classification(random_state=rng)
>>> rf = RandomForestClassifier(random_state=rng)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...                                                     random_state=rng)
>>> rf.fit(X_train, y_train).score(X_test, y_test)
0.84
```

We are now guaranteed that the result of this script will always be 0.84, no matter how many times we run it. Changing the global rng variable to a different value should affect the results, as expected.

It is also possible to declare the rng variable as an integer. This may however lead to less robust cross-validation results, as we will see in the next section.

Note
We do not recommend setting the global numpy seed by calling np.random.seed(0). See here for a discussion.

Robustness of cross-validation results

When we evaluate a randomized estimator performance by cross-validation, we want to make sure that the estimator can yield accurate predictions for new data, but we also want to make sure that the estimator is robust w.r.t. its random initialization. For example, we would like the random weights initialization of an `SGDClassifier` to be consistently good across all folds: otherwise, when we train that estimator on new data, we might get unlucky and the random initialization may lead to bad performance. Similarly, we want a random forest to be robust w.r.t. the set of randomly selected features that each tree will be using.

For these reasons, it is preferable to evaluate the cross-validation performance by letting the estimator use a different RNG on each fold. This is done by passing a RandomState instance (or None) to the estimator initialization.

When we pass an integer, the estimator will use the same RNG on each fold: if the estimator performs well (or bad), as evaluated by CV, it might just be because we got lucky (or unlucky) with that specific seed. Passing instances leads to more robust CV results, and makes the comparison between various algorithms fairer. It also helps limiting the temptation to treat the estimator's RNG as a hyper-parameter that can be tuned.

Whether we pass RandomState instances or integers to CV splitters has no impact on robustness, as long as split is only called once. When split is called multiple times, fold-to-fold comparison isn't possible anymore. As a result, passing integer to CV splitters is usually safer and covers most use-cases.

Preprocessing data

Source: *https://scikit-learn.org/stable/modules/preprocessing.html*

The `sklearn.preprocessing` package provides several common utility functions and transformer classes to change raw feature vectors into a representation that is more suitable for the downstream estimators.

In general, many learning algorithms such as linear models benefit from standardization of the data set (see sphx_glr_auto_examples_preprocessing_plot_scaling_importance.py). If some outliers are present in the set, robust scalers or other transformers can be more appropriate. The behaviors of the different scalers, transformers, and normalizers on a dataset containing marginal outliers are highlighted in sphx_glr_auto_examples_preprocessing_plot_all_scaling.py.

Standardization, or mean removal and variance scaling

Standardization of datasets is a **common requirement for many machine learning estimators** implemented in scikit-learn; they might behave badly if the individual features do not more or less look like standard normally distributed data: Gaussian with **zero mean and unit variance**.

In practice we often ignore the shape of the distribution and just transform the data to center it by removing the mean value of each feature, then scale it by dividing non-constant features by their standard deviation.

For instance, many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the l1 and l2 regularizers of linear models) may assume that all features are centered around zero or have variance in the same order. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.

The `preprocessing` module provides the `StandardScaler` utility class, which is a quick and easy way to perform the following operation on an array-like dataset:

```
>>> from sklearn import preprocessing
>>> import numpy as np
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1., -1.]])
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> scaler
StandardScaler()

>>> scaler.mean_
array([1., 0., 0.33])

>>> scaler.scale_
array([0.81, 0.81, 1.24])

>>> X_scaled = scaler.transform(X_train)
>>> X_scaled
array([[ 0.  , -1.22,  1.33 ],
       [ 1.22,  0.  , -0.267],
```

```
       [-1.22,  1.22, -1.06 ]])
```

Scaled data has zero mean and unit variance:

```
>>> X_scaled.mean(axis=0)
array([0., 0., 0.])

>>> X_scaled.std(axis=0)
array([1., 1., 1.])
```

This class implements the `Transformer` API to compute the mean and standard deviation on a training set so as to be able to later re-apply the same transformation on the testing set. This class is hence suitable for use in the early steps of a `Pipeline`:

```
>>> from sklearn.datasets import make_classification
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.preprocessing import StandardScaler

>>> X, y = make_classification(random_state=42)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
>>> pipe = make_pipeline(StandardScaler(), LogisticRegression())
>>> pipe.fit(X_train, y_train)  # apply scaling on training data
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])

>>> pipe.score(X_test, y_test)  # apply scaling on testing data, without leaking training data.
0.96
```

It is possible to disable either centering or scaling by either passing `with_mean=False` or `with_std=False` to the constructor of `StandardScaler`.

Scaling features to a range

An alternative standardization is scaling features to lie between a given minimum and maximum value, often between zero and one, or so that the maximum absolute value of each feature is scaled to unit size. This can be achieved using `MinMaxScaler` or `MaxAbsScaler`, respectively.

The motivation to use this scaling includes robustness to very small standard deviations of features and preserving zero entries in sparse data.

Here is an example to scale a toy data matrix to the `[0, 1]` range:

```
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1., -1.]])
...
>>> min_max_scaler = preprocessing.MinMaxScaler()
>>> X_train_minmax = min_max_scaler.fit_transform(X_train)
>>> X_train_minmax
array([[0.5       , 0.        , 1.        ],
       [1.        , 0.5       , 0.33333333],
       [0.        , 1.        , 0.        ]])
```

The same instance of the transformer can then be applied to some new test data unseen during the fit call: the same scaling and shifting operations will be applied to be consistent with the transformation performed on the train data:

```
>>> X_test = np.array([[-3., -1.,  4.]])
```

```
>>> X_test_minmax = min_max_scaler.transform(X_test)
>>> X_test_minmax
array([[-1.5       ,  0.        ,  1.66666667]])
```

It is possible to introspect the scaler attributes to find about the exact nature of the transformation learned on the training data:

```
>>> min_max_scaler.scale_
array([0.5       , 0.5       , 0.33])

>>> min_max_scaler.min_
array([0.        , 0.5       , 0.33])
```

If `MinMaxScaler` is given an explicit `feature_range=(min, max)` the full formula is:

```
X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))

X_scaled = X_std * (max - min) + min
```

`MaxAbsScaler` works in a very similar fashion, but scales in a way that the training data lies within the range `[-1, 1]` by dividing through the largest maximum value in each feature. It is meant for data that is already centered at zero or sparse data.

Here is how to use the toy data from the previous example with this scaler:

```
>>> X_train = np.array([[ 1., -1.,  2.],
...                     [ 2.,  0.,  0.],
...                     [ 0.,  1., -1.]])
...
>>> max_abs_scaler = preprocessing.MaxAbsScaler()
>>> X_train_maxabs = max_abs_scaler.fit_transform(X_train)
>>> X_train_maxabs
array([[ 0.5, -1. ,  1. ],
       [ 1. ,  0. ,  0. ],
       [ 0. ,  1. , -0.5]])
>>> X_test = np.array([[ -3., -1.,  4.]])
>>> X_test_maxabs = max_abs_scaler.transform(X_test)
>>> X_test_maxabs
array([[-1.5, -1. ,  2. ]])
>>> max_abs_scaler.scale_
array([2.,  1.,  2.])
```

Scaling sparse data

Centering sparse data would destroy the sparseness structure in the data, and thus rarely is a sensible thing to do. However, it can make sense to scale sparse inputs, especially if features are on different scales.

`MaxAbsScaler` was specifically designed for scaling sparse data, and is the recommended way to go about this. However, `StandardScaler` can accept `scipy.sparse` matrices as input, as long as `with_mean=False` is explicitly passed to the constructor. Otherwise a `ValueError` will be raised as silently centering would break the sparsity and would often crash the execution by allocating excessive amounts of memory unintentionally. `RobustScaler` cannot be fitted to sparse inputs, but you can use the `transform` method on sparse inputs.

Note that the scalers accept both Compressed Sparse Rows and Compressed Sparse Columns format (see `scipy.sparse.csr_matrix` and `scipy.sparse.csc_matrix`). Any other sparse input will be **converted to the Compressed Sparse Rows representation**. To avoid unnecessary memory copies, it is recommended to choose the CSR or CSC representation upstream.

Finally, if the centered data is expected to be small enough, explicitly converting the input to an array using the `toarray` method of sparse matrices is another option.

Scaling data with outliers

If your data contains many outliers, scaling using the mean and variance of the data is likely to not work very well. In these cases, you can use `RobustScaler` as a drop-in replacement instead. It uses more robust estimates for the center and range of your data.

References
Further discussion on the importance of centering and scaling data is available on this FAQ: Should I normalize/standardize/rescale the data?

Scaling vs Whitening
It is sometimes not enough to center and scale the features independently, since a downstream model can further make some assumption on the linear independence of the features. To address this issue you can use `PCA` with `whiten=True` to further remove the linear correlation across features.

Centering kernel matrices

If you have a kernel matrix of a kernel K that computes a dot product in a feature space (possibly implicitly) defined by a function $\phi(\cdot)$, a `KernelCenterer` can transform the kernel matrix so that it contains inner products in the feature space defined by ϕ followed by the removal of the mean in that space. In other words, `KernelCenterer` computes the centered Gram matrix associated to a positive semidefinite kernel K.

Mathematical formulation
We can have a look at the mathematical formulation now that we have the intuition. Let K be a kernel matrix of shape (n_samples, n_samples) computed from X, a data matrix of shape (n_samples, n_features), during the fit step. K is defined by

$$K(X, X) = \phi(X).\phi(X)^T$$

centered kernel $\tilde{K}$ is defined as:

$$\tilde{K}(X, X) = \tilde{\phi}(X).\tilde{\phi}(X)^T$$

where $\tilde{\phi}(X)$ results from centering $\phi(X)$ in the Hilbert space.
Thus, one could compute $\tilde{K}$ by mapping X using the function $\phi(\cdot)$ and center the data in this new space. However, kernels are often used because they allow some algebra calculations that avoid computing explicitly this mapping using $\phi(\cdot)$. Indeed, one can implicitly center as shown in Appendix B in [Scholkopf1998]:

$$\tilde{K} = K - 1_{\mathrm{n}_{samples}} K - K 1_{\mathrm{n}_{samples}} + 1_{\mathrm{n}_{samples}} K 1_{\mathrm{n}_{samples}}$$

all entries are equal to $\frac{1}{\mathrm{n}_{samples}}$. In the transform step, the kernel becomes $K_{test}(X, Y)$ defined as:

$$K_{test}(X, Y) = \phi(Y).\phi(X)^T$$

centering K_{test} is done as:

$$\tilde{K}_{test}(X, Y) = K_{test} - 1'_{\mathrm{n}_{samples}} K - K_{test} 1_{\mathrm{n}_{samples}} + 1'_{\mathrm{n}_{samples}} K 1_{\mathrm{n}_{samples}}$$

(n_samples_test, n_samples) where all entries are equal to
References

Non-linear transformation

Two types of transformations are available: quantile transforms and power transforms. Both quantile and power transforms are based on monotonic transformations of the features and thus preserve the rank of the values along each feature.

Quantile transforms put all features into the same desired distribution based on the formula $G^{-1}(F(X))$ where F is the cumulative distribution function of the feature and G^{-1} the quantile function of the desired output distribution G. This formula is using the two following facts: (i) if X is a random variable with a continuous cumulative distribution function F then $F(X)$ is uniformly distributed on $[0, 1]$; (ii) if U is a random variable with uniform distribution on $[0, 1]$ then $G^{-1}(U)$ has distribution G. By performing a rank transformation, a quantile transform smooths out unusual distributions and is less influenced by outliers than scaling methods. It does, however, distort correlations and distances within and across features.

Power transforms are a family of parametric transformations that aim to map data from any distribution to as close to a Gaussian distribution.

Mapping to a Uniform distribution

`QuantileTransformer` provides a non-parametric transformation to map the data to a uniform distribution with values between 0 and 1:

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import train_test_split
>>> X, y = load_iris(return_X_y=True)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
>>> quantile_transformer = preprocessing.QuantileTransformer(random_state=0)
>>> X_train_trans = quantile_transformer.fit_transform(X_train)
>>> X_test_trans = quantile_transformer.transform(X_test)
>>> np.percentile(X_train[:, 0], [0, 25, 50, 75, 100])
array([ 4.3,  5.1,  5.8,  6.5,  7.9])
```

This feature corresponds to the sepal length in cm. Once the quantile transformation is applied, those landmarks approach closely the percentiles previously defined:

```
>>> np.percentile(X_train_trans[:, 0], [0, 25, 50, 75, 100])
...
array([ 0.00 ,  0.24,  0.49,  0.73,  0.99 ])
```

This can be confirmed on an independent testing set with similar remarks:

```
>>> np.percentile(X_test[:, 0], [0, 25, 50, 75, 100])
...
array([ 4.4  ,  5.125,  5.75 ,  6.175,  7.3  ])
>>> np.percentile(X_test_trans[:, 0], [0, 25, 50, 75, 100])
...
array([ 0.01,  0.25,  0.46,  0.60 ,  0.94])
```

Mapping to a Gaussian distribution

In many modeling scenarios, normality of the features in a dataset is desirable. Power transforms are a family of parametric, monotonic transformations that aim to map data from any distribution

to as close to a Gaussian distribution as possible in order to stabilize variance and minimize skewness.

`PowerTransformer` currently provides two such power transformations, the Yeo-Johnson transform and the Box-Cox transform.

Yeo-Johnson transform

$$x_i^{(\lambda)} = \begin{cases} [(x_i + 1)^\lambda - 1]/\lambda & \text{if } \lambda \neq 0, x_i \geq 0, \\ \ln(x_i + 1) & \text{if } \lambda = 0, x_i \geq 0 \\ -[(-x_i + 1)^{2-\lambda} - 1]/(2 - \lambda) & \text{if } \lambda \neq 2, x_i < 0, \\ -\ln(-x_i + 1) & \text{if } \lambda = 2, x_i < 0 \end{cases}$$

Box-Cox transform

$$x_i^{(\lambda)} = \begin{cases} \dfrac{x_i^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln(x_i) & \text{if } \lambda = 0, \end{cases}$$

Box-Cox can only be applied to strictly positive data. In both methods, the transformation is parameterized by λ, which is determined through maximum likelihood estimation. Here is an example of using Box-Cox to map samples drawn from a lognormal distribution to a normal distribution:

```
>>> pt = preprocessing.PowerTransformer(method='box-cox', standardize=False)
>>> X_lognormal = np.random.RandomState(616).lognormal(size=(3, 3))
>>> X_lognormal
array([[1.28, 1.18 , 0.84 ],
       [0.94, 1.60 , 0.388],
       [1.35, 0.217, 1.09 ]])
>>> pt.fit_transform(X_lognormal)
array([[ 0.49 ,  0.179, -0.156],
       [-0.051,  0.589, -0.576],
       [ 0.69 , -0.849,  0.101]])
```

While the above example sets the standardize option to False, to the transformed output by default.

Below are examples of Box-Cox and Yeo-Johnson applied to various probability distributions. Note that when applied to certain distributions, the power transforms achieve very Gaussian-like results, but with others, they are ineffective. This highlights the importance of visualizing the data before and after transformation.

It is also possible to map data to a normal distribution using `QuantileTransformer` by setting `output_distribution='normal'`. Using the earlier example with the iris dataset:

```
>>> quantile_transformer = preprocessing.QuantileTransformer(
...     output_distribution='normal', random_state=0)
>>> X_trans = quantile_transformer.fit_transform(X)
>>> quantile_transformer.quantiles_
array([[4.3, 2. , 1. , 0.1],
       [4.4, 2.2, 1.1, 0.1],
       [4.4, 2.2, 1.2, 0.1],
       ...,
       [7.7, 4.1, 6.7, 2.5],
```

```
       [7.7, 4.2, 6.7, 2.5],
       [7.9, 4.4, 6.9, 2.5]])
```

Thus the median of the input becomes the mean of the output, centered at 0. The normal output is clipped so that the input's minimum and maximum ---corresponding to the 1e-7 and 1 - 1e-7 quantiles respectively --- do not become infinite under the transformation.

Normalization

Normalization is the process of **scaling individual samples to have unit norm**. This process can be useful if you plan to use a quadratic form such as the dot-product or any other kernel to quantify the similarity of any pair of samples.

This assumption is the base of the Vector Space Model often used in text classification and clustering contexts.

The function `normalize` provides a quick and easy way to perform this operation on a single array-like dataset, either using the `l1`, `l2`, or `max` norms:

```
>>> X = [[ 1., -1.,  2.],
...      [ 2.,  0.,  0.],
...      [ 0.,  1., -1.]]
>>> X_normalized = preprocessing.normalize(X, norm='l2')

>>> X_normalized
array([[ 0.408, -0.408,  0.812],
       [ 1.   ,  0.   ,  0.   ],
       [ 0.   ,  0.707, -0.707]])
```

The `preprocessing` module further provides a utility class `Normalizer` that implements the same operation using the `Transformer` API (even though the `fit` method is useless in this case: the class is stateless as this operation treats samples independently).

This class is hence suitable for use in the early steps of a `Pipeline`:

```
>>> normalizer = preprocessing.Normalizer().fit(X)  # fit does nothing
>>> normalizer
Normalizer()
```

The normalizer instance can then be used on sample vectors as any transformer:

```
>>> normalizer.transform(X)
array([[ 0.408, -0.408,  0.812],
       [ 1.   ,  0.   ,  0.   ],
       [ 0.   ,  0.707, -0.707]])

>>> normalizer.transform([[-1.,  1., 0.]])
array([[-0.707,  0.707,  0.]])
```

Note: L2 normalization is also known as spatial sign preprocessing.

Sparse input
and sparse matrices from scipy.sparse as input**.
For sparse input the data is **converted to the Compressed Sparse Rows representation** (see `scipy.sparse.csr_matrix`) before being fed to efficient Cython routines. To avoid unnecessary memory copies, it is recommended to choose the CSR representation upstream.

Encoding categorical features

Often features are not given as continuous values but categorical. For example a person could have features `["male", "female"]`, `["from Europe", "from US", "from Asia"]`, `["uses Firefox", "uses Chrome", "uses Safari", "uses Internet Explorer"]`. Such features can be efficiently coded as integers, for instance `["male", "from US", "uses Internet Explorer"]` could be expressed as `[0, 1, 3]` while `["female", "from Asia", "uses Chrome"]` would be `[1, 2, 1]`.

To convert categorical features to such integer codes, we can use the `OrdinalEncoder`. This estimator transforms each categorical feature to one new feature of integers (0 to n_categories - 1):

```
>>> enc = preprocessing.OrdinalEncoder()
>>> X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
>>> enc.fit(X)
OrdinalEncoder()
>>> enc.transform([['female', 'from US', 'uses Safari']])
array([[0., 1., 1.]])
```

Such integer representation can, however, not be used directly with all scikit-learn estimators, as these expect continuous input, and would interpret the categories as being ordered, which is often not desired (i.e. the set of browsers was ordered arbitrarily).

By default, `OrdinalEncoder` will also passthrough missing values that are indicated by np.nan.

```
>>> enc = preprocessing.OrdinalEncoder()
>>> X = [['male'], ['female'], [np.nan], ['female']]
>>> enc.fit_transform(X)
array([[ 1.],
       [ 0.],
       [nan],
       [ 0.]])
```

`OrdinalEncoder` provides a parameter encoded_missing_value to encode the missing values without the need to create a pipeline and using `SimpleImputer`.

```
>>> enc = preprocessing.OrdinalEncoder(encoded_missing_value=-1)
>>> X = [['male'], ['female'], [np.nan], ['female']]
>>> enc.fit_transform(X)
array([[ 1.],
       [ 0.],
       [-1.],
       [ 0.]])
```

The above processing is equivalent to the following pipeline:

```
>>> from sklearn.pipeline import Pipeline
>>> from sklearn.impute import SimpleImputer
>>> enc = Pipeline(steps=[
...     ("encoder", preprocessing.OrdinalEncoder()),
...     ("imputer", SimpleImputer(strategy="constant", fill_value=-1)),
... ])
>>> enc.fit_transform(X)
array([[ 1.],
       [ 0.],
       [-1.],
       [ 0.]])
```

Another possibility to convert categorical features to features that can be used with scikit-learn estimators is to use a one-of-K, also known as one-hot or dummy encoding. This type of en-

coding can be obtained with the `OneHotEncoder`, which transforms each categorical feature with `n_categories` possible values into `n_categories` binary features, with one of them 1, and all others 0.

Continuing the example above:

```
>>> enc = preprocessing.OneHotEncoder()
>>> X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
>>> enc.fit(X)
OneHotEncoder()
>>> enc.transform([['female', 'from US', 'uses Safari'],
...                ['male', 'from Europe', 'uses Safari']]).toarray()
array([[1., 0., 0., 1., 0., 1.],
       [0., 1., 1., 0., 0., 1.]])
```

By default, the values each feature can take is inferred automatically from the dataset and can be found in the `categories_` attribute:

```
>>> enc.categories_
[array(['female', 'male'], dtype=object), array(['from Europe', 'from US'], dtype=object), array(['uses Firefox
```

It is possible to specify this explicitly using the parameter `categories`. There are two genders, four possible continents and four web browsers in our dataset:

```
>>> genders = ['female', 'male']
>>> locations = ['from Africa', 'from Asia', 'from Europe', 'from US']
>>> browsers = ['uses Chrome', 'uses Firefox', 'uses IE', 'uses Safari']
>>> enc = preprocessing.OneHotEncoder(categories=[genders, locations, browsers])
>>> # Note that for there are missing categorical values for the 2nd and 3rd
>>> # feature
>>> X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
>>> enc.fit(X)
OneHotEncoder(categories=[['female', 'male'],
                          ['from Africa', 'from Asia', 'from Europe',
                           'from US'],
                          ['uses Chrome', 'uses Firefox', 'uses IE',
                           'uses Safari']])
>>> enc.transform([['female', 'from Asia', 'uses Chrome']]).toarray()
array([[1., 0., 0., 1., 0., 0., 1., 0., 0., 0.]])
```

If there is a possibility that the training data might have missing categorical features, it can often be better to specify handle_unknown='infrequent_if_exist' instead of setting the categories manually as above. When handle_unknown='infrequent_if_exist' is specified and unknown categories are encountered during transform, no error will be raised but the resulting one-hot encoded columns for this feature will be all zeros or considered as an infrequent category if enabled. (handle_unknown='infrequent_if_exist' is only supported for one-hot encoding):

```
>>> enc = preprocessing.OneHotEncoder(handle_unknown='infrequent_if_exist')
>>> X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
>>> enc.fit(X)
OneHotEncoder(handle_unknown='infrequent_if_exist')
>>> enc.transform([['female', 'from Asia', 'uses Chrome']]).toarray()
array([[1., 0., 0., 0., 0., 0.]])
```

It is also possible to encode each column into `n_categories - 1` columns instead of `n_categories` columns by using the `drop` parameter. This parameter allows the user to specify a category for each feature to be dropped. This is useful to avoid co-linearity in the input matrix in some classifiers. Such functionality is useful, for example, when using non-regularized regression (`LinearRegression`), since co-linearity would cause the covariance matrix to be non-invertible:

```
>>> X = [['male', 'from US', 'uses Safari'],
...      ['female', 'from Europe', 'uses Firefox']]
>>> drop_enc = preprocessing.OneHotEncoder(drop='first').fit(X)
>>> drop_enc.categories_
[array(['female', 'male'], dtype=object), array(['from Europe', 'from US'], dtype=object),
 array(['uses Firefox', 'uses Safari'], dtype=object)]
>>> drop_enc.transform(X).toarray()
array([[1., 1., 1.],
       [0., 0., 0.]])
```

One might want to drop one of the two columns only for features with 2 categories. In this case, you can set the parameter drop='if_binary'.

```
>>> X = [['male', 'US', 'Safari'],
...      ['female', 'Europe', 'Firefox'],
...      ['female', 'Asia', 'Chrome']]
>>> drop_enc = preprocessing.OneHotEncoder(drop='if_binary').fit(X)
>>> drop_enc.categories_
[array(['female', 'male'], dtype=object), array(['Asia', 'Europe', 'US'], dtype=object),
 array(['Chrome', 'Firefox', 'Safari'], dtype=object)]
>>> drop_enc.transform(X).toarray()
array([[1., 0., 0., 1., 0., 0., 1.],
       [0., 0., 1., 0., 0., 1., 0.],
       [0., 1., 0., 0., 1., 0., 0.]])
```

In the transformed X, the first column is the encoding of the feature with categories ”male”/”female”, while the remaining 6 columns are the encoding of the 2 features with respectively 3 categories each.

When handle_unknown='ignore' and drop is not None, unknown categories will be encoded as all zeros:

```
>>> drop_enc = preprocessing.OneHotEncoder(drop='first',
...                                        handle_unknown='ignore').fit(X)
>>> X_test = [['unknown', 'America', 'IE']]
>>> drop_enc.transform(X_test).toarray()
array([[0., 0., 0., 0., 0.]])
```

All the categories in X_test are unknown during transform and will be mapped to all zeros. This means that unknown categories will have the same mapping as the dropped category. `OneHotEncoder.inverse_transform` will map all zeros to the dropped category if a category is dropped and None if a category is not dropped:

```
>>> drop_enc = preprocessing.OneHotEncoder(drop='if_binary', sparse_output=False,
...                                        handle_unknown='ignore').fit(X)
>>> X_test = [['unknown', 'America', 'IE']]
>>> X_trans = drop_enc.transform(X_test)
>>> X_trans
array([[0., 0., 0., 0., 0., 0., 0.]])
>>> drop_enc.inverse_transform(X_trans)
array([['female', None, None]], dtype=object)
```

Support of categorical features with missing values
considering the missing values as an additional category:

```
>>> X = [['male', 'Safari'],
...      ['female', None],
...      [np.nan, 'Firefox']]
>>> enc = preprocessing.OneHotEncoder(handle_unknown='error').fit(X)
```

```
>>> enc.categories_
[array(['female', 'male', nan], dtype=object),
array(['Firefox', 'Safari', None], dtype=object)]
>>> enc.transform(X).toarray()
array([[0., 1., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0., 1.],
       [0., 0., 1., 1., 0., 0.]])
```

If a feature contains both np.nan and None, they will be considered separate categories:

```
>>> X = [['Safari'], [None], [np.nan], ['Firefox']]
>>> enc = preprocessing.OneHotEncoder(handle_unknown='error').fit(X)
>>> enc.categories_
[array(['Firefox', 'Safari', None, nan], dtype=object)]
>>> enc.transform(X).toarray()
array([[0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.],
       [1., 0., 0., 0.]])
```

See dict_feature_extraction for categorical features that are represented as a dict, not as scalars.

Infrequent categories

`OneHotEncoder` and `OrdinalEncoder` support aggregating infrequent categories into a single output for each feature. The parameters to enable the gathering of infrequent categories are min_frequency and max_categories.

1. min_frequency is either an integer greater or equal to 1, or a float in the interval (0.0, 1.0). If min_frequency is an integer, categories with a cardinality smaller than min_frequency will be considered infrequent. If min_frequency is a float, categories with a cardinality smaller than this fraction of the total number of samples will be considered infrequent. The default value is 1, which means every category is encoded separately.
2. max_categories is either None or any integer greater than 1. This parameter sets an upper limit to the number of output features for each input feature. max_categories includes the feature that combines infrequent categories.

In the following example with `OrdinalEncoder`, the categories 'dog' and 'snake' are considered infrequent:

```
>>> X = np.array([['dog'] * 5 + ['cat'] * 20 + ['rabbit'] * 10 +
...               ['snake'] * 3], dtype=object).T
>>> enc = preprocessing.OrdinalEncoder(min_frequency=6).fit(X)
>>> enc.infrequent_categories_
[array(['dog', 'snake'], dtype=object)]
>>> enc.transform(np.array([['dog'], ['cat'], ['rabbit'], ['snake']]))
array([[2.],
       [0.],
       [1.],
       [2.]])
```

`OrdinalEncoder`'s max_categories do **not** take into account missing or unknown categories. Setting unknown_value or encoded_missing_value to an integer will increase the number of unique integer codes by one each. This can result in up to max_categories + 2 integer codes. In the following example, ”a” and ”d” are considered infrequent and grouped together into a single category, ”b” and ”c” are their own categories, unknown values are encoded as 3 and missing values are encoded as 4.

```
>>> X_train = np.array(
...     [["a"] * 5 + ["b"] * 20 + ["c"] * 10 + ["d"] * 3 + [np.nan]],
...     dtype=object).T
>>> enc = preprocessing.OrdinalEncoder(
...     handle_unknown="use_encoded_value", unknown_value=3,
...     max_categories=3, encoded_missing_value=4)
>>> _ = enc.fit(X_train)
>>> X_test = np.array([["a"], ["b"], ["c"], ["d"], ["e"], [np.nan]], dtype=object)
>>> enc.transform(X_test)
array([[2.],
       [0.],
       [1.],
       [2.],
       [3.],
       [4.]])
```

Similarly, `OneHotEncoder` can be configured to group together infrequent categories:

```
>>> enc = preprocessing.OneHotEncoder(min_frequency=6, sparse_output=False).fit(X)
>>> enc.infrequent_categories_
[array(['dog', 'snake'], dtype=object)]
>>> enc.transform(np.array([['dog'], ['cat'], ['rabbit'], ['snake']]))
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

By setting handle_unknown to 'infrequent_if_exist', unknown categories will be considered infrequent:

```
>>> enc = preprocessing.OneHotEncoder(
...     handle_unknown='infrequent_if_exist', sparse_output=False, min_frequency=6)
>>> enc = enc.fit(X)
>>> enc.transform(np.array([['dragon']]))
array([[0., 0., 1.]])
```

`OneHotEncoder.get_feature_names_out` uses 'infrequent' as the infrequent feature name:

```
>>> enc.get_feature_names_out()
array(['x0_cat', 'x0_rabbit', 'x0_infrequent_sklearn'], dtype=object)
```

When 'handle_unknown' is set to 'infrequent_if_exist' and an unknown category is encountered in transform:

1. If infrequent category support was not configured or there was no infrequent category during training, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None.
2. If there is an infrequent category during training, the unknown category will be considered infrequent. In the inverse transform, 'infrequent_sklearn' will be used to represent the infrequent category.

Infrequent categories can also be configured using max_categories. In the following example, we set max_categories=2 to limit the number of features in the output. This will result in all but the 'cat' category to be considered infrequent, leading to two features, one for 'cat' and one for infrequent categories - which are all the others:

```
>>> enc = preprocessing.OneHotEncoder(max_categories=2, sparse_output=False)
>>> enc = enc.fit(X)
>>> enc.transform([['dog'], ['cat'], ['rabbit'], ['snake']])
array([[0., 1.],
```

```
       [1., 0.],
       [0., 1.],
       [0., 1.]])
```

If both max_categories and min_frequency are non-default values, then categories are selected based on min_frequency first and max_categories categories are kept. In the following example, min_frequency=4 considers only snake to be infrequent, but max_categories=3, forces dog to also be infrequent:

```
>>> enc = preprocessing.OneHotEncoder(min_frequency=4, max_categories=3, sparse_output=False)
>>> enc = enc.fit(X)
>>> enc.transform([['dog'], ['cat'], ['rabbit'], ['snake']])
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

If there are infrequent categories with the same cardinality at the cutoff of max_categories, then the first max_categories are taken based on lexicon ordering. In the following example, "b", "c", and "d", have the same cardinality and with max_categories=2, "b" and "c" are infrequent because they have a higher lexicon order.

```
>>> X = np.asarray([["a"] * 20 + ["b"] * 10 + ["c"] * 10 + ["d"] * 10], dtype=object).T
>>> enc = preprocessing.OneHotEncoder(max_categories=3).fit(X)
>>> enc.infrequent_categories_
[array(['b', 'c'], dtype=object)]
```

Target Encoder

The `TargetEncoder` uses the target mean conditioned on the categorical feature for encoding unordered categories, i.e. nominal categories [PAR] [MIC]. This encoding scheme is useful with categorical features with high cardinality, where one-hot encoding would inflate the feature space making it more expensive for a downstream model to process. A classical example of high cardinality categories are location based such as zip code or region.

Binary classification targets
For the binary classification target, the target encoding is given by:

$$S_i = \lambda_i \frac{n_{iY}}{n_i} + (1 - \lambda_i) \frac{n_Y}{n}$$

where S_i is the encoding for category i, n_{iY} is the number of observations with $Y = 1$ and category i, n_i is the number of observations with category i, n_Y is the number of observations with $Y = 1$, n is the number of observations, and factor is given by:

$$\lambda_i = \frac{n_i}{m + n_i}$$

where m is a smoothing factor, which is controlled with the smooth parameter in `TargetEncoder`. Large smoothing factors will put more weight on the global mean. When smooth="auto", the smoothing factor is computed as an empirical Bayes estimate: $m = \sigma_i^2 / \tau^2$, where

Multiclass classification targets
For multiclass classification targets, the formulation is similar to binary classification:

$$S_{ij} = \lambda_i \frac{n_{iY_j}}{n_i} + (1 - \lambda_i) \frac{n_{Y_j}}{n}$$

where S_{ij} is the encoding for category i and class j, number of observations, and λ_i is a shrinkage factor for category

Continuous targets
For continuous targets, the formulation is similar to binary classification:

$$S_i = \lambda_i \frac{\sum_{k \in L_i} Y_k}{n_i} + (1 - \lambda_i) \frac{\sum_{k=1}^{n} Y_k}{n}$$

where L_i is the set of observations with category i and

Note
In `TargetEncoder`, fit(X, y).transform(X) does not equal fit_transform(X, y).

`fit_transform` internally relies on a cross fitting scheme to prevent target information from leaking into the train-time representation, especially for non-informative high-cardinality categorical variables (features with many unique categories where each category appears only a few times), and help prevent the downstream model from overfitting spurious correlations. In `fit_transform`, the training data is split into *k* folds (determined by the cv parameter) and each fold is encoded using the encodings learnt using the *other k-1* folds. For this reason, training data should always be trained and transformed with fit_transform(X_train, y_train).

This diagram shows the cross fitting scheme in `fit_transform` with the default 'cv=5':

The `fit` method does **not** use any cross fitting schemes and learns one encoding on the entire training set. It is discouraged to use this method because it can introduce data leakage as mentioned above. Use `fit_transform` instead.

During `fit_transform`, the encoder learns category encodings from the full training data and stores them in the `encodings_` attribute. The intermediate encodings learned for each fold during the cross fitting process are temporary and not saved. The stored encodings can then be used to transform test data with encoder.transform(X_test).

Note
as another category and encodes them like any other category. Categories that are not seen during fit are encoded with the target mean, i.e. target_mean_.

Examples

- sphx_glr_auto_examples_preprocessing_plot_target_encoder.py
- sphx_glr_auto_examples_preprocessing_plot_target_encoder_cross_val.py

References

Discretization

Discretization (otherwise known as quantization or binning) provides a way to partition continuous features into discrete values. Certain datasets with continuous features may benefit from discretization, because discretization can transform the dataset of continuous attributes to one with only nominal attributes.

One-hot encoded discretized features can make a model more expressive, while maintaining interpretability. For instance, pre-processing with a discretizer can introduce nonlinearity to linear models. For more advanced possibilities, in particular smooth ones, see generating_polynomial_features further below.

K-bins discretization

`KBinsDiscretizer` discretizes features into `k` bins:

```
>>> X = np.array([[ -3., 5., 15 ],
...               [  0., 6., 14 ],
...               [  6., 3., 11 ]])
>>> est = preprocessing.KBinsDiscretizer(n_bins=[3, 2, 2], encode='ordinal').fit(X)
```

By default the output is one-hot encoded into a sparse matrix (See preprocessing_categorical_features) and this can be configured with the `encode` parameter. For each feature, the bin edges are computed during `fit` and together with the number of bins, they will define the intervals. Therefore, for the current example, these intervals are defined as:

- feature 1: $[-\infty, -1), [-1, 2), [2, \infty)$
- feature 2: $[-\infty, 5), [5, \infty)$
- feature 3: $[-\infty, 14), [14, \infty)$

Based on these bin intervals, `X` is transformed as follows:

```
>>> est.transform(X)
array([[ 0., 1., 1.],
       [ 1., 1., 1.],
       [ 2., 0., 0.]])
```

The resulting dataset contains ordinal attributes which can be further used in a `Pipeline`.

Discretization is similar to constructing histograms for continuous data. However, histograms focus on counting features which fall into particular bins, whereas discretization focuses on assigning feature values to these bins.

`KBinsDiscretizer` implements different binning strategies, which can be selected with the `strategy` parameter. The 'uniform' strategy uses constant-width bins. The 'quantile' strategy uses the quantiles values to have equally populated bins in each feature. The 'kmeans' strategy defines bins based on a k-means clustering procedure performed on each feature independently.

Be aware that one can specify custom bins by passing a callable defining the discretization strategy to `FunctionTransformer`. For instance, we can use the Pandas function `pandas.cut`:

```
>>> import pandas as pd
>>> import numpy as np
>>> from sklearn import preprocessing
>>>
>>> bins = [0, 1, 13, 20, 60, np.inf]
>>> labels = ['infant', 'kid', 'teen', 'adult', 'senior citizen']
>>> transformer = preprocessing.FunctionTransformer(
...     pd.cut, kw_args={'bins': bins, 'labels': labels, 'retbins': False}
... )
>>> X = np.array([0.2, 2, 15, 25, 97])
>>> transformer.fit_transform(X)
['infant', 'kid', 'teen', 'adult', 'senior citizen']
Categories (5, str): ['infant' < 'kid' < 'teen' < 'adult' < 'senior citizen']
```

Examples

- sphx_glr_auto_examples_preprocessing_plot_discretization.py

- sphx_glr_auto_examples_preprocessing_plot_discretization_classification.py
- sphx_glr_auto_examples_preprocessing_plot_discretization_strategies.py

Feature binarization

Feature binarization is the process of **thresholding numerical features to get boolean values**. This can be useful for downstream probabilistic estimators that make assumption that the input data is distributed according to a multi-variate Bernoulli distribution. For instance, this is the case for the `BernoulliRBM`.

It is also common among the text processing community to use binary feature values (probably to simplify the probabilistic reasoning) even if normalized counts (a.k.a. term frequencies) or TF-IDF valued features often perform slightly better in practice.

As for the `Normalizer`, the utility class `Binarizer` is meant to be used in the early stages of `Pipeline`. The `fit` method does nothing as each sample is treated independently of others:

```
>>> X = [[ 1., -1.,  2.],
...      [ 2.,  0.,  0.],
...      [ 0.,  1., -1.]]

>>> binarizer = preprocessing.Binarizer().fit(X)  # fit does nothing
>>> binarizer
Binarizer()

>>> binarizer.transform(X)
array([[1., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.]])
```

It is possible to adjust the threshold of the binarizer:

```
>>> binarizer = preprocessing.Binarizer(threshold=1.1)
>>> binarizer.transform(X)
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 0.]])
```

As for the `Normalizer` class, the preprocessing module provides a companion function `binarize` to be used when the transformer API is not necessary.

Note that the `Binarizer` is similar to the `KBinsDiscretizer` when `k = 2`, and when the bin edge is at the value `threshold`.

and sparse matrices from scipy.sparse as input**.

For sparse input the data is **converted to the Compressed Sparse Rows representation** (see `scipy.sparse.csr_matrix`). To avoid unnecessary memory copies, it is recommended to choose the CSR representation upstream.

Imputation of missing values

Tools for imputing missing values are discussed at impute.

Generating polynomial features

Often it's useful to add complexity to a model by considering nonlinear features of the input data. We show two possibilities that are both based on polynomials: The first one uses pure polynomials, the second one uses splines, i.e. piecewise polynomials.

Polynomial features

A simple and common method to use is polynomial features, which can get features' high-order and interaction terms. It is implemented in `PolynomialFeatures`:

```
>>> import numpy as np
>>> from sklearn.preprocessing import PolynomialFeatures
>>> X = np.arange(6).reshape(3, 2)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> poly = PolynomialFeatures(2)
>>> poly.fit_transform(X)
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

The features of X have been transformed from (X_1, X_2) to $(1, X_1, X_2, X_1^2, X_1X_2, X_2^2)$.

In some cases, only interaction terms among features are required, and it can be gotten with the setting `interaction_only=True`:

```
>>> X = np.arange(9).reshape(3, 3)
>>> X
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
>>> poly = PolynomialFeatures(degree=3, interaction_only=True)
>>> poly.fit_transform(X)
array([[  1.,   0.,   1.,   2.,   0.,   0.,   2.,   0.],
       [  1.,   3.,   4.,   5.,  12.,  15.,  20.,  60.],
       [  1.,   6.,   7.,   8.,  42.,  48.,  56., 336.]])
```

The features of X have been transformed from (X_1, X_2, X_3) to $(1, X_1, X_2, X_3, X_1X_2, X_1X_3, X_2X_3, X_1X_2X_3)$.

Note that polynomial features are used implicitly in kernel methods (e.g., `SVC`, `KernelPCA`) when using polynomial svm_kernels.

See sphx_glr_auto_examples_linear_model_plot_polynomial_interpolation.py for Ridge regression using created polynomial features.

Spline transformer

Another way to add nonlinear terms instead of pure polynomials of features is to generate spline basis functions for each feature with the `SplineTransformer`. Splines are piecewise polynomials, parametrized by their polynomial degree and the positions of the knots. The `SplineTransformer` implements a B-spline basis, cf. the references below.

Note
The `SplineTransformer` treats each feature separately, i.e. it won't give you interaction terms.

Some of the advantages of splines over polynomials are:

- B-splines are very flexible and robust if you keep a fixed low degree, usually 3, and parsimoniously adapt the number of knots. Polynomials would need a higher degree, which leads to the next point.
- B-splines do not have oscillatory behaviour at the boundaries as have polynomials (the higher the degree, the worse). This is known as Runge's phenomenon.
- B-splines provide good options for extrapolation beyond the boundaries, i.e. beyond the range of fitted values. Have a look at the option `extrapolation`.
- B-splines generate a feature matrix with a banded structure. For a single feature, every row contains only `degree + 1` non-zero elements, which occur consecutively and are even positive. This results in a matrix with good numerical properties, e.g. a low condition number, in sharp contrast to a matrix of polynomials, which goes under the name Vandermonde matrix. A low condition number is important for stable algorithms of linear models.

The following code snippet shows splines in action:

```
>>> import numpy as np
>>> from sklearn.preprocessing import SplineTransformer
>>> X = np.arange(5).reshape(5, 1)
>>> X
array([[0],
       [1],
       [2],
       [3],
       [4]])
>>> spline = SplineTransformer(degree=2, n_knots=3)
>>> spline.fit_transform(X)
array([[0.5  , 0.5  , 0.   , 0.   ],
       [0.125, 0.75 , 0.125, 0.   ],
       [0.   , 0.5  , 0.5  , 0.   ],
       [0.   , 0.125, 0.75 , 0.125],
       [0.   , 0.   , 0.5  , 0.5  ]])
```

As the `X` is sorted, one can easily see the banded matrix output. Only the three middle diagonals are non-zero for `degree=2`. The higher the degree, the more overlapping of the splines.

Interestingly, a `SplineTransformer` of `degree=0` is the same as `KBinsDiscretizer` with `encode='onehot-dense'` and `n_bins = n_knots - 1` if `knots = strategy`.

Examples

- sphx_glr_auto_examples_linear_model_plot_polynomial_interpolation.py
- sphx_glr_auto_examples_applications_plot_cyclical_feature_engineering.py

References

- Eilers, P., & Marx, B. (1996). `Flexible Smoothing with B-splines and Penalties <10.1214/ss/1038425655>`. Statist. Sci. 11 (1996), no. 2, 89--121.
- Perperoglou, A., Sauerbrei, W., Abrahamowicz, M. et al. `A review of spline function procedures in R <10.1186/s12874-019-0666-3>`. BMC Med Res Methodol 19, 46 (2019).

Custom transformers

Often, you will want to convert an existing Python function into a transformer to assist in data cleaning or processing. You can implement a transformer from an arbitrary function with `FunctionTransformer`. For example, to build a transformer that applies a log transformation in a pipeline, do:

```
>>> import numpy as np
>>> from sklearn.preprocessing import FunctionTransformer
>>> transformer = FunctionTransformer(np.log1p, validate=True)
>>> X = np.array([[0, 1], [2, 3]])
>>> # Since FunctionTransformer is no-op during fit, we can call transform directly
>>> transformer.transform(X)
array([[0.        , 0.69314718],
       [1.09861229, 1.38629436]])
```

You can ensure that `func` and `inverse_func` are the inverse of each other by setting `check_inverse=True` and calling `fit` before `transform`. Please note that a warning is raised and can be turned into an error with a `filterwarnings`:

```
>>> import warnings
>>> warnings.filterwarnings("error", message=".*check_inverse*.",
...                         category=UserWarning, append=False)
```

For a full code example that demonstrates using a `FunctionTransformer` to extract features from text data see sphx_glr_auto_examples_compose_plot_column_transformer.py and sphx_glr_auto_examples_applications_plot_cyclical_feature_engineering.py.

MIC Micci-Barreca, Daniele. "A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems" SIGKDD Explor. Newsl. 3, 1 (July 2001), 27-32. <10.1145/507533.507538>

PAR Pargent, F., Pfisterer, F., Thomas, J. et al. "Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features" Comput Stat 37, 2671-2692 (2022) <10.1007/s00180-022-01207-6>

Imputation of missing values

Source: *https://scikit-learn.org/stable/modules/impute.html*

For various reasons, many real world datasets contain missing values, often encoded as blanks, NaNs or other placeholders. Such datasets however are incompatible with scikit-learn estimators which assume that all values in an array are numerical, and that all have and hold meaning. A basic strategy to use incomplete datasets is to discard entire rows and/or columns containing missing values. However, this comes at the price of losing data which may be valuable (even though incomplete). A better strategy is to impute the missing values, i.e., to infer them from the known part of the data. See the glossary entry on imputation.

Univariate vs. Multivariate Imputation

One type of imputation algorithm is univariate, which imputes values in the i-th feature dimension using only non-missing values in that feature dimension (e.g. `SimpleImputer`). By contrast, multivariate imputation algorithms use the entire set of available feature dimensions to estimate the missing values (e.g. `IterativeImputer`).

Univariate feature imputation

The `SimpleImputer` class provides basic strategies for imputing missing values. Missing values can be imputed with a provided constant value, or using the statistics (mean, median or most frequent) of each column in which the missing values are located. This class also allows for different missing values encodings.

The following snippet demonstrates how to replace missing values, encoded as `np.nan`, using the mean value of the columns (axis 0) that contain the missing values:

```
>>> import numpy as np
>>> from sklearn.impute import SimpleImputer
>>> imp = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imp.fit([[1, 2], [np.nan, 3], [7, 6]])
SimpleImputer()
>>> X = [[np.nan, 2], [6, np.nan], [7, 6]]
>>> print(imp.transform(X))
[[4.          2.        ]
 [6.          3.666]
 [7.          6.        ]]
```

The `SimpleImputer` class also supports sparse matrices:

```
>>> import scipy.sparse as sp
>>> X = sp.csc_array([[1, 2], [0, -1], [8, 4]])
>>> imp = SimpleImputer(missing_values=-1, strategy='mean')
>>> imp.fit(X)
SimpleImputer(missing_values=-1)
>>> X_test = sp.csc_array([[-1, 2], [6, -1], [7, 6]])
>>> print(imp.transform(X_test).toarray())
[[3. 2.]
 [6. 3.]
 [7. 6.]]
```

Note that this format is not meant to be used to implicitly store missing values in the matrix because it would densify it at transform time. Missing values encoded by 0 must be used with dense input.

The `SimpleImputer` class also supports categorical data represented as string values or pandas categoricals when using the `'most_frequent'` or `'constant'` strategy:

```
>>> import pandas as pd
>>> df = pd.DataFrame([["a", "x"],
...                    [np.nan, "y"],
...                    ["a", np.nan],
...                    ["b", "y"]], dtype="category")
...
>>> imp = SimpleImputer(strategy="most_frequent")
>>> print(imp.fit_transform(df))
[['a' 'x']
 ['a' 'y']
 ['a' 'y']
 ['b' 'y']]
```

For another example on usage, see sphx_glr_auto_examples_impute_plot_missing_values.py.

Multivariate feature imputation

A more sophisticated approach is to use the `IterativeImputer` class, which models each feature with missing values as a function of other features, and uses that estimate for imputation. It does so in an iterated round-robin fashion: at each step, a feature column is designated as output `y` and the other feature columns are treated as inputs `X`. A regressor is fit on `(X, y)` for known `y`. Then, the regressor is used to predict the missing values of `y`. This is done for each feature in an iterative fashion, and then is repeated for `max_iter` imputation rounds. The results of the final imputation round are returned.

> Note
> This estimator is still **experimental** for now: default parameters or details of behaviour might change without any deprecation cycle. Resolving the following issues would help stabilize `IterativeImputer`: convergence criteria (`14338`) and default estimators (`13286`). To use it, you need to explicitly import `enable_iterative_imputer`.

```
>>> import numpy as np
>>> from sklearn.experimental import enable_iterative_imputer
>>> from sklearn.impute import IterativeImputer
>>> imp = IterativeImputer(max_iter=10, random_state=0)
>>> imp.fit([[1, 2], [3, 6], [4, 8], [np.nan, 3], [7, np.nan]])
IterativeImputer(random_state=0)
>>> X_test = [[np.nan, 2], [6, np.nan], [np.nan, 6]]
>>> # the model learns that the second feature is double the first
>>> print(np.round(imp.transform(X_test)))
[[ 1.  2.]
 [ 6. 12.]
 [ 3.  6.]]
```

Both `SimpleImputer` and `IterativeImputer` can be used in a Pipeline as a way to build a composite estimator that supports imputation. See sphx_glr_auto_examples_impute_plot_missing_values.py.

Flexibility of IterativeImputer

There are many well-established imputation packages in the R data science ecosystem: Amelia, mi, mice, missForest, etc. missForest is popular, and turns out to be a particular instance of different sequential imputation algorithms that can all be implemented with `IterativeImputer` by passing in different regressors to be used for predicting missing feature values. In the case of missForest, this regressor is a Random Forest. See sphx_glr_auto_examples_impute_plot_iterative_imputer_variants_comparison.py.

Multiple vs. Single Imputation

In the statistics community, it is common practice to perform multiple imputations, generating, for example, `m` separate imputations for a single feature matrix. Each of these `m` imputations is then put through the subsequent analysis pipeline (e.g. feature engineering, clustering, regression, classification). The `m` final analysis results (e.g. held-out validation errors) allow the data scientist to obtain understanding of how analytic results may differ as a consequence of the inherent uncertainty caused by the missing values. The above practice is called multiple imputation.

Our implementation of `IterativeImputer` was inspired by the R MICE package (Multivariate Imputation by Chained Equations)[1], but differs from it by returning a single imputation instead of multiple imputations. However, `IterativeImputer` can also be used for multiple imputations by applying it repeatedly to the same dataset with different random seeds when `sample_posterior=True`. See[2], chapter 4 for more discussion on multiple vs. single imputations.

It is still an open problem as to how useful single vs. multiple imputation is in the context of prediction and classification when the user is not interested in measuring uncertainty due to missing values.

Note that a call to the `transform` method of `IterativeImputer` is not allowed to change the number of samples. Therefore multiple imputations cannot be achieved by a single call to `transform`.

References

Nearest neighbors imputation

The `KNNImputer` class provides imputation for filling in missing values using the k-Nearest Neighbors approach. By default, a euclidean distance metric that supports missing values, `nan_euclidean_distances`, is used to find the nearest neighbors. Each missing feature is imputed using values from `n_neighbors` nearest neighbors that have a value for the feature. The feature of the neighbors are averaged uniformly or weighted by distance to each neighbor. If a sample has more than one feature missing, then the neighbors for that sample can be different depending on the particular feature being imputed. When the number of available neighbors is less than n_neighbors and there are no defined distances to the training set, the training set average for that feature is used during imputation. If there is at least one neighbor with a defined distance, the weighted or unweighted average of the remaining neighbors will be used during imputation. If a feature is always missing in training, it is removed during transform. For more information on the methodology, see ref. [OL2001].

The following snippet demonstrates how to replace missing values, encoded as `np.nan`, using the mean feature value of the two nearest neighbors of samples with missing values:

```
>>> import numpy as np
```

[1] Stef van Buuren, Karin Groothuis-Oudshoorn (2011). "mice: Multivariate Imputation by Chained Equations in R". Journal of Statistical Software 45: 1-67.

[2] Roderick J A Little and Donald B Rubin (1986). "Statistical Analysis with Missing Data". John Wiley & Sons, Inc., New York, NY, USA.

```
>>> from sklearn.impute import KNNImputer
>>> nan = np.nan
>>> X = [[1, 2, nan], [3, 4, 3], [nan, 6, 5], [8, 8, 7]]
>>> imputer = KNNImputer(n_neighbors=2, weights="uniform")
>>> imputer.fit_transform(X)
array([[1. , 2. , 4. ],
       [3. , 4. , 3. ],
       [5.5, 6. , 5. ],
       [8. , 8. , 7. ]])
```

For another example on usage, see sphx_glr_auto_examples_impute_plot_missing_values.py.

References

Keeping the number of features constant

By default, the scikit-learn imputers will drop fully empty features, i.e. columns containing only missing values. For instance:

```
>>> imputer = SimpleImputer()
>>> X = np.array([[np.nan, 1], [np.nan, 2], [np.nan, 3]])
>>> imputer.fit_transform(X)
array([[1.],
       [2.],
       [3.]])
```

The first feature in X containing only np.nan was dropped after the imputation. While this feature will not help in predictive setting, dropping the columns will change the shape of X which could be problematic when using imputers in a more complex machine-learning pipeline. The parameter keep_empty_features offers the option to keep the empty features by imputing with a constant value. In most of the cases, this constant value is zero:

```
>>> imputer.set_params(keep_empty_features=True)
SimpleImputer(keep_empty_features=True)
>>> imputer.fit_transform(X)
array([[0., 1.],
       [0., 2.],
       [0., 3.]])
```

Marking imputed values

The `MissingIndicator` transformer is useful to transform a dataset into corresponding binary matrix indicating the presence of missing values in the dataset. This transformation is useful in conjunction with imputation. When using imputation, preserving the information about which values had been missing can be informative. Note that both the `SimpleImputer` and `IterativeImputer` have the boolean parameter `add_indicator` (`False` by default) which when set to `True` provides a convenient way of stacking the output of the `MissingIndicator` transformer with the output of the imputer.

`NaN` is usually used as the placeholder for missing values. However, it enforces the data type to be float. The parameter `missing_values` allows to specify other placeholder such as integer. In the following example, we will use `-1` as missing values:

```
>>> from sklearn.impute import MissingIndicator
>>> X = np.array([[-1, -1, 1, 3],
...               [4, -1, 0, -1],
```

```
...               [8, -1, 1, 0]])
>>> indicator = MissingIndicator(missing_values=-1)
>>> mask_missing_values_only = indicator.fit_transform(X)
>>> mask_missing_values_only
array([[ True,  True, False],
       [False,  True,  True],
       [False,  True, False]])
```

The `features` parameter is used to choose the features for which the mask is constructed. By default, it is `'missing-only'` which returns the imputer mask of the features containing missing values at `fit` time:

```
>>> indicator.features_
array([0, 1, 3])
```

The `features` parameter can be set to `'all'` to return all features whether or not they contain missing values:

```
>>> indicator = MissingIndicator(missing_values=-1, features="all")
>>> mask_all = indicator.fit_transform(X)
>>> mask_all
array([[ True,  True, False, False],
       [False,  True, False,  True],
       [False,  True, False, False]])
>>> indicator.features_
array([0, 1, 2, 3])
```

When using the `MissingIndicator` in a `Pipeline`, be sure to use the `FeatureUnion` or `ColumnTransformer` to add the indicator features to the regular features. First we obtain the iris dataset, and add some missing values to it.

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.impute import SimpleImputer, MissingIndicator
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.pipeline import FeatureUnion, make_pipeline
>>> from sklearn.tree import DecisionTreeClassifier
>>> X, y = load_iris(return_X_y=True)
>>> mask = np.random.randint(0, 2, size=X.shape).astype(bool)
>>> X[mask] = np.nan
>>> X_train, X_test, y_train, _ = train_test_split(X, y, test_size=100,
...                                                random_state=0)
```

Now we create a `FeatureUnion`. All features will be imputed using `SimpleImputer`, in order to enable classifiers to work with this data. Additionally, it adds the indicator variables from `MissingIndicator`.

```
>>> transformer = FeatureUnion(
...     transformer_list=[
...         ('features', SimpleImputer(strategy='mean')),
...         ('indicators', MissingIndicator())])
>>> transformer = transformer.fit(X_train, y_train)
>>> results = transformer.transform(X_test)
>>> results.shape
(100, 8)
```

Of course, we cannot use the transformer to make any predictions. We should wrap this in a `Pipeline` with a classifier (e.g., a `DecisionTreeClassifier`) to be able to make predictions.

```
>>> clf = make_pipeline(transformer, DecisionTreeClassifier())
>>> clf = clf.fit(X_train, y_train)
>>> results = clf.predict(X_test)
```

```
>>> results.shape
(100,)
```

Estimators that handle NaN values

Some estimators are designed to handle NaN values without preprocessing. Below is the list of these estimators, classified by type (cluster, regressor, classifier, transform):

OL2001 Olga Troyanskaya, Michael Cantor, Gavin Sherlock, Pat Brown, Trevor Hastie, Robert Tibshirani, David Botstein and Russ B. Altman, Missing value estimation methods for DNA microarrays, BIOINFORMATICS Vol. 17 no. 6, 2001 Pages 520-525.

Pipelines and composite estimators

Source: *https://scikit-learn.org/stable/modules/compose.html*

To build a composite estimator, transformers are usually combined with other transformers or with predictors (such as classifiers or regressors). The most common tool used for composing estimators is a `Pipeline <pipeline>`. Pipelines require all steps except the last to be a transformer. The last step can be anything, a transformer, a predictor, or a clustering estimator which might have or not have a .predict(...) method. A pipeline exposes all methods provided by the last estimator: if the last step provides a transform method, then the pipeline would have a transform method and behave like a transformer. If the last step provides a predict method, then the pipeline would expose that method, and given a data X, use all steps except the last to transform the data, and then give that transformed data to the predict method of the last step of the pipeline. The class `Pipeline` is often used in combination with ColumnTransformer or FeatureUnion which concatenate the output of transformers into a composite feature space. TransformedTargetRegressor deals with transforming the target (i.e. log-transform y).

Pipeline: chaining estimators

`Pipeline` can be used to chain multiple estimators into one. This is useful as there is often a fixed sequence of steps in processing the data, for example feature selection, normalization and classification. `Pipeline` serves multiple purposes here:

Convenience and encapsulation You only have to call fit and predict once on your data to fit a whole sequence of estimators.

Joint parameter selection You can grid search over parameters of all estimators in the pipeline at once.

Safety Pipelines help avoid leaking statistics from your test data into the trained model in cross-validation, by ensuring that the same samples are used to train the transformers and predictors.

All estimators in a pipeline, except the last one, must be transformers (i.e. must have a transform method). The last estimator may be any type (transformer, classifier, etc.).

> Note
> Calling `fit` on the pipeline is the same as calling `fit` on each estimator in turn, `transform` the input and pass it on to the next step. The pipeline has all the methods that the last estimator in the pipeline has, i.e. if the last estimator is a classifier, the `Pipeline` can be used as a classifier. If the last estimator is a transformer, again, so is the pipeline.

Usage

Build a pipeline

The `Pipeline` is built using a list of `(key, value)` pairs, where the `key` is a string containing the name you want to give this step and `value` is an estimator object:

```
>>> from sklearn.pipeline import Pipeline
>>> from sklearn.svm import SVC
>>> from sklearn.decomposition import PCA
>>> estimators = [('reduce_dim', PCA()), ('clf', SVC())]
```

```
>>> pipe = Pipeline(estimators)
>>> pipe
Pipeline(steps=[('reduce_dim', PCA()), ('clf', SVC())])
```

Shorthand version using `make_pipeline`
The utility function `make_pipeline` is a shorthand for constructing pipelines; it takes a variable number of estimators and returns a pipeline, filling in the names automatically:

```
>>> from sklearn.pipeline import make_pipeline
>>> make_pipeline(PCA(), SVC())
Pipeline(steps=[('pca', PCA()), ('svc', SVC())])
```

Access pipeline steps

The estimators of a pipeline are stored as a list in the `steps` attribute. A sub-pipeline can be extracted using the slicing notation commonly used for Python Sequences such as lists or strings (although only a step of 1 is permitted). This is convenient for performing only some of the transformations (or their inverse):

```
>>> pipe[:1]
Pipeline(steps=[('reduce_dim', PCA())])
>>> pipe[-1:]
Pipeline(steps=[('clf', SVC())])
```

Accessing a step by name or position
A specific step can also be accessed by index or name by indexing (with `[idx]`) the pipeline:

```
>>> pipe.steps[0]
('reduce_dim', PCA())
>>> pipe[0]
PCA()
>>> pipe['reduce_dim']
PCA()
```

Pipeline's named_steps attribute allows accessing steps by name with tab completion in interactive environments:

```
>>> pipe.named_steps.reduce_dim is pipe['reduce_dim']
True
```

Tracking feature names in a pipeline

To enable model inspection, `Pipeline` has a `get_feature_names_out()` method, just like all transformers. You can use pipeline slicing to get the feature names going into each step:

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.feature_selection import SelectKBest
>>> iris = load_iris()
>>> pipe = Pipeline(steps=[
...    ('select', SelectKBest(k=2)),
...    ('clf', LogisticRegression())])
>>> pipe.fit(iris.data, iris.target)
Pipeline(steps=[('select', SelectKBest(...)), ('clf', LogisticRegression(...))])
>>> pipe[:-1].get_feature_names_out()
array(['x2', 'x3'], ...)
```

Customize feature names
You can also provide custom feature names for the input data using `get_feature_names_out`:

```
>>> pipe[:-1].get_feature_names_out(iris.feature_names)
array(['petal length (cm)', 'petal width (cm)'], ...)
```

Access to nested parameters

It is common to adjust the parameters of an estimator within a pipeline. This parameter is therefore nested because it belongs to a particular sub-step. Parameters of the estimators in the pipeline are accessible using the `<estimator>__<parameter>` syntax:

```
>>> pipe = Pipeline(steps=[("reduce_dim", PCA()), ("clf", SVC())])
>>> pipe.set_params(clf__C=10)
Pipeline(steps=[('reduce_dim', PCA()), ('clf', SVC(C=10))])
```

When does it matter?
This is particularly important for doing grid searches:

```
>>> from sklearn.model_selection import GridSearchCV
>>> param_grid = dict(reduce_dim__n_components=[2, 5, 10],
...                   clf__C=[0.1, 10, 100])
>>> grid_search = GridSearchCV(pipe, param_grid=param_grid)
```

Individual steps may also be replaced as parameters, and non-final steps may be ignored by setting them to `'passthrough'`:

```
>>> param_grid = dict(reduce_dim=['passthrough', PCA(5), PCA(10)],
...                   clf=[SVC(), LogisticRegression()],
...                   clf__C=[0.1, 10, 100])
>>> grid_search = GridSearchCV(pipe, param_grid=param_grid)
```

See also
- composite_grid_search

Examples

- sphx_glr_auto_examples_feature_selection_plot_feature_selection_pipeline.py
- sphx_glr_auto_examples_model_selection_plot_grid_search_text_feature_extraction.py
- sphx_glr_auto_examples_compose_plot_digits_pipe.py
- sphx_glr_auto_examples_miscellaneous_plot_kernel_approximation.py
- sphx_glr_auto_examples_svm_plot_svm_anova.py
- sphx_glr_auto_examples_compose_plot_compare_reduction.py
- sphx_glr_auto_examples_miscellaneous_plot_pipeline_display.py

Caching transformers: avoid repeated computation

Fitting transformers may be computationally expensive. With its `memory` parameter set, `Pipeline` will cache each transformer after calling `fit`. This feature is used to avoid computing the fit transformers within a pipeline if the parameters and input data are identical. A typical example is the case of a grid search in which the transformers can be fitted only once and reused for each configuration. The last step will never be cached, even if it is a transformer.

The parameter `memory` is needed in order to cache the transformers. `memory` can be either a string containing the directory where to cache the transformers or a joblib.Memory object:

```
>>> from tempfile import mkdtemp
>>> from shutil import rmtree
>>> from sklearn.decomposition import PCA
```

```
>>> from sklearn.svm import SVC
>>> from sklearn.pipeline import Pipeline
>>> estimators = [('reduce_dim', PCA()), ('clf', SVC())]
>>> cachedir = mkdtemp()
>>> pipe = Pipeline(estimators, memory=cachedir)
>>> pipe
Pipeline(memory=...,
         steps=[('reduce_dim', PCA()), ('clf', SVC())])
>>> # Clear the cache directory when you don't need it anymore
>>> rmtree(cachedir)
```

Side effect of caching transformers

Using a `Pipeline` without cache enabled, it is possible to inspect the original instance such as:

```
>>> from sklearn.datasets import load_digits
>>> X_digits, y_digits = load_digits(return_X_y=True)
>>> pca1 = PCA(n_components=10)
>>> svm1 = SVC()
>>> pipe = Pipeline([('reduce_dim', pca1), ('clf', svm1)])
>>> pipe.fit(X_digits, y_digits)
Pipeline(steps=[('reduce_dim', PCA(n_components=10)), ('clf', SVC())])
>>> # The pca instance can be inspected directly
>>> pca1.components_.shape
(10, 64)
```

Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. In the following example, accessing the `PCA` instance `pca2` will raise an `AttributeError` since `pca2` will be an unfitted transformer. Instead, use the attribute `named_steps` to inspect estimators within the pipeline:

```
>>> cachedir = mkdtemp()
>>> pca2 = PCA(n_components=10)
>>> svm2 = SVC()
>>> cached_pipe = Pipeline([('reduce_dim', pca2), ('clf', svm2)],
...                        memory=cachedir)
>>> cached_pipe.fit(X_digits, y_digits)
Pipeline(memory=...,
         steps=[('reduce_dim', PCA(n_components=10)), ('clf', SVC())])
>>> cached_pipe.named_steps['reduce_dim'].components_.shape
(10, 64)
>>> # Remove the cache directory
>>> rmtree(cachedir)
```

Examples

- sphx_glr_auto_examples_compose_plot_compare_reduction.py

Transforming target in regression

`TransformedTargetRegressor` transforms the targets `y` before fitting a regression model. The predictions are mapped back to the original space via an inverse transform. It takes as an argument the regressor that will be used for prediction, and the transformer that will be applied to the target variable:

```
>>> import numpy as np
>>> from sklearn.datasets import make_regression
>>> from sklearn.compose import TransformedTargetRegressor
```

```
>>> from sklearn.preprocessing import QuantileTransformer
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.model_selection import train_test_split
>>> # create a synthetic dataset
>>> X, y = make_regression(n_samples=20640,
...                        n_features=8,
...                        noise=100.0,
...                        random_state=0)
>>> y = np.exp( 1 + (y - y.min()) * (4 / (y.max() - y.min())))
>>> X, y = X[:2000, :], y[:2000]  # select a subset of data
>>> transformer = QuantileTransformer(output_distribution='normal')
>>> regressor = LinearRegression()
>>> regr = TransformedTargetRegressor(regressor=regressor,
...                                   transformer=transformer)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
>>> regr.fit(X_train, y_train)
TransformedTargetRegressor(...)
>>> print(f"R2 score: {regr.score(X_test, y_test):.2f}")
R2 score: 0.67
>>> raw_target_regr = LinearRegression().fit(X_train, y_train)
>>> print(f"R2 score: {raw_target_regr.score(X_test, y_test):.2f}")
R2 score: 0.64
```

For simple transformations, instead of a Transformer object, a pair of functions can be passed, defining the transformation and its inverse mapping:

```
>>> def func(x):
...     return np.log(x)
>>> def inverse_func(x):
...     return np.exp(x)
```

Subsequently, the object is created as:

```
>>> regr = TransformedTargetRegressor(regressor=regressor,
...                                   func=func,
...                                   inverse_func=inverse_func)
>>> regr.fit(X_train, y_train)
TransformedTargetRegressor(...)
>>> print(f"R2 score: {regr.score(X_test, y_test):.2f}")
R2 score: 0.67
```

By default, the provided functions are checked at each fit to be the inverse of each other. However, it is possible to bypass this checking by setting `check_inverse` to `False`:

```
>>> def inverse_func(x):
...     return x
>>> regr = TransformedTargetRegressor(regressor=regressor,
...                                   func=func,
...                                   inverse_func=inverse_func,
...                                   check_inverse=False)
>>> regr.fit(X_train, y_train)
TransformedTargetRegressor(...)
>>> print(f"R2 score: {regr.score(X_test, y_test):.2f}")
R2 score: -3.02
```

Note
The transformation can be triggered by setting either `transformer` or the pair of functions

`func` and `inverse_func`. However, setting both options will raise an error.

Examples

- sphx_glr_auto_examples_compose_plot_transformed_target.py

FeatureUnion: composite feature spaces

`FeatureUnion` combines several transformer objects into a new transformer that combines their output. A `FeatureUnion` takes a list of transformer objects. During fitting, each of these is fit to the data independently. The transformers are applied in parallel, and the feature matrices they output are concatenated side-by-side into a larger matrix.

When you want to apply different transformations to each field of the data, see the related class `ColumnTransformer` (see user guide).

`FeatureUnion` serves the same purposes as `Pipeline` -convenience and joint parameter estimation and validation.

`FeatureUnion` and `Pipeline` can be combined to create complex models.

(A `FeatureUnion` has no way of checking whether two transformers might produce identical features. It only produces a union when the feature sets are disjoint, and making sure they are is the caller's responsibility.)

Usage

A `FeatureUnion` is built using a list of `(key, value)` pairs, where the `key` is the name you want to give to a given transformation (an arbitrary string; it only serves as an identifier) and `value` is an estimator object:

```
>>> from sklearn.pipeline import FeatureUnion
>>> from sklearn.decomposition import PCA
>>> from sklearn.decomposition import KernelPCA
>>> estimators = [('linear_pca', PCA()), ('kernel_pca', KernelPCA())]
>>> combined = FeatureUnion(estimators)
>>> combined
FeatureUnion(transformer_list=[('linear_pca', PCA()),
                               ('kernel_pca', KernelPCA())])
```

Like pipelines, feature unions have a shorthand constructor called `make_union` that does not require explicit naming of the components.

Like `Pipeline`, individual steps may be replaced using `set_params`, and ignored by setting to `'drop'`:

```
>>> combined.set_params(kernel_pca='drop')
FeatureUnion(transformer_list=[('linear_pca', PCA()),
                               ('kernel_pca', 'drop')])
```

Examples

- sphx_glr_auto_examples_compose_plot_feature_union.py

ColumnTransformer for heterogeneous data

Many datasets contain features of different types, say text, floats, and dates, where each type of feature requires separate preprocessing or feature extraction steps. Often it is easiest to

preprocess data before applying scikit-learn methods, for example using pandas. Processing your data before passing it to scikit-learn might be problematic for one of the following reasons:

1. Incorporating statistics from test data into the preprocessors makes cross-validation scores unreliable (known as *data leakage*), for example in the case of scalers or imputing missing values.
2. You may want to include the parameters of the preprocessors in a parameter search.

The `ColumnTransformer` helps performing different transformations for different columns of the data, within a `Pipeline` that is safe from data leakage and that can be parametrized. `ColumnTransformer` works on arrays, sparse matrices, and pandas DataFrames.

To each column, a different transformation can be applied, such as preprocessing or a specific feature extraction method:

```
>>> import pandas as pd
>>> X = pd.DataFrame(
...     {'city': ['London', 'London', 'Paris', 'Sallisaw'],
...      'title': ["His Last Bow", "How Watson Learned the Trick",
...                "A Moveable Feast", "The Grapes of Wrath"],
...      'expert_rating': [5, 3, 4, 5],
...      'user_rating': [4, 5, 4, 3]})
```

For this data, we might want to encode the `'city'` column as a categorical variable using `OneHotEncoder` but apply a `CountVectorizer` to the `'title'` column. As we might use multiple feature extraction methods on the same column, we give each transformer a unique name, say `'city_category'` and `'title_bow'`. By default, the remaining rating columns are ignored (`remainder='drop'`):

```
>>> from sklearn.compose import ColumnTransformer
>>> from sklearn.feature_extraction.text import CountVectorizer
>>> from sklearn.preprocessing import OneHotEncoder
>>> column_trans = ColumnTransformer(
...     [('categories', OneHotEncoder(dtype='int'), ['city']),
...      ('title_bow', CountVectorizer(), 'title')],
...     remainder='drop', verbose_feature_names_out=False)

>>> column_trans.fit(X)
ColumnTransformer(transformers=[('categories', OneHotEncoder(dtype='int'),
                                 ['city']),
                                ('title_bow', CountVectorizer(), 'title')],
                  verbose_feature_names_out=False)

>>> column_trans.get_feature_names_out()
array(['city_London', 'city_Paris', 'city_Sallisaw', 'bow', 'feast',
'grapes', 'his', 'how', 'last', 'learned', 'moveable', 'of', 'the',
 'trick', 'watson', 'wrath'], ...)

>>> column_trans.transform(X).toarray()
array([[1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
       [1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0],
       [0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
       [0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1]]...)
```

In the above example, the `CountVectorizer` expects a 1D array as input and therefore the columns were specified as a string (`'title'`). However, `OneHotEncoder` as most of other transformers expects 2D data, therefore in that case you need to specify the column as a list of strings (`['city']`).

Apart from a scalar or a single item list, the column selection can be specified as a list of

multiple items, an integer array, a slice, a boolean mask, or with a `make_column_selector`. The `make_column_selector` is used to select columns based on data type or column name:

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.compose import make_column_selector
>>> ct = ColumnTransformer([
...       ('scale', StandardScaler(),
...       make_column_selector(dtype_include=np.number)),
...       ('onehot',
...       OneHotEncoder(),
...       make_column_selector(pattern='city', dtype_include=[object, "string"]))])
>>> ct.fit_transform(X)
array([[ 0.904,  0.      ,  1. ,  0. ,  0. ],
       [-1.507,  1.414,  1. ,  0. ,  0. ],
       [-0.301,  0.      ,  0. ,  1. ,  0. ],
       [ 0.904, -1.414,  0. ,  0. ,  1. ]])
```

Strings can reference columns if the input is a DataFrame, integers are always interpreted as the positional columns.

We can keep the remaining rating columns by setting `remainder='passthrough'`. The values are appended to the end of the transformation:

```
>>> column_trans = ColumnTransformer(
...     [('city_category', OneHotEncoder(dtype='int'),['city']),
...      ('title_bow', CountVectorizer(), 'title')],
...     remainder='passthrough')

>>> column_trans.fit_transform(X)
array([[1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 5, 4],
       [1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 3, 5],
       [0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 4, 4],
       [0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 5, 3]]...)
```

The `remainder` parameter can be set to an estimator to transform the remaining rating columns. The transformed values are appended to the end of the transformation:

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> column_trans = ColumnTransformer(
...     [('city_category', OneHotEncoder(), ['city']),
...      ('title_bow', CountVectorizer(), 'title')],
...     remainder=MinMaxScaler())

>>> column_trans.fit_transform(X)[:, -2:]
array([[1. , 0.5],
       [0. , 1. ],
       [0.5, 0.5],
       [1. , 0. ]])
```

The `make_column_transformer` function is available to more easily create a `ColumnTransformer` object. Specifically, the names will be given automatically. The equivalent for the above example would be:

```
>>> from sklearn.compose import make_column_transformer
>>> column_trans = make_column_transformer(
...     (OneHotEncoder(), ['city']),
...     (CountVectorizer(), 'title'),
...     remainder=MinMaxScaler())
>>> column_trans
```

```
ColumnTransformer(remainder=MinMaxScaler(),
                  transformers=[('onehotencoder', OneHotEncoder(), ['city']),
                                ('countvectorizer', CountVectorizer(),
                                 'title')])
```

If `ColumnTransformer` is fitted with a dataframe and the dataframe only has string column names, then transforming a dataframe will use the column names to select the columns:

```
>>> ct = ColumnTransformer(
...          [("scale", StandardScaler(), ["expert_rating"])]).fit(X)
>>> X_new = pd.DataFrame({"expert_rating": [5, 6, 1],
...                       "ignored_new_col": [1.2, 0.3, -0.1]})
>>> ct.transform(X_new)
array([[ 0.9],
       [ 2.1],
       [-3.9]])
```

Visualizing Composite Estimators

Estimators are displayed with an HTML representation when shown in a jupyter notebook. This is useful to diagnose or visualize a Pipeline with many estimators. This visualization is activated by default:

```
>>> column_trans
```

It can be deactivated by setting the display option in `set_config` to 'text':

```
>>> from sklearn import set_config
>>> set_config(display='text')
>>> # displays text representation in a jupyter context
>>> column_trans
```

An example of the HTML output can be seen in the **HTML representation of Pipeline** section of sphx_glr_auto_examples_compose_plot_column_transformer_mixed_types.py. As an alternative, the HTML can be written to a file using `estimator_html_repr`:

```
>>> from sklearn.utils import estimator_html_repr
>>> with open('my_estimator.html', 'w') as f:
...     f.write(estimator_html_repr(clf))
```

Examples

- sphx_glr_auto_examples_compose_plot_column_transformer.py
- sphx_glr_auto_examples_compose_plot_column_transformer_mixed_types.py

Feature selection

Source: *https://scikit-learn.org/stable/modules/feature_selection.html*

The classes in the `sklearn.feature_selection` module can be used for feature selection/dimensionality reduction on sample sets, either to improve estimators' accuracy scores or to boost their performance on very high-dimensional datasets.

Removing features with low variance

`VarianceThreshold` is a simple baseline approach to feature selection. It removes all features whose variance doesn't meet some threshold. By default, it removes all zero-variance features, i.e. features that have the same value in all samples.

As an example, suppose that we have a dataset with boolean features, and we want to remove all features that are either one or zero (on or off) in more than 80% of the samples. Boolean features are Bernoulli random variables, and the variance of such variables is given by

$$\mathrm{Var}[X] = p(1-p)$$

so we can select using the threshold `.8 * (1 - .8)`:

```
>>> from sklearn.feature_selection import VarianceThreshold
>>> X = [[0, 0, 1], [0, 1, 0], [1, 0, 0], [0, 1, 1], [0, 1, 0], [0, 1, 1]]
>>> sel = VarianceThreshold(threshold=(.8 * (1 - .8)))
>>> sel.fit_transform(X)
array([[0, 1],
       [1, 0],
       [0, 0],
       [1, 1],
       [1, 0],
       [1, 1]])
```

As expected, `VarianceThreshold` has removed the first column, which has a probability $p = 5/6 > .8$ of containing a zero.

Univariate feature selection

Univariate feature selection works by selecting the best features based on univariate statistical tests. It can be seen as a preprocessing step to an estimator. Scikit-learn exposes feature selection routines as objects that implement the `transform` method:

- `SelectKBest` removes all but the k highest scoring features
- `SelectPercentile` removes all but a user-specified highest scoring percentage of features
- using common univariate statistical tests for each feature: false positive rate `SelectFpr`, false discovery rate `SelectFdr`, or family wise error `SelectFwe`.
- `GenericUnivariateSelect` allows to perform univariate feature selection with a configurable strategy. This allows to select the best univariate selection strategy with hyper-parameter search estimator.

For instance, we can use an F-test to retrieve the two best features for a dataset as follows:

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.feature_selection import SelectKBest
>>> from sklearn.feature_selection import f_classif
>>> X, y = load_iris(return_X_y=True)
>>> X.shape
(150, 4)
>>> X_new = SelectKBest(f_classif, k=2).fit_transform(X, y)
>>> X_new.shape
(150, 2)
```

These objects take as input a scoring function that returns univariate scores and p-values (or only scores for `SelectKBest` and `SelectPercentile`):

- For regression: `r_regression`, `f_regression`, `mutual_info_regression`
- For classification: `chi2`, `f_classif`, `mutual_info_classif`

The methods based on F-test estimate the degree of linear dependency between two random variables. On the other hand, mutual information methods can capture any kind of statistical dependency, but being nonparametric, they require more samples for accurate estimation. Note that the χ^2-test should only be applied to non-negative features, such as frequencies.

If you use sparse data (i.e. data represented as sparse matrices), will deal with the data without making it dense.

> Warning
> Beware not to use a regression scoring function with a classification problem, you will get useless results.

> Note
> The `SelectPercentile` and `SelectKBest` support unsupervised feature selection as well. One needs to provide a score_func where y=None. The score_func should use internally X to compute the scores.

Examples

- sphx_glr_auto_examples_feature_selection_plot_feature_selection.py
- sphx_glr_auto_examples_feature_selection_plot_f_test_vs_mi.py

Recursive feature elimination

Given an external estimator that assigns weights to features (e.g., the coefficients of a linear model), the goal of recursive feature elimination (`RFE`) is to select features by recursively considering smaller and smaller sets of features. First, the estimator is trained on the initial set of features and the importance of each feature is obtained either through any specific attribute (such as `coef_`, `feature_importances_`) or callable. Then, the least important features are pruned from the current set of features. That procedure is recursively repeated on the pruned set until the desired number of features to select is eventually reached.

`RFECV` performs RFE in a cross-validation loop to find the optimal number of features. In more details, the number of features selected is tuned automatically by fitting an `RFE` selector on the different cross-validation splits (provided by the cv parameter). The performance of the `RFE` selector is evaluated using scorer for different numbers of selected features and aggregated together. Finally, the scores are averaged across folds and the number of features selected is set to the number of features that maximize the cross-validation score.

Examples

- sphx_glr_auto_examples_feature_selection_plot_rfe_digits.py: A recursive feature elimination example showing the relevance of pixels in a digit classification task.
- sphx_glr_auto_examples_feature_selection_plot_rfe_with_cross_validation.py: A recursive feature elimination example with automatic tuning of the number of features selected with cross-validation.

Feature selection using SelectFromModel

`SelectFromModel` is a meta-transformer that can be used alongside any estimator that assigns importance to each feature through a specific attribute (such as `coef_`, `feature_importances_`) or via an importance_getter callable after fitting. The features are considered unimportant and removed if the corresponding importance of the feature values is below the provided `threshold` parameter. Apart from specifying the threshold numerically, there are built-in heuristics for finding a threshold using a string argument. Available heuristics are ”mean”, ”median” and float multiples of these like ”0.1*mean”. In combination with the threshold criteria, one can use the max_features parameter to set a limit on the number of features to select.

For examples on how it is to be used refer to the sections below.

Examples

- sphx_glr_auto_examples_feature_selection_plot_select_from_model_diabetes.py

L1-based feature selection

Linear models penalized with the L1 norm have sparse solutions: many of their estimated coefficients are zero. When the goal is to reduce the dimensionality of the data to use with another classifier, they can be used along with `SelectFromModel` to select the non-zero coefficients. In particular, sparse estimators useful for this purpose are the `Lasso` for regression, and of `LogisticRegression` and `LinearSVC` for classification:

```
>>> from sklearn.svm import LinearSVC
>>> from sklearn.datasets import load_iris
>>> from sklearn.feature_selection import SelectFromModel
>>> X, y = load_iris(return_X_y=True)
>>> X.shape
(150, 4)
>>> lsvc = LinearSVC(C=0.01, penalty="l1", dual=False).fit(X, y)
>>> model = SelectFromModel(lsvc, prefit=True)
>>> X_new = model.transform(X)
>>> X_new.shape
(150, 3)
```

With SVMs and logistic regression, the parameter C controls the sparsity: the smaller C the fewer features selected. With Lasso, the higher the alpha parameter, the fewer features selected.

Examples

- sphx_glr_auto_examples_linear_model_plot_lasso_dense_vs_sparse_data.py.

L1-recovery and compressive sensing
For a good choice of alpha, the lasso can fully recover the exact set of non-zero variables using only few observations, provided certain specific conditions are met. In particular, the number of samples should be ”sufficiently large”, or L1 models will perform at random, where ”sufficiently large” depends on the number of non-zero coefficients, the logarithm of the number of features, the amount of noise, the smallest absolute value of non-zero coefficients, and the structure of the design matrix X. In addition, the design matrix must display certain

specific properties, such as not being too correlated. On the use of Lasso for sparse signal recovery, see this example on compressive sensing:
There is no general rule to select an alpha parameter for recovery of non-zero coefficients. It can be set by cross-validation (`LassoCV` or under-penalized models: including a small number of non-relevant variables is not detrimental to prediction score. BIC (`LassoLarsIC`) tends, on the opposite, to set high values of alpha.
References
Richard G. Baraniuk `"Compressive Sensing"`, IEEE Signal Processing Magazine [120] July 2007

Tree-based feature selection

Tree-based estimators (see the `sklearn.tree` module and forest of trees in the `sklearn.ensemble` module) can be used to compute impurity-based feature importances, which in turn can be used to discard irrelevant features (when coupled with the `SelectFromModel` meta-transformer):

```
>>> from sklearn.ensemble import ExtraTreesClassifier
>>> from sklearn.datasets import load_iris
>>> from sklearn.feature_selection import SelectFromModel
>>> X, y = load_iris(return_X_y=True)
>>> X.shape
(150, 4)
>>> clf = ExtraTreesClassifier(n_estimators=50)
>>> clf = clf.fit(X, y)
>>> clf.feature_importances_
array([ 0.04,  0.05,  0.4,  0.4])
>>> model = SelectFromModel(clf, prefit=True)
>>> X_new = model.transform(X)
>>> X_new.shape
(150, 2)
```

Examples

- sphx_glr_auto_examples_ensemble_plot_forest_importances.py: example on synthetic data showing the recovery of the actually meaningful features.
- sphx_glr_auto_examples_inspection_plot_permutation_importance.py: example discussing the caveats of using impurity-based feature importances as a proxy for feature relevance.

Sequential Feature Selection

Sequential Feature Selection [sfs] (SFS) is available in the `SequentialFeatureSelector` transformer. SFS can be either forward or backward:

Forward-SFS is a greedy procedure that iteratively finds the best new feature to add to the set of selected features. Concretely, we initially start with zero features and find the one feature that maximizes a cross-validated score when an estimator is trained on this single feature. Once that first feature is selected, we repeat the procedure by adding a new feature to the set of selected features. The procedure stops when the desired number of selected features is reached, as determined by the n_features_to_select parameter.

Backward-SFS follows the same idea but works in the opposite direction: instead of starting with no features and greedily adding features, we start with *all* the features and greedily *remove* features from the set. The direction parameter controls whether forward or backward SFS is used.

Details on Sequential Feature Selection
In general, forward and backward selection do not yield equivalent results. Also, one may be much faster than the other depending on the requested number of selected features: if we have 10 features and ask for 7 selected features, forward selection would need to perform 7 iterations while backward selection would only need to perform 3.
SFS differs from `RFE` and require the underlying model to expose a coef_ or feature_importances_ attribute. It may however be slower considering that more models need to be evaluated, compared to the other approaches. For example in backward selection, the iteration going from m features to m - 1 features using k-fold cross-validation requires fitting m * k models, while fit and requires no iterations.
References

Examples

- sphx_glr_auto_examples_feature_selection_plot_select_from_model_diabetes.py

Feature selection as part of a pipeline

Feature selection is usually used as a pre-processing step before doing the actual learning. The recommended way to do this in scikit-learn is to use a `Pipeline`:

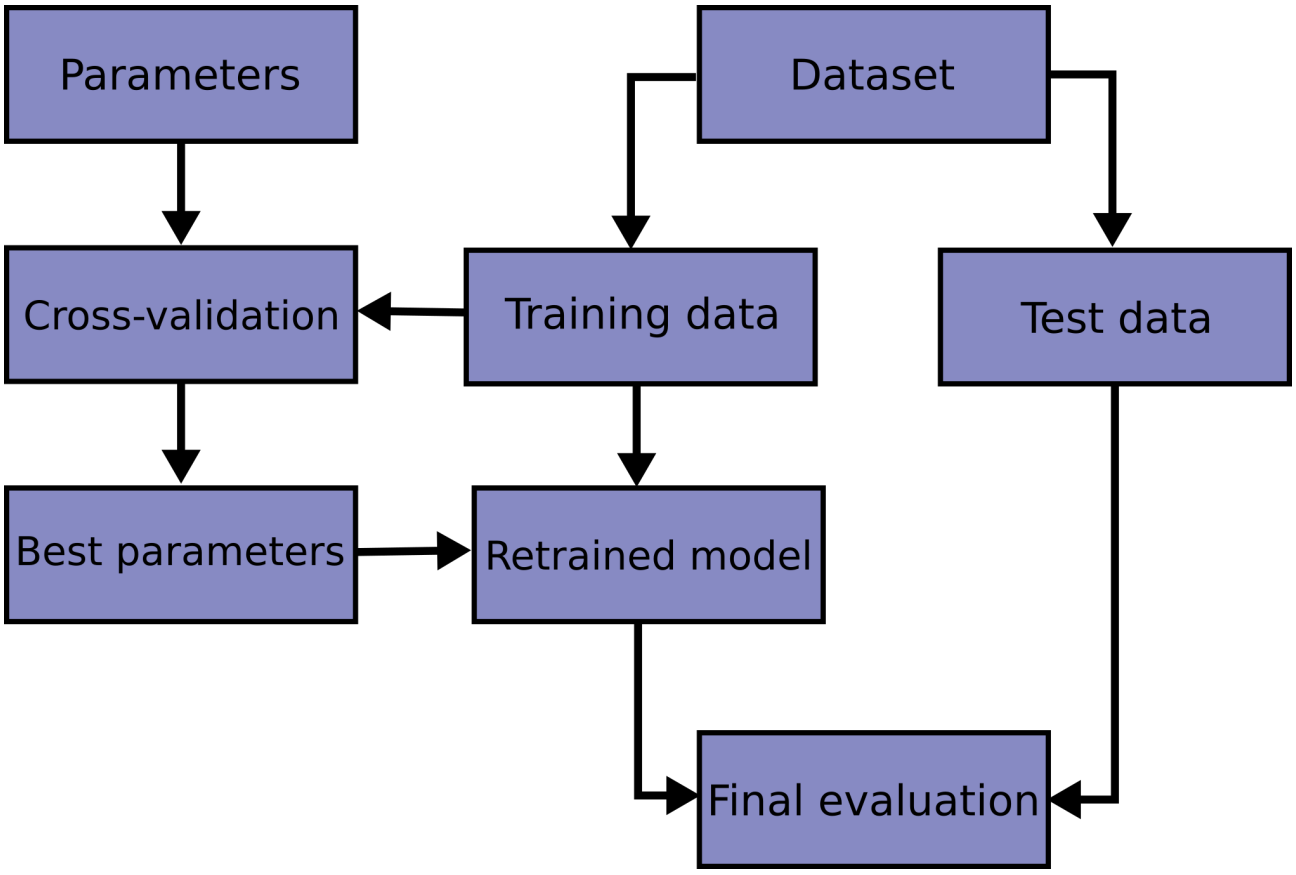
```
clf = Pipeline([
  ('feature_selection', SelectFromModel(LinearSVC(penalty="l1"))),
  ('classification', RandomForestClassifier())
])
clf.fit(X, y)
```

In this snippet we make use of a `LinearSVC` coupled with `SelectFromModel` to evaluate feature importances and select the most relevant features. Then, a `RandomForestClassifier` is trained on the transformed output, i.e. using only relevant features. You can perform similar operations with the other feature selection methods and also classifiers that provide a way to evaluate feature importances of course. See the `Pipeline` examples for more details.

Cross-validation

Source: *https://scikit-learn.org/stable/modules/cross_validation.html*

Learning the parameters of a prediction function and testing it on the same data is a methodological mistake: a model that would just repeat the labels of the samples that it has just seen would have a perfect score but would fail to predict anything useful on yet-unseen data. This situation is called **overfitting**. To avoid it, it is common practice when performing a (supervised) machine learning experiment to hold out part of the available data as a **test set** `X_test, y_test`. Note that the word "experiment" is not intended to denote academic use only, because even in commercial settings machine learning usually starts out experimentally. Here is a flowchart of typical cross validation workflow in model training. The best parameters can be determined by grid search techniques.



In scikit-learn a random split into training and test sets can be quickly computed with the `train_test_split` helper function. Let's load the iris data set to fit a linear support vector machine on it:

```
>>> import numpy as np
>>> from sklearn.model_selection import train_test_split
>>> from sklearn import datasets
>>> from sklearn import svm

>>> X, y = datasets.load_iris(return_X_y=True)
>>> X.shape, y.shape
```

```
((150, 4), (150,))
```

We can now quickly sample a training set while holding out 40% of the data for testing (evaluating) our classifier:

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.4, random_state=0)

>>> X_train.shape, y_train.shape
((90, 4), (90,))
>>> X_test.shape, y_test.shape
((60, 4), (60,))

>>> clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.96
```

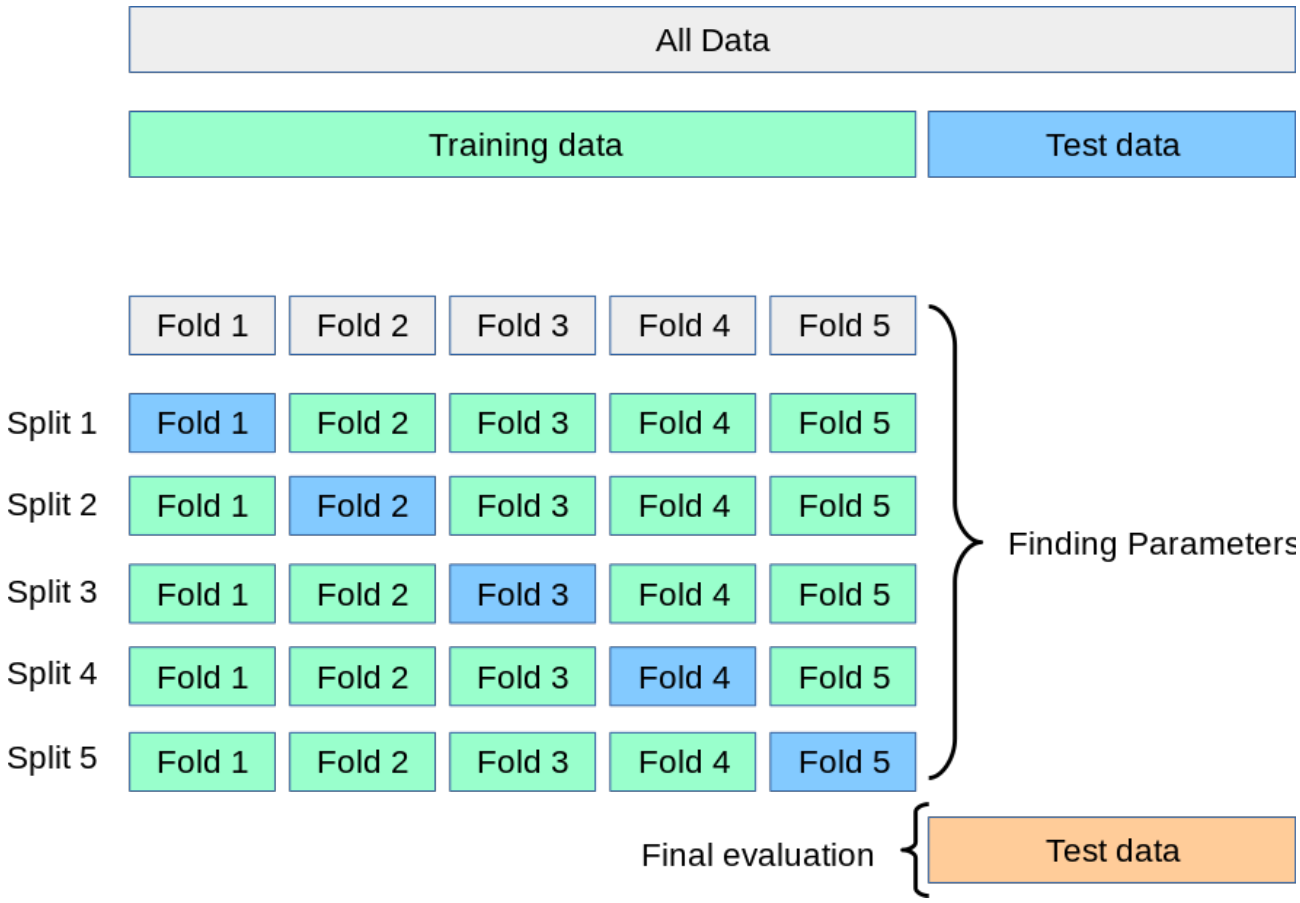
When evaluating different settings (”hyperparameters”) for estimators, such as the `C` setting that must be manually set for an SVM, there is still a risk of overfitting *on the test set* because the parameters can be tweaked until the estimator performs optimally. This way, knowledge about the test set can ”leak” into the model and evaluation metrics no longer report on generalization performance. To solve this problem, yet another part of the dataset can be held out as a so-called ”validation set”: training proceeds on the training set, after which evaluation is done on the validation set, and when the experiment seems to be successful, final evaluation can be done on the test set.

However, by partitioning the available data into three sets, we drastically reduce the number of samples which can be used for learning the model, and the results can depend on a particular random choice for the pair of (train, validation) sets.

A solution to this problem is a procedure called cross-validation (CV for short). A test set should still be held out for final evaluation, but the validation set is no longer needed when doing CV. In the basic approach, called *k*-fold CV, the training set is split into *k* smaller sets (other approaches are described below, but generally follow the same principles). The following procedure is followed for each of the *k* ”folds”:

- A model is trained using $k-1$ of the folds as training data;
- the resulting model is validated on the remaining part of the data (i.e., it is used as a test set to compute a performance measure such as accuracy).

The performance measure reported by *k*-fold cross-validation is then the average of the values computed in the loop. This approach can be computationally expensive, but does not waste too much data (as is the case when fixing an arbitrary validation set), which is a major advantage in problems such as inverse inference where the number of samples is very small.



Computing cross-validated metrics

The simplest way to use cross-validation is to call the `cross_val_score` helper function on the estimator and the dataset.

The following example demonstrates how to estimate the accuracy of a linear kernel support vector machine on the iris dataset by splitting the data, fitting a model and computing the score 5 consecutive times (with different splits each time):

```
>>> from sklearn.model_selection import cross_val_score
>>> clf = svm.SVC(kernel='linear', C=1, random_state=42)
>>> scores = cross_val_score(clf, X, y, cv=5)
>>> scores
array([0.96, 1. , 0.96, 0.96, 1. ])
```

The mean score and the standard deviation are hence given by:

```
>>> print("%0.2f accuracy with a standard deviation of %0.2f" % (scores.mean(), scores.std()))
0.98 accuracy with a standard deviation of 0.02
```

By default, the score computed at each CV iteration is the `score` method of the estimator. It is possible to change this by using the scoring parameter:

```
>>> from sklearn import metrics
>>> scores = cross_val_score(
...     clf, X, y, cv=5, scoring='f1_macro')
>>> scores
array([0.96, 1., 0.96, 0.96, 1.])
```

See scoring_parameter for details. In the case of the Iris dataset, the samples are balanced across target classes hence the accuracy and the F1-score are almost equal.

When the `cv` argument is an integer, `cross_val_score` uses the `KFold` or `StratifiedKFold` strategies by default, the latter being used if the estimator derives from `ClassifierMixin <sklearn.base.ClassifierMixin>`.

It is also possible to use other cross validation strategies by passing a cross validation iterator instead, for instance:

```
>>> from sklearn.model_selection import ShuffleSplit
>>> n_samples = X.shape[0]
>>> cv = ShuffleSplit(n_splits=5, test_size=0.3, random_state=0)
>>> cross_val_score(clf, X, y, cv=cv)
array([0.977, 0.977, 1., 0.955, 1.])
```

Another option is to use an iterable yielding (train, test) splits as arrays of indices, for example:

```
>>> def custom_cv_2folds(X):
...     n = X.shape[0]
...     i = 1
...     while i <= 2:
...         idx = np.arange(n * (i - 1) / 2, n * i / 2, dtype=int)
...         yield idx, idx
...         i += 1
...
>>> custom_cv = custom_cv_2folds(X)
>>> cross_val_score(clf, X, y, cv=custom_cv)
array([1.        , 0.973])
```

Data transformation with held-out data
Just as it is important to test a predictor on data held-out from training, preprocessing (such as standardization, feature selection, etc.) and similar data transformations similarly should be learnt from a training set and applied to held-out data for prediction:

```
>>> from sklearn import preprocessing
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.4, random_state=0)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train_transformed = scaler.transform(X_train)
>>> clf = svm.SVC(C=1).fit(X_train_transformed, y_train)
>>> X_test_transformed = scaler.transform(X_test)
>>> clf.score(X_test_transformed, y_test)
0.9333
```

A `Pipeline` makes it easier to compose estimators, providing this behavior under cross-validation:

```
>>> from sklearn.pipeline import make_pipeline
>>> clf = make_pipeline(preprocessing.StandardScaler(), svm.SVC(C=1))
>>> cross_val_score(clf, X, y, cv=cv)
array([0.977, 0.933, 0.955, 0.933, 0.977])
```

See combining_estimators.

The cross_validate function and multiple metric evaluation

The `cross_validate` function differs from `cross_val_score` in two ways:

- It allows specifying multiple metrics for evaluation.

- It returns a dict containing fit-times, score-times (and optionally training scores, fitted estimators, train-test split indices) in addition to the test score.

For single metric evaluation, where the scoring parameter is a string, callable or None, the keys will be - `['test_score', 'fit_time', 'score_time']`

And for multiple metric evaluation, the return value is a dict with the following keys - `['test_<scorer1_name>', 'test_<scorer2_name>', 'test_<scorer...>', 'fit_time', 'score_time']`

`return_train_score` is set to `False` by default to save computation time. To evaluate the scores on the training set as well you need to set it to `True`. You may also retain the estimator fitted on each training set by setting `return_estimator=True`. Similarly, you may set return_indices=True to retain the training and testing indices used to split the dataset into train and test sets for each cv split.

The multiple metrics can be specified either as a list, tuple or set of predefined scorer names:

```
>>> from sklearn.model_selection import cross_validate
>>> from sklearn.metrics import recall_score
>>> scoring = ['precision_macro', 'recall_macro']
>>> clf = svm.SVC(kernel='linear', C=1, random_state=0)
>>> scores = cross_validate(clf, X, y, scoring=scoring)
>>> sorted(scores.keys())
['fit_time', 'score_time', 'test_precision_macro', 'test_recall_macro']
>>> scores['test_recall_macro']
array([0.96, 1., 0.96, 0.96, 1.])
```

Or as a dict mapping scorer name to a predefined or custom scoring function:

```
>>> from sklearn.metrics import make_scorer
>>> scoring = {'prec_macro': 'precision_macro',
...            'rec_macro': make_scorer(recall_score, average='macro')}
>>> scores = cross_validate(clf, X, y, scoring=scoring,
...                         cv=5, return_train_score=True)
>>> sorted(scores.keys())
['fit_time', 'score_time', 'test_prec_macro', 'test_rec_macro',
 'train_prec_macro', 'train_rec_macro']
>>> scores['train_rec_macro']
array([0.97, 0.97, 0.99, 0.98, 0.98])
```

Here is an example of `cross_validate` using a single metric:

```
>>> scores = cross_validate(clf, X, y,
...                         scoring='precision_macro', cv=5,
...                         return_estimator=True)
>>> sorted(scores.keys())
['estimator', 'fit_time', 'score_time', 'test_score']
```

Obtaining predictions by cross-validation

The function `cross_val_predict` has a similar interface to `cross_val_score`, but returns, for each element in the input, the prediction that was obtained for that element when it was in the test set. Only cross-validation strategies that assign all elements to a test set exactly once can be used (otherwise, an exception is raised).

> Warning
> Note on inappropriate usage of cross_val_predict
> The result of `cross_val_predict` may be different from those obtained using `cross_val_score` as the elements are grouped in different ways. The function `cross_val_score` takes an av-

erage over cross-validation folds, whereas `cross_val_predict` simply returns the labels (or probabilities) from several distinct models undistinguished. Thus, `cross_val_predict` is not an appropriate measure of generalization error.

The function `cross_val_predict` is appropriate for:

- Visualization of predictions obtained from different models.
- Model blending: When predictions of one supervised estimator are used to train another estimator in ensemble methods.

The available cross validation iterators are introduced in the following section.

Examples

- sphx_glr_auto_examples_model_selection_plot_roc_crossval.py,
- sphx_glr_auto_examples_feature_selection_plot_rfe_with_cross_validation.py,
- sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py,
- sphx_glr_auto_examples_model_selection_plot_grid_search_text_feature_extraction.py,
- sphx_glr_auto_examples_model_selection_plot_cv_predict.py,
- sphx_glr_auto_examples_model_selection_plot_nested_cross_validation_iris.py.

Cross validation iterators

The following sections list utilities to generate indices that can be used to generate dataset splits according to different cross validation strategies.

Cross-validation iterators for i.i.d. data

Assuming that some data is Independent and Identically Distributed (i.i.d.) is making the assumption that all samples stem from the same generative process and that the generative process is assumed to have no memory of past generated samples.

The following cross-validators can be used in such cases.

Note
While i.i.d. data is a common assumption in machine learning theory, it rarely holds in practice. If one knows that the samples have been generated using a time-dependent process, it is safer to use a time-series aware cross-validation scheme. Similarly, if we know that the generative process has a group structure (samples collected from different subjects, experiments, measurement devices), it is safer to use group-wise cross-validation.

K-fold

`KFold` divides all the samples in k groups of samples, called folds (if $k = n$, this is equivalent to the *Leave One Out* strategy), of equal sizes (if possible). The prediction function is learned using $k - 1$ folds, and the fold left out is used for test.

Example of 2-fold cross-validation on a dataset with 4 samples:

```
>>> import numpy as np
>>> from sklearn.model_selection import KFold

>>> X = ["a", "b", "c", "d"]
>>> kf = KFold(n_splits=2)
>>> for train, test in kf.split(X):
...     print("%s %s" % (train, test))
[2 3] [0 1]
```

```
[0 1] [2 3]
```

Here is a visualization of the cross-validation behavior. Note that `KFold` is not affected by classes or groups.

Each fold is constituted by two arrays: the first one is related to the *training set*, and the second one to the *test set*. Thus, one can create the training/test sets using numpy indexing:

```
>>> X = np.array([[0., 0.], [1., 1.], [-1., -1.], [2., 2.]])
>>> y = np.array([0, 1, 0, 1])
>>> X_train, X_test, y_train, y_test = X[train], X[test], y[train], y[test]
```

Repeated K-Fold

`RepeatedKFold` repeats `KFold` n times, producing different splits in each repetition.

Example of 2-fold K-Fold repeated 2 times:

```
>>> import numpy as np
>>> from sklearn.model_selection import RepeatedKFold
>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
>>> random_state = 12883823
>>> rkf = RepeatedKFold(n_splits=2, n_repeats=2, random_state=random_state)
>>> for train, test in rkf.split(X):
...     print("%s %s" % (train, test))
...
[2 3] [0 1]
[0 1] [2 3]
[0 2] [1 3]
[1 3] [0 2]
```

Similarly, `RepeatedStratifiedKFold` repeats `StratifiedKFold` n times with different randomization in each repetition.

Leave One Out (LOO)

`LeaveOneOut` (or LOO) is a simple cross-validation. Each learning set is created by taking all the samples except one, the test set being the sample left out. Thus, for n samples, we have n different training sets and n different test sets. This cross-validation procedure does not waste much data as only one sample is removed from the training set:

```
>>> from sklearn.model_selection import LeaveOneOut

>>> X = [1, 2, 3, 4]
>>> loo = LeaveOneOut()
>>> for train, test in loo.split(X):
...     print("%s %s" % (train, test))
[1 2 3] [0]
[0 2 3] [1]
[0 1 3] [2]
[0 1 2] [3]
```

Potential users of LOO for model selection should weigh a few known caveats. When compared with k-fold cross validation, one builds n models from n samples instead of k models, where $n > k$. Moreover, each is trained on $n - 1$ samples rather than $(k - 1)n/k$. In both ways, assuming k is not too large and $k < n$, LOO is more computationally expensive than k-fold cross validation.

In terms of accuracy, LOO often results in high variance as an estimator for the test error. Intuitively, since $n - 1$ of the n samples are used to build each model, models constructed from folds are virtually identical to each other and to the model built from the entire training set.

However, if the learning curve is steep for the training size in question, then 5 or 10-fold cross validation can overestimate the generalization error.

As a general rule, most authors and empirical evidence suggest that 5 or 10-fold cross validation should be preferred to LOO.

References

- http://www.faqs.org/faqs/ai-faq/neural-nets/part3/section-12.html;
- T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning, Springer 2009
- L. Breiman, P. Spector Submodel selection and evaluation in regression: The X-random case, International Statistical Review 1992;
- R. Kohavi, A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection, Intl. Jnt. Conf. AI
- R. Bharat Rao, G. Fung, R. Rosales, On the Dangers of Cross-Validation. An Experimental Evaluation, SIAM 2008;
- G. James, D. Witten, T. Hastie, R. Tibshirani, An Introduction to Statistical Learning, Springer 2013.

Leave P Out (LPO)

`LeavePOut` is very similar to `LeaveOneOut` as it creates all the possible training/test sets by removing p samples from the complete set. For n samples, this produces $\binom{n}{p}$ train-test pairs. Unlike `LeaveOneOut` and `KFold`, the test sets will overlap for $p > 1$.

Example of Leave-2-Out on a dataset with 4 samples:

```
>>> from sklearn.model_selection import LeavePOut

>>> X = np.ones(4)
>>> lpo = LeavePOut(p=2)
>>> for train, test in lpo.split(X):
...     print("%s %s" % (train, test))
[2 3] [0 1]
[1 3] [0 2]
[1 2] [0 3]
[0 3] [1 2]
[0 2] [1 3]
[0 1] [2 3]
```

Random permutations cross-validation a.k.a. Shuffle & Split

The `ShuffleSplit` iterator will generate a user defined number of independent train / test dataset splits. Samples are first shuffled and then split into a pair of train and test sets.

It is possible to control the randomness for reproducibility of the results by explicitly seeding the `random_state` pseudo random number generator.

Here is a usage example:

```
>>> from sklearn.model_selection import ShuffleSplit
>>> X = np.arange(10)
>>> ss = ShuffleSplit(n_splits=5, test_size=0.25, random_state=0)
>>> for train_index, test_index in ss.split(X):
```

```
...     print("%s %s" % (train_index, test_index))
[9 1 6 7 3 0 5] [2 8 4]
[2 9 8 0 6 7 4] [3 5 1]
[4 5 1 0 6 9 7] [2 3 8]
[2 7 5 8 0 3 4] [6 1 9]
[4 1 0 6 8 9 3] [5 2 7]
```

Here is a visualization of the cross-validation behavior. Note that `ShuffleSplit` is not affected by classes or groups.

`ShuffleSplit` is thus a good alternative to `KFold` cross validation that allows a finer control on the number of iterations and the proportion of samples on each side of the train / test split.

Cross-validation iterators with stratification based on class labels

Some classification tasks can naturally exhibit rare classes: for instance, there could be orders of magnitude more negative observations than positive observations (e.g. medical screening, fraud detection, etc). As a result, cross-validation splitting can generate train or validation folds without any occurrence of a particular class. This typically leads to undefined classification metrics (e.g. ROC AUC), exceptions raised when attempting to call fit or missing columns in the output of the predict_proba or decision_function methods of multiclass classifiers trained on different folds.

To mitigate such problems, splitters such as `StratifiedKFold` and `StratifiedShuffleSplit` implement stratified sampling to ensure that relative class frequencies are approximately preserved in each fold.

> Note
> Stratified sampling was introduced in scikit-learn to workaround the aforementioned engineering problems rather than solve a statistical one.
> Stratification makes cross-validation folds more homogeneous, and as a result hides some of the variability inherent to fitting models with a limited number of observations.
> As a result, stratification can artificially shrink the spread of the metric measured across cross-validation iterations: the inter-fold variability does no longer reflect the uncertainty in the performance of classifiers in the presence of rare classes.

Stratified K-fold

`StratifiedKFold` is a variation of *K-fold* which returns *stratified* folds: each set contains approximately the same percentage of samples of each target class as the complete set.

Here is an example of stratified 3-fold cross-validation on a dataset with 50 samples from two unbalanced classes. We show the number of samples in each class and compare with `KFold`.

```
>>> from sklearn.model_selection import StratifiedKFold, KFold
>>> import numpy as np
>>> X, y = np.ones((50, 1)), np.hstack(([0] * 45, [1] * 5))
>>> skf = StratifiedKFold(n_splits=3)
>>> for train, test in skf.split(X, y):
...     print('train -  {}   |   test -  {}'.format(
...         np.bincount(y[train]), np.bincount(y[test])))
train -  [30  3]   |   test -  [15  2]
train -  [30  3]   |   test -  [15  2]
train -  [30  4]   |   test -  [15  1]
>>> kf = KFold(n_splits=3)
>>> for train, test in kf.split(X, y):
...     print('train -  {}   |   test -  {}'.format(
...         np.bincount(y[train]), np.bincount(y[test])))
```

```
train -  [28  5]   |   test -  [17]
train -  [28  5]   |   test -  [17]
train -  [34]   |   test -  [11  5]
```

We can see that `StratifiedKFold` preserves the class ratios (approximately 1 / 10) in both train and test datasets.

Here is a visualization of the cross-validation behavior.

`RepeatedStratifiedKFold` can be used to repeat Stratified K-Fold n times with different randomization in each repetition.

Stratified Shuffle Split

`StratifiedShuffleSplit` is a variation of *ShuffleSplit*, which returns stratified splits, *i.e.* which creates splits by preserving the same percentage for each target class as in the complete set.

Here is a visualization of the cross-validation behavior.

Predefined fold-splits / Validation-sets

For some datasets, a pre-defined split of the data into training- and validation fold or into several cross-validation folds already exists. Using `PredefinedSplit` it is possible to use these folds e.g. when searching for hyperparameters.

For example, when using a validation set, set the `test_fold` to 0 for all samples that are part of the validation set, and to -1 for all other samples.

Cross-validation iterators for grouped data

The i.i.d. assumption is broken if the underlying generative process yields groups of dependent samples.

Such a grouping of data is domain specific. An example would be when there is medical data collected from multiple patients, with multiple samples taken from each patient. And such data is likely to be dependent on the individual group. In our example, the patient id for each sample will be its group identifier.

In this case we would like to know if a model trained on a particular set of groups generalizes well to the unseen groups. To measure this, we need to ensure that all the samples in the validation fold come from groups that are not represented at all in the paired training fold.

The following cross-validation splitters can be used to do that. The grouping identifier for the samples is specified via the `groups` parameter.

Group K-fold

`GroupKFold` is a variation of K-fold which ensures that the same group is not represented in both testing and training sets. For example if the data is obtained from different subjects with several samples per-subject and if the model is flexible enough to learn from highly person specific features it could fail to generalize to new subjects. `GroupKFold` makes it possible to detect this kind of overfitting situations.

Imagine you have three subjects, each with an associated number from 1 to 3:

```
>>> from sklearn.model_selection import GroupKFold

>>> X = [0.1, 0.2, 2.2, 2.4, 2.3, 4.55, 5.8, 8.8, 9, 10]
>>> y = ["a", "b", "b", "b", "c", "c", "c", "d", "d", "d"]
```

```
>>> groups = [1, 1, 1, 2, 2, 2, 3, 3, 3, 3]

>>> gkf = GroupKFold(n_splits=3)
>>> for train, test in gkf.split(X, y, groups=groups):
...     print("%s %s" % (train, test))
[0 1 2 3 4 5] [6 7 8 9]
[0 1 2 6 7 8 9] [3 4 5]
[3 4 5 6 7 8 9] [0 1 2]
```

Each subject is in a different testing fold, and the same subject is never in both testing and training. Notice that the folds do not have exactly the same size due to the imbalance in the data. If class proportions must be balanced across folds, `StratifiedGroupKFold` is a better option.

Here is a visualization of the cross-validation behavior.

Similar to `KFold`, the test sets from `GroupKFold` will form a complete partition of all the data.

While `GroupKFold` attempts to place the same number of samples in each fold when `shuffle=False`, when `shuffle=True` it attempts to place an equal number of distinct groups in each fold (but does not account for group sizes).

StratifiedGroupKFold

`StratifiedGroupKFold` is a cross-validation scheme that combines both `StratifiedKFold` and `GroupKFold`. The idea is to try to preserve the distribution of classes in each split while keeping each group within a single split. That might be useful when you have an unbalanced dataset so that using just `GroupKFold` might produce skewed splits.

Example:

```
>>> from sklearn.model_selection import StratifiedGroupKFold
>>> X = list(range(18))
>>> y = [1] * 6 + [0] * 12
>>> groups = [1, 2, 3, 3, 4, 4, 1, 1, 2, 2, 3, 4, 5, 5, 5, 6, 6, 6]
>>> sgkf = StratifiedGroupKFold(n_splits=3)
>>> for train, test in sgkf.split(X, y, groups=groups):
...     print("%s %s" % (train, test))
[ 0  2  3  4  5  6  7 10 11 15 16 17] [ 1  8  9 12 13 14]
[ 0  1  4  5  6  7  8  9 11 12 13 14] [ 2  3 10 15 16 17]
[ 1  2  3  8  9 10 12 13 14 15 16 17] [ 0  4  5  6  7 11]
```

Implementation notes

- With the current implementation full shuffle is not possible in most scenarios. When shuffle=True, the following happens:
  1. All groups are shuffled.
  2. Groups are sorted by standard deviation of classes using stable sort.
  3. Sorted groups are iterated over and assigned to folds.

  That means that only groups with the same standard deviation of class distribution will be shuffled, which might be useful when each group has only a single class.
- The algorithm greedily assigns each group to one of n_splits test sets, choosing the test set that minimises the variance in class distribution across test sets. Group assignment proceeds from groups with highest to lowest variance in class frequency, i.e. large groups peaked on one or few classes are assigned first.
- This split is suboptimal in a sense that it might produce imbalanced splits even if perfect stratification is possible. If you have relatively close distribution of classes in each group, using `GroupKFold` is better.

Here is a visualization of cross-validation behavior for uneven groups:

Leave One Group Out

`LeaveOneGroupOut` is a cross-validation scheme where each split holds out samples belonging to one specific group. Group information is provided via an array that encodes the group of each sample.

Each training set is thus constituted by all the samples except the ones related to a specific group. This is the same as `LeavePGroupsOut` with n_groups=1 and the same as `GroupKFold` with n_splits equal to the number of unique labels passed to the groups parameter.

For example, in the cases of multiple experiments, `LeaveOneGroupOut` can be used to create a cross-validation based on the different experiments: we create a training set using the samples of all the experiments except one:

```
>>> from sklearn.model_selection import LeaveOneGroupOut

>>> X = [1, 5, 10, 50, 60, 70, 80]
>>> y = [0, 1, 1, 2, 2, 2, 2]
>>> groups = [1, 1, 2, 2, 3, 3, 3]
>>> logo = LeaveOneGroupOut()
>>> for train, test in logo.split(X, y, groups=groups):
...     print("%s %s" % (train, test))
[2 3 4 5 6] [0 1]
[0 1 4 5 6] [2 3]
[0 1 2 3] [4 5 6]
```

Another common application is to use time information: for instance the groups could be the year of collection of the samples and thus allow for cross-validation against time-based splits.

Leave P Groups Out

`LeavePGroupsOut` is similar to `LeaveOneGroupOut`, but removes samples related to P groups for each training/test set. All possible combinations of P groups are left out, meaning test sets will overlap for $P > 1$.

Example of Leave-2-Group Out:

```
>>> from sklearn.model_selection import LeavePGroupsOut

>>> X = np.arange(6)
>>> y = [1, 1, 1, 2, 2, 2]
>>> groups = [1, 1, 2, 2, 3, 3]
>>> lpgo = LeavePGroupsOut(n_groups=2)
>>> for train, test in lpgo.split(X, y, groups=groups):
...     print("%s %s" % (train, test))
[4 5] [0 1 2 3]
[2 3] [0 1 4 5]
[0 1] [2 3 4 5]
```

Group Shuffle Split

The `GroupShuffleSplit` iterator behaves as a combination of `ShuffleSplit` and `LeavePGroupsOut`, and generates a sequence of randomized partitions in which a subset of groups are held out for each split. Each train/test split is performed independently meaning there is no guaranteed relationship between successive test sets.

Here is a usage example:

```
>>> from sklearn.model_selection import GroupShuffleSplit
```

```
>>> X = [0.1, 0.2, 2.2, 2.4, 2.3, 4.55, 5.8, 0.001]
>>> y = ["a", "b", "b", "b", "c", "c", "c", "a"]
>>> groups = [1, 1, 2, 2, 3, 3, 4, 4]
>>> gss = GroupShuffleSplit(n_splits=4, test_size=0.5, random_state=0)
>>> for train, test in gss.split(X, y, groups=groups):
...     print("%s %s" % (train, test))
...
[0 1 2 3] [4 5 6 7]
[2 3 6 7] [0 1 4 5]
[2 3 4 5] [0 1 6 7]
[4 5 6 7] [0 1 2 3]
```

Here is a visualization of the cross-validation behavior.

This class is useful when the behavior of `LeavePGroupsOut` is desired, but the number of groups is large enough that generating all possible partitions with P groups withheld would be prohibitively expensive. In such a scenario, `GroupShuffleSplit` provides a random sample (with replacement) of the train / test splits generated by `LeavePGroupsOut`.

Using cross-validation iterators to split train and test

The above group cross-validation functions may also be useful for splitting a dataset into training and testing subsets. Note that the convenience function `train_test_split` is a wrapper around `ShuffleSplit` and thus only allows for stratified splitting (using the class labels) and cannot account for groups.

To perform the train and test split, use the indices for the train and test subsets yielded by the generator output by the split() method of the cross-validation splitter. For example:

```
>>> import numpy as np
>>> from sklearn.model_selection import GroupShuffleSplit

>>> X = np.array([0.1, 0.2, 2.2, 2.4, 2.3, 4.55, 5.8, 0.001])
>>> y = np.array(["a", "b", "b", "b", "c", "c", "c", "a"])
>>> groups = np.array([1, 1, 2, 2, 3, 3, 4, 4])
>>> train_indx, test_indx = next(
...     GroupShuffleSplit(random_state=7).split(X, y, groups)
... )
>>> X_train, X_test, y_train, y_test = \
...     X[train_indx], X[test_indx], y[train_indx], y[test_indx]
>>> X_train.shape, X_test.shape
((6,), (2,))
>>> np.unique(groups[train_indx]), np.unique(groups[test_indx])
(array([1, 2, 4]), array([3]))
```

Cross validation of time series data

Time series data is characterized by the correlation between observations that are near in time (*autocorrelation*). However, classical cross-validation techniques such as `KFold` and `ShuffleSplit` assume the samples are independent and identically distributed, and would result in unreasonable correlation between training and testing instances (yielding poor estimates of generalization error) on time series data. Therefore, it is very important to evaluate our model for time series data on the ”future” observations least like those that are used to train the model. To achieve this, one solution is provided by `TimeSeriesSplit`.

Time Series Split

`TimeSeriesSplit` is a variation of *k-fold* which returns first k folds as train set and the $(k+1)$ th fold as test set. Note that unlike standard cross-validation methods, successive training sets are supersets of those that come before them. Also, it adds all surplus data to the first training partition, which is always used to train the model.

This class can be used to cross-validate time series data samples that are observed at fixed time intervals. Indeed, the folds must represent the same duration, in order to have comparable metrics across folds.

Example of 3-split time series cross-validation on a dataset with 6 samples:

```
>>> from sklearn.model_selection import TimeSeriesSplit

>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4], [1, 2], [3, 4]])
>>> y = np.array([1, 2, 3, 4, 5, 6])
>>> tscv = TimeSeriesSplit(n_splits=3)
>>> print(tscv)
TimeSeriesSplit(gap=0, max_train_size=None, n_splits=3, test_size=None)
>>> for train, test in tscv.split(X):
...     print("%s %s" % (train, test))
[0 1 2] [3]
[0 1 2 3] [4]
[0 1 2 3 4] [5]
```

Here is a visualization of the cross-validation behavior.

A note on shuffling

If the data ordering is not arbitrary (e.g. samples with the same class label are contiguous), shuffling it first may be essential to get a meaningful cross-validation result. However, the opposite may be true if the samples are not independently and identically distributed. For example, if samples correspond to news articles, and are ordered by their time of publication, then shuffling the data will likely lead to a model that is overfit and an inflated validation score: it will be tested on samples that are artificially similar (close in time) to training samples.

Some cross validation iterators, such as `KFold`, have an inbuilt option to shuffle the data indices before splitting them. Note that:

- This consumes less memory than shuffling the data directly.
- By default no shuffling occurs, including for the (stratified) K fold cross-validation performed by specifying `cv=some_integer` to `cross_val_score`, grid search, etc. Keep in mind that `train_test_split` still returns a random split.
- The `random_state` parameter defaults to `None`, meaning that the shuffling will be different every time `KFold(..., shuffle=True)` is iterated. However, `GridSearchCV` will use the same shuffling for each set of parameters validated by a single call to its `fit` method.
- To get identical results for each split, set `random_state` to an integer.

For more details on how to control the randomness of cv splitters and avoid common pitfalls, see randomness.

Cross validation and model selection

Cross validation iterators can also be used to directly perform model selection using Grid Search for the optimal hyperparameters of the model. This is the topic of the next section: grid_search.

Permutation test score

`permutation_test_score` offers another way to evaluate the performance of a predictor. It provides a permutation-based p-value, which represents how likely an observed performance of the estimator would be obtained by chance. The null hypothesis in this test is that the estimator fails to leverage any statistical dependency between the features and the targets to make correct predictions on left-out data. `permutation_test_score` generates a null distribution by calculating n_permutations different permutations of the data. In each permutation the target values are randomly shuffled, thereby removing any dependency between the features and the targets. The p-value output is the fraction of permutations whose cross-validation score is better or equal than the true score without permuting targets. For reliable results `n_permutations` should typically be larger than 100 and `cv` between 3-10 folds.

A low p-value provides evidence that the dataset contains some real dependency between features and targets **and** that the estimator was able to utilize this dependency to obtain good results. A high p-value, in reverse, could be due to either one of these:

- a lack of dependency between features and targets (i.e., there is no systematic relationship and any observed patterns are likely due to random chance)
- **or** because the estimator was not able to use the dependency in the data (for instance because it underfit).

In the latter case, using a more appropriate estimator that is able to use the structure in the data, would result in a lower p-value.

Cross-validation provides information about how well an estimator generalizes by estimating the range of its expected scores. However, an estimator trained on a high dimensional dataset with no structure may still perform better than expected on cross-validation, just by chance. This can typically happen with small datasets with less than a few hundred samples. `permutation_test_score` provides information on whether the estimator has found a real dependency between features and targets and can help in evaluating the performance of the estimator.

It is important to note that this test has been shown to produce low p-values even if there is only weak structure in the data because in the corresponding permutated datasets there is absolutely no structure. This test is therefore only able to show whether the model reliably outperforms random guessing.

Finally, `permutation_test_score` is computed using brute force and internally fits `(n_permutations + 1) * n_cv` models. It is therefore only tractable with small datasets for which fitting an individual model is very fast. Using the n_jobs parameter parallelizes the computation and thus speeds it up.

Examples

- sphx_glr_auto_examples_model_selection_plot_permutation_tests_for_classification.py

References

- Ojala and Garriga. Permutation Tests for Studying Classifier Performance. J. Mach. Learn. Res. 2010.

Tuning hyper-parameters

Source: *https://scikit-learn.org/stable/modules/grid_search.html*

Hyper-parameters are parameters that are not directly learnt within estimators. In scikit-learn they are passed as arguments to the constructor of the estimator classes. Typical examples include `C`, `kernel` and `gamma` for Support Vector Classifier, `alpha` for Lasso, etc.

It is possible and recommended to search the hyper-parameter space for the best cross validation score.

Any parameter provided when constructing an estimator may be optimized in this manner. Specifically, to find the names and current values for all parameters for a given estimator, use:

```
estimator.get_params()
```

A search consists of:

- an estimator (regressor or classifier such as `sklearn.svm.SVC()`);
- a parameter space;
- a method for searching or sampling candidates;
- a cross-validation scheme; and
- a score function.

Two generic approaches to parameter search are provided in scikit-learn: for given values, `GridSearchCV` exhaustively considers all parameter combinations, while `RandomizedSearchCV` can sample a given number of candidates from a parameter space with a specified distribution. Both these tools have successive halving counterparts `HalvingGridSearchCV` and `HalvingRandomSearchCV`, which can be much faster at finding a good parameter combination.

After describing these tools we detail `best practices <grid_search_tips>` applicable to these approaches. Some models allow for specialized, efficient parameter search strategies, outlined in alternative_cv.

Note that it is common that a small subset of those parameters can have a large impact on the predictive or computation performance of the model while others can be left to their default values. It is recommended to read the docstring of the estimator class to get a finer understanding of their expected behavior, possibly by reading the enclosed reference to the literature.

Exhaustive Grid Search

The grid search provided by `GridSearchCV` exhaustively generates candidates from a grid of parameter values specified with the `param_grid` parameter. For instance, the following `param_grid`:

```
param_grid = [
  {'C': [1, 10, 100, 1000], 'kernel': ['linear']},
  {'C': [1, 10, 100, 1000], 'gamma': [0.001, 0.0001], 'kernel': ['rbf']},
 ]
```

specifies that two grids should be explored: one with a linear kernel and C values in [1, 10, 100, 1000], and the second one with an RBF kernel, and the cross-product of C values ranging in [1, 10, 100, 1000] and gamma values in [0.001, 0.0001].

The `GridSearchCV` instance implements the usual estimator API: when ”fitting” it on a dataset all the possible combinations of parameter values are evaluated and the best combination is retained.

Examples

- See sphx_glr_auto_examples_model_selection_plot_nested_cross_validation_iris.py for an example of Grid Search within a cross validation loop on the iris dataset. This is the best practice for evaluating the performance of a model with grid search.
- See sphx_glr_auto_examples_model_selection_plot_grid_search_text_feature_extraction.py for an example of Grid Search coupling parameters from a text documents feature extractor (n-gram count vectorizer and TF-IDF transformer) with a classifier (here a linear SVM trained with SGD with either elastic net or L2 penalty) using a `Pipeline` instance.

Advanced examples
- See sphx_glr_auto_examples_model_selection_plot_nested_cross_validation_iris.py for an example of Grid Search within a cross validation loop on the iris dataset. This is the best practice for evaluating the performance of a model with grid search.
- See sphx_glr_auto_examples_model_selection_plot_multi_metric_evaluation.py for an example of `GridSearchCV` being used to evaluate multiple metrics simultaneously.
- See sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py for an example of using `refit=callable` interface in amount of flexibility in identifying the "best" estimator. This interface can also be used in multiple metrics evaluation.
- See sphx_glr_auto_examples_model_selection_plot_grid_search_stats.py for an example of how to do a statistical comparison on the outputs of

Randomized Parameter Optimization

While using a grid of parameter settings is currently the most widely used method for parameter optimization, other search methods have more favorable properties. `RandomizedSearchCV` implements a randomized search over parameters, where each setting is sampled from a distribution over possible parameter values. This has two main benefits over an exhaustive search:

- A budget can be chosen independent of the number of parameters and possible values.
- Adding parameters that do not influence the performance does not decrease efficiency.

Specifying how parameters should be sampled is done using a dictionary, very similar to specifying parameters for `GridSearchCV`. Additionally, a computation budget, being the number of sampled candidates or sampling iterations, is specified using the `n_iter` parameter. For each parameter, either a distribution over possible values or a list of discrete choices (which will be sampled uniformly) can be specified:

```
{'C': scipy.stats.expon(scale=100), 'gamma': scipy.stats.expon(scale=.1),
  'kernel': ['rbf'], 'class_weight':['balanced', None]}
```

This example uses the `scipy.stats` module, which contains many useful distributions for sampling parameters, such as `expon`, `gamma`, `uniform`, `loguniform` or `randint`.

In principle, any function can be passed that provides a `rvs` (random variate sample) method to sample a value. A call to the `rvs` function should provide independent random samples from possible parameter values on consecutive calls.

Warning
The distributions in `scipy.stats` prior to version scipy 0.16 do not allow specifying a random state. Instead, they use the global numpy random state, that can be seeded via `np.random.seed` or set using `np.random.set_state`. However, beginning scikit-learn 0.18, the `sklearn.model_selection` module sets the random state provided by the user if scipy >= 0.16 is also available.

For continuous parameters, such as `C` above, it is important to specify a continuous distribution to take full advantage of the randomization. This way, increasing `n_iter` will always lead to a

finer search.

A continuous log-uniform random variable is the continuous version of a log-spaced parameter. For example to specify the equivalent of `C` from above, `loguniform(1, 100)` can be used instead of `[1, 10, 100]`.

Mirroring the example above in grid search, we can specify a continuous random variable that is log-uniformly distributed between `1e0` and `1e3`:

```
from sklearn.utils.fixes import loguniform
{'C': loguniform(1e0, 1e3),
 'gamma': loguniform(1e-4, 1e-3),
 'kernel': ['rbf'],
 'class_weight':['balanced', None]}
```

Examples

- sphx_glr_auto_examples_model_selection_plot_randomized_search.py compares the usage and efficiency of randomized search and grid search.

References

- Bergstra, J. and Bengio, Y., Random search for hyper-parameter optimization, The Journal of Machine Learning Research (2012)

Searching for optimal parameters with successive halving

Scikit-learn also provides the `HalvingGridSearchCV` and `HalvingRandomSearchCV` estimators that can be used to search a parameter space using successive halving. Successive halving (SH) is like a tournament among candidate parameter combinations. SH is an iterative selection process where all candidates (the parameter combinations) are evaluated with a small amount of resources at the first iteration. Only some of these candidates are selected for the next iteration, which will be allocated more resources. For parameter tuning, the resource is typically the number of training samples, but it can also be an arbitrary numeric parameter such as n_estimators in a random forest.

> Note
> The resource increase chosen should be large enough so that a large improvement in scores is obtained when taking into account statistical significance.

As illustrated in the figure below, only a subset of candidates 'survive' until the last iteration. These are the candidates that have consistently ranked among the top-scoring candidates across all iterations. Each iteration is allocated an increasing amount of resources per candidate, here the number of samples.

We here briefly describe the main parameters, but each parameter and their interactions are described more in detail in the dropdown section below. The `factor` (> 1) parameter controls the rate at which the resources grow, and the rate at which the number of candidates decreases. In each iteration, the number of resources per candidate is multiplied by `factor` and the number of candidates is divided by the same factor. Along with `resource` and `min_resources`, `factor` is the most important parameter to control the search in our implementation, though a value of 3 usually works well. `factor` effectively controls the number of iterations in `HalvingGridSearchCV` and the number of candidates (by default) and iterations in `HalvingRandomSearchCV`. `aggressive_elimination=True` can also be used if the number of available resources is small. More control is available through tuning the `min_resources` parameter.

These estimators are still **experimental**: their predictions and their API might change without any deprecation cycle. To use them, you need to explicitly import `enable_halving_search_cv`:

```
>>> from sklearn.experimental import enable_halving_search_cv  # noqa
>>> from sklearn.model_selection import HalvingGridSearchCV
>>> from sklearn.model_selection import HalvingRandomSearchCV
```

Examples

- sphx_glr_auto_examples_model_selection_plot_successive_halving_heatmap.py
- sphx_glr_auto_examples_model_selection_plot_successive_halving_iterations.py

The sections below dive into technical aspects of successive halving.

Choosing `min_resources` and the number of candidates
Beside `factor`, the two main parameters that influence the behaviour of a successive halving search are the `min_resources` parameter, and the number of candidates (or parameter combinations) that are evaluated. `min_resources` is the amount of resources allocated at the first iteration for each candidate. The number of candidates is specified directly in `HalvingRandomSearchCV`, and is determined from the `param_grid` parameter of `HalvingGridSearchCV`.
Consider a case where the resource is the number of samples, and where we have 1000 samples. In theory, with `min_resources=10` and `factor=2`, we are able to run **at most** 7 iterations with the following number of samples: `[10, 20, 40, 80, 160, 320, 640]`.
But depending on the number of candidates, we might run less than 7 iterations: if we start with a **small** number of candidates, the last iteration might use less than 640 samples, which means not using all the available resources (samples). For example if we start with 5 candidates, we only need 2 iterations: 5 candidates for the first iteration, then 5 // 2 = 2 candidates at the second iteration, after which we know which candidate performs the best (so we don't need a third one). We would only be using at most 20 samples which is a waste since we have 1000 samples at our disposal. On the other hand, if we start with a **high** number of candidates, we might end up with a lot of candidates at the last iteration, which may not always be ideal: it means that many candidates will run with the full resources, basically reducing the procedure to standard search.
In the case of `HalvingRandomSearchCV`, the number of candidates is set by default such that the last iteration uses as much of the available resources as possible. For `HalvingGridSearchCV`, the number of candidates is determined by the param_grid parameter. Changing the value of `min_resources` will impact the number of possible iterations, and as a result will also have an effect on the ideal number of candidates.
Another consideration when choosing `min_resources` is whether or not it is easy to discriminate between good and bad candidates with a small amount of resources. For example, if you need a lot of samples to distinguish between good and bad parameters, a high `min_resources` is recommended. On the other hand if the distinction is clear even with a small amount of samples, then a small `min_resources` may be preferable since it would speed up the computation.
Notice in the example above that the last iteration does not use the maximum amount of resources available: 1000 samples are available, yet only 640 are used, at most. By default, both `HalvingRandomSearchCV` and last iteration, with the constraint that this amount of resources must be a multiple of both min_resources and factor (this constraint will be clear in the next section). `HalvingRandomSearchCV` achieves this by sampling the right amount of candidates, while `HalvingGridSearchCV` achieves this by properly setting min_resources.

Amount of resource and number of candidates at each iteration

At any iteration i, each candidate is allocated a given amount of resources which we denote n_resources_i. This quantity is controlled by the parameters `factor` and `min_resources` as follows (factor is strictly greater than 1):

```
n_resources_i = factor**i * min_resources,
```

or equivalently:

```
n_resources_{i+1} = n_resources_i * factor
```

where `min_resources == n_resources_0` is the amount of resources used at the first iteration. `factor` also defines the proportions of candidates that will be selected for the next iteration:

```
n_candidates_i = n_candidates // (factor ** i)
```

or equivalently:

```
n_candidates_0 = n_candidates
n_candidates_{i+1} = n_candidates_i // factor
```

So in the first iteration, we use `min_resources` resources `n_candidates` times. In the second iteration, we use `min_resources * factor` resources `n_candidates // factor` times. The third again multiplies the resources per candidate and divides the number of candidates. This process stops when the maximum amount of resource per candidate is reached, or when we have identified the best candidate. The best candidate is identified at the iteration that is evaluating factor or less candidates (see just below for an explanation).

Here is an example with `min_resources=3` and `factor=2`, starting with 70 candidates:

n_resources_i	n_candidates_i
3 (=min_resources)	70 (=n_candidates)
3 * 2 = 6	70 // 2 = 35
6 * 2 = 12	35 // 2 = 17
12 * 2 = 24	17 // 2 = 8
24 * 2 = 48	8 // 2 = 4
48 * 2 = 96	4 // 2 = 2

We can note that:

- the process stops at the first iteration which evaluates factor=2 candidates: the best candidate is the best out of these 2 candidates. It is not necessary to run an additional iteration, since it would only evaluate one candidate (namely the best one, which we have already identified). For this reason, in general, we want the last iteration to run at most `factor` candidates. If the last iteration evaluates more than factor candidates, then this last iteration reduces to a regular search (as in `RandomizedSearchCV` or `GridSearchCV`).
- each `n_resources_i` is a multiple of both `factor` and `min_resources` (which is confirmed by its definition above).

The amount of resources that is used at each iteration can be found in the n_resources_ attribute.

Choosing a resource
By default, the resource is defined in terms of number of samples. That is, each iteration will use an increasing amount of samples to train on. You can however manually specify a parameter to use as the resource with the `resource` parameter. Here is an example where the resource is defined in terms of the number of estimators of a random forest:

```
>>> from sklearn.datasets import make_classification
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.experimental import enable_halving_search_cv  # noqa
>>> from sklearn.model_selection import HalvingGridSearchCV
>>> import pandas as pd
>>> param_grid = {'max_depth': [3, 5, 10],
...               'min_samples_split': [2, 5, 10]}
>>> base_estimator = RandomForestClassifier(random_state=0)
>>> X, y = make_classification(n_samples=1000, random_state=0)
>>> sh = HalvingGridSearchCV(base_estimator, param_grid, cv=5,
```

```
...                          factor=2, resource='n_estimators',
...                          max_resources=30).fit(X, y)
>>> sh.best_estimator_
RandomForestClassifier(max_depth=5, n_estimators=24, random_state=0)
```

Note that it is not possible to budget on a parameter that is part of the parameter grid.

Exhausting the available resources

As mentioned above, the number of resources that is used at each iteration depends on the min_resources parameter. If you have a lot of resources available but start with a low number of resources, some of them might be wasted (i.e. not used):

```
>>> from sklearn.datasets import make_classification
>>> from sklearn.svm import SVC
>>> from sklearn.experimental import enable_halving_search_cv  # noqa
>>> from sklearn.model_selection import HalvingGridSearchCV
>>> import pandas as pd
>>> param_grid= {'kernel': ('linear', 'rbf'),
...              'C': [1, 10, 100]}
>>> base_estimator = SVC(gamma='scale')
>>> X, y = make_classification(n_samples=1000)
>>> sh = HalvingGridSearchCV(base_estimator, param_grid, cv=5,
...                          factor=2, min_resources=20).fit(X, y)
>>> sh.n_resources_
[20, 40, 80]
```

The search process will only use 80 resources at most, while our maximum amount of available resources is `n_samples=1000`. Here, we have `min_resources = r_0 = 20`.

For `HalvingGridSearchCV`, by default, the min_resources parameter is set to 'exhaust'. This means that min_resources is automatically set such that the last iteration can use as many resources as possible, within the max_resources limit:

```
>>> sh = HalvingGridSearchCV(base_estimator, param_grid, cv=5,
...                          factor=2, min_resources='exhaust').fit(X, y)
>>> sh.n_resources_
[250, 500, 1000]
```

min_resources was here automatically set to 250, which results in the last iteration using all the resources. The exact value that is used depends on the number of candidate parameters, on max_resources and on factor.

For `HalvingRandomSearchCV`, exhausting the resources can be done in 2 ways:

- by setting min_resources='exhaust', just like for
- by setting n_candidates='exhaust'.

Both options are mutually exclusive: using min_resources='exhaust' requires knowing the number of candidates, and symmetrically n_candidates='exhaust' requires knowing min_resources.

In general, exhausting the total number of resources leads to a better final candidate parameter, and is slightly more time-intensive.

Aggressive elimination of candidates

Using the `aggressive_elimination` parameter, you can force the search process to end up with less than `factor` candidates at the last iteration.

Code example of aggressive elimination

Ideally, we want the last iteration to evaluate `factor` candidates. We then just have to pick the best one. When the number of available resources is small with respect to the number of candidates, the last iteration may have to evaluate more than `factor` candidates:

```
>>> from sklearn.datasets import make_classification
>>> from sklearn.svm import SVC
>>> from sklearn.experimental import enable_halving_search_cv  # noqa
>>> from sklearn.model_selection import HalvingGridSearchCV
>>> import pandas as pd
>>> param_grid = {'kernel': ('linear', 'rbf'),
...               'C': [1, 10, 100]}
>>> base_estimator = SVC(gamma='scale')
>>> X, y = make_classification(n_samples=1000)
>>> sh = HalvingGridSearchCV(base_estimator, param_grid, cv=5,
...                          factor=2, max_resources=40,
...                          aggressive_elimination=False).fit(X, y)
>>> sh.n_resources_
[20, 40]
>>> sh.n_candidates_
[6, 3]
```

Since we cannot use more than `max_resources=40` resources, the process has to stop at the second iteration which evaluates more than `factor=2` candidates.

When using `aggressive_elimination`, the process will eliminate as many candidates as necessary using `min_resources` resources:

```
>>> sh = HalvingGridSearchCV(base_estimator, param_grid, cv=5,
...                          factor=2,
...                          max_resources=40,
...                          aggressive_elimination=True,
...                          ).fit(X, y)
>>> sh.n_resources_
[20, 20, 40]
>>> sh.n_candidates_
[6, 3, 2]
```

Notice that we end with 2 candidates at the last iteration since we have eliminated enough candidates during the first iterations, using `n_resources = min_resources = 20`.

Analyzing results with the cv_results_ attribute

The `cv_results_` attribute contains useful information for analyzing the results of a search. It can be converted to a pandas dataframe with `df = pd.DataFrame(est.cv_results_)`. The `cv_results_` attribute of `HalvingGridSearchCV` and `HalvingRandomSearchCV` is similar to that of `GridSearchCV` and `RandomizedSearchCV`, with additional information related to the successive halving process.

Example of a (truncated) output dataframe:

0	0	125	0.983667	{'criterion': 'log_loss', 'max_depth': None, 'max_features': 9, 'min_samples_split': 5}
1	0	125	0.983667	{'criterion': 'gini', 'max_depth': None, 'max_features': 8, 'min_samples_split': 7}
2	0	125	0.983667	{'criterion': 'gini', 'max_depth': None, 'max_features': 10, 'min_samples_split': 10}
3	0	125	0.983667	{'criterion': 'log_loss', 'max_depth': None, 'max_features': 6, 'min_samples_split': 6}
...	...	...	...	...
15	2	500	0.951958	{'criterion': 'log_loss', 'max_depth': None, 'max_features': 9, 'min_samples_split': 10}

16	2	500	0.947958	{'criterion': 'gini', 'max_depth': None, 'max_features': 10, 'min_samples_split': 10}
17	2	500	0.951958	{'criterion': 'gini', 'max_depth': None, 'max_features': 10, 'min_samples_split': 4}
18	3	1000	0.961009	{'criterion': 'log_loss', 'max_depth': None, 'max_features': 9, 'min_samples_split': 10}
19	3	1000	0.955989	{'criterion': 'gini', 'max_depth': None, 'max_features': 10, 'min_samples_split': 4}

Each row corresponds to a given parameter combination (a candidate) and a given iteration. The iteration is given by the `iter` column. The `n_resources` column tells you how many resources were used.

In the example above, the best parameter combination is `{'criterion': 'log_loss', 'max_depth': None, 'max_features': 9, 'min_samples_split': 10}` since it has reached the last iteration (3) with the highest score: 0.96.

References

Tips for parameter search

Specifying an objective metric

By default, parameter search uses the `score` function of the estimator to evaluate a parameter setting. These are the `sklearn.metrics.accuracy_score` for classification and `sklearn.metrics.r2_score` for regression. For some applications, other scoring functions are better suited (for example in unbalanced classification, the accuracy score is often uninformative), see which_scoring_function for some guidance. An alternative scoring function can be specified via the `scoring` parameter of most parameter search tools, see scoring_parameter for more details.

Specifying multiple metrics for evaluation

`GridSearchCV` and `RandomizedSearchCV` allow specifying multiple metrics for the `scoring` parameter.

Multimetric scoring can either be specified as a list of strings of predefined scores names or a dict mapping the scorer name to the scorer function and/or the predefined scorer name(s). See multimetric_scoring for more details.

When specifying multiple metrics, the `refit` parameter must be set to the metric (string) for which the `best_params_` will be found and used to build the `best_estimator_` on the whole dataset. If the search should not be refit, set `refit=False`. Leaving refit to the default value `None` will result in an error when using multiple metrics.

See sphx_glr_auto_examples_model_selection_plot_multi_metric_evaluation.py for an example usage.

`HalvingRandomSearchCV` and `HalvingGridSearchCV` do not support multimetric scoring.

Composite estimators and parameter spaces

`GridSearchCV` and `RandomizedSearchCV` allow searching over parameters of composite or nested estimators such as `Pipeline`, `ColumnTransformer`, `VotingClassifier` or `CalibratedClassifierCV` using a dedicated `<estimator>__<parameter>` syntax:

```
>>> from sklearn.model_selection import GridSearchCV
>>> from sklearn.calibration import CalibratedClassifierCV
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.datasets import make_moons
>>> X, y = make_moons()
>>> calibrated_forest = CalibratedClassifierCV(
...    estimator=RandomForestClassifier(n_estimators=10))
>>> param_grid = {
...    'estimator__max_depth': [2, 4, 6, 8]}
>>> search = GridSearchCV(calibrated_forest, param_grid, cv=5)
>>> search.fit(X, y)
GridSearchCV(cv=5,
          estimator=CalibratedClassifierCV(estimator=RandomForestClassifier(n_estimators=10)),
            param_grid={'estimator__max_depth': [2, 4, 6, 8]})
```

Here, `<estimator>` is the parameter name of the nested estimator, in this case `estimator`. If the meta-estimator is constructed as a collection of estimators as in pipeline.Pipeline, then `<estimator>` refers to the name of the estimator, see pipeline_nested_parameters. In practice, there can be several levels of nesting:

```
>>> from sklearn.pipeline import Pipeline
>>> from sklearn.feature_selection import SelectKBest
>>> pipe = Pipeline([
...    ('select', SelectKBest()),
...    ('model', calibrated_forest)])
>>> param_grid = {
...    'select__k': [1, 2],
...    'model__estimator__max_depth': [2, 4, 6, 8]}
>>> search = GridSearchCV(pipe, param_grid, cv=5).fit(X, y)
```

Please refer to pipeline for performing parameter searches over pipelines.

Model selection: development and evaluation

Model selection by evaluating various parameter settings can be seen as a way to use the labeled data to ”train” the parameters of the grid.

When evaluating the resulting model it is important to do it on held-out samples that were not seen during the grid search process: it is recommended to split the data into a **development set** (to be fed to the `GridSearchCV` instance) and an **evaluation set** to compute performance metrics.

This can be done by using the `train_test_split` utility function.

Parallelism

The parameter search tools evaluate each parameter combination on each data fold independently. Computations can be run in parallel by using the keyword `n_jobs=-1`. See function signature for more details, and also the Glossary entry for n_jobs.

Robustness to failure

Some parameter settings may result in a failure to `fit` one or more folds of the data. By default, the score for those settings will be np.nan. This can be controlled by setting error_score=”raise” to raise an exception if one fit fails, or for example error_score=0 to set another value for the score of failing parameter combinations.

Alternatives to brute force parameter search

Model specific cross-validation

Some models can fit data for a range of values of some parameter almost as efficiently as fitting the estimator for a single value of the parameter. This feature can be leveraged to perform a more efficient cross-validation used for model selection of this parameter.

The most common parameter amenable to this strategy is the parameter encoding the strength of the regularizer. In this case we say that we compute the **regularization path** of the estimator.

Here is the list of such models:

Information Criterion

Some models can offer an information-theoretic closed-form formula of the optimal estimate of the regularization parameter by computing a single regularization path (instead of several when using cross-validation).

Here is the list of models benefiting from the Akaike Information Criterion (AIC) or the Bayesian Information Criterion (BIC) for automated model selection:

Out of Bag Estimates

When using ensemble methods based upon bagging, i.e. generating new training sets using sampling with replacement, part of the training set remains unused. For each classifier in the ensemble, a different part of the training set is left out.

This left out portion can be used to estimate the generalization error without having to rely on a separate validation set. This estimate comes ”for free” as no additional data is needed and can be used for model selection.

This is currently implemented in the following classes:

Metrics and scoring

Source: *https://scikit-learn.org/stable/modules/model_evaluation.html*

Which scoring function should I use?

Before we take a closer look into the details of the many scores and evaluation metrics, we want to give some guidance, inspired by statistical decision theory, on the choice of **scoring functions** for **supervised learning**, see [Gneiting2009]:

- *Which scoring function should I use?*
- *Which scoring function is a good one for my task?*

In a nutshell, if the scoring function is given, e.g. in a kaggle competition or in a business context, use that one. If you are free to choose, it starts by considering the ultimate goal and application of the prediction. It is useful to distinguish two steps:

- Predicting
- Decision making

Predicting: Usually, the response variable Y is a random variable, in the sense that there is *no deterministic* function $Y = g(X)$ of the features X. Instead, there is a probability distribution F of Y. One can aim to predict the whole distribution, known as *probabilistic prediction*, or---more the focus of scikit-learn---issue a *point prediction* (or point forecast) by choosing a property or functional of that distribution F. Typical examples are the mean (expected value), the median or a quantile of the response variable Y (conditionally on X).

Once that is settled, use a **strictly consistent** scoring function for that (target) functional, see [Gneiting2009]. This means using a scoring function that is aligned with *measuring the distance between predictions* y_pred *and the true target functional using observations of* Y, i.e. y_true. For classification **strictly proper scoring rules**, see Wikipedia entry for Scoring rule and [Gneiting2007], coincide with strictly consistent scoring functions. The table further below provides examples. One could say that consistent scoring functions act as *truth serum* in that they guarantee *"that truth telling [. . .] is an optimal strategy in expectation"* [Gneiting2014].

Once a strictly consistent scoring function is chosen, it is best used for both: as loss function for model training and as metric/score in model evaluation and model comparison.

Note that for regressors, the prediction is done with predict while for classifiers it is usually predict_proba.

Decision Making: The most common decisions are done on binary classification tasks, where the result of predict_proba is turned into a single outcome, e.g., from the predicted probability of rain a decision is made on how to act (whether to take mitigating measures like an umbrella or not). For classifiers, this is what predict returns. See also threshold_tuning. There are many scoring functions which measure different aspects of such a decision, most of them are covered with or derived from the `metrics.confusion_matrix`.

List of strictly consistent scoring functions: Here, we list some of the most relevant statistical functionals and corresponding strictly consistent scoring functions for tasks in practice. Note that the list is not complete and that there are more of them. For further criteria on how to select a specific one, see [Fissler2022].

functional	scoring or loss function	response y	prediction
Classification mean mean mode	Brier score 1 multi-class predi log loss multi-class zero-one loss2multi-classpr	ct_proba‘predict_probaedict‘, categorical	
Regression mean mean mean	squared error 3 all reals predict Poisson deviance non-negativepredict Gamma deviance strictly positive “predict	, all reals, strictly positive “, strictly positive	
mean median quantile mode	Tweedie deviance depends on power predict absolute error all realspredi pinball loss all reals no consistent one exists	, depends onpower ct, all realspredict“, all reals reals	“

1 The Brier score is just a different name for the squared error in case of classification with one-hot encoded targets.

2 The zero-one loss is only consistent but not strictly consistent for the mode. The zero-one loss is equivalent to one minus the accuracy score, meaning it gives different score values but the same ranking.

3 R^2 gives the same ranking as squared error.

Fictitious Example: Let's make the above arguments more tangible. Consider a setting in network reliability engineering, such as maintaining stable internet or Wi-Fi connections. As provider of the network, you have access to the dataset of log entries of network connections containing network load over time and many interesting features. Your goal is to improve the reliability of the connections. In fact, you promise your customers that on at least 99% of all days there are no connection discontinuities larger than 1 minute. Therefore, you are interested in a prediction of the 99% quantile (of longest connection interruption duration per day) in order to know in advance when to add more bandwidth and thereby satisfy your customers. So the *target functional* is the 99% quantile. From the table above, you choose the pinball loss as scoring function (fair enough, not much choice given), for model training (e.g. HistGradientBoostingRegressor(loss=”quantile”, quantile=0.99)) as well as model evaluation (mean_pinball_loss(..., alpha=0.99) - we apologize for the different argument names, quantile and alpha) be it in grid search for finding hyperparameters or in comparing to other models like QuantileRegressor(quantile=0.99).

References

Scoring API overview

There are 3 different APIs for evaluating the quality of a model's predictions:

- **Estimator score method**: Estimators have a `score` method providing a default evaluation criterion for the problem they are designed to solve. Most commonly this is accuracy for classifiers and the coefficient of determination (R^2) for regressors. Details for each estimator can be found in its documentation.
- **Scoring parameter**: Model-evaluation tools that use cross-validation (such as `model_selection.GridSearchCV`, `model_selection.validation_curve` and `linear_model.LogisticRegressionCV`) rely on an internal *scoring* strategy. This can be specified using the scoring parameter of that tool and is discussed in the section scoring_parameter.
- **Metric functions**: The `sklearn.metrics` module implements functions assessing prediction error for specific purposes. These metrics are detailed in sections on classification_metrics, multilabel_ranking_metrics, regression_metrics and clustering_metrics.

Finally, dummy_estimators are useful to get a baseline value of those metrics for random predictions.

See also
For ”pairwise” metrics, between *samples* and not estimators or predictions, see the metrics section.

The `scoring` parameter: defining model evaluation rules

Model selection and evaluation tools that internally use cross-validation (such as `model_selection.GridSearchCV`, `model_selection.validation_curve` and `linear_model.LogisticRegressionCV`) take a `scoring` parameter that controls what metric they apply to the estimators evaluated.

They can be specified in several ways:

- ‘None‘: the estimator's default evaluation criterion (i.e., the metric used in the estimator's score method) is used.
- String name: common metrics can be passed via a string name.
- Callable: more complex metrics can be passed via a custom metric callable (e.g., function).

Some tools do also accept multiple metric evaluation. See multimetric_scoring for details.

String name scorers

For the most common use cases, you can designate a scorer object with the `scoring` parameter via a string name; the table below shows all possible values. All scorer objects follow the convention that **higher return values are better than lower return values**. Thus metrics which measure the distance between the model and the data, like `metrics.mean_squared_error`, are available as 'neg_mean_squared_error' which return the negated value of the metric.

Scoring string name	Function	Comment
Classification 'accuracy' 'balanced_accuracy' 'top_k_accuracy' 'average_precision'	`metrics.accuracy_score` `metrics.balanced_accuracy_score` `metrics.top_k_accuracy_score` `metrics.average_precision_score`	
'neg_brier_score'	`metrics.brier_score_loss` requ	ires `predict_proba` sup- port
'f1'	`metrics.f1_score` for	binary tar- gets
'f1_micro'	`metrics.f1_score` micr	o- averaged
'f1_macro'	`metrics.f1_score` macr	o- averaged
'f1_weighted'	`metrics.f1_score` weig	hted av- er- age
'f1_samples'	`metrics.f1_score` by m	ultilabel sam- ple

Scoring string name	Function	Comment
'neg_log_loss'	`metrics.log_loss` requ	ires `predict_proba` sup- port
'precision' etc.	`metrics.precision_score` suff	ixes ap- ply as with 'f1'
'recall' etc.	`metrics.recall_score` suff	ixes ap- ply as with 'f1'
'jaccard' etc. 'roc_auc' 'roc_auc_ovr'	`metrics.jaccard_score` suff	ixes
'roc_auc_ovo' 'roc_auc_ovr_weighted'	`metrics.roc_auc_score` `metrics.roc_auc_score`	ap-
'roc_auc_ovo_weighted'	`metrics.roc_auc_score` `metrics.roc_auc_score` `metrics.roc_auc_score`	ply as with 'f1'
'd2_log_loss_score'	`metrics.d2_log_loss_score` requ	ires `predict_proba` sup- port
'd2_brier_score'	`metrics.d2_brier_score` requ	ires
Clustering 'adjusted_mutual_info_score'	`metrics.adjusted_mutual_info_score`	`predict_proba`
'adjusted_rand_score'	`metrics.adjusted_rand_score`	sup-
'completeness_score'	`metrics.completeness_score`	port
'fowlkes_mallows_score'	`metrics.fowlkes_mallows_score`	
'homogeneity_score' 'mutual_info_score'	`metrics.homogeneity_score`	
'normalized_mutual_info_score'	`metrics.mutual_info_score`	
'rand_score' 'v_measure_score'	`metrics.normalized_mutual_info_score`	
Regression 'explained_variance'	`metrics.rand_score` `metrics.v_measure_score`	
'neg_max_error'	`metrics.explained_variance_score`	
'neg_mean_absolute_error'	`metrics.max_error` `metrics.mean_absolute_error`	
'neg_mean_squared_error'	`metrics.mean_squared_error`	
'neg_root_mean_squared_error'	`metrics.root_mean_squared_error`	
'neg_mean_squared_log_error'	`metrics.mean_squared_log_error`	
'neg_root_mean_squared_log_error'	`metrics.root_mean_squared_log_error`	
'neg_median_absolute_error' 'r2'	`metrics.median_absolute_error`	
'neg_mean_poisson_deviance'	`metrics.r2_score`	
'neg_mean_gamma_deviance'	`metrics.mean_poisson_deviance`	
'neg_mean_absolute_percentage_error'	`metrics.mean_gamma_deviance`	
'd2_absolute_error_score'	`metrics.mean_absolute_percentage_error`	
	`metrics.d2_absolute_error_score`	

Usage examples:

```
>>> from sklearn import svm, datasets
>>> from sklearn.model_selection import cross_val_score
>>> X, y = datasets.load_iris(return_X_y=True)
```

```
>>> clf = svm.SVC(random_state=0)
>>> cross_val_score(clf, X, y, cv=5, scoring='recall_macro')
array([0.96, 0.96, 0.96, 0.93, 1.        ])
```

Note
If a wrong scoring name is passed, an `InvalidParameterError` is raised. You can retrieve the names of all available scorers by calling

Callable scorers

For more complex use cases and more flexibility, you can pass a callable to the scoring parameter. This can be done by:

- scoring_adapt_metric
- scoring_custom (most flexible)

Adapting predefined metrics via make_scorer

The following metric functions are not implemented as named scorers, sometimes because they require additional parameters, such as `fbeta_score`. They cannot be passed to the `scoring` parameters; instead their callable needs to be passed to `make_scorer` together with the value of the user-settable parameters.

Function	Parameter	Example usage
Classification		
`metrics.fbeta_score` “be	ta“ma	ke_scorer(fbeta_score, beta=2)“
Regression		
`metrics.mean_tweedie_deviance` “po	wer“ma	ke_scorer(mean_tweedie_deviance, power=1.5)“
`metrics.mean_pinball_loss` “al	pha“ma	ke_scorer(mean_pinball_loss, alpha=0.95)“
`metrics.d2_tweedie_score` “po	wer“ma	ke_scorer(d2_tweedie_score, power=1.5)“
`metrics.d2_pinball_score` “al	pha“ma	ke_scorer(d2_pinball_score, alpha=0.95)“

One typical use case is to wrap an existing metric function from the library with non-default values for its parameters, such as the `beta` parameter for the `fbeta_score` function:

```
>>> from sklearn.metrics import fbeta_score, make_scorer
>>> ftwo_scorer = make_scorer(fbeta_score, beta=2)
>>> from sklearn.model_selection import GridSearchCV
>>> from sklearn.svm import LinearSVC
>>> grid = GridSearchCV(LinearSVC(), param_grid={'C': [1, 10]},
...                     scoring=ftwo_scorer, cv=5)
```

The module `sklearn.metrics` also exposes a set of simple functions measuring a prediction error given ground truth and prediction:

- functions ending with `_score` return a value to maximize, the higher the better.
- functions ending with `_error`, `_loss`, or `_deviance` return a value to minimize, the lower the better. When converting into a scorer object using `make_scorer`, set the `greater_is_better` parameter to `False` (`True` by default; see the parameter description below).

Creating a custom scorer object

You can create your own custom scorer object using `make_scorer`.

Custom scorer objects using make_scorer

You can build a completely custom scorer object from a simple python function using `make_scorer`, which can take several parameters:

- the python function you want to use (`my_custom_loss_func` in the example below)
- whether the python function returns a score (`greater_is_better=True`, the default) or a loss (`greater_is_better=False`). If a loss, the output of the python function is negated by the scorer object, conforming to the cross validation convention that scorers return higher values for better models.
- for classification metrics only: whether the python function you provided requires continuous decision certainties. If the scoring function only accepts probability estimates (e.g. `metrics.log_loss`), then one needs to set the parameter response_method="predict_proba". Some scoring functions do not necessarily require probability estimates but rather non-thresholded decision values (e.g. `metrics.roc_auc_score`). In this case, one can provide a list (e.g., response_method=["decision_function", "predict_proba"]), and scorer will use the first available method, in the order given in the list, to compute the scores.
- any additional parameters of the scoring function, such as `beta` or `labels`.

Here is an example of building custom scorers, and of using the `greater_is_better` parameter:

```
>>> import numpy as np
>>> def my_custom_loss_func(y_true, y_pred):
...     diff = np.abs(y_true - y_pred).max()
...     return float(np.log1p(diff))
...
>>> # score will negate the return value of my_custom_loss_func,
>>> # which will be np.log(2), 0.693, given the values for X
>>> # and y defined below.
>>> score = make_scorer(my_custom_loss_func, greater_is_better=False)
>>> X = [[1], [1]]
>>> y = [0, 1]
>>> from sklearn.dummy import DummyClassifier
>>> clf = DummyClassifier(strategy='most_frequent', random_state=0)
>>> clf = clf.fit(X, y)
>>> my_custom_loss_func(y, clf.predict(X))
0.69
>>> score(clf, X, y)
-0.69
```

Using custom scorers in functions where n_jobs > 1

While defining the custom scoring function alongside the calling function should work out of the box with the default joblib backend (loky), importing it from another module will be a more robust approach and work independently of the joblib backend.

For example, to use `n_jobs` greater than 1 in the example below, `custom_scoring_function` function is saved in a user-created module (`custom_scorer_module.py`) and imported:

```
>>> from custom_scorer_module import custom_scoring_function
>>> cross_val_score(model,
...  X_train,
...  y_train,
...  scoring=make_scorer(custom_scoring_function, greater_is_better=False),
...  cv=5,
...  n_jobs=-1)
```

Using multiple metric evaluation

Scikit-learn also permits evaluation of multiple metrics in `GridSearchCV`, `RandomizedSearchCV` and `cross_validate`.

There are three ways to specify multiple scoring metrics for the `scoring` parameter:

- As an iterable of string metrics:

```
>>> scoring = ['accuracy', 'precision']
```

- As a `dict` mapping the scorer name to the scoring function:

```
>>> from sklearn.metrics import accuracy_score
>>> from sklearn.metrics import make_scorer
>>> scoring = {'accuracy': make_scorer(accuracy_score),
...            'prec': 'precision'}
```

  Note that the dict values can either be scorer functions or one of the predefined metric strings.

- As a callable that returns a dictionary of scores:

```
>>> from sklearn.model_selection import cross_validate
>>> from sklearn.metrics import confusion_matrix
>>> # A sample toy binary classification dataset
>>> X, y = datasets.make_classification(n_classes=2, random_state=0)
>>> svm = LinearSVC(random_state=0)
>>> def confusion_matrix_scorer(clf, X, y):
...      y_pred = clf.predict(X)
...      cm = confusion_matrix(y, y_pred)
...      return {'tn': cm[0, 0], 'fp': cm[0, 1],
...              'fn': cm[1, 0], 'tp': cm[1, 1]}
>>> cv_results = cross_validate(svm, X, y, cv=5,
...                             scoring=confusion_matrix_scorer)
>>> # Getting the test set true positive scores
>>> print(cv_results['test_tp'])
[10  9  8  7  8]
>>> # Getting the test set false negative scores
>>> print(cv_results['test_fn'])
[0 1 2 3 2]
```

Classification metrics

The `sklearn.metrics` module implements several loss, score, and utility functions to measure classification performance. Some metrics might require probability estimates of the positive class or non-thresholded decision values (as returned by decision_function on some classifiers). Most implementations allow each sample to provide a weighted contribution to the overall score, through the `sample_weight` parameter.

Some of these are restricted to the binary classification case:

Others also work in the multiclass case:

Some also work in the multilabel case:

And some work with binary and multilabel (but not multiclass) problems:

In the following sub-sections, we will describe each of those functions, preceded by some notes on common API and metric definition.

From binary to multiclass and multilabel

Some metrics are essentially defined for binary classification tasks (e.g. `f1_score`, `roc_auc_score`). In these cases, by default only the positive label is evaluated, assuming by default that the positive class is labelled `1` (though this may be configurable through the `pos_label` parameter).

In extending a binary metric to multiclass or multilabel problems, the data is treated as a collection of binary problems, one for each class. There are then a number of ways to average binary metric calculations across the set of classes, each of which may be useful in some scenario. Where available, you should select among these using the `average` parameter.

- `"macro"` simply calculates the mean of the binary metrics, giving equal weight to each class. In problems where infrequent classes are nonetheless important, macro-averaging may be a means of highlighting their performance. On the other hand, the assumption that all classes are equally important is often untrue, such that macro-averaging will over-emphasize the typically low performance on an infrequent class.
- `"weighted"` accounts for class imbalance by computing the average of binary metrics in which each class's score is weighted by its presence in the true data sample.
- `"micro"` gives each sample-class pair an equal contribution to the overall metric (except as a result of sample-weight). Rather than summing the metric per class, this sums the dividends and divisors that make up the per-class metrics to calculate an overall quotient. Micro-averaging may be preferred in multilabel settings, including multiclass classification where a majority class is to be ignored.
- `"samples"` applies only to multilabel problems. It does not calculate a per-class measure, instead calculating the metric over the true and predicted classes for each sample in the evaluation data, and returning their (`sample_weight`-weighted) average.
- Selecting `average=None` will return an array with the score for each class.

While multiclass data is provided to the metric, like binary targets, as an array of class labels, multilabel data is specified as an indicator matrix, in which cell `[i, j]` has value 1 if sample `i` has label `j` and value 0 otherwise.

Accuracy score

The `accuracy_score` function computes the accuracy, either the fraction (default) or the count (normalize=False) of correct predictions.

In multilabel classification, the function returns the subset accuracy. If the entire set of predicted labels for a sample strictly match with the true set of labels, then the subset accuracy is 1.0; otherwise it is 0.0.

If $\hat{y}_i$ is the predicted value of the i-th sample and y_i is the corresponding true value, then the fraction of correct predictions over n_{samples} is defined as

$$\texttt{accuracy}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} 1(\hat{y}_i = y_i)$$

where $1(x)$ is the indicator function.

```
>>> import numpy as np
>>> from sklearn.metrics import accuracy_score
>>> y_pred = [0, 2, 1, 3]
>>> y_true = [0, 1, 2, 3]
>>> accuracy_score(y_true, y_pred)
0.5
>>> accuracy_score(y_true, y_pred, normalize=False)
2.0
```

In the multilabel case with binary label indicators:

```
>>> accuracy_score(np.array([[0, 1], [1, 1]]), np.ones((2, 2)))
0.5
```

Examples

- See sphx_glr_auto_examples_model_selection_plot_permutation_tests_for_classification.py for an example of accuracy score usage using permutations of the dataset.

Top-k accuracy score

The `top_k_accuracy_score` function is a generalization of `accuracy_score`. The difference is that a prediction is considered correct as long as the true label is associated with one of the `k` highest predicted scores. `accuracy_score` is the special case of k = 1.

The function covers the binary and multiclass classification cases but not the multilabel case.

If $\hat{f}_{i,j}$ is the predicted class for the i-th sample corresponding to the j-th largest predicted score and y_i is the corresponding true value, then the fraction of correct predictions over n_{samples} is defined as

$$\texttt{top-k accuracy}(y, \hat{f}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \sum_{j=1}^{k} 1(\hat{f}_{i,j} = y_i)$$

where k is the number of guesses allowed and $1(x)$ is the indicator function.

```
>>> import numpy as np
>>> from sklearn.metrics import top_k_accuracy_score
>>> y_true = np.array([0, 1, 2, 2])
>>> y_score = np.array([[0.5, 0.2, 0.2],
...                     [0.3, 0.4, 0.2],
...                     [0.2, 0.4, 0.3],
...                     [0.7, 0.2, 0.1]])
>>> top_k_accuracy_score(y_true, y_score, k=2)
0.75
>>> # Not normalizing gives the number of "correctly" classified samples
>>> top_k_accuracy_score(y_true, y_score, k=2, normalize=False)
3.0
```

Balanced accuracy score

The `balanced_accuracy_score` function computes the balanced accuracy, which avoids inflated performance estimates on imbalanced datasets. It is the macro-average of recall scores per class or, equivalently, raw accuracy where each sample is weighted according to the inverse prevalence of its true class. Thus for balanced datasets, the score is equal to accuracy.

In the binary case, balanced accuracy is equal to the arithmetic mean of sensitivity (true positive rate) and specificity (true negative rate), or the area under the ROC curve with binary predictions rather than scores:

$$\texttt{balanced-accuracy} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right)$$

If the classifier performs equally well on either class, this term reduces to the conventional accuracy (i.e., the number of correct predictions divided by the total number of predictions).

In contrast, if the conventional accuracy is above chance only because the classifier takes advantage of an imbalanced test set, then the balanced accuracy, as appropriate, will drop to $\frac{1}{n_classes}$.

The score ranges from 0 to 1, or when `adjusted=True` is used, it is rescaled to the range $\frac{1}{1-n_classes}$ to 1, inclusive, with performance at random scoring 0.

If y_i is the true value of the i-th sample, and w_i is the corresponding sample weight, then we adjust the sample weight to:

$$\hat{w}_i = \frac{w_i}{\sum_j 1(y_j = y_i) w_j}$$

where $1(x)$ is the indicator function. Given predicted $\hat{y}_i$ for sample i, balanced accuracy is defined as:

$$\texttt{balanced-accuracy}(y, \hat{y}, w) = \frac{1}{\sum \hat{w}_i} \sum_i 1(\hat{y}_i = y_i) \hat{w}_i$$

With `adjusted=True`, balanced accuracy reports the relative increase from $\texttt{balanced-accuracy}(y, \mathbf{0}, w) = \frac{1}{n_classes}$. In the binary case, this is also known as Youden's J statistic, or *informedness*.

Note
The multiclass definition here seems the most reasonable extension of the metric used in binary classification, though there is no certain consensus in the literature:

- Our definition: [Mosley2013], [Kelleher2015] and [Guyon2015], where [Guyon2015] adopt the adjusted version to ensure that random predictions have a score of 0 and perfect predictions have a score of 1.
- Class balanced accuracy as described in [Mosley2013]: the minimum between the precision and the recall for each class is computed. Those values are then averaged over the total number of classes to get the balanced accuracy.
- Balanced Accuracy as described in [Urbanowicz2015]: the average of sensitivity and specificity is computed for each class and then averaged over total number of classes.

References

Cohen's kappa

The function `cohen_kappa_score` computes Cohen's kappa statistic. This measure is intended to compare labelings by different human annotators, not a classifier versus a ground truth.

The kappa score is a number between -1 and 1. Scores above .8 are generally considered good agreement; zero or lower means no agreement (practically random labels).

Kappa scores can be computed for binary or multiclass problems, but not for multilabel problems (except by manually computing a per-label score) and not for more than two annotators.

```
>>> from sklearn.metrics import cohen_kappa_score
>>> labeling1 = [2, 0, 2, 2, 0, 1]
>>> labeling2 = [0, 0, 2, 2, 0, 2]
>>> cohen_kappa_score(labeling1, labeling2)
0.4285714285714286
```

Confusion matrix

The `confusion_matrix` function evaluates classification accuracy by computing the confusion matrix with each row corresponding to the true class (Wikipedia and other references may use different convention for axes).

By definition, entry i, j in a confusion matrix is the number of observations actually in group i, but predicted to be in group j. Here is an example:

```
>>> from sklearn.metrics import confusion_matrix
>>> y_true = [2, 0, 2, 2, 0, 1]
>>> y_pred = [0, 0, 2, 2, 0, 2]
>>> confusion_matrix(y_true, y_pred)
array([[2, 0, 0],
       [0, 0, 1],
       [1, 0, 2]])
```

`ConfusionMatrixDisplay` can be used to visually represent a confusion matrix as shown in the sphx_glr_auto_examples_model_selection_plot_confusion_matrix.py example, which creates the following figure:

The parameter `normalize` allows to report ratios instead of counts. The confusion matrix can be normalized in 3 different ways: `'pred'`, `'true'`, and `'all'` which will divide the counts by the sum of each columns, rows, or the entire matrix, respectively.

```
>>> y_true = [0, 0, 0, 1, 1, 1, 1, 1]
>>> y_pred = [0, 1, 0, 1, 0, 1, 0, 1]
>>> confusion_matrix(y_true, y_pred, normalize='all')
array([[0.25 , 0.125],
       [0.25 , 0.375]])
```

For binary problems, we can get counts of true negatives, false positives, false negatives and true positives as follows:

```
>>> y_true = [0, 0, 0, 1, 1, 1, 1, 1]
>>> y_pred = [0, 1, 0, 1, 0, 1, 0, 1]
>>> tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel().tolist()
>>> tn, fp, fn, tp
(2, 1, 2, 3)
```

With `confusion_matrix_at_thresholds` we can get true negatives, false positives, false negatives and true positives for different thresholds:

```
>>> from sklearn.metrics import confusion_matrix_at_thresholds
>>> y_true = np.array([0., 0., 1., 1.])
>>> y_score = np.array([0.1, 0.4, 0.35, 0.8])
>>> tns, fps, fns, tps, thresholds = confusion_matrix_at_thresholds(y_true, y_score)
>>> tns
array([2., 1., 1., 0.])
>>> fps
array([0., 1., 1., 2.])
>>> fns
array([1., 1., 0., 0.])
>>> tps
array([1., 1., 2., 2.])
>>> thresholds
array([0.8, 0.4, 0.35, 0.1])
```

Note that the thresholds consist of distinct y_score values, in decreasing order.

Examples

- See sphx_glr_auto_examples_model_selection_plot_confusion_matrix.py for an example of using a confusion matrix to evaluate classifier output quality.
- See sphx_glr_auto_examples_classification_plot_digits_classification.py for an example of using a confusion matrix to classify hand-written digits.
- See sphx_glr_auto_examples_text_plot_document_classification_20newsgroups.py for an example of using a confusion matrix to classify text documents.

Classification report

The `classification_report` function builds a text report showing the main classification metrics. Here is a small example with custom `target_names` and inferred labels:

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 0]
>>> y_pred = [0, 0, 2, 1, 0]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
              precision    recall  f1-score   support
<BLANKLINE>
     class 0       0.67      1.00      0.80         2
     class 1       0.00      0.00      0.00         1
     class 2       1.00      0.50      0.67         2
<BLANKLINE>
    accuracy                           0.60         5
   macro avg       0.56      0.50      0.49         5
weighted avg       0.67      0.60      0.59         5
<BLANKLINE>
```

Examples

- See sphx_glr_auto_examples_classification_plot_digits_classification.py for an example of classification report usage for hand-written digits.
- See sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py for an example of classification report usage for grid search with nested cross-validation.

Hamming loss

The `hamming_loss` computes the average Hamming loss or Hamming distance between two sets of samples.

If $\hat{y}_{i,j}$ is the predicted value for the j-th label of a given sample i, $y_{i,j}$ is the corresponding true value, n_{samples} is the number of samples and n_{labels} is the number of labels, then the Hamming loss $L_{Hamming}$ is defined as:

$$L_{Hamming}(y, \hat{y}) = \frac{1}{n_{\text{samples}} * n_{\text{labels}}} \sum_{i=0}^{n_{\text{samples}}-1} \sum_{j=0}^{n_{\text{labels}}-1} 1(\hat{y}_{i,j} \neq y_{i,j})$$

where $1(x)$ is the indicator function.

The equation above does not hold true in the case of multiclass classification. Please refer to the note below for more information. :

```
>>> from sklearn.metrics import hamming_loss
>>> y_pred = [1, 2, 3, 4]
>>> y_true = [2, 2, 3, 4]
>>> hamming_loss(y_true, y_pred)
```

```
0.25
```

In the multilabel case with binary label indicators:

```
>>> hamming_loss(np.array([[0, 1], [1, 1]]), np.zeros((2, 2)))
0.75
```

> Note
> In multiclass classification, the Hamming loss corresponds to the Hamming distance between `y_true` and `y_pred` which is similar to the prediction sets that do not strictly match true sets, the Hamming loss penalizes individual labels. Thus the Hamming loss, upper bounded by the zero-one loss, is always between zero and one, inclusive; and predicting a proper subset or superset of the true labels will give a Hamming loss between zero and one, exclusive.

Precision, recall and F-measures

Intuitively, precision is the ability of the classifier not to label as positive a sample that is negative, and recall is the ability of the classifier to find all the positive samples.

The F-measure (F_β and F_1 measures) can be interpreted as a weighted harmonic mean of the precision and recall. A F_β measure reaches its best value at 1 and its worst score at 0. With $\beta = 1$, F_β and F_1 are equivalent, and the recall and the precision are equally important.

The `precision_recall_curve` computes a precision-recall curve from the ground truth label and a score given by the classifier by varying a decision threshold.

The `average_precision_score` function computes the average precision (AP) from prediction scores. The value is between 0 and 1 and higher is better. AP is defined as

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n$$

where P_n and R_n are the precision and recall at the nth threshold. With random predictions, the AP is the fraction of positive samples.

References [Manning2008] and [Everingham2010] present alternative variants of AP that interpolate the precision-recall curve. Currently, `average_precision_score` does not implement any interpolated variant. References [Davis2006] and [Flach2015] describe why a linear interpolation of points on the precision-recall curve provides an overly-optimistic measure of classifier performance. This linear interpolation is used when computing area under the curve with the trapezoidal rule in `auc`. [Chen2024] benchmarks different interpolation strategies to demonstrate the effects.

Several functions allow you to analyze the precision, recall and F-measures score:

Note that the `precision_recall_curve` function is restricted to the binary case. The `average_precision_score` function supports multiclass and multilabel formats by computing each class score in a One-vs-the-rest (OvR) fashion and averaging them or not depending of its `average` argument value.

The `PrecisionRecallDisplay.from_estimator` and `PrecisionRecallDisplay.from_predictions` functions will plot the precision-recall curve as follows.

Examples

- See sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py for an example of `precision_score` and `recall_score` usage to estimate parameters using grid search with nested cross-validation.

- See sphx_glr_auto_examples_model_selection_plot_precision_recall.py for an example of `precision_recall_curve` usage to evaluate classifier output quality.

References

Binary classification

In a binary classification task, the terms ''positive'' and ''negative'' refer to the classifier's prediction, and the terms ''true'' and ''false'' refer to whether that prediction corresponds to the external judgment (sometimes known as the ''observation''). Given these definitions, we can formulate the following table:

	Actual class (observation)	
Predicted class (expectation)	tp (true positive) Correct result fn (false negative) Missing result	fp (false positive) Unexpected result tn (true negative) Correct absence of result

In this context, we can define the notions of precision and recall:

$$\text{precision} = \frac{\text{tp}}{\text{tp} + \text{fp}},$$

$$\text{recall} = \frac{\text{tp}}{\text{tp} + \text{fn}},$$

(Sometimes recall is also called ''sensitivity'')

F-measure is the weighted harmonic mean of precision and recall, with precision's contribution to the mean weighted by some parameter β:

$$F_\beta = (1 + \beta^2) \frac{\text{precision} \times \text{recall}}{\beta^2 \text{precision} + \text{recall}}$$

To avoid division by zero when precision and recall are zero, Scikit-Learn calculates F-measure with this otherwise-equivalent formula:

$$F_\beta = \frac{(1 + \beta^2)\text{tp}}{(1 + \beta^2)\text{tp} + \text{fp} + \beta^2\text{fn}}$$

Note that this formula is still undefined when there are no true positives, false positives, or false negatives. By default, F-1 for a set of exclusively true negatives is calculated as 0, however this behavior can be changed using the zero_division parameter. Here are some small examples in binary classification:

```
>>> from sklearn import metrics
>>> y_pred = [0, 1, 0, 0]
>>> y_true = [0, 1, 0, 1]
>>> metrics.precision_score(y_true, y_pred)
1.0
>>> metrics.recall_score(y_true, y_pred)
0.5
>>> metrics.f1_score(y_true, y_pred)
0.66
```

```
>>> metrics.fbeta_score(y_true, y_pred, beta=0.5)
0.83
>>> metrics.fbeta_score(y_true, y_pred, beta=1)
0.66
>>> metrics.fbeta_score(y_true, y_pred, beta=2)
0.55
>>> metrics.precision_recall_fscore_support(y_true, y_pred, beta=0.5)
(array([0.66, 1.        ]), array([1. , 0.5]), array([0.71, 0.83]), array([2, 2]))


>>> import numpy as np
>>> from sklearn.metrics import precision_recall_curve
>>> from sklearn.metrics import average_precision_score
>>> y_true = np.array([0, 0, 1, 1])
>>> y_scores = np.array([0.1, 0.4, 0.35, 0.8])
>>> precision, recall, threshold = precision_recall_curve(y_true, y_scores)
>>> precision
array([0.5       , 0.66, 0.5       , 1.        , 1.        ])
>>> recall
array([1. , 1. , 0.5, 0.5, 0. ])
>>> threshold
array([0.1 , 0.35, 0.4 , 0.8 ])
>>> average_precision_score(y_true, y_scores)
0.83
```

Multiclass and multilabel classification

In a multiclass and multilabel classification task, the notions of precision, recall, and F-measures can be applied to each label independently. There are a few ways to combine results across labels, specified by the `average` argument to the `average_precision_score`, `f1_score`, `fbeta_score`, `precision_recall_fscore_support`, `precision_score` and `recall_score` functions, as described above.

Note the following behaviors when averaging:

- If all labels are included, "micro"-averaging in a multiclass setting will produce precision, recall and F that are all identical to accuracy.
- "weighted" averaging may produce an F-score that is not between precision and recall.
- "macro" averaging for F-measures is calculated as the arithmetic mean over per-label/class F-measures, not the harmonic mean over the arithmetic precision and recall means. Both calculations can be seen in the literature but are not equivalent, see [OB2019] for details.

To make this more explicit, consider the following notation:

- y the set of *true* $(sample, label)$ pairs
- $\hat{y}$ the set of *predicted* $(sample, label)$ pairs
- L the set of labels
- S the set of samples
- y_s the subset of y with sample s, i.e. $y_s := \{(s', l) \in y | s' = s\}$
- y_l the subset of y with label l
- similarly, $\hat{y}_s$ and $\hat{y}_l$ are subsets of $\hat{y}$
- $P(A, B) := \frac{|A \cap B|}{|B|}$ for some sets A and B
- $R(A, B) := \frac{|A \cap B|}{|A|}$ (Conventions vary on handling $A = \emptyset$; this implementation uses $R(A, B) := 0$, and similar for P.)
- $F_\beta(A, B) := (1 + \beta^2) \frac{P(A,B) \times R(A,B)}{\beta^2 P(A,B) + R(A,B)}$

Then the metrics are defined as:

average	Precision	Recall	F_beta
"micro"	$P(y, \hat{y})$	$R(y, \hat{y})$	$F_\beta(y, \hat{y})$
"samples"	$\frac{1}{\lvert S\rvert}\sum_{s\in S} P(y_s, \hat{y}_s)$	$\frac{1}{\lvert S\rvert}\sum_{s\in S} R(y_s, \hat{y}_s)$	$\frac{1}{\lvert S\rvert}\sum_{s\in S} F_\beta(y_s, \hat{y}_s)$
"macro"	$\frac{1}{\lvert L\rvert}\sum_{l\in L} P(y_l, \hat{y}_l)$	$\frac{1}{\lvert L\rvert}\sum_{l\in L} R(y_l, \hat{y}_l)$	$\frac{1}{\lvert L\rvert}\sum_{l\in L} F_\beta(y_l, \hat{y}_l)$
"weighted"	$\frac{1}{\sum_{l\in L}\lvert y_l\rvert}\sum_{l\in L} \lvert y_l\rvert P(y_l, \hat{y}_l)$	$\frac{1}{\sum_{l\in L}\lvert y_l\rvert}\sum_{l\in L} \lvert y_l\rvert R(y_l, \hat{y}_l)$	$\frac{1}{\sum_{l\in L}\lvert y_l\rvert}\sum_{l\in L} \lvert y_l\rvert F_\beta(y_l, \hat{y}_l)$
None	$\langle P(y_l, \hat{y}_l) \mid l \in L\rangle$	$\langle R(y_l, \hat{y}_l) \mid l \in L\rangle$	$\langle F_\beta(y_l, \hat{y}_l) \mid l \in L\rangle$

```
>>> from sklearn import metrics
>>> y_true = [0, 1, 2, 0, 1, 2]
>>> y_pred = [0, 2, 1, 0, 0, 1]
>>> metrics.precision_score(y_true, y_pred, average='macro')
0.22
>>> metrics.recall_score(y_true, y_pred, average='micro')
0.33
>>> metrics.f1_score(y_true, y_pred, average='weighted')
0.267
>>> metrics.fbeta_score(y_true, y_pred, average='macro', beta=0.5)
0.238
>>> metrics.precision_recall_fscore_support(y_true, y_pred, beta=0.5, average=None)
(array([0.667, 0., 0.]), array([1., 0., 0.]), array([0.714, 0., 0.]), array([2, 2, 2]))
```

For multiclass classification with a ”negative class”, it is possible to exclude some labels:

```
>>> metrics.recall_score(y_true, y_pred, labels=[1, 2], average='micro')
... # excluding 0, no labels were correctly recalled
0.0
```

Similarly, labels not present in the data sample may be accounted for in macro-averaging.

```
>>> metrics.precision_score(y_true, y_pred, labels=[0, 1, 2, 3], average='macro')
0.166
```

References

Jaccard similarity coefficient score

The `jaccard_score` function computes the average of Jaccard similarity coefficients, also called the Jaccard index, between pairs of label sets.

The Jaccard similarity coefficient with a ground truth label set y and predicted label set $\hat{y}$, is defined as

$$J(y, \hat{y}) = \frac{|y \cap \hat{y}|}{|y \cup \hat{y}|}.$$

The `jaccard_score` (like `precision_recall_fscore_support`) applies natively to binary targets. By computing it set-wise it can be extended to apply to multilabel and multiclass through the use of average (see above).

In the binary case:

```
>>> import numpy as np
>>> from sklearn.metrics import jaccard_score
>>> y_true = np.array([[0, 1, 1],
...                    [1, 1, 0]])
```

```
>>> y_pred = np.array([[1, 1, 1],
...                    [1, 0, 0]])
>>> jaccard_score(y_true[0], y_pred[0])
0.6666
```

In the 2D comparison case (e.g. image similarity):

```
>>> jaccard_score(y_true, y_pred, average="micro")
0.6
```

In the multilabel case with binary label indicators:

```
>>> jaccard_score(y_true, y_pred, average='samples')
0.5833
>>> jaccard_score(y_true, y_pred, average='macro')
0.6666
>>> jaccard_score(y_true, y_pred, average=None)
array([0.5, 0.5, 1. ])
```

Multiclass problems are binarized and treated like the corresponding multilabel problem:

```
>>> y_pred = [0, 2, 1, 2]
>>> y_true = [0, 1, 2, 2]
>>> jaccard_score(y_true, y_pred, average=None)
array([1. , 0. , 0.33])
>>> jaccard_score(y_true, y_pred, average='macro')
0.44
>>> jaccard_score(y_true, y_pred, average='micro')
0.33
```

Hinge loss

The `hinge_loss` function computes the average distance between the model and the data using hinge loss, a one-sided metric that considers only prediction errors. (Hinge loss is used in maximal margin classifiers such as support vector machines.)

If the true label y_i of a binary classification task is encoded as $y_i = \{-1, +1\}$ for every sample i; and w_i is the corresponding predicted decision (an array of shape (n_samples,) as output by the decision_function method), then the hinge loss is defined as:

$$L_{\text{Hinge}}(y, w) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \max\{1 - w_i y_i, 0\}$$

If there are more than two labels, `hinge_loss` uses a multiclass variant due to Crammer & Singer. Here is the paper describing it.

In this case the predicted decision is an array of shape (n_samples, n_labels). If w_{i,y_i} is the predicted decision for the true label y_i of the i-th sample; and $\hat{w}_{i,y_i} = \max\left\{w_{i,y_j} \mid y_j \neq y_i\right\}$ is the maximum of the predicted decisions for all the other labels, then the multi-class hinge loss is defined by:

$$L_{\text{Hinge}}(y, w) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \max\left\{1 + \hat{w}_{i,y_i} - w_{i,y_i}, 0\right\}$$

Here is a small example demonstrating the use of the `hinge_loss` function with an svm classifier in a binary class problem:

```
>>> from sklearn import svm
>>> from sklearn.metrics import hinge_loss
>>> X = [[0], [1]]
>>> y = [-1, 1]
>>> est = svm.LinearSVC(random_state=0)
>>> est.fit(X, y)
LinearSVC(random_state=0)
>>> pred_decision = est.decision_function([[-2], [3], [0.5]])
>>> pred_decision
array([-2.18,  2.36,  0.09])
>>> hinge_loss([-1, 1, 1], pred_decision)
0.3
```

Here is an example demonstrating the use of the `hinge_loss` function with an svm classifier in a multiclass problem:

```
>>> X = np.array([[0], [1], [2], [3]])
>>> Y = np.array([0, 1, 2, 3])
>>> labels = np.array([0, 1, 2, 3])
>>> est = svm.LinearSVC()
>>> est.fit(X, Y)
LinearSVC()
>>> pred_decision = est.decision_function([[-1], [2], [3]])
>>> y_true = [0, 2, 3]
>>> hinge_loss(y_true, pred_decision, labels=labels)
0.56
```

Log loss

Log loss, also called logistic regression loss or cross-entropy loss, is defined on probability estimates. It is commonly used in (multinomial) logistic regression and neural networks, as well as in some variants of expectation-maximization, and can be used to evaluate the probability outputs (`predict_proba`) of a classifier instead of its discrete predictions.

For binary classification with a true label $y \in \{0,1\}$ and a probability estimate $\hat{p} \approx \Pr(y = 1)$, the log loss per sample is the negative log-likelihood of the classifier given the true label:

$$L_{\log}(y, \hat{p}) = -\log \Pr(y|\hat{p}) = -(y \log(\hat{p}) + (1-y) \log(1-\hat{p}))$$

This extends to the multiclass case as follows. Let the true labels for a set of samples be encoded as a 1-of-K binary indicator matrix Y, i.e., $y_{i,k} = 1$ if sample i has label k taken from a set of K labels. Let $\hat{P}$ be a matrix of probability estimates, with elements $\hat{p}_{i,k} \approx \Pr(y_{i,k} = 1)$. Then the log loss of the whole set is

$$L_{\log}(Y, \hat{P}) = -\log \Pr(Y|\hat{P}) = -\frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{K-1} y_{i,k} \log \hat{p}_{i,k}$$

To see how this generalizes the binary log loss given above, note that in the binary case, $\hat{p}_{i,0} = 1 - \hat{p}_{i,1}$ and $y_{i,0} = 1 - y_{i,1}$, so expanding the inner sum over $y_{i,k} \in \{0,1\}$ gives the binary log loss.

The `log_loss` function computes log loss given a list of ground-truth labels and a probability matrix, as returned by an estimator's `predict_proba` method.

```
>>> from sklearn.metrics import log_loss
>>> y_true = [0, 0, 1, 1]
>>> y_proba = [[.9, .1], [.8, .2], [.3, .7], [.01, .99]]
>>> log_loss(y_true, y_proba)
0.1738
```

The first `[.9, .1]` in `y_proba` denotes 90% probability that the first sample has label 0. The log loss is non-negative.

Matthews correlation coefficient

The `matthews_corrcoef` function computes the Matthew's correlation coefficient (MCC) for binary classes. Quoting Wikipedia:

> "The Matthews correlation coefficient is used in machine learning as a measure of the quality of binary (two-class) classifications. It takes into account true and false positives and negatives and is generally regarded as a balanced measure which can be used even if the classes are of very different sizes. The MCC is in essence a correlation coefficient value between -1 and +1. A coefficient of +1 represents a perfect prediction, 0 an average random prediction and -1 an inverse prediction. The statistic is also known as the phi coefficient."

In the binary (two-class) case, tp, tn, fp and fn are respectively the number of true positives, true negatives, false positives and false negatives, the MCC is defined as

$$MCC = \frac{tp \times tn - fp \times fn}{\sqrt{(tp + fp)(tp + fn)(tn + fp)(tn + fn)}}.$$

In the multiclass case, the Matthews correlation coefficient can be defined in terms of a `confusion_matrix` C for K classes. To simplify the definition consider the following intermediate variables:

- $t_k = \sum_i^K C_{ik}$ the number of times class k truly occurred,
- $p_k = \sum_i^K C_{ki}$ the number of times class k was predicted,
- $c = \sum_k^K C_{kk}$ the total number of samples correctly predicted,
- $s = \sum_i^K \sum_j^K C_{ij}$ the total number of samples.

Then the multiclass MCC is defined as:

$$MCC = \frac{c \times s - \sum_k^K p_k \times t_k}{\sqrt{(s^2 - \sum_k^K p_k^2) \times (s^2 - \sum_k^K t_k^2)}}$$

When there are more than two labels, the value of the MCC will no longer range between -1 and +1. Instead the minimum value will be somewhere between -1 and 0 depending on the number and distribution of ground truth labels. The maximum value is always +1. For additional information, see [WikipediaMCC2021].

Here is a small example illustrating the usage of the `matthews_corrcoef` function:

```
>>> from sklearn.metrics import matthews_corrcoef
>>> y_true = [+1, +1, +1, -1]
>>> y_pred = [+1, -1, +1, +1]
>>> matthews_corrcoef(y_true, y_pred)
-0.33
```

References

Multi-label confusion matrix

The `multilabel_confusion_matrix` function computes class-wise (default) or sample-wise (samplewise=True) multilabel confusion matrix to evaluate the accuracy of a classification. multilabel_confusion_matrix also treats multiclass data as if it were multilabel, as this is a transformation commonly applied to evaluate multiclass problems with binary classification metrics (such as precision, recall, etc.).

When calculating class-wise multilabel confusion matrix C, the count of true negatives for class i is $C_{i,0,0}$, false negatives is $C_{i,1,0}$, true positives is $C_{i,1,1}$ and false positives is $C_{i,0,1}$.

Here is an example demonstrating the use of the `multilabel_confusion_matrix` function with multilabel indicator matrix input:

```
>>> import numpy as np
>>> from sklearn.metrics import multilabel_confusion_matrix
>>> y_true = np.array([[1, 0, 1],
...                    [0, 1, 0]])
>>> y_pred = np.array([[1, 0, 0],
...                    [0, 1, 1]])
>>> multilabel_confusion_matrix(y_true, y_pred)
array([[[1, 0],
        [0, 1]],
<BLANKLINE>
       [[1, 0],
        [0, 1]],
<BLANKLINE>
       [[0, 1],
        [1, 0]]])
```

Or a confusion matrix can be constructed for each sample's labels:

```
>>> multilabel_confusion_matrix(y_true, y_pred, samplewise=True)
array([[[1, 0],
        [1, 1]],
<BLANKLINE>
       [[1, 1],
        [0, 1]]])
```

Here is an example demonstrating the use of the `multilabel_confusion_matrix` function with multiclass input:

```
>>> y_true = ["cat", "ant", "cat", "cat", "ant", "bird"]
>>> y_pred = ["ant", "ant", "cat", "cat", "ant", "cat"]
>>> multilabel_confusion_matrix(y_true, y_pred,
...                             labels=["ant", "bird", "cat"])
array([[[3, 1],
        [0, 2]],
<BLANKLINE>
       [[5, 0],
```

```
        [1, 0]],
<BLANKLINE>
       [[2, 1],
        [1, 2]]])
```

Here are some examples demonstrating the use of the `multilabel_confusion_matrix` function to calculate recall (or sensitivity), specificity, fall out and miss rate for each class in a problem with multilabel indicator matrix input.

Calculating recall (also called the true positive rate or the sensitivity) for each class:

```
>>> y_true = np.array([[0, 0, 1],
...                    [0, 1, 0],
...                    [1, 1, 0]])
>>> y_pred = np.array([[0, 1, 0],
...                    [0, 0, 1],
...                    [1, 1, 0]])
>>> mcm = multilabel_confusion_matrix(y_true, y_pred)
>>> tn = mcm[:, 0, 0]
>>> tp = mcm[:, 1, 1]
>>> fn = mcm[:, 1, 0]
>>> fp = mcm[:, 0, 1]
>>> tp / (tp + fn)
array([1. , 0.5, 0. ])
```

Calculating specificity (also called the true negative rate) for each class:

```
>>> tn / (tn + fp)
array([1. , 0. , 0.5])
```

Calculating fall out (also called the false positive rate) for each class:

```
>>> fp / (fp + tn)
array([0. , 1. , 0.5])
```

Calculating miss rate (also called the false negative rate) for each class:

```
>>> fn / (fn + tp)
array([0. , 0.5, 1. ])
```

Receiver operating characteristic (ROC)

The function `roc_curve` computes the receiver operating characteristic curve, or ROC curve. Quoting Wikipedia :

> "A receiver operating characteristic (ROC), or simply ROC curve, is a graphical plot which illustrates the performance of a binary classifier system as its discrimination threshold is varied. It is created by plotting the fraction of true positives out of the positives (TPR = true positive rate) vs. the fraction of false positives out of the negatives (FPR = false positive rate), at various threshold settings. TPR is also known as sensitivity, and FPR is one minus the specificity or true negative rate."

This function requires the true binary value and the target scores, which can either be probability estimates of the positive class or non-thresholded decision values (as returned by decision_function on some classifiers). Here is a small example of how to use the `roc_curve` function:

```
>>> import numpy as np
>>> from sklearn.metrics import roc_curve
>>> y = np.array([1, 1, 2, 2])
>>> scores = np.array([0.1, 0.4, 0.35, 0.8])
```

```
>>> fpr, tpr, thresholds = roc_curve(y, scores, pos_label=2)
>>> fpr
array([0. , 0. , 0.5, 0.5, 1. ])
>>> tpr
array([0. , 0.5, 0.5, 1. , 1. ])
>>> thresholds
array([ inf, 0.8 , 0.4 , 0.35, 0.1 ])
```

Compared to metrics such as the subset accuracy, the Hamming loss, or the F1 score, ROC doesn't require optimizing a threshold for each label.

The `roc_auc_score` function, denoted by ROC-AUC or AUROC, computes the area under the ROC curve. By doing so, the curve information is summarized in one number.

The following figure shows the ROC curve and ROC-AUC score for a classifier aimed to distinguish the virginica flower from the rest of the species in the iris_dataset:

For more information see the Wikipedia article on AUC.

Binary case

In the **binary case**, you can either provide the probability estimates, using the classifier.predict_proba() method, or the non-thresholded decision values given by the classifier.decision_function() method. In the case of providing the probability estimates, the probability of the class with the ”greater label” should be provided. The ”greater label” corresponds to classifier.classes_[1] and thus classifier.predict_proba(X)[:, 1]. Therefore, the y_score parameter is of size (n_samples,).

```
>>> from sklearn.datasets import load_breast_cancer
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.metrics import roc_auc_score
>>> X, y = load_breast_cancer(return_X_y=True)
>>> clf = LogisticRegression().fit(X, y)
>>> clf.classes_
array([0, 1])
```

We can use the probability estimates corresponding to clf.classes_[1].

```
>>> y_score = clf.predict_proba(X)[:, 1]
>>> roc_auc_score(y, y_score)
0.99
```

Otherwise, we can use the non-thresholded decision values

```
>>> roc_auc_score(y, clf.decision_function(X))
0.99
```

Multi-class case

The `roc_auc_score` function can also be used in **multi-class classification**. Two averaging strategies are currently supported: the one-vs-one algorithm computes the average of the pairwise ROC AUC scores, and the one-vs-rest algorithm computes the average of the ROC AUC scores for each class against all other classes. In both cases, the predicted labels are provided in an array with values from 0 to `n_classes`, and the scores correspond to the probability estimates that a sample belongs to a particular class. The OvO and OvR algorithms support weighting uniformly (`average='macro'`) and by prevalence (`average='weighted'`).

One-vs-one Algorithm
Computes the average AUC of all possible pairwise combinations of classes. [HT2001] defines a multiclass AUC metric weighted uniformly:

$$\frac{1}{c(c-1)}\sum_{j=1}^{c}\sum_{k>j}^{c}(\text{AUC}(j|k)+\text{AUC}(k|j))$$

where c is the number of classes and $\text{AUC}(j|k)$ is the AUC with class j as the positive class and class k as the negative class. In general, case. This algorithm is used by setting the keyword argument `multiclass` to `'ovo'` and `average` to `'macro'`.
The [HT2001] multiclass AUC metric can be extended to be weighted by the prevalence:

$$\frac{1}{c(c-1)}\sum_{j=1}^{c}\sum_{k>j}^{c}p(j\cup k)(\text{AUC}(j|k)+\text{AUC}(k|j))$$

where c is the number of classes. This algorithm is used by setting the keyword argument `multiclass` to `'ovo'` and `average` to `'weighted'`. The `'weighted'` option returns a prevalence-weighted average as described in [FC2009].

One-vs-rest Algorithm
Computes the AUC of each class against the rest [PD2000]. The algorithm is functionally the same as the multilabel case. To enable this algorithm set the keyword argument `multiclass` to `'ovr'`. Additionally to `'macro'` [F2006] and `'weighted'` [F2001] averaging, OvR supports `'micro'` averaging.
In applications where a high false positive rate is not tolerable the parameter `max_fpr` of `roc_auc_score` can be used to summarize the ROC curve up to the given limit.
The following figure shows the micro-averaged ROC curve and its corresponding ROC-AUC score for a classifier aimed to distinguish the different species in the iris_dataset:

Multi-label case

In **multi-label classification**, the `roc_auc_score` function is extended by averaging over the labels as above. In this case, you should provide a y_score of shape (n_samples, n_classes). Thus, when using the probability estimates, one needs to select the probability of the class with the greater label for each output.

```
>>> from sklearn.datasets import make_multilabel_classification
>>> from sklearn.multioutput import MultiOutputClassifier
>>> X, y = make_multilabel_classification(random_state=0)
>>> inner_clf = LogisticRegression(random_state=0)
>>> clf = MultiOutputClassifier(inner_clf).fit(X, y)
>>> y_score = np.transpose([y_pred[:, 1] for y_pred in clf.predict_proba(X)])
>>> roc_auc_score(y, y_score, average=None)
array([0.828, 0.851, 0.94, 0.87, 0.95])
```

And the decision values do not require such processing.

```
>>> from sklearn.linear_model import RidgeClassifierCV
>>> clf = RidgeClassifierCV().fit(X, y)
>>> y_score = clf.decision_function(X)
>>> roc_auc_score(y, y_score, average=None)
array([0.82, 0.85, 0.93, 0.87, 0.94])
```

Examples

- See sphx_glr_auto_examples_model_selection_plot_roc.py for an example of using ROC to evaluate the quality of the output of a classifier.
- See sphx_glr_auto_examples_model_selection_plot_roc_crossval.py for an example of using ROC to evaluate classifier output quality, using cross-validation.
- See sphx_glr_auto_examples_applications_plot_species_distribution_modeling.py for an example of using ROC to model species distribution.

References

Detection error tradeoff (DET)

The function `det_curve` computes the detection error tradeoff curve (DET) curve [WikipediaDET2017]. Quoting Wikipedia:

> "A detection error tradeoff (DET) graph is a graphical plot of error rates for binary classification systems, plotting false reject rate vs. false accept rate. The x- and y-axes are scaled non-linearly by their standard normal deviates (or just by logarithmic transformation), yielding tradeoff curves that are more linear than ROC curves, and use most of the image area to highlight the differences of importance in the critical operating region."

DET curves are a variation of receiver operating characteristic (ROC) curves where False Negative Rate is plotted on the y-axis instead of True Positive Rate. DET curves are commonly plotted in normal deviate scale by transformation with ϕ^{-1} (with ϕ being the cumulative distribution function). The resulting performance curves explicitly visualize the tradeoff of error types for given classification algorithms. See [Martin1997] for examples and further motivation.

This figure compares the ROC and DET curves of two example classifiers on the same classification task:

Properties
- DET curves form a linear curve in normal deviate scale if the detection scores are normally (or close-to normally) distributed. It was shown by [Navratil2007] that the reverse is not necessarily true and even more general distributions are able to produce linear DET curves.
- The normal deviate scale transformation spreads out the points such that a comparatively larger space of plot is occupied. Therefore curves with similar classification performance might be easier to distinguish on a DET plot.
- With False Negative Rate being "inverse" to True Positive Rate the point of perfection for DET curves is the origin (in contrast to the top left corner for ROC curves).

Applications and limitations
DET curves are intuitive to read and hence allow quick visual assessment of a classifier's performance. Additionally DET curves can be consulted for threshold analysis and operating point selection. This is particularly helpful if a comparison of error types is required.
On the other hand DET curves do not provide their metric as a single number. Therefore for either automated evaluation or comparison to other classification tasks metrics like the derived area under ROC curve might be better suited.

Examples

- See sphx_glr_auto_examples_model_selection_plot_det.py for an example comparison between receiver operating characteristic (ROC) curves and Detection error tradeoff (DET) curves.

References

Zero one loss

The `zero_one_loss` function computes the sum or the average of the 0-1 classification loss (L_{0-1}) over n_{samples}. By default, the function normalizes over the sample. To get the sum of the L_{0-1}, set `normalize` to `False`.

In multilabel classification, the `zero_one_loss` scores a subset as one if its labels strictly match the predictions, and as a zero if there are any errors. By default, the function returns the percentage of imperfectly predicted subsets. To get the count of such subsets instead, set `normalize` to `False`.

If $\hat{y}_i$ is the predicted value of the i-th sample and y_i is the corresponding true value, then the 0-1 loss L_{0-1} is defined as:

$$L_{0-1}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} 1(\hat{y}_i \neq y_i)$$

where $1(x)$ is the indicator function. The zero-one loss can also be computed as zero-one loss $=$ $1 -$ accuracy.

```
>>> from sklearn.metrics import zero_one_loss
>>> y_pred = [1, 2, 3, 4]
>>> y_true = [2, 2, 3, 4]
>>> zero_one_loss(y_true, y_pred)
0.25
>>> zero_one_loss(y_true, y_pred, normalize=False)
1.0
```

In the multilabel case with binary label indicators, where the first label set [0,1] has an error:

```
>>> zero_one_loss(np.array([[0, 1], [1, 1]]), np.ones((2, 2)))
0.5

>>> zero_one_loss(np.array([[0, 1], [1, 1]]), np.ones((2, 2)),  normalize=False)
1.0
```

Examples

- See sphx_glr_auto_examples_feature_selection_plot_rfe_with_cross_validation.py for an example of zero one loss usage to perform recursive feature elimination with cross-validation.

Brier score loss

The `brier_score_loss` function computes the Brier score for binary and multiclass probabilistic predictions and is equivalent to the mean squared error. Quoting Wikipedia:

> "The Brier score is a strictly proper scoring rule that measures the accuracy of probabilistic predictions. [...] [It] is applicable to tasks in which predictions must assign probabilities to a set of mutually exclusive discrete outcomes or classes."

Let the true labels for a set of N data points be encoded as a 1-of-K binary indicator matrix Y, i.e., $y_{i,k} = 1$ if sample i has label k taken from a set of K labels. Let $\hat{P}$ be a matrix of probability estimates with elements $\hat{p}_{i,k} \approx \Pr(y_{i,k} = 1)$. Following the original definition by [Brier1950], the Brier score is given by:

$$BS(Y, \hat{P}) = \frac{1}{N} \sum_{i=0}^{N-1} \sum_{k=0}^{K-1} (y_{i,k} - \hat{p}_{i,k})^2$$

The Brier score lies in the interval $[0, 2]$ and the lower the value the better the probability estimates are (the mean squared difference is smaller). Actually, the Brier score is a strictly proper scoring rule, meaning that it achieves the best score only when the estimated probabilities equal the true ones.

Note that in the binary case, the Brier score is usually divided by two and ranges between $[0, 1]$. For binary targets $y_i \in \{0, 1\}$ and probability estimates $\hat{p}_i \approx \Pr(y_i = 1)$ for the positive class, the Brier score is then equal to:

$$BS(y, \hat{p}) = \frac{1}{N} \sum_{i=0}^{N-1} (y_i - \hat{p}_i)^2$$

The `brier_score_loss` function computes the Brier score given the ground-truth labels and predicted probabilities, as returned by an estimator's `predict_proba` method. The scale_by_half parameter controls which of the two above definitions to follow.

```
>>> import numpy as np
>>> from sklearn.metrics import brier_score_loss
>>> y_true = np.array([0, 1, 1, 0])
>>> y_true_categorical = np.array(["spam", "ham", "ham", "spam"])
>>> y_prob = np.array([0.1, 0.9, 0.8, 0.4])
>>> brier_score_loss(y_true, y_prob)
0.055
>>> brier_score_loss(y_true, 1 - y_prob, pos_label=0)
0.055
>>> brier_score_loss(y_true_categorical, y_prob, pos_label="ham")
0.055
>>> brier_score_loss(
...     ["eggs", "ham", "spam"],
...     [[0.8, 0.1, 0.1], [0.2, 0.7, 0.1], [0.2, 0.2, 0.6]],
...     labels=["eggs", "ham", "spam"],
... )
0.146
```

Note
As a strictly proper scoring rules for probabilistic predictions, the Brier score assesses calibration (reliability) and discriminative power (resolution) of a model, as well as the randomness of the data (uncertainty) at the same time. This follows from the well-known Brier score decomposition of Murphy [Murphy1973]. As it is not clear which term dominates, the score is of limited use for assessing calibration alone (unless one computes each term of the decomposition). A lower Brier loss, for instance, does not necessarily mean a better calibrated model, it could also mean a worse calibrated model with much more discriminatory power, e.g. using many more features.

Examples

- See sphx_glr_auto_examples_calibration_plot_calibration.py for an example of Brier score loss usage to perform probability calibration of classifiers.

References

Class likelihood ratios

The `class_likelihood_ratios` function computes the positive and negative likelihood ratios $LR_{\pm}$ for binary classes, which can be interpreted as the ratio of post-test to pre-test odds as explained below. As a consequence, this metric is invariant w.r.t. the class prevalence (the number of

samples in the positive class divided by the total number of samples) and **can be extrapolated between populations regardless of any possible class imbalance.**

The $LR_{\pm}$ metrics are therefore very useful in settings where the data available to learn and evaluate a classifier is a study population with nearly balanced classes, such as a case-control study, while the target application, i.e. the general population, has very low prevalence.

The positive likelihood ratio LR_{+} is the probability of a classifier to correctly predict that a sample belongs to the positive class divided by the probability of predicting the positive class for a sample belonging to the negative class:

$$LR_{+} = \frac{\text{PR}(P+|T+)}{\text{PR}(P+|T-)}.$$

The notation here refers to predicted (P) or true (T) label and the sign $+$ and $-$ refer to the positive and negative class, respectively, e.g. $P+$ stands for "predicted positive".

Analogously, the negative likelihood ratio LR_{-} is the probability of a sample of the positive class being classified as belonging to the negative class divided by the probability of a sample of the negative class being correctly classified:

$$LR_{-} = \frac{\text{PR}(P-|T+)}{\text{PR}(P-|T-)}.$$

For classifiers above chance LR_{+} above 1 **higher is better**, while LR_{-} ranges from 0 to 1 and **lower is better**. Values of $LR_{\pm} \approx 1$ correspond to chance level.

Notice that probabilities differ from counts, for instance $\text{PR}(P+|T+)$ is not equal to the number of true positive counts `tp` (see the wikipedia page for the actual formulas).

Examples

- sphx_glr_auto_examples_model_selection_plot_likelihood_ratios.py

Interpretation across varying prevalence
Both class likelihood ratios are interpretable in terms of an odds ratio (pre-test and post-tests):

$$\text{post-test odds} = \text{Likelihood ratio} \times \text{pre-test odds}.$$

Odds are in general related to probabilities via

$$\text{odds} = \frac{\text{probability}}{1 - \text{probability}},$$

or equivalently

$$\text{probability} = \frac{\text{odds}}{1 + \text{odds}}.$$

On a given population, the pre-test probability is given by the prevalence. By converting odds to probabilities, the likelihood ratios can be translated into a probability of truly belonging to either class before and after a classifier prediction:

$$\text{post-test odds} = \text{Likelihood ratio} \times \frac{\text{pre-test probability}}{1 - \text{pre-test probability}},$$

$$\text{post-test probability} = \frac{\text{post-test odds}}{1 + \text{post-test odds}}.$$

Mathematical divergences
The positive likelihood ratio (LR+) is undefined when $fp = 0$, meaning the classifier does not misclassify any negative labels as positives. This condition can either indicate a perfect identification of all the negative cases or, if there are also no true positive predictions ($tp = 0$), that the classifier does not predict the positive class at all. In the first case, LR+ can be interpreted as np.inf, in the second case (for instance, with highly imbalanced data) it can be interpreted as np.nan.
The negative likelihood ratio (LR-) is undefined when $tn = 0$. Such divergence is invalid, as $LR_- > 1.0$ would indicate an increase in the odds of a sample belonging to the positive class after being classified as negative, as if the act of classifying caused the positive condition. This includes the case of a (i.e. when $tn = fn = 0$).
Both class likelihood ratios (LR+ and LR-) are undefined when $tp = fn = 0$, which means that no samples of the positive class were present in the test set. This can happen when cross-validating on highly imbalanced data and also leads to a division by zero.
If a division by zero occurs and raise_warning is set to True (default), np.nan by default to avoid pollution when averaging over cross-validation folds. Users can set return values in case of a division by zero with the replace_undefined_by param.
For a worked-out demonstration of the `class_likelihood_ratios` function, see the example below.

References
- Wikipedia entry for Likelihood ratios in diagnostic testing
- Brenner, H., & Gefeller, O. (1997). Variation of sensitivity, specificity, likelihood ratios and predictive values with disease prevalence. Statistics in medicine, 16(9), 981-991.

D² score for classification

The D² score computes the fraction of deviance explained. It is a generalization of R², where the squared error is generalized and replaced by a classification deviance of choice $\text{dev}(y, \hat{y})$ (e.g., Log loss, Brier score,). D² is a form of a *skill score*. It is calculated as

$$D^2(y, \hat{y}) = 1 - \frac{\text{dev}(y, \hat{y})}{\text{dev}(y, y_{\text{null}})} .$$

Where y_{null} is the optimal prediction of an intercept-only model (e.g., the per-class proportion of y_true in the case of the Log loss and Brier score).

Like R², the best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts y_{null}, disregarding the input features, would get a D² score of 0.0.

D2 log loss score
The `d2_log_loss_score` function implements the special case of D² with the log loss, see log_loss, i.e.:

$$\text{dev}(y, \hat{y}) = \text{log_loss}(y, \hat{y}).$$

Here are some usage examples of the `d2_log_loss_score` function:

```
>>> from sklearn.metrics import d2_log_loss_score
>>> y_true = [1, 1, 2, 3]
>>> y_proba = [
...     [0.5, 0.25, 0.25],
...     [0.5, 0.25, 0.25],
...     [0.5, 0.25, 0.25],
```

```
...     [0.5, 0.25, 0.25],
... ]
>>> d2_log_loss_score(y_true, y_proba)
0.0
>>> y_true = [1, 2, 3]
>>> y_proba = [
...     [0.98, 0.01, 0.01],
...     [0.01, 0.98, 0.01],
...     [0.01, 0.01, 0.98],
... ]
>>> d2_log_loss_score(y_true, y_proba)
0.981
>>> y_true = [1, 2, 3]
>>> y_proba = [
...     [0.1, 0.6, 0.3],
...     [0.1, 0.6, 0.3],
...     [0.4, 0.5, 0.1],
... ]
>>> d2_log_loss_score(y_true, y_proba)
-0.552
```

D2 Brier score

The `d2_brier_score` function implements the special case of D^2 with the Brier score, see brier_score_loss, i.e.:

$$\text{dev}(y, \hat{y}) = \text{brier_score_loss}(y, \hat{y}).$$

This is also referred to as the Brier Skill Score (BSS) and scaled Brier score.

Here are some usage examples of the `d2_brier_score` function:

```
>>> from sklearn.metrics import d2_brier_score
>>> y_true = [1, 1, 2, 3]
>>> y_pred = [
...     [0.5, 0.25, 0.25],
...     [0.5, 0.25, 0.25],
...     [0.5, 0.25, 0.25],
...     [0.5, 0.25, 0.25],
... ]
>>> d2_brier_score(y_true, y_pred)
0.0
>>> y_true = [1, 2, 3]
>>> y_pred = [
...     [0.98, 0.01, 0.01],
...     [0.01, 0.98, 0.01],
...     [0.01, 0.01, 0.98],
... ]
>>> d2_brier_score(y_true, y_pred)
0.9991
>>> y_true = [1, 2, 3]
>>> y_pred = [
...     [0.1, 0.6, 0.3],
...     [0.1, 0.6, 0.3],
...     [0.4, 0.5, 0.1],
... ]
>>> d2_brier_score(y_true, y_pred)
-0.370...
```

Multilabel ranking metrics

In multilabel learning, each sample can have any number of ground truth labels associated with it. The goal is to give high scores and better rank to the ground truth labels.

Coverage error

The `coverage_error` function computes the average number of labels that have to be included in the final prediction such that all true labels are predicted. This is useful if you want to know how many top-scored-labels you have to predict in average without missing any true one. The best value of this metric is thus the average number of true labels.

> Note
> Our implementation's score is 1 greater than the one given in Tsoumakas et al., 2010. This extends it to handle the degenerate case in which an instance has 0 true labels.

Formally, given a binary indicator matrix of the ground truth labels $y \in \{0,1\}^{n_{\text{samples}} \times n_{\text{labels}}}$ and the score associated with each label $\hat{f} \in \mathbb{R}^{n_{\text{samples}} \times n_{\text{labels}}}$, the coverage is defined as

$$coverage(y, \hat{f}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \max_{j:y_{ij}=1} \text{rank}_{ij}$$

with $\text{rank}_{ij} = \left|\left\{k : \hat{f}_{ik} \geq \hat{f}_{ij}\right\}\right|$. Given the rank definition, ties in `y_scores` are broken by giving the maximal rank that would have been assigned to all tied values.

Here is a small example of usage of this function:

```
>>> import numpy as np
>>> from sklearn.metrics import coverage_error
>>> y_true = np.array([[1, 0, 0], [0, 0, 1]])
>>> y_score = np.array([[0.75, 0.5, 1], [1, 0.2, 0.1]])
>>> coverage_error(y_true, y_score)
2.5
```

Label ranking average precision

The `label_ranking_average_precision_score` function implements label ranking average precision (LRAP). This metric is linked to the `average_precision_score` function, but is based on the notion of label ranking instead of precision and recall.

Label ranking average precision (LRAP) averages over the samples the answer to the following question: for each ground truth label, what fraction of higher-ranked labels were true labels? This performance measure will be higher if you are able to give better rank to the labels associated with each sample. The obtained score is always strictly greater than 0, and the best value is 1. If there is exactly one relevant label per sample, label ranking average precision is equivalent to the mean reciprocal rank.

Formally, given a binary indicator matrix of the ground truth labels $y \in \{0,1\}^{n_{\text{samples}} \times n_{\text{labels}}}$ and the score associated with each label $\hat{f} \in \mathbb{R}^{n_{\text{samples}} \times n_{\text{labels}}}$, the average precision is defined as

$$LRAP(y, \hat{f}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \frac{1}{||y_i||_0} \sum_{j:y_{ij}=1} \frac{|\mathcal{L}_{ij}|}{\text{rank}_{ij}}$$

where $\mathcal{L}_{ij} = \left\{k : y_{ik} = 1, \hat{f}_{ik} \geq \hat{f}_{ij}\right\}$, $\text{rank}_{ij} = \left|\left\{k : \hat{f}_{ik} \geq \hat{f}_{ij}\right\}\right|$, $|\cdot|$ computes the cardinality of the set (i.e., the number of elements in the set), and $||\cdot||_0$ is the ℓ_0 "norm" (which computes the number of nonzero elements in a vector).

Here is a small example of usage of this function:

```
>>> import numpy as np
>>> from sklearn.metrics import label_ranking_average_precision_score
>>> y_true = np.array([[1, 0, 0], [0, 0, 1]])
>>> y_score = np.array([[0.75, 0.5, 1], [1, 0.2, 0.1]])
>>> label_ranking_average_precision_score(y_true, y_score)
0.416
```

Ranking loss

The `label_ranking_loss` function computes the ranking loss which averages over the samples the number of label pairs that are incorrectly ordered, i.e. true labels have a lower score than false labels, weighted by the inverse of the number of ordered pairs of false and true labels. The lowest achievable ranking loss is zero.

Formally, given a binary indicator matrix of the ground truth labels $y \in \{0, 1\}^{n_{\text{samples}} \times n_{\text{labels}}}$ and the score associated with each label $\hat{f} \in \mathbb{R}^{n_{\text{samples}} \times n_{\text{labels}}}$, the ranking loss is defined as

$$ranking_loss(y, \hat{f}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \frac{1}{||y_i||_0(n_{\text{labels}} - ||y_i||_0)} \left|\left\{(k, l) : \hat{f}_{ik} \leq \hat{f}_{il}, y_{ik} = 1, y_{il} = 0 \right\}\right|$$

where $|\cdot|$ computes the cardinality of the set (i.e., the number of elements in the set) and $||\cdot||_0$ is the ℓ_0 "norm" (which computes the number of nonzero elements in a vector).

Here is a small example of usage of this function:

```
>>> import numpy as np
>>> from sklearn.metrics import label_ranking_loss
>>> y_true = np.array([[1, 0, 0], [0, 0, 1]])
>>> y_score = np.array([[0.75, 0.5, 1], [1, 0.2, 0.1]])
>>> label_ranking_loss(y_true, y_score)
0.75
>>> # With the following prediction, we have perfect and minimal loss
>>> y_score = np.array([[1.0, 0.1, 0.2], [0.1, 0.2, 0.9]])
>>> label_ranking_loss(y_true, y_score)
0.0
```

References

- Tsoumakas, G., Katakis, I., & Vlahavas, I. (2010). Mining multi-label data. In Data mining and knowledge discovery handbook (pp. 667-685). Springer US.

Normalized Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) and Normalized Discounted Cumulative Gain (NDCG) are ranking metrics implemented in `dcg_score` and `ndcg_score` ; they compare a predicted order to ground-truth scores, such as the relevance of answers to a query.

From the Wikipedia page for Discounted Cumulative Gain:

”Discounted cumulative gain (DCG) is a measure of ranking quality. In information retrieval, it is often used to measure effectiveness of web search engine algorithms or related applications. Using a graded relevance scale of documents in a search-engine result set, DCG measures the usefulness, or gain, of a document based on its position in the result list. The gain is accumulated from the top of the result list to the bottom, with the gain of each result discounted at lower ranks.”

DCG orders the true targets (e.g. relevance of query answers) in the predicted order, then multiplies them by a logarithmic decay and sums the result. The sum can be truncated after the first K results, in which case we call it DCG@K. NDCG, or NDCG@K is DCG divided by the DCG obtained by a perfect prediction, so that it is always between 0 and 1. Usually, NDCG is preferred to DCG.

Compared with the ranking loss, NDCG can take into account relevance scores, rather than a ground-truth ranking. So if the ground-truth consists only of an ordering, the ranking loss should be preferred; if the ground-truth consists of actual usefulness scores (e.g. 0 for irrelevant, 1 for relevant, 2 for very relevant), NDCG can be used.

For one sample, given the vector of continuous ground-truth values for each target $y \in \mathbb{R}^M$, where M is the number of outputs, and the prediction $\hat{y}$, which induces the ranking function f, the DCG score is

$$\sum_{r=1}^{\min(K,M)} \frac{y_{f(r)}}{\log(1+r)}$$

and the NDCG score is the DCG score divided by the DCG score obtained for y.

References

- Wikipedia entry for Discounted Cumulative Gain
- Jarvelin, K., & Kekalainen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems (TOIS), 20(4), 422-446.
- Wang, Y., Wang, L., Li, Y., He, D., Chen, W., & Liu, T. Y. (2013, May). A theoretical analysis of NDCG ranking measures. In Proceedings of the 26th Annual Conference on Learning Theory (COLT 2013)
- McSherry, F., & Najork, M. (2008, March). Computing information retrieval performance measures efficiently in the presence of tied scores. In European conference on information retrieval (pp. 414-421). Springer, Berlin, Heidelberg.

Regression metrics

The `sklearn.metrics` module implements several loss, score, and utility functions to measure regression performance. Some of those have been enhanced to handle the multioutput case: `mean_squared_error`, `mean_absolute_error`, `r2_score`, `explained_variance_score`, `mean_pinball_loss`, `d2_pinball_score` and `d2_absolute_error_score`.

These functions have a `multioutput` keyword argument which specifies the way the scores or losses for each individual target should be averaged. The default is `'uniform_average'`, which specifies a uniformly weighted mean over outputs. If an `ndarray` of shape `(n_outputs,)` is passed, then its entries are interpreted as weights and an according weighted average is returned. If `multioutput` is `'raw_values'`, then all unaltered individual scores or losses will be returned in an array of shape `(n_outputs,)`.

The `r2_score` and `explained_variance_score` accept an additional value `'variance_weighted'` for the `multioutput` parameter. This option leads to a weighting of each individual score by the variance

of the corresponding target variable. This setting quantifies the globally captured unscaled variance. If the target variables are of different scale, then this score puts more importance on explaining the higher variance variables.

R^2 score, the coefficient of determination

The `r2_score` function computes the coefficient of determination, usually denoted as R^2.

It represents the proportion of variance (of y) that has been explained by the independent variables in the model. It provides an indication of goodness of fit and therefore a measure of how well unseen samples are likely to be predicted by the model, through the proportion of explained variance.

As such variance is dataset dependent, R^2 may not be meaningfully comparable across different datasets. Best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected (average) value of y, disregarding the input features, would get an R^2 score of 0.0.

Note: when the prediction residuals have zero mean, the R^2 score and the explained_variance_score are identical.

If $\hat{y}_i$ is the predicted value of the i-th sample and y_i is the corresponding true value for total n samples, the estimated R^2 is defined as:

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ and $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} \epsilon_i^2$.

Note that `r2_score` calculates unadjusted R^2 without correcting for bias in sample variance of y.

In the particular case where the true target is constant, the R^2 score is not finite: it is either `NaN` (perfect predictions) or `-Inf` (imperfect predictions). Such non-finite scores may prevent correct model optimization such as grid-search cross-validation to be performed correctly. For this reason the default behaviour of `r2_score` is to replace them with 1.0 (perfect predictions) or 0.0 (imperfect predictions). If `force_finite` is set to `False`, this score falls back on the original R^2 definition.

Here is a small example of usage of the `r2_score` function:

```
>>> from sklearn.metrics import r2_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> r2_score(y_true, y_pred)
0.948
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> r2_score(y_true, y_pred, multioutput='variance_weighted')
0.938
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> r2_score(y_true, y_pred, multioutput='uniform_average')
0.936
>>> r2_score(y_true, y_pred, multioutput='raw_values')
array([0.965, 0.908])
>>> r2_score(y_true, y_pred, multioutput=[0.3, 0.7])
```

```
0.925
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2]
>>> r2_score(y_true, y_pred)
1.0
>>> r2_score(y_true, y_pred, force_finite=False)
nan
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2 + 1e-8]
>>> r2_score(y_true, y_pred)
0.0
>>> r2_score(y_true, y_pred, force_finite=False)
-inf
```

Examples

- See sphx_glr_auto_examples_linear_model_plot_lasso_and_elasticnet.py for an example of R^2 score usage to evaluate Lasso and Elastic Net on sparse signals.

Mean absolute error

The `mean_absolute_error` function computes mean absolute error, a risk metric corresponding to the expected value of the absolute error loss or $l1$-norm loss.

If $\hat{y}_i$ is the predicted value of the i-th sample, and y_i is the corresponding true value, then the mean absolute error (MAE) estimated over n_{samples} is defined as

$$\text{MAE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} |y_i - \hat{y}_i| .$$

Here is a small example of usage of the `mean_absolute_error` function:

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> mean_absolute_error(y_true, y_pred)
0.5
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> mean_absolute_error(y_true, y_pred)
0.75
>>> mean_absolute_error(y_true, y_pred, multioutput='raw_values')
array([0.5, 1. ])
>>> mean_absolute_error(y_true, y_pred, multioutput=[0.3, 0.7])
0.85
```

Mean squared error

The `mean_squared_error` function computes mean squared error, a risk metric corresponding to the expected value of the squared (quadratic) error or loss.

If $\hat{y}_i$ is the predicted value of the i-th sample, and y_i is the corresponding true value, then the mean squared error (MSE) estimated over n_{samples} is defined as

$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (y_i - \hat{y}_i)^2.$$

Here is a small example of usage of the `mean_squared_error` function:

```
>>> from sklearn.metrics import mean_squared_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> mean_squared_error(y_true, y_pred)
0.375
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> mean_squared_error(y_true, y_pred)
0.7083
```

Examples

- See sphx_glr_auto_examples_ensemble_plot_gradient_boosting_regression.py for an example of mean squared error usage to evaluate gradient boosting regression.

Taking the square root of the MSE, called the root mean squared error (RMSE), is another common metric that provides a measure in the same units as the target variable. RMSE is available through the `root_mean_squared_error` function.

Mean squared logarithmic error

The `mean_squared_log_error` function computes a risk metric corresponding to the expected value of the squared logarithmic (quadratic) error or loss.

If $\hat{y}_i$ is the predicted value of the i-th sample, and y_i is the corresponding true value, then the mean squared logarithmic error (MSLE) estimated over n_{samples} is defined as

$$\text{MSLE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (\log_e(1 + y_i) - \log_e(1 + \hat{y}_i))^2.$$

Where $\log_e(x)$ means the natural logarithm of x. This metric is best to use when targets having exponential growth, such as population counts, average sales of a commodity over a span of years etc. Note that this metric penalizes an under-predicted estimate greater than an over-predicted estimate.

Here is a small example of usage of the `mean_squared_log_error` function:

```
>>> from sklearn.metrics import mean_squared_log_error
>>> y_true = [3, 5, 2.5, 7]
>>> y_pred = [2.5, 5, 4, 8]
>>> mean_squared_log_error(y_true, y_pred)
0.0397
>>> y_true = [[0.5, 1], [1, 2], [7, 6]]
>>> y_pred = [[0.5, 2], [1, 2.5], [8, 8]]
>>> mean_squared_log_error(y_true, y_pred)
0.044
```

The root mean squared logarithmic error (RMSLE) is available through the `root_mean_squared_log_error` function.

Mean absolute percentage error

The `mean_absolute_percentage_error` (MAPE), also known as mean absolute percentage deviation (MAPD), is an evaluation metric for regression problems. The idea of this metric is to be sensitive to relative errors. It is for example not changed by a global scaling of the target variable.

If $\hat{y}_i$ is the predicted value of the i-th sample and y_i is the corresponding true value, then the mean absolute percentage error (MAPE) estimated over n_{samples} is defined as

$$\text{MAPE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \frac{|y_i - \hat{y}_i|}{\max(\epsilon, |y_i|)}$$

where ϵ is an arbitrary small yet strictly positive number to avoid undefined results when y is zero.

The `mean_absolute_percentage_error` function supports multioutput.

Here is a small example of usage of the `mean_absolute_percentage_error` function:

```
>>> from sklearn.metrics import mean_absolute_percentage_error
>>> y_true = [1, 10, 1e6]
>>> y_pred = [0.9, 15, 1.2e6]
>>> mean_absolute_percentage_error(y_true, y_pred)
0.2666
```

In above example, if we had used mean_absolute_error, it would have ignored the small magnitude values and only reflected the error in prediction of highest magnitude value. But that problem is resolved in case of MAPE because it calculates relative percentage error with respect to actual output.

Note
The MAPE formula here does not represent the common ”percentage” definition: the percentage in the range [0, 100] is converted to a relative value in the range [0, 1] by dividing by 100. Thus, an error of 200% corresponds to a relative error of 2. The motivation here is to have a range of values that is more consistent with other error metrics in scikit-learn, such as accuracy_score.
To obtain the mean absolute percentage error as per the Wikipedia formula, multiply the mean_absolute_percentage_error computed here by 100.

References
- Wikipedia entry for Mean Absolute Percentage Error

Median absolute error

The `median_absolute_error` is particularly interesting because it is robust to outliers. The loss is calculated by taking the median of all absolute differences between the target and the prediction.

If $\hat{y}_i$ is the predicted value of the i-th sample and y_i is the corresponding true value, then the median absolute error (MedAE) estimated over n_{samples} is defined as

$$\text{MedAE}(y, \hat{y}) = \text{median}(\mid y_1 - \hat{y}_1 \mid, \ldots, \mid y_n - \hat{y}_n \mid).$$

The `median_absolute_error` does not support multioutput.

Here is a small example of usage of the `median_absolute_error` function:

```
>>> from sklearn.metrics import median_absolute_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> median_absolute_error(y_true, y_pred)
0.5
```

Max error

The `max_error` function computes the maximum residual error , a metric that captures the worst case error between the predicted value and the true value. In a perfectly fitted single output regression model, `max_error` would be `0` on the training set and though this would be highly unlikely in the real world, this metric shows the extent of error that the model had when it was fitted.

If $\hat{y}_i$ is the predicted value of the i-th sample, and y_i is the corresponding true value, then the max error is defined as

$$\text{Max Error}(y, \hat{y}) = \max(|y_i - \hat{y}_i|)$$

Here is a small example of usage of the `max_error` function:

```
>>> from sklearn.metrics import max_error
>>> y_true = [3, 2, 7, 1]
>>> y_pred = [9, 2, 7, 1]
>>> max_error(y_true, y_pred)
6.0
```

The `max_error` does not support multioutput.

Explained variance score

The `explained_variance_score` computes the explained variance regression score.

If $\hat{y}$ is the estimated target output, y the corresponding (correct) target output, and Var is Variance, the square of the standard deviation, then the explained variance is estimated as follow:

$$explained_variance(y, \hat{y}) = 1 - \frac{Var\{y - \hat{y}\}}{Var\{y\}}$$

The best possible score is 1.0, lower values are worse.

The difference between the explained variance score and the r2_score is that the explained variance score does not account for systematic offset in the prediction. For this reason, the

In the particular case where the true target is constant, the Explained Variance score is not finite: it is either `NaN` (perfect predictions) or `-Inf` (imperfect predictions). Such non-finite scores may prevent correct model optimization such as grid-search cross-validation to be performed correctly. For this reason the default behaviour of `explained_variance_score` is to replace them with 1.0 (perfect predictions) or 0.0 (imperfect predictions). You can set the `force_finite` parameter to `False` to prevent this fix from happening and fallback on the original Explained Variance score.

Here is a small example of usage of the `explained_variance_score` function:

```
>>> from sklearn.metrics import explained_variance_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> explained_variance_score(y_true, y_pred)
```

```
0.957
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> explained_variance_score(y_true, y_pred, multioutput='raw_values')
array([0.967, 1.        ])
>>> explained_variance_score(y_true, y_pred, multioutput=[0.3, 0.7])
0.990
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2]
>>> explained_variance_score(y_true, y_pred)
1.0
>>> explained_variance_score(y_true, y_pred, force_finite=False)
nan
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2 + 1e-8]
>>> explained_variance_score(y_true, y_pred)
0.0
>>> explained_variance_score(y_true, y_pred, force_finite=False)
-inf
```

Mean Poisson, Gamma, and Tweedie deviances

The `mean_tweedie_deviance` function computes the mean Tweedie deviance error with a `power` parameter (p). This is a metric that elicits predicted expectation values of regression targets.

Following special cases exist,

- when `power=0` it is equivalent to `mean_squared_error`.
- when `power=1` it is equivalent to `mean_poisson_deviance`.
- when `power=2` it is equivalent to `mean_gamma_deviance`.

If $\hat{y}_i$ is the predicted value of the i-th sample, and y_i is the corresponding true value, then the mean Tweedie deviance error (D) for power p, estimated over n_{samples} is defined as

$$\mathrm{D}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \begin{cases} (y_i - \hat{y}_i)^2, & \text{for } p = 0 \text{ (Normal)} \\ 2(y_i \log(y_i/\hat{y}_i) + \hat{y}_i - y_i), & \text{for } p = 1 \text{ (Poisson)} \\ 2(\log(\hat{y}_i/y_i) + y_i/\hat{y}_i - 1), & \text{for } p = 2 \text{ (Gamma)} \\ 2\left(\frac{\max(y_i,0)^{2-p}}{(1-p)(2-p)} - \frac{y_i\,\hat{y}_i^{1-p}}{1-p} + \frac{\hat{y}_i^{2-p}}{2-p}\right), & \text{otherwise} \end{cases}$$

Tweedie deviance is a homogeneous function of degree `2-power`. Thus, Gamma distribution with `power=2` means that simultaneously scaling `y_true` and `y_pred` has no effect on the deviance. For Poisson distribution `power=1` the deviance scales linearly, and for Normal distribution (`power=0`), quadratically. In general, the higher `power` the less weight is given to extreme deviations between true and predicted targets.

For instance, let's compare the two predictions 1.5 and 150 that are both 50% larger than their corresponding true value.

The mean squared error (`power=0`) is very sensitive to the prediction difference of the second point,:

```
>>> from sklearn.metrics import mean_tweedie_deviance
>>> mean_tweedie_deviance([1.0], [1.5], power=0)
0.25
>>> mean_tweedie_deviance([100.], [150.], power=0)
2500.0
```

If we increase `power` to 1,:

```
>>> mean_tweedie_deviance([1.0], [1.5], power=1)
0.189
>>> mean_tweedie_deviance([100.], [150.], power=1)
18.9
```

the difference in errors decreases. Finally, by setting, `power=2`:

```
>>> mean_tweedie_deviance([1.0], [1.5], power=2)
0.144
>>> mean_tweedie_deviance([100.], [150.], power=2)
0.144
```

we would get identical errors. The deviance when `power=2` is thus only sensitive to relative errors.

Pinball loss

The `mean_pinball_loss` function is used to evaluate the predictive performance of quantile regression models.

$$\text{pinball}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \alpha \max(y_i - \hat{y}_i, 0) + (1 - \alpha) \max(\hat{y}_i - y_i, 0)$$

The value of pinball loss is equivalent to half of `mean_absolute_error` when the quantile parameter `alpha` is set to 0.5.

Here is a small example of usage of the `mean_pinball_loss` function:

```
>>> from sklearn.metrics import mean_pinball_loss
>>> y_true = [1, 2, 3]
>>> mean_pinball_loss(y_true, [0, 2, 3], alpha=0.1)
0.033
>>> mean_pinball_loss(y_true, [1, 2, 4], alpha=0.1)
0.3
>>> mean_pinball_loss(y_true, [0, 2, 3], alpha=0.9)
0.3
>>> mean_pinball_loss(y_true, [1, 2, 4], alpha=0.9)
0.033
>>> mean_pinball_loss(y_true, y_true, alpha=0.1)
0.0
>>> mean_pinball_loss(y_true, y_true, alpha=0.9)
0.0
```

It is possible to build a scorer object with a specific choice of `alpha`:

```
>>> from sklearn.metrics import make_scorer
>>> mean_pinball_loss_95p = make_scorer(mean_pinball_loss, alpha=0.95)
```

Such a scorer can be used to evaluate the generalization performance of a quantile regressor via cross-validation:

```
>>> from sklearn.datasets import make_regression
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.ensemble import GradientBoostingRegressor
>>>
>>> X, y = make_regression(n_samples=100, random_state=0)
>>> estimator = GradientBoostingRegressor(
...     loss="quantile",
```

```
...     alpha=0.95,
...     random_state=0,
... )
>>> cross_val_score(estimator, X, y, cv=5, scoring=mean_pinball_loss_95p)
array([14.3,  9.8, 23.9,  9.4, 10.8])
```

It is also possible to build scorer objects for hyper-parameter tuning. The sign of the loss must be switched to ensure that greater means better as explained in the example linked below.

Examples

- See sphx_glr_auto_examples_ensemble_plot_gradient_boosting_quantile.py for an example of using the pinball loss to evaluate and tune the hyper-parameters of quantile regression models on data with non-symmetric noise and outliers.

D^2 score

The D^2 score computes the fraction of deviance explained. It is a generalization of R^2, where the squared error is generalized and replaced by a deviance of choice $\text{dev}(y, \hat{y})$ (e.g., Tweedie, pinball or mean absolute error). D^2 is a form of a *skill score*. It is calculated as

$$D^2(y, \hat{y}) = 1 - \frac{\text{dev}(y, \hat{y})}{\text{dev}(y, y_{\text{null}})} .$$

Where y_{null} is the optimal prediction of an intercept-only model (e.g., the mean of y_true for the Tweedie case, the median for absolute error and the alpha-quantile for pinball loss).

Like R^2, the best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts y_{null}, disregarding the input features, would get a D^2 score of 0.0.

D^2 Tweedie score
The `d2_tweedie_score` function implements the special case of D^2 where $\text{dev}(y, \hat{y})$ is the Tweedie deviance, see mean_tweedie_deviance. It is also known as D^2 Tweedie and is related to McFadden's likelihood ratio index.
The argument `power` defines the Tweedie power as for
A scorer object with a specific choice of `power` can be built by:

```
>>> from sklearn.metrics import d2_tweedie_score, make_scorer
>>> d2_tweedie_score_15 = make_scorer(d2_tweedie_score, power=1.5)
```

D^2 pinball score
The `d2_pinball_score` function implements the special case of D^2 with the pinball loss, see pinball_loss, i.e.:

$$\text{dev}(y, \hat{y}) = \text{pinball}(y, \hat{y}).$$

The argument `alpha` defines the slope of the pinball loss as for quantile level `alpha` for which the pinball loss and also D^2 are optimal. Note that for alpha=0.5 (the default) `d2_pinball_score` equals `d2_absolute_error_score`.
A scorer object with a specific choice of `alpha` can be built by:

```
>>> from sklearn.metrics import d2_pinball_score, make_scorer
>>> d2_pinball_score_08 = make_scorer(d2_pinball_score, alpha=0.8)
```

D² absolute error score
The `d2_absolute_error_score` function implements the special case of the mean_absolute_error:

$$\text{dev}(y, \hat{y}) = \text{MAE}(y, \hat{y}).$$

Here are some usage examples of the `d2_absolute_error_score` function:

```
>>> from sklearn.metrics import d2_absolute_error_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> d2_absolute_error_score(y_true, y_pred)
0.764
>>> y_true = [1, 2, 3]
>>> y_pred = [1, 2, 3]
>>> d2_absolute_error_score(y_true, y_pred)
1.0
>>> y_true = [1, 2, 3]
>>> y_pred = [2, 2, 2]
>>> d2_absolute_error_score(y_true, y_pred)
0.0
```

Visual evaluation of regression models

Among methods to assess the quality of regression models, scikit-learn provides the `PredictionErrorDisplay` class. It allows to visually inspect the prediction errors of a model in two different manners.

The plot on the left shows the actual values vs predicted values. For a noise-free regression task aiming to predict the (conditional) expectation of y, a perfect regression model would display data points on the diagonal defined by predicted equal to actual values. The further away from this optimal line, the larger the error of the model. In a more realistic setting with irreducible noise, that is, when not all the variations of y can be explained by features in X, then the best model would lead to a cloud of points densely arranged around the diagonal.

Note that the above only holds when the predicted values is the expected value of y given X. This is typically the case for regression models that minimize the mean squared error objective function or more generally the mean Tweedie deviance for any value of its ”power” parameter.

When plotting the predictions of an estimator that predicts a quantile of y given X, e.g. `QuantileRegressor` or any other model minimizing the pinball loss, a fraction of the points are either expected to lie above or below the diagonal depending on the estimated quantile level.

All in all, while intuitive to read, this plot does not really inform us on what to do to obtain a better model.

The right-hand side plot shows the residuals (i.e. the difference between the actual and the predicted values) vs. the predicted values.

This plot makes it easier to visualize if the residuals follow and homoscedastic or heteroschedastic distribution.

In particular, if the true distribution of y|X is Poisson or Gamma distributed, it is expected that the variance of the residuals of the optimal model would grow with the predicted value of E[y|X] (either linearly for Poisson or quadratically for Gamma).

When fitting a linear least squares regression model (see `LinearRegression` and `Ridge`), we can use this plot to check if some of the model assumptions are met, in particular that the residuals

should be uncorrelated, their expected value should be null and that their variance should be constant (homoschedasticity).

If this is not the case, and in particular if the residuals plot show some banana-shaped structure, this is a hint that the model is likely misspecified and that non-linear feature engineering or switching to a non-linear regression model might be useful.

Refer to the example below to see a model evaluation that makes use of this display.

Examples

- See sphx_glr_auto_examples_compose_plot_transformed_target.py for an example on how to use `PredictionErrorDisplay` to visualize the prediction quality improvement of a regression model obtained by transforming the target before learning.

Clustering metrics

The `sklearn.metrics` module implements several loss, score, and utility functions to measure clustering performance. For more information see the clustering_evaluation section for instance clustering, and biclustering_evaluation for biclustering.

Dummy estimators

When doing supervised learning, a simple sanity check consists of comparing one's estimator against simple rules of thumb. `DummyClassifier` implements several such simple strategies for classification:

- `stratified` generates random predictions by respecting the training set class distribution.
- `most_frequent` always predicts the most frequent label in the training set.
- `prior` always predicts the class that maximizes the class prior (like `most_frequent`) and `predict_proba` returns the class prior.
- `uniform` generates predictions uniformly at random.
- **`constant` always predicts a constant label that is provided by the user.** A major motivation of this method is F1-scoring, when the positive class is in the minority.

Note that with all these strategies, the `predict` method completely ignores the input data!

To illustrate `DummyClassifier`, first let's create an imbalanced dataset:

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import train_test_split
>>> X, y = load_iris(return_X_y=True)
>>> y[y != 1] = -1
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

Next, let's compare the accuracy of `SVC` and `most_frequent`:

```
>>> from sklearn.dummy import DummyClassifier
>>> from sklearn.svm import SVC
>>> clf = SVC(kernel='linear', C=1).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.63
>>> clf = DummyClassifier(strategy='most_frequent', random_state=0)
>>> clf.fit(X_train, y_train)
DummyClassifier(random_state=0, strategy='most_frequent')
>>> clf.score(X_test, y_test)
```

```
0.579
```

We see that `SVC` doesn't do much better than a dummy classifier. Now, let's change the kernel:

```
>>> clf = SVC(kernel='rbf', C=1).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.94
```

We see that the accuracy was boosted to almost 100%. A cross validation strategy is recommended for a better estimate of the accuracy, if it is not too CPU costly. For more information see the cross_validation section. Moreover if you want to optimize over the parameter space, it is highly recommended to use an appropriate methodology; see the grid_search section for details.

More generally, when the accuracy of a classifier is too close to random, it probably means that something went wrong: features are not helpful, a hyperparameter is not correctly tuned, the classifier is suffering from class imbalance, etc...

`DummyRegressor` also implements four simple rules of thumb for regression:

- `mean` always predicts the mean of the training targets.
- `median` always predicts the median of the training targets.
- `quantile` always predicts a user provided quantile of the training targets.
- `constant` always predicts a constant value that is provided by the user.

In all these strategies, the `predict` method completely ignores the input data.

Brier1950 G. Brier (1950). "Verification of forecasts expressed in terms of probability" <10.1175/1520-0493(1950)078%3C0001:VOFEIT%3E2.0.CO;2>. Monthly Weather Review 78(1), 1-3

Chen2024 W. Chen, C. Miao, Z. Zhang, C.S. Fung, R. Wang, Y. Chen, Y. Qian, L. Cheng, K.Y. Yip, S.K Tsui, Q. Cao, Commonly used software tools produce conflicting and overly-optimistic AUPRC values, Genome Biology 2024.

Davis2006 J. Davis, M. Goadrich, The Relationship Between Precision-Recall and ROC Curves, ICML 2006.

Everingham2010 M. Everingham, L. Van Gool, C.K.I. Williams, J. Winn, A. Zisserman, The Pascal Visual Object Classes (VOC) Challenge, IJCV 2010.

F2001 Fawcett, T., 2001. Using rule sets to maximize ROC performance In Data Mining, 2001. Proceedings IEEE International Conference, pp. 131-138.

F2006 Fawcett, T., 2006. An introduction to ROC analysis. Pattern Recognition Letters, 27(8), pp. 861-874.

FC2009 Ferri, Cèsar & Hernandez-Orallo, Jose & Modroiu, R. (2009). An Experimental Comparison of Performance Measures for Classification. Pattern Recognition Letters. 30. 27-38.

Fissler2022 T. Fissler, C. Lorentzen and M. Mayer. Model Comparison and Calibration Assessment: User Guide for Consistent Scoring Functions in Machine Learning and Actuarial Practice. <2202.12780>

Flach2015 P.A. Flach, M. Kull, Precision-Recall-Gain Curves: PR Analysis Done Right, NIPS 2015.

Gneiting2007 T. Gneiting and A. E. Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation <10.1198/016214506000001437> In: Journal of the American Statistical Association 102 (2007), pp. 359– 378. link to pdf

Gneiting2009 T. Gneiting. Making and Evaluating Point Forecasts <0912.0902> Journal of the American Statistical Association 106 (2009): 746 - 762.

Gneiting2014 T. Gneiting and M. Katzfuss. Probabilistic Forecasting <10.1146/annurev-statistics-062713-085831>. In: Annual Review of Statistics and Its Application 1.1 (2014), pp. 125–151.

Guyon2015 I. Guyon, K. Bennett, G. Cawley, H.J. Escalante, S. Escalera, T.K. Ho, N. Macià, B. Ray, M. Saeed, A.R. Statnikov, E. Viegas, Design of the 2015 ChaLearn AutoML Challenge, IJCNN 2015.

HT2001 Hand, D.J. and Till, R.J., (2001). A simple generalisation of the area under the ROC curve for multiple class classification problems. Machine learning, 45(2), pp. 171-186.

Kelleher2015 John. D. Kelleher, Brian Mac Namee, Aoife D'Arcy, Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies, 2015.

Manning2008 C.D. Manning, P. Raghavan, H. Schütze, Introduction to Information Retrieval, 2008.

Martin1997 A. Martin, G. Doddington, T. Kamm, M. Ordowski, and M. Przybocki, The DET Curve in Assessment of Detection Task Performance, NIST 1997.

Mosley2013 L. Mosley, A balanced approach to the multi-class imbalance problem, IJCV 2010.

Murphy1973 Allan H. Murphy (1973). "A New Vector Partition of the Probability Score" <10.1175/1520-0450(1973)012%3C0595:ANVPOT%3E2.0.CO;2> Journal of Applied Meteorology and Climatology, 12(4), 595-600

Navratil2007 J. Navratil and D. Klusacek, "On Linear DETs", 2007 IEEE International Conference on Acoustics, Speech and Signal Processing - ICASSP '07, Honolulu, HI, 2007, pp. IV-229-IV-232.

OB2019 Opitz, J., & Burst, S. (2019). "Macro f1 and macro f1." <1911.03347>

PD2000 Provost, F., Domingos, P. (2000). Well-trained PETs: Improving probability estimation trees (Section 6.2), CeDER Working Paper #IS-00-04, Stern School of Business, New York University.

Urbanowicz2015 Urbanowicz R.J., Moore, J.H. ExSTraCS 2.0: description and evaluation of a scalable learning classifier system <10.1007/s12065-015-0128-8>, Evol. Intel. (2015) 8: 89.

WikipediaDET2017 Wikipedia contributors. Detection error tradeoff. Wikipedia, The Free Encyclopedia. September 4, 2017, 23:33 UTC. Available at: https://en.wikipedia.org/w/index.php?title=Detection_error_tradeoff&oldid=798982054. Accessed February 19, 2018.

WikipediaMCC2021 Wikipedia contributors. Phi coefficient. Wikipedia, The Free Encyclopedia. April 21, 2021, 12:21 CEST. Available at: https://en.wikipedia.org/wiki/Phi_coefficient Accessed April 21, 2021.

Validation and learning curves

Source: *https://scikit-learn.org/stable/modules/learning_curve.html*

Every estimator has its advantages and drawbacks. Its generalization error can be decomposed in terms of bias, variance and noise. The **bias** of an estimator is its average error for different training sets. The **variance** of an estimator indicates how sensitive it is to varying training sets. Noise is a property of the data.

In the following plot, we see a function $f(x) = \cos(\frac{3}{2}\pi x)$ and some noisy samples from that function. We use three different estimators to fit the function: linear regression with polynomial features of degree 1, 4 and 15. We see that the first estimator can at best provide only a poor fit to the samples and the true function because it is too simple (high bias), the second estimator approximates it almost perfectly and the last estimator approximates the training data perfectly but does not fit the true function very well, i.e. it is very sensitive to varying training data (high variance).

Bias and variance are inherent properties of estimators and we usually have to select learning algorithms and hyperparameters so that both bias and variance are as low as possible (see Bias-variance dilemma). Another way to reduce the variance of a model is to use more training data. However, you should only collect more training data if the true function is too complex to be approximated by an estimator with a lower variance.

In the simple one-dimensional problem that we have seen in the example it is easy to see whether the estimator suffers from bias or variance. However, in high-dimensional spaces, models can become very difficult to visualize. For this reason, it is often helpful to use the tools described below.

Examples

- sphx_glr_auto_examples_model_selection_plot_underfitting_overfitting.py
- sphx_glr_auto_examples_model_selection_plot_train_error_vs_test_error.py
- sphx_glr_auto_examples_model_selection_plot_learning_curve.py

Validation curve

To validate a model we need a scoring function (see model_evaluation), for example accuracy for classifiers. The proper way of choosing multiple hyperparameters of an estimator is of course grid search or similar methods (see grid_search) that select the hyperparameter with the maximum score on a validation set or multiple validation sets. Note that if we optimize the hyperparameters based on a validation score the validation score is biased and not a good estimate of the generalization any longer. To get a proper estimate of the generalization we have to compute the score on another test set.

However, it is sometimes helpful to plot the influence of a single hyperparameter on the training score and the validation score to find out whether the estimator is overfitting or underfitting for some hyperparameter values.

The function `validation_curve` can help in this case:

```
>>> import numpy as np
>>> from sklearn.model_selection import validation_curve
>>> from sklearn.datasets import load_iris
>>> from sklearn.svm import SVC
```

```
>>> np.random.seed(0)
>>> X, y = load_iris(return_X_y=True)
>>> indices = np.arange(y.shape[0])
>>> np.random.shuffle(indices)
>>> X, y = X[indices], y[indices]

>>> train_scores, valid_scores = validation_curve(
...     SVC(kernel="linear"), X, y, param_name="C", param_range=np.logspace(-7, 3, 3),
... )
>>> train_scores
array([[0.90, 0.94, 0.91, 0.89, 0.92],
       [0.9 , 0.92, 0.93, 0.92, 0.93],
       [0.97, 1   , 0.98, 0.97, 0.99]])
>>> valid_scores
array([[0.9, 0.9 , 0.9 , 0.96, 0.9 ],
       [0.9, 0.83, 0.96, 0.96, 0.93],
       [1. , 0.93, 1   , 1   , 0.9 ]])
```

If you intend to plot the validation curves only, the class `ValidationCurveDisplay` is more direct than using matplotlib manually on the results of a call to `validation_curve`. You can use the method `from_estimator` similarly to `validation_curve` to generate and plot the validation curve:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import ValidationCurveDisplay
from sklearn.svm import SVC
from sklearn.utils import shuffle
X, y = load_iris(return_X_y=True)
X, y = shuffle(X, y, random_state=0)
ValidationCurveDisplay.from_estimator(
   SVC(kernel="linear"), X, y, param_name="C", param_range=np.logspace(-7, 3, 10)
)
```

If the training score and the validation score are both low, the estimator will be underfitting. If the training score is high and the validation score is low, the estimator is overfitting and otherwise it is working very well. A low training score and a high validation score is usually not possible.

Learning curve

A learning curve shows the validation and training score of an estimator for varying numbers of training samples. It is a tool to find out how much we benefit from adding more training data and whether the estimator suffers more from a variance error or a bias error. Consider the following example where we plot the learning curve of a naive Bayes classifier and an SVM.

For the naive Bayes, both the validation score and the training score converge to a value that is quite low with increasing size of the training set. Thus, we will probably not benefit much from more training data.

In contrast, for small amounts of data, the training score of the SVM is much greater than the validation score. Adding more training samples will most likely increase generalization.

We can use the function `learning_curve` to generate the values that are required to plot such a learning curve (number of samples that have been used, the average scores on the training sets and the average scores on the validation sets):

```
>>> from sklearn.model_selection import learning_curve
>>> from sklearn.svm import SVC
```

```
>>> train_sizes, train_scores, valid_scores = learning_curve(
...     SVC(kernel='linear'), X, y, train_sizes=[50, 80, 110], cv=5)
>>> train_sizes
array([ 50, 80, 110])
>>> train_scores
array([[0.98, 0.98 , 0.98, 0.98, 0.98],
       [0.98, 1.   , 0.98, 0.98, 0.98],
       [0.98, 1.   , 0.98, 0.98, 0.99]])
>>> valid_scores
array([[1. ,  0.93,  1. ,  1. ,  0.96],
       [1. ,  0.96,  1. ,  1. ,  0.96],
       [1. ,  0.96,  1. ,  1. ,  0.96]])
```

If you intend to plot the learning curves only, the class `LearningCurveDisplay` will be easier to use. You can use the method `from_estimator` similarly to `learning_curve` to generate and plot the learning curve:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import LearningCurveDisplay
from sklearn.svm import SVC
from sklearn.utils import shuffle
X, y = load_iris(return_X_y=True)
X, y = shuffle(X, y, random_state=0)
LearningCurveDisplay.from_estimator(
   SVC(kernel="linear"), X, y, train_sizes=[50, 80, 110], cv=5)
```

Examples

- See sphx_glr_auto_examples_model_selection_plot_learning_curve.py for an example of using learning curves to check the scalability of a predictive model.

Linear models

Source: *https://scikit-learn.org/stable/modules/linear_model.html*

The following are a set of methods intended for regression in which the target value is expected to be a linear combination of the features. In mathematical notation, the predicted value $\hat{y}$ can be written as:

$$\hat{y}(w, x) = w_0 + w_1 x_1 + ... + w_p x_p$$

Across the module, we designate the vector $w = (w_1, ..., w_p)$ as `coef_` and w_0 as `intercept_`.

To perform classification with generalized linear models, see Logistic_regression.

Ordinary Least Squares

`LinearRegression` fits a linear model with coefficients $w = (w_1, ..., w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation. Mathematically it solves a problem of the form:

$$\min_w ||Xw - y||_2^2$$

`LinearRegression` takes in its `fit` method arguments `X`, `y`, `sample_weight` and stores the coefficients w of the linear model in its `coef_` and `intercept_` attributes:

```
>>> from sklearn import linear_model
>>> reg = linear_model.LinearRegression()
>>> reg.fit([[0, 0], [1, 1], [2, 2]], [0, 1, 2])
LinearRegression()
>>> reg.coef_
array([0.5, 0.5])
>>> reg.intercept_
0.0
```

The coefficient estimates for Ordinary Least Squares rely on the independence of the features. When features are correlated and some columns of the design matrix X have an approximately linear dependence, the design matrix becomes close to singular and as a result, the least-squares estimate becomes highly sensitive to random errors in the observed target, producing a large variance. This situation of *multicollinearity* can arise, for example, when data are collected without an experimental design.

Examples

- sphx_glr_auto_examples_linear_model_plot_ols_ridge.py

Non-Negative Least Squares

It is possible to constrain all the coefficients to be non-negative, which may be useful when they represent some physical or naturally non-negative quantities (e.g., frequency counts or prices of goods). `LinearRegression` accepts a boolean `positive` parameter: when set to True Non-Negative Least Squares are then applied.

Examples

- sphx_glr_auto_examples_linear_model_plot_nnls.py

Ordinary Least Squares Complexity

The least squares solution is computed using the singular value decomposition of X. If X is a matrix of shape (n_samples, n_features) this method has a cost of $O(n_{\text{samples}} n_{\text{features}}^2)$, assuming that $n_{\text{samples}} \geq n_{\text{features}}$.

Ridge regression and classification

Regression

`Ridge` regression addresses some of the problems of ordinary_least_squares by imposing a penalty on the size of the coefficients. The ridge coefficients minimize a penalized residual sum of squares:

$$\min_w ||Xw - y||_2^2 + \alpha ||w||_2^2$$

The complexity parameter $\alpha \geq 0$ controls the amount of shrinkage: the larger the value of α, the greater the amount of shrinkage and thus the coefficients become more robust to collinearity.

As with other linear models, `Ridge` will take in its `fit` method arrays `X`, `y` and will store the coefficients w of the linear model in its `coef_` member:

```
>>> from sklearn import linear_model
>>> reg = linear_model.Ridge(alpha=.5)
>>> reg.fit([[0, 0], [0, 0], [1, 1]], [0, .1, 1])
Ridge(alpha=0.5)
>>> reg.coef_
array([0.34545455, 0.34545455])
>>> reg.intercept_
np.float64(0.13636)
```

Note that the class `Ridge` allows for the user to specify that the solver be automatically chosen by setting solver="auto". When this option is specified, `Ridge` will choose between the "lbfgs", "cholesky", and "sparse_cg" solvers. `Ridge` will begin checking the conditions shown in the following table from top to bottom. If the condition is true, the corresponding solver is chosen.

Solver	Condition
'lbfgs'	The `positive=True` option is specified.
'cholesky'	The input array X is not sparse.
'sparse_cg'	None of the above conditions are fulfilled.

Examples

- sphx_glr_auto_examples_linear_model_plot_ols_ridge.py
- sphx_glr_auto_examples_linear_model_plot_ridge_path.py
- sphx_glr_auto_examples_inspection_plot_linear_model_coefficient_interpretation.py
- sphx_glr_auto_examples_linear_model_plot_ridge_coeffs.py

Classification

The `Ridge` regressor has a classifier variant: `RidgeClassifier`. This classifier first converts binary targets to `{-1, 1}` and then treats the problem as a regression task, optimizing the same objective as above. The predicted class corresponds to the sign of the regressor's prediction. For multiclass classification, the problem is treated as multi-output regression, and the predicted class corresponds to the output with the highest value.

It might seem questionable to use a (penalized) Least Squares loss to fit a classification model instead of the more traditional logistic or hinge losses. However, in practice, all those models can lead to similar cross-validation scores in terms of accuracy or precision/recall, while the penalized least squares loss used by the `RidgeClassifier` allows for a very different choice of the numerical solvers with distinct computational performance profiles.

The `RidgeClassifier` can be significantly faster than e.g. `LogisticRegression` with a high number of classes because it can compute the projection matrix $(X^T X)^{-1} X^T$ only once.

This classifier is sometimes referred to as a Least Squares Support Vector Machine with a linear kernel.

Examples

- sphx_glr_auto_examples_text_plot_document_classification_20newsgroups.py

Ridge Complexity

This method has the same order of complexity as ordinary_least_squares.

Setting the regularization parameter: leave-one-out Cross-Validation

`RidgeCV` and `RidgeClassifierCV` implement ridge regression/classification with built-in cross-validation of the alpha parameter. They work in the same way as `GridSearchCV` except that it defaults to efficient Leave-One-Out cross-validation. When using the default cross-validation, alpha cannot be 0 due to the formulation used to calculate Leave-One-Out error. See [RL2007] for details.

Usage example:

```
>>> import numpy as np
>>> from sklearn import linear_model
>>> reg = linear_model.RidgeCV(alphas=np.logspace(-6, 6, 13))
>>> reg.fit([[0, 0], [0, 0.1], [1, 1]], [0, -0.1, 1])
RidgeCV(alphas=array([1.e-06, 1.e-05, 1.e-04, 1.e-03, 1.e-02, 1.e-01, 1.e+00, 1.e+01,
      1.e+02, 1.e+03, 1.e+04, 1.e+05, 1.e+06]))
>>> reg.alpha_
np.float64(0.1)
```

Specifying the value of the cv attribute will trigger the use of cross-validation with `GridSearchCV`, for example cv=10 for 10-fold cross-validation, rather than Leave-One-Out Cross-Validation.

References

Lasso

The `Lasso` is a linear model that estimates sparse coefficients, i.e., it is able to set coefficients exactly to zero. It is useful in some contexts due to its tendency to prefer solutions with fewer non-zero coefficients, effectively reducing the number of features upon which the given solution

is dependent. For this reason, Lasso and its variants are fundamental to the field of compressed sensing. Under certain conditions, it can recover the exact set of non-zero coefficients (see sphx_glr_auto_examples_applications_plot_tomography_l1_reconstruction.py).

Mathematically, it consists of a linear model with an added regularization term. The objective function to minimize is:

$$\min_w P(w) = \frac{1}{2n_{\text{samples}}}||Xw - y||_2^2 + \alpha||w||_1$$

The lasso estimate thus solves the least-squares with added penalty $\alpha||w||_1$, where α is a constant and $||w||_1$ is the ℓ_1-norm of the coefficient vector.

The implementation in the class `Lasso` uses coordinate descent as the algorithm to fit the coefficients. See least_angle_regression for another implementation:

```
>>> from sklearn import linear_model
>>> reg = linear_model.Lasso(alpha=0.1)
>>> reg.fit([[0, 0], [1, 1]], [0, 1])
Lasso(alpha=0.1)
>>> reg.predict([[1, 1]])
array([0.8])
```

The function `lasso_path` is useful for lower-level tasks, as it computes the coefficients along the full path of possible values.

Examples

- sphx_glr_auto_examples_linear_model_plot_lasso_and_elasticnet.py
- sphx_glr_auto_examples_applications_plot_tomography_l1_reconstruction.py
- sphx_glr_auto_examples_inspection_plot_linear_model_coefficient_interpretation.py
- sphx_glr_auto_examples_linear_model_plot_lasso_model_selection.py

Note
Feature selection with Lasso
As the Lasso regression yields sparse models, it can thus be used to perform feature selection, as detailed in

References
The following references explain the origin of the Lasso as well as properties of the Lasso problem and the duality gap computation used for convergence control.

- `Robert Tibshirani. (1996) Regression Shrinkage and Selection Via the Lasso. J. R. Stat. Soc. Ser. B Stat. Methodol., 58(1):267-288 <10.1111/j.2517-6161.1996.tb02080.x>`
- ”An Interior-Point Method for Large-Scale L1-Regularized Least Squares,” S. J. Kim, K. Koh, M. Lustig, S. Boyd and D. Gorinevsky, in IEEE Journal of Selected Topics in Signal Processing, 2007 (Paper)

Coordinate Descent with Gap Safe Screening Rules

Coordinate descent (CD) is a strategy to solve a minimization problem that considers a single feature j at a time. This way, the optimization problem is reduced to a 1-dimensional problem which is easier to solve:

$$\min_{w_j} \frac{1}{2n_{\text{samples}}}||x_j w_j + X_{-j}w_{-j} - y||_2^2 + \alpha|w_j|$$

with index $-j$ meaning all features but j. The solution is

$$w_j = \frac{S(x_j^T(y - X_{-j}w_{-j}), \alpha)}{||x_j||_2^2}$$

with the soft-thresholding function $S(z, \alpha) = \text{sign}(z)\max(0, |z| - \alpha)$. Note that the soft-thresholding function is exactly zero whenever $\alpha \geq |z|$. The CD solver then loops over the features either in a cycle, picking one feature after the other in the order given by X (selection="cyclic"), or by randomly picking features (selection="random"). It stops if the duality gap is smaller than the provided tolerance tol.

Mathematical details
The duality gap $G(w, v)$ is an upper bound of the difference between the current primal objective function of the Lasso, $P(w)$, and its minimum

$$D(v) = \frac{1}{2n_{\text{samples}}}(y^T v - ||v||_2^2)$$

subject to $v \in ||X^T v||_\infty \leq n_{\text{samples}}\alpha$. At optimum, the duality gap is zero, $G(w^\star, v^\star) = 0$ (a property called strong duality). With (scaled) dual variable $v = cr$, current residual $r = y - Xw$ and dual scaling

$$c = \begin{cases} 1, & ||X^T r||_\infty \leq n_{\text{samples}}\alpha, \\ \frac{n_{\text{samples}}\alpha}{||X^T r||_\infty}, & \text{otherwise} \end{cases}$$

the stopping criterion is

$$\text{tol}\frac{||y||_2^2}{n_{\text{samples}}} < G(w, cr)\,.$$

A clever method to speedup the coordinate descent algorithm is to screen features such that at optimum $w_j = 0$. Gap safe screening rules are such a tool. Anywhere during the optimization algorithm, they can tell which feature we can safely exclude, i.e., set to zero with certainty.

References
The first reference explains the coordinate descent solver used in scikit-learn, the others treat gap safe screening rules.

- Friedman, Hastie & Tibshirani. (2010). Regularization Path For Generalized linear Models by Coordinate Descent. J Stat Softw 33(1), 1-22 <10.18637/jss.v033.i01>
- O. Fercoq, A. Gramfort, J. Salmon. (2015). Mind the duality gap: safer rules for the Lasso. Proceedings of Machine Learning Research 37:333-342, 2015. <1505.03410>
- E. Ndiaye, O. Fercoq, A. Gramfort, J. Salmon. (2017). Gap Safe Screening Rules for Sparsity Enforcing Penalties. Journal of Machine Learning Research 18(128):1-33, 2017. <1611.05780>

Setting regularization parameter

The `alpha` parameter controls the degree of sparsity of the estimated coefficients.

Using cross-validation

scikit-learn exposes objects that set the Lasso `alpha` parameter by cross-validation: `LassoCV` and `LassoLarsCV`. `LassoLarsCV` is based on the least_angle_regression algorithm explained below.

For high-dimensional datasets with many collinear features, `LassoCV` is most often preferable. However, `LassoLarsCV` has the advantage of exploring more relevant values of alpha parameter, and if the number of samples is very small compared to the number of features, it is often faster than `LassoCV`.

target ../auto_examples/linear_model/plot_lasso_model_selection.html
scale 48%
target ../auto_examples/linear_model/plot_lasso_model_selection.html
scale 48%

Information-criteria based model selection

Alternatively, the estimator `LassoLarsIC` proposes to use the Akaike information criterion (AIC) and the Bayes Information criterion (BIC). It is a computationally cheaper alternative to find the optimal value of alpha as the regularization path is computed only once instead of k+1 times when using k-fold cross-validation.

Indeed, these criteria are computed on the in-sample training set. In short, they penalize the over-optimistic scores of the different Lasso models by their flexibility (cf. to "Mathematical details" section below).

However, such criteria need a proper estimation of the degrees of freedom of the solution, are derived for large samples (asymptotic results) and assume the correct model is candidates under investigation. They also tend to break when the problem is badly conditioned (e.g. more features than samples).

Examples

- sphx_glr_auto_examples_linear_model_plot_lasso_model_selection.py
- sphx_glr_auto_examples_linear_model_plot_lasso_lars_ic.py

AIC and BIC criteria

The definition of AIC (and thus BIC) might differ in the literature. In this section, we give more information regarding the criterion computed in scikit-learn.

Mathematical details
The AIC criterion is defined as:

$$AIC = -2\log(\hat{L}) + 2d$$

where $\hat{L}$ is the maximum likelihood of the model and freedom in the previous section).
The definition of BIC replaces the constant 2 by $\log(N)$:

$$BIC = -2\log(\hat{L}) + \log(N)d$$

where N is the number of samples.
For a linear Gaussian model, the maximum log-likelihood is defined as:

$$\log(\hat{L}) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{2\sigma^2}$$

where σ^2 is an estimate of the noise variance, targets, and n is the number of samples.
Plugging the maximum log-likelihood in the AIC formula yields:

$$AIC = n\log(2\pi\sigma^2) + \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sigma^2} + 2d$$

The first term of the above expression is sometimes discarded since it is a constant when σ^2 is provided. In addition, it is sometimes stated that the AIC is equivalent to the C_p statistic . In a strict sense, however, it is equivalent only up to some constant and a multiplicative factor.
At last, we mentioned above that σ^2 is an estimate of the noise variance. In `LassoLarsIC` when the parameter noise_variance is not provided (default), the noise variance is estimated via the unbiased estimator defined as:

$$\sigma^2 = \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n - p}$$

where p is the number of features and $\hat{y}_i$ is the predicted target using an ordinary least squares regression. Note, that this formula is valid only when n_samples > n_features.
References

Comparison with the regularization parameter of SVM

The equivalence between `alpha` and the regularization parameter of SVM, `C` is given by `alpha = 1 / C` or `alpha = 1 / (n_samples * C)`, depending on the estimator and the exact objective function optimized by the model.

Multi-task Lasso

The `MultiTaskLasso` is a linear model that estimates sparse coefficients for multiple regression problems jointly: `y` is a 2D array, of shape `(n_samples, n_tasks)`. The constraint is that the selected features are the same for all the regression problems, also called tasks.

The following figure compares the location of the non-zero entries in the coefficient matrix W obtained with a simple Lasso or a MultiTaskLasso. The Lasso estimates yield scattered non-zeros while the non-zeros of the MultiTaskLasso are full columns.

target ../auto_examples/linear_model/plot_multi_task_lasso_support.html
scale 48%
target ../auto_examples/linear_model/plot_multi_task_lasso_support.html
scale 48%

Examples

- sphx_glr_auto_examples_linear_model_plot_multi_task_lasso_support.py

Mathematical details
Mathematically, it consists of a linear model trained with a mixed The objective function to minimize is:

$$\min_W \frac{1}{2n_{\text{samples}}}||XW - Y||^2_{\text{Fro}} + \alpha||W||_{21}$$

where Fro indicates the Frobenius norm

$$||A||_{\text{Fro}} = \sqrt{\sum_{ij} a_{ij}^2}$$

and ℓ_1 ℓ_2 reads

$$||A||_{21} = \sum_i \sqrt{\sum_j a_{ij}^2}.$$

The implementation in the class `MultiTaskLasso` uses coordinate descent as the algorithm to fit the coefficients.

Elastic-Net

`ElasticNet` is a linear regression model trained with both ℓ_1 and ℓ_2-norm regularization of the coefficients. This combination allows for learning a sparse model where few of the weights are non-zero like `Lasso`, while still maintaining the regularization properties of `Ridge`. We control the convex combination of ℓ_1 and ℓ_2 using the `l1_ratio` parameter.

Elastic-net is useful when there are multiple features that are correlated with one another. Lasso is likely to pick one of these at random, while elastic-net is likely to pick both.

A practical advantage of trading-off between Lasso and Ridge is that it allows Elastic-Net to inherit some of Ridge's stability under rotation.

The objective function to minimize is in this case

$$\min_w \frac{1}{2n_{\text{samples}}}||Xw - y||_2^2 + \alpha\rho||w||_1 + \frac{\alpha(1-\rho)}{2}||w||_2^2$$

The class `ElasticNetCV` can be used to set the parameters `alpha` (α) and `l1_ratio` (ρ) by cross-validation.

Examples

- sphx_glr_auto_examples_linear_model_plot_lasso_and_elasticnet.py
- sphx_glr_auto_examples_linear_model_plot_lasso_lasso_lars_elasticnet_path.py
- sphx_glr_auto_examples_linear_model_plot_elastic_net_precomputed_gram_matrix_with_weighted_sam

References
The following two references explain the iterations used in the coordinate descent solver of scikit-learn, as well as the duality gap computation used for convergence control.

- ”Regularization Path For Generalized linear Models by Coordinate Descent”, Friedman, Hastie & Tibshirani, J Stat Softw, 2010 (Paper).
- ”An Interior-Point Method for Large-Scale L1-Regularized Least Squares,” S. J. Kim, K. Koh, M. Lustig, S. Boyd and D. Gorinevsky, in IEEE Journal of Selected Topics in Signal Processing, 2007 (Paper)

Multi-task Elastic-Net

The `MultiTaskElasticNet` is an elastic-net model that estimates sparse coefficients for multiple regression problems jointly: `Y` is a 2D array of shape `(n_samples, n_tasks)`. The constraint is that the selected features are the same for all the regression problems, also called tasks.

Mathematically, it consists of a linear model trained with a mixed ℓ_1 ℓ_2-norm and ℓ_2-norm for regularization. The objective function to minimize is:

$$\min_W \frac{1}{2n_{\text{samples}}}||XW - Y||_{\text{Fro}}^2 + \alpha\rho||W||_{21} + \frac{\alpha(1-\rho)}{2}||W||_{\text{Fro}}^2$$

The implementation in the class `MultiTaskElasticNet` uses coordinate descent as the algorithm to fit the coefficients.

The class `MultiTaskElasticNetCV` can be used to set the parameters `alpha` (α) and `l1_ratio` (ρ) by cross-validation.

Least Angle Regression

Least-angle regression (LARS) is a regression algorithm for high-dimensional data, developed by Bradley Efron, Trevor Hastie, Iain Johnstone and Robert Tibshirani. LARS is similar to forward stepwise regression. At each step, it finds the feature most correlated with the target. When there are multiple features having equal correlation, instead of continuing along the same feature, it proceeds in a direction equiangular between the features.

The advantages of LARS are:

- It is numerically efficient in contexts where the number of features is significantly greater than the number of samples.
- It is computationally just as fast as forward selection and has the same order of complexity as ordinary least squares.
- It produces a full piecewise linear solution path, which is useful in cross-validation or similar attempts to tune the model.
- If two features are almost equally correlated with the target, then their coefficients should increase at approximately the same rate. The algorithm thus behaves as intuition would expect, and also is more stable.
- It is easily modified to produce solutions for other estimators, like the Lasso.

The disadvantages of the LARS method include:

- Because LARS is based upon an iterative refitting of the residuals, it would appear to be especially sensitive to the effects of noise. This problem is discussed in detail by Weisberg in the discussion section of the Efron et al. (2004) Annals of Statistics article.

The LARS model can be used via the estimator `Lars`, or its low-level implementation `lars_path` or `lars_path_gram`.

LARS Lasso

`LassoLars` is a lasso model implemented using the LARS algorithm, and unlike the implementation based on coordinate descent, this yields the exact solution, which is piecewise linear as a function of the norm of its coefficients.

```
>>> from sklearn import linear_model
>>> reg = linear_model.LassoLars(alpha=.1)
>>> reg.fit([[0, 0], [1, 1]], [0, 1])
LassoLars(alpha=0.1)
>>> reg.coef_
array([0.6, 0.        ])
```

Examples

- sphx_glr_auto_examples_linear_model_plot_lasso_lasso_lars_elasticnet_path.py

The LARS algorithm provides the full path of the coefficients along the regularization parameter almost for free, thus a common operation is to retrieve the path with one of the functions `lars_path` or `lars_path_gram`.

Mathematical formulation
The algorithm is similar to forward stepwise regression, but instead of including features at each step, the estimated coefficients are increased in a direction equiangular to each one's correlations with the residual.
Instead of giving a vector result, the LARS solution consists of a curve denoting the solution for each value of the ℓ_1 norm of the parameter vector. The full coefficients path is stored in the array `coef_path_` of shape (n_features, max_features + 1). The first column is always zero.
References
- Original Algorithm is detailed in the paper Least Angle Regression by Hastie et al.

Orthogonal Matching Pursuit (OMP)

`OrthogonalMatchingPursuit` and `orthogonal_mp` implement the OMP algorithm for approximating the fit of a linear model with constraints imposed on the number of non-zero coefficients (i.e. the ℓ_0 pseudo-norm).

Being a forward feature selection method like least_angle_regression, orthogonal matching pursuit can approximate the optimum solution vector with a fixed number of non-zero elements:

$$\arg\min_w ||y - Xw||_2^2 \text{ subject to } ||w||_0 \leq n_{\text{nonzero_coefs}}$$

Alternatively, orthogonal matching pursuit can target a specific error instead of a specific number of non-zero coefficients. This can be expressed as:

$$\arg\min_w ||w||_0 \text{ subject to } ||y - Xw||_2^2 \leq \text{tol}$$

OMP is based on a greedy algorithm that includes at each step the atom most highly correlated with the current residual. It is similar to the simpler matching pursuit (MP) method, but better in that at each iteration, the residual is recomputed using an orthogonal projection on the space of the previously chosen dictionary elements.

Examples

- sphx_glr_auto_examples_linear_model_plot_omp.py

References
- https://www.cs.technion.ac.il/~ronrubin/Publications/KSVD-OMP-v2.pdf
- Matching pursuits with time-frequency dictionaries, S. G. Mallat, Z. Zhang, 1993.

Bayesian Regression

Bayesian regression techniques can be used to include regularization parameters in the estimation procedure: the regularization parameter is not set in a hard sense but tuned to the data at hand.

This can be done by introducing uninformative priors over the hyper parameters of the model. The ℓ_2 regularization used in ridge_regression is equivalent to finding a maximum a posteriori estimation under a Gaussian prior over the coefficients w with precision λ^{-1}. Instead of setting lambda manually, it is possible to treat it as a random variable to be estimated from the data.

To obtain a fully probabilistic model, the output y is assumed to be Gaussian distributed around Xw:

$$p(y|X, w, \alpha) = \mathcal{N}(y|Xw, \alpha^{-1})$$

where α is again treated as a random variable that is to be estimated from the data.

The advantages of Bayesian Regression are:

- It adapts to the data at hand.
- It can be used to include regularization parameters in the estimation procedure.

The disadvantages of Bayesian regression include:

- Inference of the model can be time consuming.

References
- A good introduction to Bayesian methods is given in C. Bishop: Pattern Recognition and Machine Learning.
- Original Algorithm is detailed in the book Bayesian learning for neural networks by Radford M. Neal.

Bayesian Ridge Regression

`BayesianRidge` estimates a probabilistic model of the regression problem as described above. The prior for the coefficient w is given by a spherical Gaussian:

$$p(w|\lambda) = \mathcal{N}(w|0, \lambda^{-1}\mathbf{I}_p)$$

The priors over α and λ are chosen to be gamma distributions, the conjugate prior for the precision of the Gaussian. The resulting model is called *Bayesian Ridge Regression*, and is similar to the classical `Ridge`.

The parameters w, α and λ are estimated jointly during the fit of the model, the regularization parameters α and λ being estimated by maximizing the *log marginal likelihood*. The scikit-learn implementation is based on the algorithm described in Appendix A of (Tipping, 2001) where the update of the parameters α and λ is done as suggested in (MacKay, 1992). The initial value of the maximization procedure can be set with the hyperparameters `alpha_init` and `lambda_init`.

There are four more hyperparameters, α_1, α_2, λ_1 and λ_2 of the gamma prior distributions over α and λ. These are usually chosen to be *non-informative*. By default $\alpha_1 = \alpha_2 = \lambda_1 = \lambda_2 = 10^{-6}$.

Bayesian Ridge Regression is used for regression:

```
>>> from sklearn import linear_model
>>> X = [[0., 0.], [1., 1.], [2., 2.], [3., 3.]]
>>> Y = [0., 1., 2., 3.]
>>> reg = linear_model.BayesianRidge()
>>> reg.fit(X, Y)
BayesianRidge()
```

After being fitted, the model can then be used to predict new values:

```
>>> reg.predict([[1, 0.]])
array([0.50000013])
```

The coefficients w of the model can be accessed:

```
>>> reg.coef_
array([0.49999993, 0.49999993])
```

Due to the Bayesian framework, the weights found are slightly different from the ones found by ordinary_least_squares. However, Bayesian Ridge Regression is more robust to ill-posed problems.

Examples

- sphx_glr_auto_examples_linear_model_plot_bayesian_ridge_curvefit.py

References
- Section 3.3 in Christopher M. Bishop: Pattern Recognition and Machine Learning, 2006
- David J. C. MacKay, Bayesian Interpolation, 1992.
- Michael E. Tipping, Sparse Bayesian Learning and the Relevance Vector Machine, 2001.

Automatic Relevance Determination - ARD

The Automatic Relevance Determination (as being implemented in `ARDRegression`) is a kind of linear model which is very similar to the Bayesian Ridge Regression, but that leads to sparser coefficients w [3][4].

`ARDRegression` poses a different prior over w: it drops the spherical Gaussian distribution for a centered elliptic Gaussian distribution. This means each coefficient w_i can itself be drawn from a Gaussian distribution, centered on zero and with a precision λ_i:

$$p(w|\lambda) = \mathcal{N}(w|0, A^{-1})$$

with A being a positive definite diagonal matrix and $\text{diag}(A) = \lambda = \{\lambda_1, ..., \lambda_p\}$.

In contrast to the Bayesian Ridge Regression, each coordinate of w_i has its own standard deviation $\frac{1}{\lambda_i}$. The prior over all λ_i is chosen to be the same gamma distribution given by the hyperparameters λ_1 and λ_2.

ARD is also known in the literature as *Sparse Bayesian Learning* and *Relevance Vector Machine*[5][6].

See sphx_glr_auto_examples_linear_model_plot_ard.py for a worked-out comparison between ARD and Bayesian Ridge Regression.

See sphx_glr_auto_examples_linear_model_plot_lasso_and_elasticnet.py for a comparison between various methods - Lasso, ARD and ElasticNet - on correlated data.

References

Logistic regression

The logistic regression is implemented in `LogisticRegression`. Despite its name, it is implemented as a linear model for classification rather than regression in terms of the scikit-learn/ML nomenclature. The logistic regression is also known in the literature as logit regression, maximum-entropy classification (MaxEnt) or the log-linear classifier. In this model, the probabilities describing the possible outcomes of a single trial are modeled using a logistic function.

[3] Christopher M. Bishop: Pattern Recognition and Machine Learning, Chapter 7.2.1
[4] David Wipf and Srikantan Nagarajan: A New View of Automatic Relevance Determination
[5] Michael E. Tipping: Sparse Bayesian Learning and the Relevance Vector Machine
[6] Tristan Fletcher: Relevance Vector Machines Explained

This implementation can fit binary, One-vs-Rest, or multinomial logistic regression with optional ℓ_1, ℓ_2 or Elastic-Net regularization.

Note
Regularization
Regularization is applied by default, which is common in machine learning but not in statistics. Another advantage of regularization is that it improves numerical stability. No regularization amounts to setting C to a very high value.

Note
Logistic Regression as a special case of the Generalized Linear Models (GLM)
Logistic regression is a special case of distribution and a Logit link. The numerical output of the logistic regression, which is the predicted probability, can be used as a classifier by applying a threshold (by default 0.5) to it. This is how it is implemented in scikit-learn, so it expects a categorical target, making the Logistic Regression a classifier.

Examples

- sphx_glr_auto_examples_linear_model_plot_logistic_l1_l2_sparsity.py
- sphx_glr_auto_examples_linear_model_plot_logistic_path.py
- sphx_glr_auto_examples_linear_model_plot_logistic_multinomial.py
- sphx_glr_auto_examples_linear_model_plot_sparse_logistic_regression_20newsgroups.py
- sphx_glr_auto_examples_linear_model_plot_sparse_logistic_regression_mnist.py
- sphx_glr_auto_examples_classification_plot_classification_probability.py

Binary Case

For notational ease, we assume that the target y_i takes values in the set $\{0, 1\}$ for data point i. Once fitted, the `predict_proba` method of `LogisticRegression` predicts the probability of the positive class $P(y_i = 1|X_i)$ as

$$\hat{p}(X_i) = \operatorname{expit}(X_i w + w_0) = \frac{1}{1 + \exp(-X_i w - w_0)}.$$

As an optimization problem, binary class logistic regression with regularization term $r(w)$ minimizes the following cost function:

$$\min_w \frac{1}{S} \sum_{i=1}^{n} s_i \left(-y_i \log(\hat{p}(X_i)) - (1 - y_i) \log(1 - \hat{p}(X_i))\right) + \frac{r(w)}{SC},$$

where s_i corresponds to the weights assigned by the user to a specific training sample (the vector s is formed by element-wise multiplication of the class weights and sample weights), and the sum $S = \sum_{i=1}^{n} s_i$.

We currently provide four choices for the regularization or penalty term $r(w)$ via the arguments C and 'l1_ratio':

penalty	$r(w)$
none (C=np.inf)	0
ℓ_1 (l1_ratio=1)	$\|w\|_1$
ℓ_2 (l1_ratio=0)	$\frac{1}{2}\|w\|_2^2 = \frac{1}{2} w^T w$
ElasticNet (0<l1_ratio<1)	$\frac{1-\rho}{2} w^T w + \rho \|w\|_1$

penalty	$r(w)$

For ElasticNet, ρ (which corresponds to the l1_ratio parameter) controls the strength of ℓ_1 regularization vs. ℓ_2 regularization. Elastic-Net is equivalent to ℓ_1 when $\rho = 1$ and equivalent to ℓ_2 when $\rho = 0$.

Note that the scale of the class weights and the sample weights will influence the optimization problem. For instance, multiplying the sample weights by a constant $b > 0$ is equivalent to multiplying the (inverse) regularization strength C by b.

Multinomial Case

The binary case can be extended to K classes leading to the multinomial logistic regression, see also log-linear model.

> Note
> It is possible to parameterize a K-class classification model using only $K-1$ weight vectors, leaving one class probability fully determined by the other class probabilities by leveraging the fact that all class probabilities must sum to one. We deliberately choose to overparameterize the model using K weight vectors for ease of implementation and to preserve the symmetrical inductive bias regarding ordering of classes, see. This effect becomes especially important when using regularization. The choice of overparameterization can be detrimental for unpenalized models since then the solution may not be unique, as shown in.

Mathematical details

Let $y_i \in \{1, \ldots, K\}$ be the label (ordinal) encoded target variable for observation i. Instead of a single coefficient vector, we now have a matrix of coefficients W where each row vector W_k corresponds to class

$$\hat{p}_k(X_i) = \frac{\exp(X_i W_k + W_{0,k})}{\sum_{l=0}^{K-1} \exp(X_i W_l + W_{0,l})}.$$

The objective for the optimization becomes

$$\min_W -\frac{1}{S} \sum_{i=1}^{n} \sum_{k=0}^{K-1} s_{ik} [y_i = k] \log(\hat{p}_k(X_i)) + \frac{r(W)}{SC},$$

where $[P]$ represents the Iverson bracket which evaluates to 0 if P is false, otherwise it evaluates to 1.

Again, s_{ik} are the weights assigned by the user (multiplication of sample weights and class weights) with their sum $S = \sum_{i=1}^{n} \sum_{k=0}^{K-1} s_{ik}$.

We currently provide four choices for the regularization or penalty term $r(W)$ via the arguments C and l1_ratio, where m is the number of features:

penalty	$r(W)$
none (C=np.inf)	0
ℓ_1 (l1_ratio=1)	$\|W\|_{1,1} = \sum_{i=1}^{m} \sum_{j=1}^{K} \|W_{i,j}\|$

penalty	$r(W)$
ℓ_2 (l1_ratio=0)	$\frac{1}{2}\|W\|_F^2 = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{K} W_{i,j}^2$
ElasticNet (0<l1_ratio<1)	$\frac{1-\rho}{2}\|W\|_F^2 + \rho\|W\|_{1,1}$

Solvers

The solvers implemented in the class `LogisticRegression` are ”lbfgs”, ”liblinear”, ”newton-cg”, ”newton-cholesky”, ”sag” and ”saga”:

The following table summarizes the penalties and multinomial multiclass supported by each solver:

Penalties	**Solvers** **'lbfgs'**	**'liblinear'**	**'newton-cg'**	**'newton-cholesky'**	**'sag'**	**'saga'**
L2 penalty	yes	yes	yes	yes	yes	yes
L1 penalty	no	yes	no	no	no	yes
Elastic-Net (L1 + L2)	no	no	no	no	no	yes
No penalty	yes	no	yes	yes	yes	yes
Multiclass support						
multinomial multiclass	yes	no	yes	yes	yes	yes
Behaviors						
Penalize the intercept (bad)	no	yes	no	no	no	no
Faster for large datasets	no	no	no	no	yes	yes
Robust to unscaled datasets	yes	yes	yes	yes	no	no

The ”lbfgs” solver is used by default for its robustness. For n_samples » n_features, ”newton-cholesky” is a good choice and can reach high precision (tiny tol values). For large datasets the ”saga” solver is usually faster (than ”lbfgs”), in particular for low precision (high tol). For large dataset, you may also consider using `SGDClassifier` with loss=”log_loss”, which might be even faster but requires more tuning.

References
The following references explain some of the solvers.

- For ”lbfgs”, see documentation of
- ”liblinear”
  - Fan, R. E., Chang, K. W., Hsieh, C. J., Wang, X. R., & Lin, C. J. (2008). LIBLINEAR: A library for large linear classification. the Journal of machine Learning research, 9, 1871-1874. https://www.jmlr.org/papers/volume9/fan08a/fan08a.pdf
  - Yu, H. F., Huang, F. L., & Lin, C. J. (2011). Dual coordinate descent methods for logistic regression and maximum entropy models. Machine Learning, 85(1), 41-75. <10.1007/s10994-010-5221-8>
- SAG -- Schmidt, M., Le Roux, N., & Bach, F. (2017). Minimizing finite sums with the stochastic average gradient. Mathematical Programming, 162(1), 83-112. <1309.2388>
- SAGA -- Defazio, A., Bach, F., & Lacoste-Julien, S. (2014). SAGA: A fast incremental

```
    gradient method with support for non-strongly convex    composite objectives. Advances
    in neural information processing systems, 27.    <1407.0202>
```

Differences between solvers

There might be a difference in the scores obtained between `LogisticRegression` with `solver=liblinear` or `LinearSVC` and the external liblinear library directly, when `fit_intercept=False` and the fit `coef_` (or) the data to be predicted are zeroes. This is because for the sample(s) with `decision_function` zero, `LogisticRegression` and `LinearSVC` predict the negative class, while liblinear predicts the positive class. Note that a model with `fit_intercept=False` and having many samples with `decision_function` zero, is likely to be an underfit, bad model and you are advised to set `fit_intercept=True` and increase the `intercept_scaling`.

Solvers' details

- The solver "liblinear" uses a coordinate descent (CD) algorithm, and relies on the excellent C++ LIBLINEAR library, which is shipped with scikit-learn. However, the CD algorithm implemented in liblinear cannot learn a true multinomial (multiclass) model. If you still want to use "liblinear" on multiclass problems, you can use a "one-vs-rest" scheme OneVsRestClassifier(LogisticRegression(solver="liblinear")), see `OneVsRestClassifier. Note that minimizing the multinomial loss is expected to give better calibrated results as compared to a "one-vs-rest" scheme. For $\ell_1$ regularization `sklearn.svm.l1_min_c` allows to calculate the lower bound for C in order to get a non "null" (all feature weights to zero) model.
- The "lbfgs", "newton-cg", "newton-cholesky" and "sag" solvers only support faster for some high-dimensional data. These solvers (and "saga") learn a true multinomial logistic regression model.
- The "sag" solver uses Stochastic Average Gradient descent. It is faster than other solvers for large datasets, when both the number of samples and the number of features are large.
- The "saga" solver is a variant of "sag" that also supports the non-smooth sparse multinomial logistic regression. It is also the only solver that supports Elastic-Net (0 < l1_ratio < 1).
- The "lbfgs" is an optimization algorithm that approximates the Broyden–Fletcher–Goldfarb–Shanno algorithm, which belongs to quasi-Newton methods. As such, it can deal with a wide range of different training data and is therefore the default solver. Its performance, however, suffers on poorly scaled datasets and on datasets with one-hot encoded categorical features with rare categories.
- The "newton-cholesky" solver is an exact Newton solver that calculates the Hessian matrix and solves the resulting linear system. It is a very good choice for n_samples » n_features and can reach high precision (tiny values of tol), but has a few shortcomings: Only ℓ_2 regularization is supported. Furthermore, because the Hessian matrix is explicitly computed, the memory usage has a quadratic dependency on n_features as well as on n_classes.

For a comparison of some of these solvers, see.
References

Note
Feature selection with sparse logistic regression
A logistic regression with ℓ_1 penalty yields sparse models, and can thus be used to perform feature selection, as detailed in

Note
P-value estimation

It is possible to obtain the p-values and confidence intervals for coefficients in cases of regression without penalization. The statsmodels package natively supports this. Within sklearn, one could use bootstrapping instead as well.

`LogisticRegressionCV` implements Logistic Regression with built-in cross-validation support, to find the optimal C and l1_ratio parameters according to the `scoring` attribute. The "newton-cg", "sag", "saga" and "lbfgs" solvers are found to be faster for high-dimensional dense data, due to warm-starting (see Glossary).

Generalized Linear Models

Generalized Linear Models (GLM) extend linear models in two ways [7]. First, the predicted values $\hat{y}$ are linked to a linear combination of the input variables X via an inverse link function h as

$$\hat{y}(w, X) = h(Xw).$$

Secondly, the squared loss function is replaced by the unit deviance d of a distribution in the exponential family (or more precisely, a reproductive exponential dispersion model (EDM)[8]).

The minimization problem becomes:

$$\min_{w} \frac{1}{2n_{\text{samples}}} \sum_i d(y_i, \hat{y}_i) + \frac{\alpha}{2}||w||_2^2,$$

where α is the L2 regularization penalty. When sample weights are provided, the average becomes a weighted average.

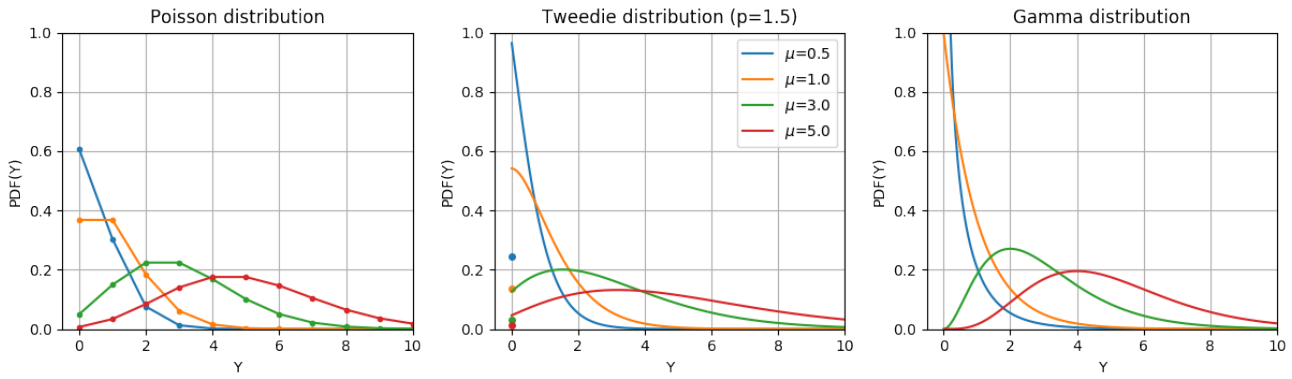
The following table lists some specific EDMs and their unit deviance :

Distribution	Target Domain	Unit Deviance $d(y, \hat{y})$
Normal	$y \in (-\infty, \infty)$	$(y - \hat{y})^2$
Bernoulli	$y \in \{0, 1\}$	$2(y \log \frac{y}{\hat{y}} + (1 - y) \log \frac{1-y}{1-\hat{y}})$
Categorical	$y \in \{0, 1, ..., k\}$	$2 \sum_{i \in \{0,1,...,k\}} I(y = i) y_{\text{i}} \log \frac{I(y=i)}{I(\hat{y}=i)}$
Poisson	$y \in [0, \infty)$	$2(y \log \frac{y}{\hat{y}} - y + \hat{y})$
Gamma	$y \in (0, \infty)$	$2(\log \frac{\hat{y}}{y} + \frac{y}{\hat{y}} - 1)$
Inverse Gaussian	$y \in (0, \infty)$	$\frac{(y-\hat{y})^2}{y\hat{y}^2}$

The Probability Density Functions (PDF) of these distributions are illustrated in the following figure,

[7] McCullagh, Peter; Nelder, John (1989). Generalized Linear Models, Second Edition. Boca Raton: Chapman and Hall/CRC. ISBN 0-412-31760-5.

[8] Jørgensen, B. (1992). The theory of exponential dispersion models and analysis of deviance. Monografias de matemática, no. 51. See also Exponential dispersion model.



PDF of a random variable Y following Poisson, Tweedie (power=1.5) and Gamma distributions with different mean values (μ). Observe the point mass at $Y = 0$ for the Poisson distribution and the Tweedie (power=1.5) distribution, but not for the Gamma distribution which has a strictly positive target domain.

The Bernoulli distribution is a discrete probability distribution modelling a Bernoulli trial - an event that has only two mutually exclusive outcomes. The Categorical distribution is a generalization of the Bernoulli distribution for a categorical random variable. While a random variable in a Bernoulli distribution has two possible outcomes, a Categorical random variable can take on one of K possible categories, with the probability of each category specified separately.

The choice of the distribution depends on the problem at hand:

- If the target values y are counts (non-negative integer valued) or relative frequencies (non-negative), you might use a Poisson distribution with a log-link.
- If the target values are positive valued and skewed, you might try a Gamma distribution with a log-link.
- If the target values seem to be heavier tailed than a Gamma distribution, you might try an Inverse Gaussian distribution (or even higher variance powers of the Tweedie family).
- If the target values y are probabilities, you can use the Bernoulli distribution. The Bernoulli distribution with a logit link can be used for binary classification. The Categorical distribution with a softmax link can be used for multiclass classification.

Examples of use cases

- Agriculture / weather modeling: number of rain events per year (Poisson), amount of rainfall per event (Gamma), total rainfall per year (Tweedie / Compound Poisson Gamma).
- Risk modeling / insurance policy pricing: number of claim events / policyholder per year (Poisson), cost per event (Gamma), total cost per policyholder per year (Tweedie / Compound Poisson Gamma).
- Credit Default: probability that a loan can't be paid back (Bernoulli).
- Fraud Detection: probability that a financial transaction like a cash transfer is a fraudulent transaction (Bernoulli).
- Predictive maintenance: number of production interruption events per year (Poisson), duration of interruption (Gamma), total interruption time per year (Tweedie / Compound Poisson Gamma).
- Medical Drug Testing: probability of curing a patient in a set of trials or probability that a patient will experience side effects (Bernoulli).
- News Classification: classification of news articles into three categories namely Business News, Politics and Entertainment news (Categorical).

References

Usage

`TweedieRegressor` implements a generalized linear model for the Tweedie distribution, that allows to model any of the above mentioned distributions using the appropriate `power` parameter. In particular:

- `power = 0`: Normal distribution. Specific estimators such as `Ridge`, `ElasticNet` are generally more appropriate in this case.
- `power = 1`: Poisson distribution. `PoissonRegressor` is exposed for convenience. However, it is strictly equivalent to TweedieRegressor(power=1, link='log').
- `power = 2`: Gamma distribution. `GammaRegressor` is exposed for convenience. However, it is strictly equivalent to TweedieRegressor(power=2, link='log').
- `power = 3`: Inverse Gaussian distribution.

The link function is determined by the link parameter.

Usage example:

```
>>> from sklearn.linear_model import TweedieRegressor
>>> reg = TweedieRegressor(power=1, alpha=0.5, link='log')
>>> reg.fit([[0, 0], [0, 1], [2, 2]], [0, 1, 2])
TweedieRegressor(alpha=0.5, link='log', power=1)
>>> reg.coef_
array([0.2463, 0.4337])
>>> reg.intercept_
np.float64(-0.7638)
```

Examples

- sphx_glr_auto_examples_linear_model_plot_poisson_regression_non_normal_loss.py
- sphx_glr_auto_examples_linear_model_plot_tweedie_regression_insurance_claims.py

Practical considerations
The feature matrix X should be standardized before fitting. This ensures that the penalty treats features equally.
Since the linear predictor Xw can be negative and Poisson, Gamma and Inverse Gaussian distributions don't support negative values, it is necessary to apply an inverse link function that guarantees the non-negativeness. For example with link='log', the inverse link function becomes $h(Xw) = \exp(Xw)$.
If you want to model a relative frequency, i.e. counts per exposure (time, volume, ...) you can do so by using a Poisson distribution and passing together with exposure as sample weights. For a concrete example see e.g.
When performing cross-validation for the power parameter of TweedieRegressor, it is advisable to specify an explicit scoring function, because the default scorer `TweedieRegressor.score` is a function of power itself.

Stochastic Gradient Descent - SGD

Stochastic gradient descent is a simple yet very efficient approach to fit linear models. It is particularly useful when the number of samples (and the number of features) is very large. The `partial_fit` method allows online/out-of-core learning.

The classes `SGDClassifier` and `SGDRegressor` provide functionality to fit linear models for classification and regression using different (convex) loss functions and different penalties. E.g., with `loss="log"`, `SGDClassifier` fits a logistic regression model, while with `loss="hinge"` it fits a linear support vector machine (SVM).

You can refer to the dedicated sgd documentation section for more details.

Perceptron

The `Perceptron` is another simple classification algorithm suitable for large scale learning and derives from SGD. By default:

- It does not require a learning rate.
- It is not regularized (penalized).
- It updates its model only on mistakes.

The last characteristic implies that the Perceptron is slightly faster to train than SGD with the hinge loss and that the resulting models are sparser.

In fact, the `Perceptron` is a wrapper around the `SGDClassifier` class using a perceptron loss and a constant learning rate. Refer to mathematical section of the SGD procedure for more details.

Passive Aggressive Algorithms

The passive-aggressive (PA) algorithms are another family of 2 algorithms (PA-I and PA-II) for large-scale online learning that derive from SGD. They are similar to the Perceptron in that they do not require a learning rate. However, contrary to the Perceptron, they include a regularization parameter `eta0` (C in the reference paper).

For classification, `SGDClassifier(loss="hinge", penalty=None, learning_rate="pa1", eta0=1.0)` can be used for PA-I or with `learning_rate="pa2"` for PA-II. For regression, `SGDRegressor(loss="epsilon_insensitive", penalty=None, learning_rate="pa1", eta0=1.0)` can be used for PA-I or with `learning_rate="pa2"` for PA-II.

References
- "Online Passive-Aggressive Algorithms"
  K. Crammer, O. Dekel, J. Keshat, S. Shalev-Shwartz, Y. Singer - JMLR 7 (2006)

Robustness regression: outliers and modeling errors

Robust regression aims to fit a regression model in the presence of corrupt data: either outliers, or error in the model.

Different scenario and useful concepts

There are different things to keep in mind when dealing with data corrupted by outliers:

target ../auto_examples/linear_model/plot_robust_fit.html
scale 60%
target ../auto_examples/linear_model/plot_robust_fit.html
scale 60%
target ../auto_examples/linear_model/plot_robust_fit.html
scale 60%

- **Outliers in X or in y**?

Outliers in the y direction	Outliers in the X direction
y_outliers X	outliers

- **Fraction of outliers versus amplitude of error**

The number of outlying points matters, but also how much they are outliers.

Small outliers	Large outliers
y_outliers la	rge_y_outliers

An important notion of robust fitting is that of breakdown point: the fraction of data that can be outlying for the fit to start missing the inlying data.

Note that in general, robust fitting in high-dimensional setting (large n_features) is very hard. The robust models here will probably not work in these settings.

Scikit-learn provides 3 robust regression estimators:

- HuberRegressor should be faster than unless the number of samples is very large, i.e. `n_samples` » `n_features`. This is because RANSAC and Theil Sen fit on smaller subsets of the data. However, both Theil Sen and RANSAC are unlikely to be as robust as
- RANSAC is faster than Theil Sen and scales much better with the number of samples.
- RANSAC will deal better with large outliers in the y direction (most common situation).
- Theil Sen will cope better with medium-size outliers in the X direction, but this property will disappear in high-dimensional settings.

When in doubt, use RANSAC.

RANSAC: RANdom SAmple Consensus

RANSAC (RANdom SAmple Consensus) fits a model from random subsets of inliers from the complete data set.

RANSAC is a non-deterministic algorithm producing only a reasonable result with a certain probability, which is dependent on the number of iterations (see max_trials parameter). It is typically used for linear and non-linear regression problems and is especially popular in the field of photogrammetric computer vision.

The algorithm splits the complete input sample data into a set of inliers, which may be subject to noise, and outliers, which are e.g. caused by erroneous measurements or invalid hypotheses about the data. The resulting model is then estimated only from the determined inliers.

Examples

- sphx_glr_auto_examples_linear_model_plot_ransac.py
- sphx_glr_auto_examples_linear_model_plot_robust_fit.py

Details of the algorithm
Each iteration performs the following steps:

1. Select `min_samples` random samples from the original data and check whether the set of data is valid (see `is_data_valid`).
2. Fit a model to the random subset (`estimator.fit`) and check whether the estimated model is valid (see `is_model_valid`).
3. Classify all data as inliers or outliers by calculating the residuals to the estimated model (`estimator.predict(X) - y`) - all data samples with absolute residuals smaller than or equal to the `residual_threshold` are considered as inliers.
4. Save fitted model as best model if number of inlier samples is maximal. In case the current estimated model has the same number of inliers, it is only considered as the best model if it has better score.

These steps are performed either a maximum number of times (`max_trials`) or until one of the special stop criteria are met (see `stop_n_inliers` and `stop_score`). The final model is estimated using all inlier samples (consensus set) of the previously determined best model.

The `is_data_valid` and `is_model_valid` functions allow to identify and reject degenerate combinations of random sub-samples. If the estimated model is not needed for identifying degenerate cases, `is_data_valid` should be used as it is called prior to fitting the model and thus leading to better computational performance.

References
- https://en.wikipedia.org/wiki/RANSAC
- "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography" Martin A. Fischler and Robert C. Bolles - SRI International (1981)
- "Performance Evaluation of RANSAC Family" Sunglok Choi, Taemin Kim and Wonpil Yu - BMVC (2009)

Theil-Sen estimator: generalized-median-based estimator

The `TheilSenRegressor` estimator uses a generalization of the median in multiple dimensions. It is thus robust to multivariate outliers. Note however that the robustness of the estimator decreases quickly with the dimensionality of the problem. It loses its robustness properties and becomes no better than an ordinary least squares in high dimension.

Examples

- sphx_glr_auto_examples_linear_model_plot_theilsen.py
- sphx_glr_auto_examples_linear_model_plot_robust_fit.py

Theoretical considerations
(OLS) <ordinary_least_squares>' in terms of asymptotic efficiency and as an unbiased estimator. In contrast to OLS, Theil-Sen is a non-parametric method which means it makes no assumption about the underlying distribution of the data. Since Theil-Sen is a median-based estimator, it is more robust against corrupted data aka outliers. In univariate setting, Theil-Sen has a breakdown point of about 29.3% in case of a simple linear regression which means that it can tolerate arbitrary corrupted data of up to 29.3%.
The implementation of `TheilSenRegressor` in scikit-learn follows a generalization to a multivariate linear regression model using the spatial median which is a generalization of the median to multiple dimensions.
In terms of time and space complexity, Theil-Sen scales according to

$$\binom{n_{\text{samples}}}{n_{\text{subsamples}}}$$

which makes it infeasible to be applied exhaustively to problems with a large number of samples and features. Therefore, the magnitude of a subpopulation can be chosen to limit the time and space complexity by considering only a random subset of all possible combinations.
References
Also see the Wikipedia page

Huber Regression

The `HuberRegressor` is different from `Ridge` because it applies a linear loss to samples that are defined as outliers by the epsilon parameter. A sample is classified as an inlier if the absolute error of that sample is less than the threshold epsilon. It differs from `TheilSenRegressor` and `RANSACRegressor` because it does not ignore the effect of the outliers but gives a lesser weight to them.

Examples

- sphx_glr_auto_examples_linear_model_plot_huber_vs_ridge.py

Mathematical details

$$\min_{w,\sigma} \sum_{i=1}^{n} \left(\sigma + H_\epsilon \left(\frac{X_i w - y_i}{\sigma} \right) \sigma \right) + \alpha ||w||_2^2$$

where the loss function is given by

$$H_\epsilon(z) = \begin{cases} z^2, & \text{if } |z| < \epsilon, \\ 2\epsilon|z| - \epsilon^2, & \text{otherwise} \end{cases}$$

It is advised to set the parameter `epsilon` to 1.35 to achieve 95% statistical efficiency.

References

- Peter J. Huber, Elvezio M. Ronchetti: Robust Statistics, Concomitant scale estimates, p. 172.

The `HuberRegressor` differs from using `SGDRegressor` with loss set to huber in the following ways.

- `HuberRegressor` is scaling invariant. Once `epsilon` is set, scaling `X` and `y` down or up by different values would produce the same robustness to outliers as before. as compared to `SGDRegressor` where `epsilon` has to be set again when `X` and `y` are scaled.
- `HuberRegressor` should be more efficient to use on data with small number of samples while `SGDRegressor` needs a number of passes on the training data to produce the same robustness.

Note that this estimator is different from the R implementation of Robust Regression because the R implementation does a weighted least squares implementation with weights given to each sample on the basis of how much the residual is greater than a certain threshold.

Quantile Regression

Quantile regression estimates the median or other quantiles of y conditional on X, while ordinary least squares (OLS) estimates the conditional mean.

Quantile regression may be useful if one is interested in predicting an interval instead of point prediction. Sometimes, prediction intervals are calculated based on the assumption that prediction error is distributed normally with zero mean and constant variance. Quantile regression provides sensible prediction intervals even for errors with non-constant (but predictable) variance or non-normal distribution.

Based on minimizing the pinball loss, conditional quantiles can also be estimated by models other than linear models. For example, `GradientBoostingRegressor` can predict conditional quantiles if its parameter `loss` is set to `"quantile"` and parameter `alpha` is set to the quantile that should be predicted. See the example in sphx_glr_auto_examples_ensemble_plot_gradient_boosting_quantile.py.

Most implementations of quantile regression are based on linear programming problem. The current implementation is based on `scipy.optimize.linprog`.

Examples

- sphx_glr_auto_examples_linear_model_plot_quantile_regression.py

Mathematical details
As a linear model, the `QuantileRegressor` gives linear predictions The weights or coefficients w are then found by the following minimization problem:

$$\min_w \frac{1}{n_{\text{samples}}} \sum_i PB_q(y_i - X_i w) + \alpha ||w||_1.$$

This consists of the pinball loss (also known as linear loss), see also `mean_pinball_loss`,

$$PB_q(t) = q \max(t, 0) + (1 - q) \max(-t, 0) = \begin{cases} qt, & t > 0, \\ 0, & t = 0, \\ (q-1)t, & t < 0 \end{cases}$$

and the L1 penalty controlled by parameter `alpha`, similar to
As the pinball loss is only linear in the residuals, quantile regression is much more robust to outliers than squared error based estimation of the mean. Somewhat in between is the `HuberRegressor`.

References

- Koenker, R., & Bassett Jr, G. (1978). Regression quantiles. Econometrica: journal of the Econometric Society, 33-50.
- Portnoy, S., & Koenker, R. (1997). `The Gaussian hare and the Laplacian tortoise: computability of squared-error versus absolute-error estimators. Statistical Science, 12, 279-300 <10.1214/ss/1030037960>`.
- Koenker, R. (2005). `Quantile Regression`. Cambridge University Press.

Polynomial regression: extending linear models with basis functions

One common pattern within machine learning is to use linear models trained on nonlinear functions of the data. This approach maintains the generally fast performance of linear methods, while allowing them to fit a much wider range of data.

Mathematical details
For example, a simple linear regression can be extended by constructing **polynomial features** from the coefficients. In the standard linear regression case, you might have a model that looks like this for two-dimensional data:

$$\hat{y}(w, x) = w_0 + w_1 x_1 + w_2 x_2$$

If we want to fit a paraboloid to the data instead of a plane, we can combine the features in second-order polynomials, so that the model looks like this:

$$\hat{y}(w, x) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1 x_2 + w_4 x_1^2 + w_5 x_2^2$$

The (sometimes surprising) observation is that this is *still a linear model*: to see this, imagine creating a new set of features

$$z = [x_1, x_2, x_1 x_2, x_1^2, x_2^2]$$

With this re-labeling of the data, our problem can be written

$$\hat{y}(w, z) = w_0 + w_1 z_1 + w_2 z_2 + w_3 z_3 + w_4 z_4 + w_5 z_5$$

We see that the resulting *polynomial regression* is in the same class of linear models we considered above (i.e. the model is linear in w) and can be solved by the same techniques. By considering linear fits within a higher-dimensional space built with these basis functions, the model has the flexibility to fit a much broader range of data.

Here is an example of applying this idea to one-dimensional data, using polynomial features of varying degrees:

This figure is created using the `PolynomialFeatures` transformer, which transforms an input data matrix into a new data matrix of a given degree. It can be used as follows:

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> import numpy as np
>>> X = np.arange(6).reshape(3, 2)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> poly = PolynomialFeatures(degree=2)
>>> poly.fit_transform(X)
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

The features of `X` have been transformed from $[x_1, x_2]$ to $[1, x_1, x_2, x_1^2, x_1 x_2, x_2^2]$, and can now be used within any linear model.

This sort of preprocessing can be streamlined with the Pipeline tools. A single object representing a simple polynomial regression can be created and used as follows:

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.pipeline import Pipeline
>>> import numpy as np
>>> model = Pipeline([('poly', PolynomialFeatures(degree=3)),
...                   ('linear', LinearRegression(fit_intercept=False))])
>>> # fit to an order-3 polynomial data
>>> x = np.arange(5)
>>> y = 3 - 2 * x + x ** 2 - x ** 3
>>> model = model.fit(x[:, np.newaxis], y)
>>> model.named_steps['linear'].coef_
array([ 3., -2.,  1., -1.])
```

The linear model trained on polynomial features is able to exactly recover the input polynomial coefficients.

In some cases it's not necessary to include higher powers of any single feature, but only the so-called *interaction features* that multiply together at most d distinct features. These can be gotten from `PolynomialFeatures` with the setting `interaction_only=True`.

For example, when dealing with boolean features, $x_i^n = x_i$ for all n and is therefore useless; but $x_i x_j$ represents the conjunction of two booleans. This way, we can solve the XOR problem with a linear classifier:

```
>>> from sklearn.linear_model import Perceptron
>>> from sklearn.preprocessing import PolynomialFeatures
>>> import numpy as np
>>> X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
>>> y = X[:, 0] ^ X[:, 1]
>>> y
array([0, 1, 1, 0])
>>> X = PolynomialFeatures(interaction_only=True).fit_transform(X).astype(int)
>>> X
array([[1, 0, 0, 0],
```

```
       [1, 0, 1, 0],
       [1, 1, 0, 0],
       [1, 1, 1, 1]])
>>> clf = Perceptron(fit_intercept=False, max_iter=10, tol=None,
...                  shuffle=False).fit(X, y)
```

And the classifier ”predictions” are perfect:

```
>>> clf.predict(X)
array([0, 1, 1, 0])
>>> clf.score(X, y)
1.0
```

Ensembles: gradient boosting, random forests, bagging, voting, stacking

Source: https://scikit-learn.org/stable/modules/ensemble.html

Ensemble methods combine the predictions of several base estimators built with a given learning algorithm in order to improve generalizability / robustness over a single estimator.

Two very famous examples of ensemble methods are `gradient-boosted trees <gradient_boosting>` and random forests.

More generally, ensemble models can be applied to any base learner beyond trees, in averaging methods such as Bagging methods, model stacking, or Voting, or in boosting, as AdaBoost.

Gradient-boosted trees

Gradient Tree Boosting or Gradient Boosted Decision Trees (GBDT) is a generalization of boosting to arbitrary differentiable loss functions, see the seminal work of [Friedman2001]. GBDT is an excellent model for both regression and classification, in particular for tabular data.

Scikit-learn provides two implementations of gradient-boosted trees: corresponding classes for regression. The former can be **orders of magnitude faster** than the latter when the number of samples is larger than tens of thousands of samples.

Missing values and categorical data are natively supported by the Hist... version, removing the need for additional preprocessing such as imputation.

sizes since binning may lead to split points that are too approximate in this setting.

Histogram-Based Gradient Boosting

Scikit-learn 0.21 introduced two new implementations of gradient boosted trees, namely `HistGradientBoostingClassifier` and `HistGradientBoostingRegressor`, inspired by LightGBM (See [LightGBM]).

These histogram-based estimators can be **orders of magnitude faster** than `GradientBoostingClassifier` and `GradientBoostingRegressor` when the number of samples is larger than tens of thousands of samples.

They also have built-in support for missing values, which avoids the need for an imputer.

These fast estimators first bin the input samples `X` into integer-valued bins (typically 256 bins) which tremendously reduces the number of splitting points to consider, and allows the algorithm to leverage integer-based data structures (histograms) instead of relying on sorted continuous values when building the trees. The API of these estimators is slightly different, and some of the features from `GradientBoostingClassifier` and `GradientBoostingRegressor` are not yet supported, for instance some loss functions.

Examples

- sphx_glr_auto_examples_inspection_plot_partial_dependence.py
- sphx_glr_auto_examples_ensemble_plot_forest_hist_grad_boosting_comparison.py

Usage

Most of the parameters are unchanged from `GradientBoostingClassifier` and `GradientBoostingRegressor`. One exception is the `max_iter` parameter that replaces `n_estimators`, and controls the number of iterations of the boosting process:

```
>>> from sklearn.ensemble import HistGradientBoostingClassifier
>>> from sklearn.datasets import make_hastie_10_2

>>> X, y = make_hastie_10_2(random_state=0)
>>> X_train, X_test = X[:2000], X[2000:]
>>> y_train, y_test = y[:2000], y[2000:]

>>> clf = HistGradientBoostingClassifier(max_iter=100).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.8965
```

Available losses for **regression** are:

- 'squared_error', which is the default loss;
- 'absolute_error', which is less sensitive to outliers than the squared error;
- 'gamma', which is well suited to model strictly positive outcomes;
- 'poisson', which is well suited to model counts and frequencies;
- 'quantile', which allows for estimating a conditional quantile that can later be used to obtain prediction intervals.

For **classification**, 'log_loss' is the only option. For binary classification it uses the binary log loss, also known as binomial deviance or binary cross-entropy. For n_classes >= 3, it uses the multi-class log loss function, with multinomial deviance and categorical cross-entropy as alternative names. The appropriate loss version is selected based on y passed to fit.

The size of the trees can be controlled through the `max_leaf_nodes`, `max_depth`, and `min_samples_leaf` parameters.

The number of bins used to bin the data is controlled with the `max_bins` parameter. Using less bins acts as a form of regularization. It is generally recommended to use as many bins as possible (255), which is the default.

The `l2_regularization` parameter acts as a regularizer for the loss function, and corresponds to λ in the following expression (see equation (2) in [XGBoost]):

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \frac{1}{2}\sum_k \lambda ||w_k||^2$$

Details on l2 regularization
It is important to notice that the loss term $l(\hat{y}_i, y_i)$ describes only half of the actual loss function except for the pinball loss and absolute error.
The index k refers to the k-th tree in the ensemble of trees. In the case of regression and binary classification, gradient boosting models grow one tree per iteration, then k runs up to max_iter. In the case of multiclass classification problems, the maximal value of the index k is n_classes $\times$ max_iter.
If T_k denotes the number of leaves in the k-th tree, then w_k is a vector of length T_k, which contains the leaf values of the form w = -sum_gradient / (sum_hessian + l2_regularization) (see equation (5) in [XGBoost]).
The leaf values w_k are derived by dividing the sum of the gradients of the loss function by the combined sum of hessians. Adding the regularization to the denominator penalizes the leaves with small hessians (flat regions), resulting in smaller updates. Those w_k values contribute

then to the model's prediction for a given input that ends up in the corresponding leaf. The final prediction is the sum of the base prediction and the contributions from each tree. The result of that sum is then transformed by the inverse link function depending on the choice of the loss function (see
Notice that the original paper [XGBoost] introduces a term $\gamma \sum_k T_k$ that penalizes the number of leaves (making it a smooth version of max_leaf_nodes) not presented here as it is not implemented in scikit-learn; whereas λ penalizes the magnitude of the individual tree predictions before being rescaled by the learning rate, see

Note that **early-stopping is enabled by default if the number of samples is larger than 10,000**. The early-stopping behaviour is controlled via the `early_stopping`, `scoring`, `validation_fraction`, `n_iter_no_change`, and `tol` parameters. It is possible to early-stop using an arbitrary scorer, or just the training or validation loss. Note that for technical reasons, using a callable as a scorer is significantly slower than using the loss. By default, early-stopping is performed if there are at least 10,000 samples in the training set, using the validation loss.

Missing values support

`HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` have built-in support for missing values (NaNs).

During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently:

```
>>> from sklearn.ensemble import HistGradientBoostingClassifier
>>> import numpy as np

>>> X = np.array([0, 1, 2, np.nan]).reshape(-1, 1)
>>> y = [0, 0, 1, 1]

>>> gbdt = HistGradientBoostingClassifier(min_samples_leaf=1).fit(X, y)
>>> gbdt.predict(X)
array([0, 0, 1, 1])
```

When the missingness pattern is predictive, the splits can be performed on whether the feature value is missing or not:

```
>>> X = np.array([0, np.nan, 1, 2, np.nan]).reshape(-1, 1)
>>> y = [0, 1, 0, 0, 1]
>>> gbdt = HistGradientBoostingClassifier(min_samples_leaf=1,
...                                       max_depth=2,
...                                       learning_rate=1,
...                                       max_iter=1).fit(X, y)
>>> gbdt.predict(X)
array([0, 1, 0, 0, 1])
```

If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

Examples

- sphx_glr_auto_examples_ensemble_plot_hgbt_regression.py

Sample weight support

`HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` support sample weights during fit.

The following toy example demonstrates that samples with a sample weight of zero are ignored:

```
>>> X = [[1, 0],
...      [1, 0],
...      [1, 0],
...      [0, 1]]
>>> y = [0, 0, 1, 0]
>>> # ignore the first 2 training samples by setting their weight to 0
>>> sample_weight = [0, 0, 1, 1]
>>> gb = HistGradientBoostingClassifier(min_samples_leaf=1)
>>> gb.fit(X, y, sample_weight=sample_weight)
HistGradientBoostingClassifier(...)
>>> gb.predict([[1, 0]])
array([1])
>>> gb.predict_proba([[1, 0]])[0, 1]
np.float64(0.999)
```

As you can see, the [1, 0] is comfortably classified as 1 since the first two samples are ignored due to their sample weights.

Implementation detail: taking sample weights into account amounts to multiplying the gradients (and the hessians) by the sample weights. Note that the binning stage (specifically the quantiles computation) does not take the weights into account.

Categorical Features Support

`HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` have native support for categorical features: they can consider splits on non-ordered, categorical data.

For datasets with categorical features, using the native categorical support is often better than relying on one-hot encoding (`OneHotEncoder`), because one-hot encoding requires more tree depth to achieve equivalent splits. It is also usually better to rely on the native categorical support rather than to treat categorical features as continuous (ordinal), which happens for ordinal-encoded categorical data, since categories are nominal quantities where order does not matter.

To enable categorical support, a boolean mask can be passed to the categorical_features parameter, indicating which feature is categorical. In the following, the first feature will be treated as categorical and the second feature as numerical:

```
>>> gbdt = HistGradientBoostingClassifier(categorical_features=[True, False])
```

Equivalently, one can pass a list of integers indicating the indices of the categorical features:

```
>>> gbdt = HistGradientBoostingClassifier(categorical_features=[0])
```

When the input is a DataFrame, it is also possible to pass a list of column names:

```
>>> gbdt = HistGradientBoostingClassifier(categorical_features=["site", "manufacturer"])
```

Finally, when the input is a DataFrame we can use categorical_features=”from_dtype” in which case all columns with a categorical dtype will be treated as categorical features.

The cardinality of each categorical feature must be less than the max_bins parameter. For an example using histogram-based gradient boosting on categorical features, see sphx_glr_auto_examples_ensemble_plot_gradient_boosting_categorical.py.

If there are missing values during training, the missing values will be treated as a proper category. If there are no missing values during training, then at prediction time, missing values are mapped to the child node that has the most samples (just like for continuous features). When predicting, categories that were not seen during fit time will be treated as missing values.

Split finding with categorical features
The canonical way of considering categorical splits in a tree is to consider all of the $2^{K-1}-1$ partitions, where K is the number of categories. This can quickly become prohibitive when K is large. Fortunately, since gradient boosting trees are always regression trees (even for classification problems), there exists a faster strategy that can yield equivalent splits. First, the categories of a feature are sorted according to the variance of the target, for each category k. Once the categories are sorted, one can consider *continuous partitions*, i.e. treat the categories as if they were ordered continuous values (see Fisher [Fisher1958] for a formal proof). As a result, only $K-1$ splits need to be considered instead of $2^{K-1}-1$. The initial sorting is a

Examples

- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_categorical.py

Monotonic Constraints

Depending on the problem at hand, you may have prior knowledge indicating that a given feature should in general have a positive (or negative) effect on the target value. For example, all else being equal, a higher credit score should increase the probability of getting approved for a loan. Monotonic constraints allow you to incorporate such prior knowledge into the model.

For a predictor F with two features:

- a **monotonic increase constraint** is a constraint of the form:

$$x_1 \leq x_1' \implies F(x_1, x_2) \leq F(x_1', x_2)$$

- a **monotonic decrease constraint** is a constraint of the form:

$$x_1 \leq x_1' \implies F(x_1, x_2) \geq F(x_1', x_2)$$

You can specify a monotonic constraint on each feature using the monotonic_cst parameter. For each feature, a value of 0 indicates no constraint, while 1 and -1 indicate a monotonic increase and monotonic decrease constraint, respectively:

```
>>> from sklearn.ensemble import HistGradientBoostingRegressor

... # monotonic increase, monotonic decrease, and no constraint on the 3 features
>>> gbdt = HistGradientBoostingRegressor(monotonic_cst=[1, -1, 0])
```

In a binary classification context, imposing a monotonic increase (decrease) constraint means that higher values of the feature are supposed to have a positive (negative) effect on the probability of samples to belong to the positive class.

Nevertheless, monotonic constraints only marginally constrain feature effects on the output. For instance, monotonic increase and decrease constraints cannot be used to enforce the following modelling constraint:

$$x_1 \leq x_1' \implies F(x_1, x_2) \leq F(x_1', x_2')$$

Also, monotonic constraints are not supported for multiclass classification.

For a practical implementation of monotonic constraints with the histogram-based gradient boosting, including how they can improve generalization when domain knowledge is available, see sphx_glr_auto_examples_ensemble_plot_monotonic_constraints.py.

Note
Since categories are unordered quantities, it is not possible to enforce monotonic constraints on categorical features.

Examples

- sphx_glr_auto_examples_ensemble_plot_hgbt_regression.py

Interaction constraints

A priori, the histogram gradient boosted trees are allowed to use any feature to split a node into child nodes. This creates so called interactions between features, i.e. usage of different features as split along a branch. Sometimes, one wants to restrict the possible interactions, see [Mayer2022]. This can be done by the parameter `interaction_cst`, where one can specify the indices of features that are allowed to interact. For instance, with 3 features in total, `interaction_cst=[{0}, {1}, {2}]` forbids all interactions. The constraints `[{0, 1}, {1, 2}]` specify two groups of possibly interacting features. Features 0 and 1 may interact with each other, as well as features 1 and 2. But note that features 0 and 2 are forbidden to interact. The following depicts a tree and the possible splits of the tree:

```
1      <- Both constraint groups could be applied from now on
/ \
1   2    <- Left split still fulfills both constraint groups.
/ \ / \      Right split at feature 2 has only group {1, 2} from now on.
```

LightGBM uses the same logic for overlapping groups.

Note that features not listed in `interaction_cst` are automatically assigned an interaction group for themselves. With again 3 features, this means that `[{0}]` is equivalent to `[{0}, {1, 2}]`.

Examples

- sphx_glr_auto_examples_inspection_plot_partial_dependence.py

References

Low-level parallelism

`HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` use OpenMP for parallelization through Cython. For more details on how to control the number of threads, please refer to our parallelism notes.

The following parts are parallelized:

- mapping samples from real values to integer-valued bins (finding the bin thresholds is however sequential)
- building histograms is parallelized over features
- finding the best split point at a node is parallelized over features
- during fit, mapping samples into the left and right children is parallelized over samples
- gradient and hessians computations are parallelized over samples
- predicting is parallelized over samples

Why it's faster

The bottleneck of a gradient boosting procedure is building the decision trees. Building a traditional decision tree (as in the other GBDTs `GradientBoostingClassifier` and `GradientBoostingRegressor`) requires sorting the samples at each node (for each feature). Sorting is needed so that the potential gain of a split point can be computed efficiently. Splitting a

single node has thus a complexity of $\mathcal{O}(n_{\text{features}} \times n \log(n))$ where n is the number of samples at the node.

`HistGradientBoostingClassifier` and `HistGradientBoostingRegressor`, in contrast, do not require sorting the feature values and instead use a data-structure called histogram , where the samples are implicitly ordered. Building a histogram has a $\mathcal{O}(n)$ complexity, so the node splitting procedure has a $\mathcal{O}(n_{\text{features}} \times n)$ complexity, much smaller than the previous one. In addition, instead of considering n split points, we consider only `max_bins` split points, which might be much smaller.

In order to build histograms, the input data X needs to be binned into integer-valued bins. This binning procedure does require sorting the feature values, but it only happens once at the very beginning of the boosting process (not at each node, like in `GradientBoostingClassifier` and `GradientBoostingRegressor`).

A second major reason for being faster is the fact that `HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` both use second order information about the loss, i.e. the Hessian. This is called Newton boosting and avoids the line-search step of the ordinary gradient boosting.

Finally, many parts of the implementation of `HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` are parallelized.

References

`GradientBoostingClassifier` and `GradientBoostingRegressor`

The usage and the parameters of `GradientBoostingClassifier` and `GradientBoostingRegressor` are described below. The 2 most important parameters of these estimators are n_estimators and learning_rate.

Classification
classification. The following example shows how to fit a gradient boosting classifier with 100 decision stumps as weak learners:

```
>>> from sklearn.datasets import make_hastie_10_2
>>> from sklearn.ensemble import GradientBoostingClassifier

>>> X, y = make_hastie_10_2(random_state=0)
>>> X_train, X_test = X[:2000], X[2000:]
>>> y_train, y_test = y[:2000], y[2000:]

>>> clf = GradientBoostingClassifier(n_estimators=100, learning_rate=1.0,
...     max_depth=1, random_state=0).fit(X_train, y_train)
>>> clf.score(X_test, y_test)
0.913
```

The number of weak learners (i.e. regression trees) is controlled by the parameter `n_estimators`; `The size of each tree <gradient_boosting_tree_size>` can be controlled either by setting the tree depth via `max_depth` or by setting the number of leaf nodes via `max_leaf_nodes`. The `learning_rate` is a hyper-parameter in the range (0.0, 1.0] that controls overfitting via `shrinkage <gradient_boosting_shrinkage>` .

Note
Classification with more than 2 classes requires the induction of `n_classes` regression trees at each iteration, thus, the total number of induced trees equals `n_classes * n_estimators`. For datasets with a large number of classes we strongly recommend to use

Regression
for regression which can be specified via the argument `loss`; the default loss function for regression is squared error (`'squared_error'`).

```
>>> import numpy as np
>>> from sklearn.metrics import mean_squared_error
>>> from sklearn.datasets import make_friedman1
>>> from sklearn.ensemble import GradientBoostingRegressor

>>> X, y = make_friedman1(n_samples=1200, random_state=0, noise=1.0)
>>> X_train, X_test = X[:200], X[200:]
>>> y_train, y_test = y[:200], y[200:]
>>> est = GradientBoostingRegressor(
...     n_estimators=100, learning_rate=0.1, max_depth=1, random_state=0,
...     loss='squared_error'
... ).fit(X_train, y_train)
>>> mean_squared_error(y_test, est.predict(X_test))
5.00
```

The figure below shows the results of applying `GradientBoostingRegressor` with least squares loss and 500 base learners to the diabetes dataset (`sklearn.datasets.load_diabetes`). The plot shows the train and test error at each iteration. The train error at each iteration is stored in the train_score_ attribute of the gradient boosting model. The test error at each iteration can be obtained via the `staged_predict` method which returns a generator that yields the predictions at each stage. Plots like these can be used to determine the optimal number of trees (i.e. `n_estimators`) by early stopping.

Examples

- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_regression.py
- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_oob.py

Fitting additional weak-learners

Both `GradientBoostingRegressor` and `GradientBoostingClassifier` support `warm_start=True` which allows you to add more estimators to an already fitted model.

```
>>> import numpy as np
>>> from sklearn.metrics import mean_squared_error
>>> from sklearn.datasets import make_friedman1
>>> from sklearn.ensemble import GradientBoostingRegressor

>>> X, y = make_friedman1(n_samples=1200, random_state=0, noise=1.0)
>>> X_train, X_test = X[:200], X[200:]
>>> y_train, y_test = y[:200], y[200:]
>>> est = GradientBoostingRegressor(
...     n_estimators=100, learning_rate=0.1, max_depth=1, random_state=0,
...     loss='squared_error'
... )
>>> est = est.fit(X_train, y_train)  # fit with 100 trees
>>> mean_squared_error(y_test, est.predict(X_test))
5.00
>>> _ = est.set_params(n_estimators=200, warm_start=True)  # set warm_start and increase num of trees
>>> _ = est.fit(X_train, y_train) # fit additional 100 trees to est
>>> mean_squared_error(y_test, est.predict(X_test))
3.84
```

Controlling the tree size

The size of the regression tree base learners defines the level of variable interactions that can be captured by the gradient boosting model. In general, a tree of depth `h` can capture interactions of order `h` . There are two ways in which the size of the individual regression trees can be controlled.

If you specify `max_depth=h` then complete binary trees of depth `h` will be grown. Such trees will have (at most) `2**h` leaf nodes and `2**h - 1` split nodes.

Alternatively, you can control the tree size by specifying the number of leaf nodes via the parameter `max_leaf_nodes`. In this case, trees will be grown using best-first search where nodes with the highest improvement in impurity will be expanded first. A tree with `max_leaf_nodes=k` has `k - 1` split nodes and thus can model interactions of up to order `max_leaf_nodes - 1` .

We found that `max_leaf_nodes=k` gives comparable results to `max_depth=k-1` but is significantly faster to train at the expense of a slightly higher training error. The parameter `max_leaf_nodes` corresponds to the variable `J` in the chapter on gradient boosting in [Friedman2001] and is related to the parameter `interaction.depth` in R's gbm package where `max_leaf_nodes == interaction.depth + 1` .

Mathematical formulation

We first present GBRT for regression, and then detail the classification case.

Regression
GBRT regressors are additive models whose prediction $\hat{y}_i$ for a given input x_i is of the following form:

$$\hat{y}_i = F_M(x_i) = \sum_{m=1}^{M} h_m(x_i)$$

where the h_m are estimators called *weak learners* in the context of boosting. Gradient Tree Boosting uses `decision tree regressors <tree>` of fixed size as weak learners. The constant M corresponds to the n_estimators parameter.
Similar to other boosting algorithms, a GBRT is built in a greedy fashion:

$$F_m(x) = F_{m-1}(x) + h_m(x),$$

where the newly added tree h_m is fitted in order to minimize a sum of losses L_m, given the previous ensemble F_{m-1}:

$$h_m = \arg\min_h L_m = \arg\min_h \sum_{i=1}^{n} l(y_i, F_{m-1}(x_i) + h(x_i)),$$

where $l(y_i, F(x_i))$ is defined by the loss parameter, detailed in the next section.
By default, the initial model F_0 is chosen as the constant that minimizes the loss: for a least-squares loss, this is the empirical mean of the target values. The initial model can also be specified via the `init` argument.
Using a first-order Taylor approximation, the value of l can be approximated as follows:

$$l(y_i, F_{m-1}(x_i) + h_m(x_i)) \approx l(y_i, F_{m-1}(x_i)) + h_m(x_i) \left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}.$$

Note
Briefly, a first-order Taylor approximation says that Here, z corresponds to $F_{m-1}(x_i) + h_m(x_i)$, and

The quantity $\left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$ is the derivative of the loss with respect to its second parameter, evaluated at $F_{m-1}(x)$. It is easy to compute for any given $F_{m-1}(x_i)$ in a closed form since the loss is differentiable. We will denote it by g_i.
Removing the constant terms, we have:

$$h_m \approx \arg\min_h \sum_{i=1}^{n} h(x_i) g_i$$

This is minimized if $h(x_i)$ is fitted to predict a value that is proportional to the negative gradient $-g_i$. Therefore, at each iteration, **the estimator h_m is fitted to predict the negative gradients of the samples**. The gradients are updated at each iteration. This can be considered as some kind of gradient descent in a functional space.

Note
For some losses, e.g. `'absolute_error'` where the gradients are ± 1, the values predicted by a fitted h_m are not accurate enough: the tree can only output integer values. As a result, the leaves values of the tree h_m are modified once the tree is fitted, such that the leaves values minimize the loss L_m. The update is loss-dependent: for the absolute error loss, the value of a leaf is updated to the median of the samples in that leaf.

Classification
Gradient boosting for classification is very similar to the regression case. However, the sum of the trees $F_M(x_i) = \sum_m h_m(x_i)$ is not homogeneous to a prediction: it cannot be a class, since the trees predict continuous values.
The mapping from the value $F_M(x_i)$ to a class or a probability is loss-dependent. For the log-loss, the probability that x_i) = sigma(F_M(x_i)) where :math:sigma‘ is the sigmoid or expit function.
For multiclass classification, K trees (for K classes) are built at each of the M iterations. The probability that x_i belongs to class k is modeled as a softmax of the $F_{M,k}(x_i)$ values.
Note that even for a classification task, the h_m sub-estimator is still a regressor, not a classifier. This is because the sub-estimators are trained to predict (negative) *gradients*, which are always continuous quantities.

Loss Functions

The following loss functions are supported and can be specified using the parameter `loss`:

Regression
- Squared error (`'squared_error'`): The natural choice for regression due to its superior computational properties. The initial model is given by the mean of the target values.
- Absolute error (`'absolute_error'`): A robust loss function for regression. The initial model is given by the median of the target values.
- Huber (`'huber'`): Another robust loss function that combines least squares and least absolute deviation; use `alpha` to control the sensitivity with regards to outliers (see [Friedman2001] for more details).
- Quantile (`'quantile'`): A loss function for quantile regression. Use `0 < alpha < 1` to

specify the quantile. This loss function can be used to create prediction intervals (see sphx_glr_auto_examples_ensemble_plot_gradient_boosting_quantile.py).

Classification

- Binary log-loss (`'log-loss'`): The binomial negative log-likelihood loss function for binary classification. It provides probability estimates. The initial model is given by the log odds-ratio.
- Multi-class log-loss (`'log-loss'`): The multinomial negative log-likelihood loss function for multi-class classification with `n_classes` mutually exclusive classes. It provides probability estimates. The initial model is given by the prior probability of each class. At each iteration `n_classes` regression trees have to be constructed which makes GBRT rather inefficient for data sets with a large number of classes.
- Exponential loss (`'exponential'`): The same loss function as `AdaBoostClassifier`. Less robust to mislabeled examples than `'log-loss'`; can only be used for binary classification.

Shrinkage via learning rate

[Friedman2001] proposed a simple regularization strategy that scales the contribution of each weak learner by a constant factor ν:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

The parameter ν is also called the **learning rate** because it scales the step length of the gradient descent procedure; it can be set via the `learning_rate` parameter.

The parameter `learning_rate` strongly interacts with the parameter `n_estimators`, the number of weak learners to fit. Smaller values of `learning_rate` require larger numbers of weak learners to maintain a constant training error. Empirical evidence suggests that small values of `learning_rate` favor better test error. [HTF] recommend to set the learning rate to a small constant (e.g. `learning_rate <= 0.1`) and choose `n_estimators` large enough that early stopping applies, see sphx_glr_auto_examples_ensemble_plot_gradient_boosting_early_stopping.py for a more detailed discussion of the interaction between `learning_rate` and `n_estimators` see [R2007].

Subsampling

[Friedman2002] proposed stochastic gradient boosting, which combines gradient boosting with bootstrap averaging (bagging). At each iteration the base classifier is trained on a fraction `subsample` of the available training data. The subsample is drawn without replacement. A typical value of `subsample` is 0.5.

The figure below illustrates the effect of shrinkage and subsampling on the goodness-of-fit of the model. We can clearly see that shrinkage outperforms no-shrinkage. Subsampling with shrinkage can further increase the accuracy of the model. Subsampling without shrinkage, on the other hand, does poorly.

Another strategy to reduce the variance is by subsampling the features analogous to the random splits in `RandomForestClassifier`. The number of subsampled features can be controlled via the `max_features` parameter.

Note
Using a small `max_features` value can significantly decrease the runtime.

Stochastic gradient boosting allows to compute out-of-bag estimates of the test deviance by computing the improvement in deviance on the examples that are not included in the bootstrap sample (i.e. the out-of-bag examples). The improvements are stored in the attribute

oob_improvement_. `oob_improvement_[i]` holds the improvement in terms of the loss on the OOB samples if you add the i-th stage to the current predictions. Out-of-bag estimates can be used for model selection, for example to determine the optimal number of iterations. OOB estimates are usually very pessimistic thus we recommend to use cross-validation instead and only use OOB if cross-validation is too time consuming.

Examples

- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_regularization.py
- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_oob.py
- sphx_glr_auto_examples_ensemble_plot_ensemble_oob.py

Interpretation with feature importance

Individual decision trees can be interpreted easily by simply visualizing the tree structure. Gradient boosting models, however, comprise hundreds of regression trees thus they cannot be easily interpreted by visual inspection of the individual trees. Fortunately, a number of techniques have been proposed to summarize and interpret gradient boosting models.

Often features do not contribute equally to predict the target response; in many situations the majority of the features are in fact irrelevant. When interpreting a model, the first question usually is: what are those important features and how do they contribute in predicting the target response?

Individual decision trees intrinsically perform feature selection by selecting appropriate split points. This information can be used to measure the importance of each feature; the basic idea is: the more often a feature is used in the split points of a tree the more important that feature is. This notion of importance can be extended to decision tree ensembles by simply averaging the impurity-based feature importance of each tree (see random_forest_feature_importance for more details).

The feature importance scores of a fit gradient boosting model can be accessed via the `feature_importances_` property:

```
>>> from sklearn.datasets import make_hastie_10_2
>>> from sklearn.ensemble import GradientBoostingClassifier

>>> X, y = make_hastie_10_2(random_state=0)
>>> clf = GradientBoostingClassifier(n_estimators=100, learning_rate=1.0,
...     max_depth=1, random_state=0).fit(X, y)
>>> clf.feature_importances_
array([0.107, 0.105, 0.113, 0.0987, 0.0947,
       0.107, 0.0916, 0.0972, 0.0958, 0.0906])
```

Note that this computation of feature importance is based on entropy, and it is distinct from `sklearn.inspection.permutation_importance` which is based on permutation of the features.

Examples

- sphx_glr_auto_examples_ensemble_plot_gradient_boosting_regression.py

References

Random forests and other randomized tree ensembles

The `sklearn.ensemble` module includes two averaging algorithms based on randomized decision trees: the RandomForest algorithm and the Extra-Trees method. Both algorithms are perturb-and-combine techniques [B1998] specifically designed for trees. This means a diverse set of

classifiers is created by introducing randomness in the classifier construction. The prediction of the ensemble is given as the averaged prediction of the individual classifiers.

As other classifiers, forest classifiers have to be fitted with two arrays: a sparse or dense array X of shape `(n_samples, n_features)` holding the training samples, and an array Y of shape `(n_samples,)` holding the target values (class labels) for the training samples:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> X = [[0, 0], [1, 1]]
>>> Y = [0, 1]
>>> clf = RandomForestClassifier(n_estimators=10)
>>> clf = clf.fit(X, Y)
```

Like decision trees, forests of trees also extend to multi-output problems (if Y is an array of shape `(n_samples, n_outputs)`).

Random Forests

In random forests (see `RandomForestClassifier` and `RandomForestRegressor` classes), each tree in the ensemble is built from a sample drawn with replacement (i.e., a bootstrap sample) from the training set.

During the construction of each tree in the forest, a random subset of the features is considered. The size of this subset is controlled by the max_features parameter; it may include either all input features or a random subset of them (see the `parameter tuning guidelines <random_forest_parameters>` for more details).

The purpose of these two sources of randomness (bootstrapping the samples and randomly selecting features at each split) is to decrease the variance of the forest estimator. Indeed, individual decision trees typically exhibit high variance and tend to overfit. The injected randomness in forests yield decision trees with somewhat decoupled prediction errors. By taking an average of those predictions, some errors can cancel out. Random forests achieve a reduced variance by combining diverse trees, sometimes at the cost of a slight increase in bias. In practice the variance reduction is often significant hence yielding an overall better model.

When growing each tree in the forest, the ”best” split (i.e. equivalent to passing splitter=”best” to the underlying decision trees) is chosen according to the impurity criterion. See the `CART mathematical formulation <tree_mathematical_formulation>` for more details.

In contrast to the original publication [B2001], the scikit-learn implementation combines classifiers by averaging their probabilistic prediction, instead of letting each classifier vote for a single class.

A competitive alternative to random forests are histogram_based_gradient_boosting (HGBT) models:

- Building trees: Random forests typically rely on deep trees (that overfit individually) which uses much computational resources, as they require several splittings and evaluations of candidate splits. Boosting models build shallow trees (that underfit individually) which are faster to fit and predict.
- Sequential boosting: In HGBT, the decision trees are built sequentially, where each tree is trained to correct the errors made by the previous ones. This allows them to iteratively improve the model's performance using relatively few trees. In contrast, random forests use a majority vote to predict the outcome, which can require a larger number of trees to achieve the same level of accuracy.
- Efficient binning: HGBT uses an efficient binning algorithm that can handle large datasets with a high number of features. The binning algorithm can pre-process the data to speed up the subsequent tree construction (see Why it's faster). In contrast, the scikit-learn implementation of random forests does not use binning and relies on exact splitting, which

can be computationally expensive.

Overall, the computational cost of HGBT versus RF depends on the specific characteristics of the dataset and the modeling task. It's a good idea to try both models and compare their performance and computational efficiency on your specific problem to determine which model is the best fit.

Examples

- sphx_glr_auto_examples_ensemble_plot_forest_hist_grad_boosting_comparison.py

Extremely Randomized Trees

In extremely randomized trees (see `ExtraTreesClassifier` and `ExtraTreesRegressor` classes), randomness goes one step further in the way splits are computed. As in random forests, a random subset of candidate features is used, but instead of looking for the most discriminative thresholds, thresholds are drawn at random for each candidate feature and the best of these randomly-generated thresholds is picked as the splitting rule. This usually allows to reduce the variance of the model a bit more, at the expense of a slightly greater increase in bias:

```
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.datasets import make_blobs
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.ensemble import ExtraTreesClassifier
>>> from sklearn.tree import DecisionTreeClassifier

>>> X, y = make_blobs(n_samples=10000, n_features=10, centers=100,
...     random_state=0)

>>> clf = DecisionTreeClassifier(max_depth=None, min_samples_split=2,
...     random_state=0)
>>> scores = cross_val_score(clf, X, y, cv=5)
>>> scores.mean()
np.float64(0.98)

>>> clf = RandomForestClassifier(n_estimators=10, max_depth=None,
...     min_samples_split=2, random_state=0)
>>> scores = cross_val_score(clf, X, y, cv=5)
>>> scores.mean()
np.float64(0.999)

>>> clf = ExtraTreesClassifier(n_estimators=10, max_depth=None,
...     min_samples_split=2, random_state=0)
>>> scores = cross_val_score(clf, X, y, cv=5)
>>> scores.mean() > 0.999
np.True_
```

Parameters

The main parameters to adjust when using these methods is `n_estimators` and `max_features`. The former is the number of trees in the forest. The larger the better, but also the longer it will take to compute. In addition, note that results will stop getting significantly better beyond a critical number of trees. The latter is the size of the random subsets of features to consider when splitting a node. The lower the greater the reduction of variance, but also the greater the increase in bias. Empirical good default values are `max_features=1.0` or equivalently `max_features=None` (always considering all features instead of a random subset) for regression problems, and `max_features="sqrt"` (using a random subset of size `sqrt(n_features)`) for classification tasks (where `n_features` is the number of features in the data). The default value of

`max_features=1.0` is equivalent to bagged trees and more randomness can be achieved by setting smaller values (e.g. 0.3 is a typical default in the literature). Good results are often achieved when setting `max_depth=None` in combination with `min_samples_split=2` (i.e., when fully developing the trees). Bear in mind though that these values are usually not optimal, and might result in models that consume a lot of RAM. The best parameter values should always be cross-validated. In addition, note that in random forests, bootstrap samples are used by default (`bootstrap=True`) while the default strategy for extra-trees is to use the whole dataset (`bootstrap=False`). When using bootstrap sampling the generalization error can be estimated on the left out or out-of-bag samples. This can be enabled by setting `oob_score=True`.

Note
The size of the model with the default parameters is $O(M * N * log(N))$, where M is the number of trees and N is the number of samples. In order to reduce the size of the model, you can change these parameters: `min_samples_split`, `max_leaf_nodes`, `max_depth` and `min_samples_leaf`.

Parallelization

Finally, this module also features the parallel construction of the trees and the parallel computation of the predictions through the `n_jobs` parameter. If `n_jobs=k` then computations are partitioned into `k` jobs, and run on `k` cores of the machine. If `n_jobs=-1` then all cores available on the machine are used. Note that because of inter-process communication overhead, the speedup might not be linear (i.e., using `k` jobs will unfortunately not be `k` times as fast). Significant speedup can still be achieved though when building a large number of trees, or when building a single tree requires a fair amount of time (e.g., on large datasets).

Examples

- sphx_glr_auto_examples_ensemble_plot_forest_iris.py
- sphx_glr_auto_examples_miscellaneous_plot_multioutput_face_completion.py

References

- P. Geurts, D. Ernst., and L. Wehenkel, "Extremely randomized trees", Machine Learning, 63(1), 3-42, 2006.

Feature importance evaluation

The relative rank (i.e. depth) of a feature used as a decision node in a tree can be used to assess the relative importance of that feature with respect to the predictability of the target variable. Features used at the top of the tree contribute to the final prediction decision of a larger fraction of the input samples. The **expected fraction of the samples** they contribute to can thus be used as an estimate of the **relative importance of the features**. In scikit-learn, the fraction of samples a feature contributes to is combined with the decrease in impurity from splitting them to create a normalized estimate of the predictive power of that feature.

By **averaging** the estimates of predictive ability over several randomized trees one can **reduce the variance** of such an estimate and use it for feature selection. This is known as the mean decrease in impurity, or MDI. Refer to [L2014] for more information on MDI and feature importance evaluation with Random Forests.

Warning
The impurity-based feature importances computed on tree-based models suffer from two flaws that can lead to misleading conclusions. First they are computed on statistics derived from the training dataset and therefore **do not necessarily inform us on which features are most important to make good predictions on held-out dataset**. Secondly, **they favor high cardinality features**, that is features with many unique values. importance

that does not suffer from these flaws. These two methods of obtaining feature importance are explored in:

In practice those estimates are stored as an attribute named `feature_importances_` on the fitted model. This is an array with shape `(n_features,)` whose values are positive and sum to 1.0. The higher the value, the more important is the contribution of the matching feature to the prediction function.

Examples

- sphx_glr_auto_examples_ensemble_plot_forest_importances.py

References

Totally Random Trees Embedding

`RandomTreesEmbedding` implements an unsupervised transformation of the data. Using a forest of completely random trees, `RandomTreesEmbedding` encodes the data by the indices of the leaves a data point ends up in. This index is then encoded in a one-of-K manner, leading to a high dimensional, sparse binary coding. This coding can be computed very efficiently and can then be used as a basis for other learning tasks. The size and sparsity of the code can be influenced by choosing the number of trees and the maximum depth per tree. For each tree in the ensemble, the coding contains one entry of one. The size of the coding is at most `n_estimators * 2 ** max_depth`, the maximum number of leaves in the forest.

As neighboring data points are more likely to lie within the same leaf of a tree, the transformation performs an implicit, non-parametric density estimation.

Examples

- sphx_glr_auto_examples_ensemble_plot_random_forest_embedding.py
- sphx_glr_auto_examples_manifold_plot_lle_digits.py compares non-linear dimensionality reduction techniques on handwritten digits.
- sphx_glr_auto_examples_ensemble_plot_feature_transformation.py compares supervised and unsupervised tree based feature transformations.

See also
representations of feature space, also these approaches focus also on dimensionality reduction.

Fitting additional trees

RandomForest, Extra-Trees and `RandomTreesEmbedding` estimators all support `warm_start=True` which allows you to add more trees to an already fitted model.

```
>>> from sklearn.datasets import make_classification
>>> from sklearn.ensemble import RandomForestClassifier

>>> X, y = make_classification(n_samples=100, random_state=1)
>>> clf = RandomForestClassifier(n_estimators=10)
>>> clf = clf.fit(X, y)  # fit with 10 trees
>>> len(clf.estimators_)
10
>>> # set warm_start and increase num of estimators
>>> _ = clf.set_params(n_estimators=20, warm_start=True)
>>> _ = clf.fit(X, y) # fit additional 10 trees
>>> len(clf.estimators_)
20
```

When `random_state` is also set, the internal random state is also preserved between `fit` calls. This means that training a model once with `n` estimators is the same as building the model iteratively via multiple `fit` calls, where the final number of estimators is equal to `n`.

```
>>> clf = RandomForestClassifier(n_estimators=20)  # set `n_estimators` to 10 + 10
>>> _ = clf.fit(X, y)  # fit `estimators_` will be the same as `clf` above
```

Note that this differs from the usual behavior of random_state in that it does *not* result in the same result across different calls.

Bagging meta-estimator

In ensemble algorithms, bagging methods form a class of algorithms which build several instances of a black-box estimator on random subsets of the original training set and then aggregate their individual predictions to form a final prediction. These methods are used as a way to reduce the variance of a base estimator (e.g., a decision tree), by introducing randomization into its construction procedure and then making an ensemble out of it. In many cases, bagging methods constitute a very simple way to improve with respect to a single model, without making it necessary to adapt the underlying base algorithm. As they provide a way to reduce overfitting, bagging methods work best with strong and complex models (e.g., fully developed decision trees), in contrast with boosting methods which usually work best with weak models (e.g., shallow decision trees).

Bagging methods come in many flavours but mostly differ from each other by the way they draw random subsets of the training set:

- When random subsets of the dataset are drawn as random subsets of the samples, then this algorithm is known as Pasting [B1999].
- When samples are drawn with replacement, then the method is known as Bagging [B1996].
- When random subsets of the dataset are drawn as random subsets of the features, then the method is known as Random Subspaces [H1998].
- Finally, when base estimators are built on subsets of both samples and features, then the method is known as Random Patches [LG2012].

In scikit-learn, bagging methods are offered as a unified `BaggingClassifier` meta-estimator (resp. `BaggingRegressor`), taking as input a user-specified estimator along with parameters specifying the strategy to draw random subsets. In particular, `max_samples` and `max_features` control the size of the subsets (in terms of samples and features), while `bootstrap` and `bootstrap_features` control whether samples and features are drawn with or without replacement. When using a subset of the available samples the generalization accuracy can be estimated with the out-of-bag samples by setting `oob_score=True`. As an example, the snippet below illustrates how to instantiate a bagging ensemble of `KNeighborsClassifier` estimators, each built on random subsets of 50% of the samples and 50% of the features.

```
>>> from sklearn.ensemble import BaggingClassifier
>>> from sklearn.neighbors import KNeighborsClassifier
>>> bagging = BaggingClassifier(KNeighborsClassifier(),
...                             max_samples=0.5, max_features=0.5)
```

Examples

- sphx_glr_auto_examples_ensemble_plot_bias_variance.py

References

Voting Classifier

The idea behind the `VotingClassifier` is to combine conceptually different machine learning classifiers and use a majority vote or the average predicted probabilities (soft vote) to predict the class labels. Such a classifier can be useful for a set of equally well performing models in order to balance out their individual weaknesses.

Majority Class Labels (Majority/Hard Voting)

In majority voting, the predicted class label for a particular sample is the class label that represents the majority (mode) of the class labels predicted by each individual classifier.

E.g., if the prediction for a given sample is

- classifier 1 -> class 1
- classifier 2 -> class 1
- classifier 3 -> class 2

the VotingClassifier (with `voting='hard'`) would classify the sample as "class 1" based on the majority class label.

In the cases of a tie, the `VotingClassifier` will select the class based on the ascending sort order. E.g., in the following scenario

- classifier 1 -> class 2
- classifier 2 -> class 1

the class label 1 will be assigned to the sample.

Usage

The following example shows how to fit the majority rule classifier:

```
>>> from sklearn import datasets
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.naive_bayes import GaussianNB
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.ensemble import VotingClassifier

>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, 1:3], iris.target

>>> clf1 = LogisticRegression(random_state=1)
>>> clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
>>> clf3 = GaussianNB()

>>> eclf = VotingClassifier(
...     estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3)],
...     voting='hard')

>>> for clf, label in zip([clf1, clf2, clf3, eclf], ['Logistic Regression', 'Random Forest', 'naive Bayes', 'Ense
...     scores = cross_val_score(clf, X, y, scoring='accuracy', cv=5)
...     print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(), scores.std(), label))
Accuracy: 0.95 (+/- 0.04) [Logistic Regression]
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [naive Bayes]
Accuracy: 0.95 (+/- 0.04) [Ensemble]
```

Weighted Average Probabilities (Soft Voting)

In contrast to majority voting (hard voting), soft voting returns the class label as argmax of the sum of predicted probabilities.

Specific weights can be assigned to each classifier via the `weights` parameter. When weights are provided, the predicted class probabilities for each classifier are collected, multiplied by the classifier weight, and averaged. The final class label is then derived from the class label with the highest average probability.

To illustrate this with a simple example, let's assume we have 3 classifiers and a 3-class classification problem where we assign equal weights to all classifiers: w1=1, w2=1, w3=1.

The weighted average probabilities for a sample would then be calculated as follows:

classifier	class 1	class 2	class 3
classifier 1	w1 * 0.2	w1 * 0.5	w1 * 0.3
classifier 2	w2 * 0.6	w2 * 0.3	w2 * 0.1
classifier 3	w3 * 0.3	w3 * 0.4	w3 * 0.3
weighted average	0.37	0.4	0.23

Here, the predicted class label is 2, since it has the highest average predicted probability. See the example on sphx_glr_auto_examples_ensemble_plot_voting_decision_regions.py for a demonstration of how the predicted class label can be obtained from the weighted average of predicted probabilities.

The following figure illustrates how the decision regions may change when a soft `VotingClassifier` is trained with weights on three linear models:

Usage

In order to predict the class labels based on the predicted class-probabilities (scikit-learn estimators in the VotingClassifier must support `predict_proba` method):

```
>>> eclf = VotingClassifier(
...     estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3)],
...     voting='soft'
... )
```

Optionally, weights can be provided for the individual classifiers:

```
>>> eclf = VotingClassifier(
...     estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3)],
...     voting='soft', weights=[2,5,1]
... )
```

Using the `VotingClassifier` with `GridSearchCV`
The `VotingClassifier` can also be used together with hyperparameters of the individual estimators:

```
>>> from sklearn.model_selection import GridSearchCV
>>> clf1 = LogisticRegression(random_state=1)
>>> clf2 = RandomForestClassifier(random_state=1)
>>> clf3 = GaussianNB()
>>> eclf = VotingClassifier(
...     estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3)],
...     voting='soft'
... )
```

```
>>> params = {'lr__C': [1.0, 100.0], 'rf__n_estimators': [20, 200]}

>>> grid = GridSearchCV(estimator=eclf, param_grid=params, cv=5)
>>> grid = grid.fit(iris.data, iris.target)
```

Voting Regressor

The idea behind the `VotingRegressor` is to combine conceptually different machine learning regressors and return the average predicted values. Such a regressor can be useful for a set of equally well performing models in order to balance out their individual weaknesses.

Usage

The following example shows how to fit the VotingRegressor:

```
>>> from sklearn.datasets import load_diabetes
>>> from sklearn.ensemble import GradientBoostingRegressor
>>> from sklearn.ensemble import RandomForestRegressor
>>> from sklearn.linear_model import LinearRegression
>>> from sklearn.ensemble import VotingRegressor

>>> # Loading some example data
>>> X, y = load_diabetes(return_X_y=True)

>>> # Training classifiers
>>> reg1 = GradientBoostingRegressor(random_state=1)
>>> reg2 = RandomForestRegressor(random_state=1)
>>> reg3 = LinearRegression()
>>> ereg = VotingRegressor(estimators=[('gb', reg1), ('rf', reg2), ('lr', reg3)])
>>> ereg = ereg.fit(X, y)
```

Examples

- sphx_glr_auto_examples_ensemble_plot_voting_regressor.py

Stacked generalization

Stacked generalization is a method for combining estimators to reduce their biases [W1992] [HTF]. More precisely, the predictions of each individual estimator are stacked together and used as input to a final estimator to compute the prediction. This final estimator is trained through cross-validation.

The `StackingClassifier` and `StackingRegressor` provide such strategies which can be applied to classification and regression problems.

The estimators parameter corresponds to the list of the estimators which are stacked together in parallel on the input data. It should be given as a list of names and estimators:

```
>>> from sklearn.linear_model import RidgeCV, LassoCV
>>> from sklearn.neighbors import KNeighborsRegressor
>>> estimators = [('ridge', RidgeCV()),
...               ('lasso', LassoCV(random_state=42)),
...               ('knr', KNeighborsRegressor(n_neighbors=20,
...                                           metric='euclidean'))]
```

The final_estimator will use the predictions of the estimators as input. It needs to be a classifier or a regressor when using `StackingClassifier` or `StackingRegressor`, respectively:

```
>>> from sklearn.ensemble import GradientBoostingRegressor
>>> from sklearn.ensemble import StackingRegressor
>>> final_estimator = GradientBoostingRegressor(
...     n_estimators=25, subsample=0.5, min_samples_leaf=25, max_features=1,
...     random_state=42)
>>> reg = StackingRegressor(
...     estimators=estimators,
...     final_estimator=final_estimator)
```

To train the estimators and final_estimator, the fit method needs to be called on the training data:

```
>>> from sklearn.datasets import load_diabetes
>>> X, y = load_diabetes(return_X_y=True)
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...                                                     random_state=42)
>>> reg.fit(X_train, y_train)
StackingRegressor(...)
```

During training, the estimators are fitted on the whole training data X_train. They will be used when calling predict or predict_proba. To generalize and avoid over-fitting, the final_estimator is trained on out-samples using `sklearn.model_selection.cross_val_predict` internally.

For `StackingClassifier`, note that the output of the `estimators` is controlled by the parameter stack_method and it is called by each estimator. This parameter is either a string, being estimator method names, or 'auto' which will automatically identify an available method depending on the availability, tested in the order of preference: predict_proba, decision_function and predict.

A `StackingRegressor` and `StackingClassifier` can be used as any other regressor or classifier, exposing a predict, predict_proba, or decision_function method, e.g.:

```
>>> y_pred = reg.predict(X_test)
>>> from sklearn.metrics import r2_score
>>> print('R2 score: {:.2f}'.format(r2_score(y_test, y_pred)))
R2 score: 0.53
```

Note that it is also possible to get the output of the stacked estimators using the transform method:

```
>>> reg.transform(X_test[:5])
array([[142, 138, 146],
       [179, 182, 151],
       [139, 132, 158],
       [286, 292, 225],
       [126, 124, 164]])
```

In practice, a stacking predictor predicts as good as the best predictor of the base layer and even sometimes outperforms it by combining the different strengths of these predictors. However, training a stacking predictor is computationally expensive.

> Note
> For `StackingClassifier`, when using stack_method_='predict_proba', the first column is dropped when the problem is a binary classification problem. Indeed, both probability columns predicted by each estimator are perfectly collinear.

Note
Multiple stacking layers can be achieved by assigning final_estimator to a `StackingClassifier` or `StackingRegressor`:

```
>>> final_layer_rfr = RandomForestRegressor(
...     n_estimators=10, max_features=1, max_leaf_nodes=5,random_state=42)
>>> final_layer_gbr = GradientBoostingRegressor(
...     n_estimators=10, max_features=1, max_leaf_nodes=5,random_state=42)
>>> final_layer = StackingRegressor(
...     estimators=[('rf', final_layer_rfr),
...                 ('gbrt', final_layer_gbr)],
...     final_estimator=RidgeCV()
...     )
>>> multi_layer_regressor = StackingRegressor(
...     estimators=[('ridge', RidgeCV()),
...                 ('lasso', LassoCV(random_state=42)),
...                 ('knr', KNeighborsRegressor(n_neighbors=20,
...                                             metric='euclidean'))],
...     final_estimator=final_layer
... )
>>> multi_layer_regressor.fit(X_train, y_train)
StackingRegressor(...)
>>> print('R2 score: {:.2f}'
...       .format(multi_layer_regressor.score(X_test, y_test)))
R2 score: 0.53
```

Examples

- sphx_glr_auto_examples_ensemble_plot_stack_predictors.py

References

AdaBoost

The module `sklearn.ensemble` includes the popular boosting algorithm AdaBoost, introduced in 1995 by Freund and Schapire [FS1995].

The core principle of AdaBoost is to fit a sequence of weak learners (i.e., models that are only slightly better than random guessing, such as small decision trees) on repeatedly modified versions of the data. The predictions from all of them are then combined through a weighted majority vote (or sum) to produce the final prediction. The data modifications at each so-called boosting iteration consists of applying weights w_1, w_2, ..., w_N to each of the training samples. Initially, those weights are all set to $w_i = 1/N$, so that the first step simply trains a weak learner on the original data. For each successive iteration, the sample weights are individually modified and the learning algorithm is reapplied to the reweighted data. At a given step, those training examples that were incorrectly predicted by the boosted model induced at the previous step have their weights increased, whereas the weights are decreased for those that were predicted correctly. As iterations proceed, examples that are difficult to predict receive ever-increasing influence. Each subsequent weak learner is thereby forced to concentrate on the examples that are missed by the previous ones in the sequence [HTF].

AdaBoost can be used both for classification and regression problems:

- For multi-class classification, `AdaBoostClassifier` implements AdaBoost.SAMME [ZZRH2009].
- For regression, `AdaBoostRegressor` implements AdaBoost.R2 [D1997].

Usage

The following example shows how to fit an AdaBoost classifier with 100 weak learners:

```
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.datasets import load_iris
>>> from sklearn.ensemble import AdaBoostClassifier

>>> X, y = load_iris(return_X_y=True)
>>> clf = AdaBoostClassifier(n_estimators=100)
>>> scores = cross_val_score(clf, X, y, cv=5)
>>> scores.mean()
np.float64(0.95)
```

The number of weak learners is controlled by the parameter `n_estimators`. The `learning_rate` parameter controls the contribution of the weak learners in the final combination. By default, weak learners are decision stumps. Different weak learners can be specified through the `estimator` parameter. The main parameters to tune to obtain good results are `n_estimators` and the complexity of the base estimators (e.g., its depth `max_depth` or minimum required number of samples to consider a split `min_samples_split`).

Examples

- sphx_glr_auto_examples_ensemble_plot_adaboost_multiclass.py shows the performance of AdaBoost on a multi-class problem.
- sphx_glr_auto_examples_ensemble_plot_adaboost_twoclass.py shows the decision boundary and decision function values for a non-linearly separable two-class problem using AdaBoost-SAMME.
- sphx_glr_auto_examples_ensemble_plot_adaboost_regression.py demonstrates regression with the AdaBoost.R2 algorithm.

References

B1996 L. Breiman, ”Bagging predictors”, Machine Learning, 24(2), 123-140, 1996.

B1998 L. Breiman, ”Arcing Classifiers”, Annals of Statistics 1998.

B1999 L. Breiman, ”Pasting small votes for classification in large databases and on-line”, Machine Learning, 36(1), 85-103, 1999.

B2001 L. Breiman, ”Random Forests”, Machine Learning, 45(1), 5-32, 2001.

D1997 H. Drucker. ”Improving Regressors using Boosting Techniques”, 1997.

FS1995 Y. Freund, and R. Schapire, ”A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting”, 1997.

Fisher1958 Fisher, W.D. (1958). "On Grouping for Maximum Homogeneity" <10.1080/01621459.1958.10501479> Journal of the American Statistical Association, 53, 789-798.

Friedman2001 Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine <10.1214/aos/1013203451>. Annals of Statistics, 29, 1189-1232.

Friedman2002 Friedman, J.H. (2002). Stochastic gradient boosting. <10.1016/S0167-9473(01)00065-2>. Computational Statistics & Data Analysis, 38, 367-378.

H1998 T. Ho, ”The random subspace method for constructing decision forests”, Pattern Analysis and Machine Intelligence, 20(8), 832-844, 1998.

HTF T. Hastie, R. Tibshirani and J. Friedman, ”Elements of Statistical Learning Ed. 2”, Springer, 2009.

L2014 G. Louppe, "Understanding Random Forests: From Theory to Practice" <1407.7502>, PhD Thesis, U. of Liege, 2014.

LG2012 G. Louppe and P. Geurts, ”Ensembles on Random Patches”, Machine Learning and Knowledge Discovery in Databases, 346-361, 2012.

LightGBM Ke et. al. ”LightGBM: A Highly Efficient Gradient BoostingDecision Tree” <https://papers.nips.cc/paper/ 6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>

Mayer2022 M. Mayer, S.C. Bourassa, M. Hoesli, and D.F. Scognamiglio. 2022. Machine Learning Applications to Land and Structure Valuation <10.3390/jrfm15050193>. Journal of Risk and Financial Management 15, no. 5: 193

R2007 G. Ridgeway (2006). Generalized Boosted Models: A guide to the gbm package

W1992 Wolpert, David H. ”Stacked generalization.” Neural networks 5.2 (1992): 241-259.

XGBoost Tianqi Chen, Carlos Guestrin, "XGBoost: A Scalable Tree Boosting System" <1603.02754>

ZZRH2009
J. Zhu, H. Zou, S. Rosset, T. Hastie. ”Multi-class AdaBoost”, 2009.

Model persistence

Source: *https://scikit-learn.org/stable/model_persistence.html*

Table 16: Summary of model persistence methods

Persistence method	Pros	Risks / Cons
ONNX	• Serve models without a Python environment • Serving and training environments independent of one another • Most secure option	• Not all scikit-learn models are supported • Custom estimators require more work to support • Original Python object is lost and cannot be reconstructed
skops_persistence	• More secure than pickle based formats • Contents can be partly validated without loading	• Not as fast as pickle based formats • Supports less types than pickle based formats • Requires the same environment as the training environment
`pickle`	• Native to Python • Can serialize most Python objects • Efficient memory usage with protocol=5	• Loading can execute arbitrary code • Requires the same environment as the training environment
`joblib`	• Efficient memory usage • Supports memory mapping • Easy shortcuts for compression and decompression	• Pickle based format • Loading can execute arbitrary code • Requires the same environment as the training environment
cloudpickle	• Can serialize non-packaged, custom Python code • Comparable loading efficiency as `pickle` with protocol=5	• Pickle based format • Loading can execute arbitrary code • No forward compatibility guarantees • Requires the same environment as the training environment

After training a scikit-learn model, it is desirable to have a way to persist the model for future use without having to retrain. Based on your use-case, there are a few different ways to persist a scikit-learn model, and here we help you decide which one suits you best. In order to make a decision, you need to answer the following questions:

1. Do you need the Python object after persistence, or do you only need to persist in order to serve the model and get predictions out of it?

If you only need to serve the model and no further investigation on the Python object itself is required, then ONNX might be the best fit for you. Note that not all models are supported by

ONNX.

In case ONNX is not suitable for your use-case, the next question is:

2. Do you absolutely trust the source of the model, or are there any security concerns regarding where the persisted model comes from?

If you have security concerns, then you should consider using `skops.io <skops_persistence>` which gives you back the Python object, but unlike pickle based persistence solutions, loading the persisted model doesn't automatically allow arbitrary code execution. Note that this requires manual investigation of the persisted file, which `skops.io` allows you to do.

The other solutions assume you absolutely trust the source of the file to be loaded, as they are all susceptible to arbitrary code execution upon loading the persisted file since they all use the pickle protocol under the hood.

3. Do you care about the performance of loading the model, and sharing it between processes where a memory mapped object on disk is beneficial?

If yes, then you can consider using joblib. If this is not a major concern for you, then you can use the built-in `pickle` module.

4. Did you try `pickle` or `joblib` and found that the model cannot be persisted? It can happen for instance when you have user defined functions in your model.

If yes, then you can use cloudpickle which can serialize certain objects which cannot be serialized by `pickle` or `joblib`.

Workflow Overview

In a typical workflow, the first step is to train the model using scikit-learn and scikit-learn compatible libraries. Note that support for scikit-learn and third party estimators varies across the different persistence methods.

Train and Persist the Model

Creating an appropriate model depends on your use-case. As an example, here we train a `sklearn.ensemble.HistGradientBoostingClassifier` on the iris dataset:

```
>>> from sklearn import ensemble
>>> from sklearn import datasets
>>> clf = ensemble.HistGradientBoostingClassifier()
>>> X, y = datasets.load_iris(return_X_y=True)
>>> clf.fit(X, y)
HistGradientBoostingClassifier()
```

Once the model is trained, you can persist it using your desired method, and then you can load the model in a separate environment and get predictions from it given input data. Here there are two major paths depending on how you persist and plan to serve the model:

- ONNX: You need an ONNX runtime and an environment with appropriate dependencies installed to load the model and use the runtime to get predictions. This environment can be minimal and does not necessarily even require Python to be installed to load the model and compute predictions. Also note that onnxruntime typically requires much less RAM than Python to compute predictions from small models.
- `skops.io`, `pickle`, `joblib`, cloudpickle: You need a Python environment with the appropriate dependencies installed to load the model and get predictions from it. This environment should have the same **packages** and the same **versions** as the environment where the model was trained. Note that none of these methods support loading a model trained with

a different version of scikit-learn, and possibly different versions of other dependencies such as numpy and scipy. Another concern would be running the persisted model on a different hardware, and in most cases you should be able to load your persisted model on a different hardware.

ONNX

ONNX, or Open Neural Network Exchange format is best suitable in use-cases where one needs to persist the model and then use the persisted artifact to get predictions without the need to load the Python object itself. It is also useful in cases where the serving environment needs to be lean and minimal, since the ONNX runtime does not require python.

ONNX is a binary serialization of the model. It has been developed to improve the usability of the interoperable representation of data models. It aims to facilitate the conversion of the data models between different machine learning frameworks, and to improve their portability on different computing architectures. More details are available from the ONNX tutorial. To convert scikit-learn model to ONNX sklearn-onnx has been developed. However, not all scikit-learn models are supported, and it is limited to the core scikit-learn and does not support most third party estimators. One can write a custom converter for third party or custom estimators, but the documentation to do that is sparse and it might be challenging to do so.

Using ONNX
To convert the model to ONNX format, you need to give the converter some information about the input as well, about which you can read more here:

```
from skl2onnx import to_onnx
onx = to_onnx(clf, X[:1].astype(numpy.float32), target_opset=12)
with open("filename.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

You can load the model in Python and use the ONNX runtime to get predictions:

```
from onnxruntime import InferenceSession
with open("filename.onnx", "rb") as f:
    onx = f.read()
sess = InferenceSession(onx, providers=["CPUExecutionProvider"])
pred_ort = sess.run(None, {"X": X_test.astype(numpy.float32)})[0]
```

skops.io

`skops.io` avoids using `pickle` and only loads files which have types and references to functions which are trusted either by default or by the user. Therefore it provides a more secure format than `pickle`, `joblib`, and cloudpickle.

Using skops
The API is very similar to `pickle`, and you can persist your models as explained in the documentation using

```
import skops.io as sio obj = sio.dump(clf, ”filename.skops”)
```

And you can load them back using `skops.io.load` and trusted by you. You can get existing unknown types in a dumped object / file using `skops.io.get_untrusted_types`, and after checking its contents, pass it to the load function:

```
unknown_types = sio.get_untrusted_types(file="filename.skops")
# investigate the contents of unknown_types, and only load if you trust
```

```
# everything you see.
clf = sio.load("filename.skops", trusted=unknown_types)
```
Please report issues and feature requests related to this format on the skops issue tracker.

pickle, joblib, and cloudpickle

These three modules / packages, use the pickle protocol under the hood, but come with slight variations:

- `pickle` is a module from the Python Standard Library. It can serialize and deserialize any Python object, including custom Python classes and objects.
- `joblib` is more efficient than pickle when working with large machine learning models or large numpy arrays.
- cloudpickle can serialize certain objects which cannot be serialized by `pickle` or `joblib`, such as user defined functions and lambda functions. This can happen for instance, when using a `FunctionTransformer` and using a custom function to transform the data.

Using pickle, joblib, or cloudpickle
Depending on your use-case, you can choose one of these three methods to persist and load your scikit-learn model, and they all follow the same API:

```
# Here you can replace pickle with joblib or cloudpickle
from pickle import dump
with open("filename.pkl", "wb") as f:
    dump(clf, f, protocol=5)
```

Using protocol=5 is recommended to reduce memory usage and make it faster to store and load any large NumPy array stored as a fitted attribute in the model. You can alternatively pass protocol=pickle.HIGHEST_PROTOCOL which is equivalent to protocol=5 in Python 3.8 and later (at the time of writing).
And later when needed, you can load the same object from the persisted file:

```
# Here you can replace pickle with joblib or cloudpickle
from pickle import load
with open("filename.pkl", "rb") as f:
    clf = load(f)
```

Security & Maintainability Limitations

`pickle` (and `joblib` and `cloudpickle` by extension), has many documented security vulnerabilities by design and should only be used if the artifact, i.e. the pickle-file, is coming from a trusted and verified source. You should never load a pickle file from an untrusted source, similarly to how you should never execute code from an untrusted source.

Also note that arbitrary computations can be represented using the ONNX format, and it is therefore recommended to serve models using ONNX in a sandboxed environment to safeguard against computational and memory exploits.

Also note that there are no supported ways to load a model trained with a different version of scikit-learn. While using `skops.io`, `joblib`, `pickle`, or cloudpickle, models saved using one version of scikit-learn might load in other versions, however, this is entirely unsupported and inadvisable. It should also be kept in mind that operations performed on such data could give different and unexpected results, or even crash your Python process.

In order to rebuild a similar model with future versions of scikit-learn, additional metadata should be saved along the pickled model:

- The training data, e.g. a reference to an immutable snapshot
- The Python source code used to generate the model
- The versions of scikit-learn and its dependencies
- The cross validation score obtained on the training data

This should make it possible to check that the cross-validation score is in the same range as before.

Aside for a few exceptions, persisted models should be portable across operating systems and hardware architectures assuming the same versions of dependencies and Python are used. If you encounter an estimator that is not portable, please open an issue on GitHub. Persisted models are often deployed in production using containers like Docker, in order to freeze the environment and dependencies.

If you want to know more about these issues, please refer to these talks:

- Adrin Jalali: Let's exploit pickle, and skops to the rescue! | PyData Amsterdam 2023.
- Alex Gaynor: Pickles are for Delis, not Software - PyCon 2014.

Replicating the training environment in production

If the versions of the dependencies used may differ from training to production, it may result in unexpected behaviour and errors while using the trained model. To prevent such situations it is recommended to use the same dependencies and versions in both the training and production environment. These transitive dependencies can be pinned with the help of package management tools like pip, mamba, conda, poetry, conda-lock, pixi, etc.

It is not always possible to load a model trained with older versions of the scikit-learn library and its dependencies in an updated software environment. Instead, you might need to retrain the model with the new versions of all the libraries. So when training a model, it is important to record the training recipe (e.g. a Python script) and training set information, and metadata about all the dependencies to be able to automatically reconstruct the same training environment for the updated software.

InconsistentVersionWarning
When an estimator is loaded with a scikit-learn version that is inconsistent with the version the estimator was pickled with, an can be caught to obtain the original version the estimator was pickled with:

```
from sklearn.exceptions import InconsistentVersionWarning
warnings.simplefilter("error", InconsistentVersionWarning)

try:
    with open("model_from_previous_version.pickle", "rb") as f:
        est = pickle.load(f)
except InconsistentVersionWarning as w:
    print(w.original_sklearn_version)
```

Serving the model artifact

The last step after training a scikit-learn model is serving the model. Once the trained model is successfully loaded, it can be served to manage different prediction requests. This can involve deploying the model as a web service using containerization, or other model deployment strategies, according to the specifications.

Summarizing the key points

Based on the different approaches for model persistence, the key points for each approach can be summarized as follows:

- 'ONNX': It provides a uniform format for persisting any machine learning or deep learning model (other than scikit-learn) and is useful for model inference (predictions). It can however, result in compatibility issues with different frameworks.
- `skops.io`: Trained scikit-learn models can be easily shared and put into production using `skops.io`. It is more secure compared to alternate approaches based on `pickle` because it does not load arbitrary code unless explicitly asked for by the user. Such code needs to be packaged and importable in the target Python environment.
- `joblib`: Efficient memory mapping techniques make it faster when using the same persisted model in multiple Python processes when using mmap_mode="r". It also gives easy shortcuts to compress and decompress the persisted object without the need for extra code. However, it may trigger the execution of malicious code when loading a model from an untrusted source as any other pickle-based persistence mechanism.
- `pickle`: It is native to Python and most Python objects can be serialized and deserialized using `pickle`, including custom Python classes and functions as long as they are defined in a package that can be imported in the target environment. While `pickle` can be used to easily save and load scikit-learn models, it may trigger the execution of malicious code while loading a model from an untrusted source. `pickle` can also be very efficient memorywise if the model was persisted with protocol=5 but it does not support memory mapping.
- cloudpickle: It has comparable loading efficiency as `pickle` and `joblib` (without memory mapping), but offers additional flexibility to serialize custom Python code such as lambda expressions and interactively defined functions and classes. It might be a last resort to persist pipelines with custom Python components such as a `sklearn.preprocessing.FunctionTransformer` that wraps a function defined in the training script itself or more generally outside of any importable Python package. Note that cloudpickle offers no forward compatibility guarantees and you might need the same version of cloudpickle to load the persisted model along with the same version of all the libraries used to define the model. As the other pickle-based persistence mechanisms, it may trigger the execution of malicious code while loading a model from an untrusted source.

API quick reference

Generated from the docstrings of the installed scikit-learn 1.9.1 package, which are the text the official API reference pages are built from. Full reference: https://scikit-learn.org/stable/api/index.html

Model selection

sklearn.model_selection.train_test_split

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None,
↪  shuffle=True, stratify=None)
```

Split arrays or matrices into random train and test subsets.

Quick utility that wraps input validation, `next(ShuffleSplit().split(X, y))`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner.

Read more in the User Guide.

Parameters

`*arrays` : *sequence of array-like of shape (n_samples,) or (n_samples, n_outputs)*
Indexable data-structures can be arrays, lists, dataframes, scipy sparse matrices or pandas dataframes with consistent first dimension.

`test_size` : *float or int, default=None* If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the test split. If int, represents the absolute number of test samples. If None, the value is set to the complement of the train size. If `train_size` is also None, it will be set to 0.25.

`train_size` : *float or int, default=None* If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the train split. If int, represents the absolute number of train samples. If None, the value is automatically set to the complement of the test size.

`random_state` : *int, RandomState instance or None, default=None* Controls the shuffling applied to the data before applying the split. Pass an int for reproducible output across multiple function calls. See Glossary.

`shuffle` : *bool, default=True* Whether or not to shuffle the data before splitting. If shuffle=False then stratify must be None.

`stratify` : *array-like of shape (n_samples,), default=None* If not None, data is split in a stratified fashion, using this as the class labels. Read more in the User Guide.

Returns

`splitting` : *list, length=2 len(arrays)* List containing train-test split of inputs.

New in version 0.16. If the input is sparse, the output will be a `scipy.sparse.csr_matrix`. Else, output type is the same as the input type.

Examples

```
>>> from sklearn.model_selection import train_test_split
>>> X, y = np.arange(10).reshape((5, 2)), range(5)
```

```
>>> X
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> list(y)
[0, 1, 2, 3, 4]
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.33, random_state=42)
...
>>> X_train
array([[4, 5],
       [0, 1],
       [6, 7]])
>>> y_train
[2, 0, 3]
>>> X_test
array([[2, 3],
       [8, 9]])
>>> y_test
...
```

sklearn.model_selection.cross_val_score

```
sklearn.model_selection.cross_val_score(estimator, X, y=None, *, groups=None, scoring=None, cv=None,
↪  n_jobs=None, verbose=0, params=None, pre_dispatch='2*n_jobs', error_score=nan)
```

Evaluate a score by cross-validation.

Read more in the User Guide.

Parameters

`estimator` : ***estimator object implementing 'fit'*** The object to use to fit the data.

`X` : ***{array-like, sparse matrix} of shape (n_samples, n_features)*** The data to fit. Can be for example a list, or an array.

`y` : ***array-like of shape (n_samples,) or (n_samples, n_outputs), default=None*** The target variable to try to predict in the case of supervised learning.

`groups` : ***array-like of shape (n_samples,), default=None*** Group labels for the samples used while splitting the dataset into train/test set. Only used in conjunction with a "Group" cv instance (e.g., `GroupKFold`).

Changed in version 1.4. `groups` can only be passed if metadata routing is not enabled via `sklearn.set_config(enable_metadata_routing=True)`. When routing is enabled, pass `groups` alongside other metadata via the `params` argument instead. E.g.: `cross_val_score(..., params={'groups': groups})`.

`scoring` : ***str or callable, default=None*** Strategy to evaluate the performance of the estimator across cross-validation splits.

- str: see scoring_string_names for options.
- callable: a scorer callable object (e.g., function) with signature `scorer(estimator, X, y)`, which should return only a single value. See scoring_callable for details.
- 'None': the estimator's default evaluation criterion is used.

Similar to the use of scoring in `cross_validate` but only a single metric is permitted.

`cv` : *int, cross-validation generator or an iterable, default=None* Determines the cross-validation splitting strategy. Possible inputs for cv are:

- None, to use the default 5-fold cross validation,
- int, to specify the number of folds in a (Stratified)KFold,
- CV splitter,
- An iterable that generates (train, test) splits as arrays of indices.

For int/None inputs, if the estimator is a classifier and y is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with shuffle=False so the splits will be the same across calls.

Refer User Guide for the various cross-validation strategies that can be used here.

Changed in version 0.22. cv default value if None changed from 3-fold to 5-fold.

`n_jobs` : *int, default=None* Number of jobs to run in parallel. Training the estimator and computing the score are parallelized over the cross-validation splits. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

`verbose` : *int, default=0* The verbosity level.

`params` : *dict, default=None* Parameters to pass to the underlying estimator's `fit`, the scorer, and the CV splitter.

New in version 1.4.

`pre_dispatch` : *int or str, default='2n_jobs'* Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

- `None`, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs
- An int, giving the exact number of total jobs that are spawned
- A str, giving an expression as a function of n_jobs, as in '2*n_jobs'

`error_score` : *'raise' or numeric, default=np.nan* Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised.

New in version 0.20.

Returns

`scores` : *ndarray of float of shape=(len(list(cv)),)* Array of scores of the estimator for each run of the cross validation.

See also: `cross_validate`, `cross_val_predict`, `sklearn.metrics.make_scorer`

Examples

```
>>> from sklearn import datasets, linear_model
>>> from sklearn.model_selection import cross_val_score
>>> diabetes = datasets.load_diabetes()
>>> X = diabetes.data[:150]
>>> y = diabetes.target[:150]
>>> lasso = linear_model.Lasso()
>>> print(cross_val_score(lasso, X, y, cv=3))
[0.3315057  0.08022103 0.03531816]
```

sklearn.model_selection.cross_validate

```
sklearn.model_selection.cross_validate(estimator, X, y=None, *, groups=None, scoring=None, cv=None,
↪  n_jobs=None, verbose=0, params=None, pre_dispatch='2*n_jobs', return_train_score=False,
↪  return_estimator=False, return_indices=False, error_score=nan)
```

Evaluate metric(s) by cross-validation and also record fit/score times.

Read more in the User Guide.

Parameters

estimator : *estimator object implementing 'fit'* The object to use to fit the data.

X : *{array-like, sparse matrix} of shape (n_samples, n_features)* The data to fit. Can be for example a list, or an array.

y : *array-like of shape (n_samples,) or (n_samples, n_outputs), default=None* The target variable to try to predict in the case of supervised learning.

groups : *array-like of shape (n_samples,), default=None* Group labels for the samples used while splitting the dataset into train/test set. Only used in conjunction with a "Group" cv instance (e.g., `GroupKFold`).

Changed in version 1.4. `groups` can only be passed if metadata routing is not enabled via `sklearn.set_config(enable_metadata_routing=True)`. When routing is enabled, pass `groups` alongside other metadata via the `params` argument instead. E.g.: `cross_validate(..., params={'groups': groups})`.

scoring : *str, callable, list, tuple, or dict, default=None* Strategy to evaluate the performance of the estimator across cross-validation splits.

If scoring represents a single score, one can use:

- a single string (see scoring_string_names);
- a callable (see scoring_callable) that returns a single value.
- None, the estimator's default evaluation criterion is used.

If scoring represents multiple scores, one can use:

- a list or tuple of unique strings;
- a callable returning a dictionary where the keys are the metric names and the values are the metric scores;
- a dictionary with metric names as keys and callables as values.

See multimetric_grid_search for an example.

cv : *int, cross-validation generator or an iterable, default=None* Determines the cross-validation splitting strategy. Possible inputs for cv are:

- None, to use the default 5-fold cross validation,
- int, to specify the number of folds in a (Stratified)KFold,
- CV splitter,
- an iterable yielding (train, test) splits as arrays of indices.

For int/None inputs, if the estimator is a classifier and `y` is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with shuffle=False so the splits will be the same across calls.

Refer User Guide for the various cross-validation strategies that can be used here.

Changed in version 0.22. `cv` default value if None changed from 3-fold to 5-fold.

n_jobs : *int, default=None* Number of jobs to run in parallel. Training the estimator and computing the score are parallelized over the cross-validation splits. `None` means 1 unless in

a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

`verbose` : ***int, default=0*** The verbosity level.

`params` : ***dict, default=None*** Parameters to pass to the underlying estimator's `fit`, the scorer, and the CV splitter.

New in version 1.4.

`pre_dispatch` : ***int or str, default='2n_jobs'*** Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

- An int, giving the exact number of total jobs that are spawned
- A str, giving an expression as a function of n_jobs, as in '2*n_jobs'

`return_train_score` : ***bool, default=False*** Whether to include train scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance.

New in version 0.19.

Changed in version 0.21. Default value was changed from `True` to `False`

`return_estimator` : ***bool, default=False*** Whether to return the estimators fitted on each split.

New in version 0.20.

`return_indices` : ***bool, default=False*** Whether to return the train-test indices selected for each split.

New in version 1.3.

`error_score` : ***'raise' or numeric, default=np.nan*** Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised.

New in version 0.20.

Returns

`scores` : ***dict of float arrays of shape (n_splits,)*** Array of scores of the estimator for each run of the cross validation.

A dict of arrays containing the score/time arrays for each scorer is returned. The possible keys for this `dict` are:

`test_score` The score array for test scores on each cv split. Suffix `_score` in `test_score` changes to a specific metric like `test_r2` or `test_auc` if there are multiple scoring metrics in the scoring parameter.

`train_score` The score array for train scores on each cv split. Suffix `_score` in `train_score` changes to a specific metric like `train_r2` or `train_auc` if there are multiple scoring metrics in the scoring parameter. This is available only if `return_train_score` parameter is `True`.

`fit_time` The time for fitting the estimator on the train set for each cv split.

`score_time` The time for scoring the estimator on the test set for each cv split. (Note: time for scoring on the train set is not included even if `return_train_score` is set to `True`).

estimator The estimator objects for each cv split. This is available only if `return_estimator` parameter is set to `True`.

indices The train/test positional indices for each cv split. A dictionary is returned where the keys are either "train" or "test" and the associated values are a list of integer-dtyped NumPy arrays with the indices. Available only if return_indices=True.

See also: `cross_val_score`, `cross_val_predict`, `sklearn.metrics.make_scorer`

Examples

```
>>> from sklearn import datasets, linear_model
>>> from sklearn.model_selection import cross_validate
>>> diabetes = datasets.load_diabetes()
>>> X = diabetes.data[:150]
>>> y = diabetes.target[:150]
>>> lasso = linear_model.Lasso()
```

Single metric evaluation using `cross_validate`

```
>>> cv_results = cross_validate(lasso, X, y, cv=3)
>>> sorted(cv_results.keys())
['fit_time', 'score_time', 'test_score']
>>> cv_results['test_score']
array([0.3315057 , 0.08022103, 0.03531816])
```

Multiple metric evaluation using `cross_validate` (please refer the `scoring` parameter doc for more information) ...

sklearn.model_selection.KFold

```
sklearn.model_selection.KFold(n_splits=5, *, shuffle=False, random_state=None)
```

K-Fold cross-validator.

Provides train/test indices to split data in train/test sets. Split dataset into k consecutive folds (without shuffling by default).

Each fold is then used once as a validation while the k - 1 remaining folds form the training set.

Read more in the User Guide.

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to sphx_glr_auto_examples_model_selection_plot_cv_indices.py

Parameters

n_splits : *int, default=5* Number of folds. Must be at least 2.

Changed in version 0.22. `n_splits` default value changed from 3 to 5.

shuffle : *bool, default=False* Whether to shuffle the data before splitting into batches. Note that the samples within each split will not be shuffled.

random_state : *int, RandomState instance or None, default=None* When shuffle is True, random_state affects the ordering of the indices, which controls the randomness of each fold. Otherwise, this parameter has no effect. Pass an int for reproducible output across multiple function calls. See Glossary.

See also: `StratifiedKFold`, `GroupKFold`, `RepeatedKFold`

Notes

The first `n_samples % n_splits` folds have size `n_samples // n_splits + 1`, other folds have size `n_samples // n_splits`, where `n_samples` is the number of samples.

Randomized CV splitters may return different results for each call of split. You can make the results identical by setting random_state to an integer.

Examples

```
>>> from sklearn.model_selection import KFold
>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
>>> y = np.array([1, 2, 3, 4])
>>> kf = KFold(n_splits=2)
>>> kf.get_n_splits()
2
>>> print(kf)
KFold(n_splits=2, random_state=None, shuffle=False)
>>> for i, (train_index, test_index) in enumerate(kf.split(X)):
...     print(f"Fold {i}:")
...     print(f"  Train: index={train_index}")
...     print(f"  Test:  index={test_index}")
Fold 0:
  Train: index=[2 3]
  Test:  index=[0 1]
Fold 1:
  Train: index=[0 1]
  Test:  index=[2 3]
...
```

sklearn.model_selection.StratifiedKFold

```
sklearn.model_selection.StratifiedKFold(n_splits=5, *, shuffle=False, random_state=None)
```

Class-wise stratified K-Fold cross-validator.

Provides train/test indices to split data in train/test sets.

This cross-validation object is a variation of KFold that returns stratified folds. The folds are made by preserving the percentage of samples for each class in y in a binary or multiclass classification setting.

Read more in the User Guide.

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to sphx_glr_auto_examples_model_selection_plot_cv_indices.py

> Note
> Stratification on the class label solves an engineering problem rather than a statistical one. See stratification for more details.

Parameters

`n_splits` : ***int, default=5*** Number of folds. Must be at least 2.

Changed in version 0.22. `n_splits` default value changed from 3 to 5.

`shuffle` : ***bool, default=False*** Whether to shuffle each class's samples before splitting into batches. Note that the samples within each split will not be shuffled.

`random_state` : ***int, RandomState instance or None, default=None*** When shuffle is True, random_state affects the ordering of the indices, which controls the randomness of each

fold for each class. Otherwise, leave random_state as None. Pass an int for reproducible output across multiple function calls. See Glossary.

See also: `RepeatedStratifiedKFold`

Notes

The implementation is designed to:

- Generate test sets such that all contain the same distribution of classes, or as close as possible.
- Be invariant to class label: relabelling `y = ["Happy", "Sad"]` to `y = [1, 0]` should not change the indices generated.
- Preserve order dependencies in the dataset ordering, when `shuffle=False`: all samples from class k in some test set were contiguous in y, or separated in y by samples from classes other than k.
- Generate test sets where the smallest and largest differ by at most one sample.

Changed in version 0.22. The previous implementation did not follow the last constraint.

Examples

```
>>> from sklearn.model_selection import StratifiedKFold
>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4]])
>>> y = np.array([0, 0, 1, 1])
>>> skf = StratifiedKFold(n_splits=2)
>>> skf.get_n_splits()
2
>>> print(skf)
StratifiedKFold(n_splits=2, random_state=None, shuffle=False)
>>> for i, (train_index, test_index) in enumerate(skf.split(X, y)):
...     print(f"Fold {i}:")
...     print(f"  Train: index={train_index}")
...     print(f"  Test:  index={test_index}")
Fold 0:
  Train: index=[1 3]
  Test:  index=[0 2]
Fold 1:
  Train: index=[0 2]
  Test:  index=[1 3]
...
```

sklearn.model_selection.TimeSeriesSplit

```
sklearn.model_selection.TimeSeriesSplit(n_splits=5, *, max_train_size=None, test_size=None, gap=0)
```

Time Series cross-validator.

Provides train/test indices to split time-ordered data, where other cross-validation methods are inappropriate, as they would lead to training on future data and evaluating on past data. To ensure comparable metrics across folds, samples must be equally spaced. Once this condition is met, each test set covers the same time duration, while the train set size accumulates data from previous splits.

This cross-validation object is a variation of `KFold`. In the k-th split, it returns the first k folds as the train set and the (k+1)-th fold as the test set.

Note that, unlike standard cross-validation methods, successive training sets are supersets of those that come before them.

Read more in the User Guide.

For visualisation of cross-validation behaviour and comparison between common scikit-learn split methods refer to sphx_glr_auto_examples_model_selection_plot_cv_indices.py

New in version 0.18.

Parameters

`n_splits` : *int, default=5* Number of splits. Must be at least 2.

Changed in version 0.22. `n_splits` default value changed from 3 to 5.

`max_train_size` : *int, default=None* Maximum size for a single training set.

`test_size` : *int, default=None* Used to limit the size of the test set. Defaults to `n_samples // (n_splits + 1)`, which is the maximum allowed value with `gap=0`.

New in version 0.24.

`gap` : *int, default=0* Number of samples to exclude from the end of each train set before the test set.

New in version 0.24.

Notes

The training set has size `i * n_samples // (n_splits + 1) + n_samples % (n_splits + 1)` in the `i` th split, with a test set of size `n_samples//(n_splits + 1)` by default, where `n_samples` is the number of samples. Note that this formula is only valid when `test_size` and `max_train_size` are left to their default values.

Examples

```
>>> from sklearn.model_selection import TimeSeriesSplit
>>> X = np.array([[1, 2], [3, 4], [1, 2], [3, 4], [1, 2], [3, 4]])
>>> y = np.array([1, 2, 3, 4, 5, 6])
>>> tscv = TimeSeriesSplit()
>>> print(tscv)
TimeSeriesSplit(gap=0, max_train_size=None, n_splits=5, test_size=None)
>>> for i, (train_index, test_index) in enumerate(tscv.split(X)):
...     print(f"Fold {i}:")
...     print(f"  Train: index={train_index}")
...     print(f"  Test:  index={test_index}")
Fold 0:
  Train: index=[0]
  Test:  index=[1]
Fold 1:
  Train: index=[0 1]
  Test:  index=[2]
Fold 2:
  Train: index=[0 1 2]
  Test:  index=[3]
Fold 3:
  Train: index=[0 1 2 3]
  Test:  index=[4]
Fold 4:
  Train: index=[0 1 2 3 4]
...
```

sklearn.model_selection.GridSearchCV

```
sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None, n_jobs=None, refit=True,
↪  cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)
```

Exhaustive search over specified parameter values for an estimator.

Important members are fit, predict.

GridSearchCV implements a ”fit” and a ”score” method. It also implements ”score_samples”, ”predict”, ”predict_proba”, ”decision_function”, ”transform” and ”inverse_transform” if they are implemented in the estimator used.

The parameters of the estimator used to apply these methods are optimized by cross-validated grid-search over a parameter grid.

Read more in the User Guide.

Parameters

`estimator` : *estimator object* This is assumed to implement the scikit-learn estimator interface. Either estimator needs to provide a `score` function, or `scoring` must be passed.

`param_grid` : *dict or list of dictionaries* Dictionary with parameters names (str) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.

`scoring` : *str, callable, list, tuple or dict, default=None* Strategy to evaluate the performance of the cross-validated model on the test set.

If scoring represents a single score, one can use:

- a single string (see scoring_string_names);
- a callable (see scoring_callable) that returns a single value;
- None, the estimator's default evaluation criterion is used.

If scoring represents multiple scores, one can use:

- a list or tuple of unique strings;
- a callable returning a dictionary where the keys are the metric names and the values are the metric scores;
- a dictionary with metric names as keys and callables as values.

See multimetric_grid_search for an example.

`n_jobs` : *int, default=None* Number of jobs to run in parallel. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

Changed in version v0.20. n_jobs default changed from 1 to None

`refit` : *bool, str, or callable, default=True* Refit an estimator using the best found parameters on the whole dataset.

For multiple metric evaluation, this needs to be a str denoting the scorer that would be used to find the best parameters for refitting the estimator at the end.

Where there are considerations other than maximum score in choosing a best estimator, `refit` can be set to a function which returns the selected `best_index_` given `cv_results_`. In that case, the `best_estimator_` and `best_params_` will be set according to the returned `best_index_` while the `best_score_` attribute will not be available.

The refitted estimator is made available at the `best_estimator_` attribute and permits using `predict` directly on this `GridSearchCV` instance.

Also for multiple metric evaluation, the attributes `best_index_`, `best_score_` and `best_params_` will only be available if `refit` is set and all of them will be determined w.r.t this specific scorer.

See `scoring` parameter to know more about multiple metric evaluation.

See sphx_glr_auto_examples_model_selection_plot_grid_search_digits.py to see how to design a custom selection strategy using a callable via refit.

See `this example <sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py>` for an example of how to use `refit=callable` to balance model complexity and cross-validated score.

Changed in version 0.20. Support for callable added.

`cv` : *int, cross-validation generator or an iterable, default=None* Determines the cross-validation splitting strategy. Possible inputs for cv are:

- None, to use the default 5-fold cross validation,
- integer, to specify the number of folds in a (Stratified)KFold,
- CV splitter,
- an iterable yielding (train, test) splits as arrays of indices.

For integer/None inputs, if the estimator is a classifier and `y` is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with shuffle=False so the splits will be the same across calls.

Refer User Guide for the various cross-validation strategies that can be used here.

Changed in version 0.22. `cv` default value if None changed from 3-fold to 5-fold.

`verbose` : *int, default=0* Controls the verbosity of information printed during fitting, with higher values yielding more detailed logging.

- 0 : no messages are printed;
- >=1 : summary of the total number of fits;
- >=2 : computation time for each fold and parameter candidate;
- >=3 : fold indices and scores;
- >=10 : parameter candidate indices and START messages before each fit.

`pre_dispatch` : *int, or str, default='2n_jobs'* Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

- None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs
- An int, giving the exact number of total jobs that are spawned
- A str, giving an expression as a function of n_jobs, as in '2*n_jobs'

`error_score` : *'raise' or numeric, default=np.nan* Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised. This parameter does not affect the refit step, which will always raise the error.

`return_train_score` : *bool, default=False* If `False`, the `cv_results_` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance.

New in version 0.19.

Changed in version 0.21. Default value was changed from `True` to `False`

See also: `ParameterGrid`, `train_test_split`, `sklearn.metrics.make_scorer`

Notes

The parameters selected are those that maximize the score of the left out data, unless an explicit score is passed in which case it is used instead.

If n_jobs was set to a value higher than one, the data is copied for each point in the grid (and not n_jobs times). This is done for efficiency reasons if individual jobs take very little time, but may raise errors if the dataset is large and not enough memory is available. A workaround in this case is to set pre_dispatch. Then, the memory is copied only pre_dispatch many times. A reasonable value for pre_dispatch is 2 * n_jobs.

Examples

```
>>> from sklearn import svm, datasets
>>> from sklearn.model_selection import GridSearchCV
>>> iris = datasets.load_iris()
>>> parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
>>> svc = svm.SVC()
>>> clf = GridSearchCV(svc, parameters)
>>> clf.fit(iris.data, iris.target)
GridSearchCV(estimator=SVC(),
             param_grid={'C': [1, 10], 'kernel': ('linear', 'rbf')})
>>> sorted(clf.cv_results_.keys())
['mean_fit_time', 'mean_score_time', 'mean_test_score',...
 'param_C', 'param_kernel', 'params',...
 'rank_test_score', 'split0_test_score',...
 'split2_test_score', ...
 'std_fit_time', 'std_score_time', 'std_test_score']
```

sklearn.model_selection.RandomizedSearchCV

```
sklearn.model_selection.RandomizedSearchCV(estimator, param_distributions, *, n_iter=10, scoring=None,
↪   n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', random_state=None,
↪   error_score=nan, return_train_score=False)
```

Randomized search on hyper parameters.

RandomizedSearchCV implements a ”fit” and a ”score” method. It also implements ”score_samples”, ”predict”, ”predict_proba”, ”decision_function”, ”transform” and ”inverse_transform” if they are implemented in the estimator used.

The parameters of the estimator used to apply these methods are optimized by cross-validated search over parameter settings.

In contrast to GridSearchCV, not all parameter values are tried out, but rather a fixed number of parameter settings is sampled from the specified distributions. The number of parameter settings that are tried is given by n_iter.

If all parameters are presented as a list, sampling without replacement is performed. If at least one parameter is given as a distribution, sampling with replacement is used. It is highly recommended to use continuous distributions for continuous parameters.

Read more in the User Guide.

New in version 0.14.

Parameters

`estimator` : *estimator object* An object of that type is instantiated for each grid point. This is assumed to implement the scikit-learn estimator interface. Either estimator needs to provide a `score` function, or `scoring` must be passed.

param_distributions : *dict or list of dicts* Dictionary with parameters names (str) as keys and distributions or lists of parameters to try. Distributions must provide a `rvs` method for sampling (such as those from scipy.stats.distributions). If a list is given, it is sampled uniformly. If a list of dicts is given, first a dict is sampled uniformly, and then a parameter is sampled using that dict as above.

n_iter : *int, default=10* Number of parameter settings that are sampled. n_iter trades off runtime vs quality of the solution.

scoring : *str, callable, list, tuple or dict, default=None* Strategy to evaluate the performance of the cross-validated model on the test set.

If scoring represents a single score, one can use:

- a single string (see scoring_string_names);
- a callable (see scoring_callable) that returns a single value;
- None, the estimator's default evaluation criterion is used.

If scoring represents multiple scores, one can use:

- a list or tuple of unique strings;
- a callable returning a dictionary where the keys are the metric names and the values are the metric scores;
- a dictionary with metric names as keys and callables as values.

See multimetric_grid_search for an example.

If None, the estimator's score method is used.

n_jobs : *int, default=None* Number of jobs to run in parallel. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

Changed in version v0.20. n_jobs default changed from 1 to None

refit : *bool, str, or callable, default=True* Refit an estimator using the best found parameters on the whole dataset.

For multiple metric evaluation, this needs to be a str denoting the scorer that would be used to find the best parameters for refitting the estimator at the end.

Where there are considerations other than maximum score in choosing a best estimator, `refit` can be set to a function which returns the selected `best_index_` given the `cv_results_`. In that case, the `best_estimator_` and `best_params_` will be set according to the returned `best_index_` while the `best_score_` attribute will not be available.

The refitted estimator is made available at the `best_estimator_` attribute and permits using `predict` directly on this `RandomizedSearchCV` instance.

Also for multiple metric evaluation, the attributes `best_index_`, `best_score_` and `best_params_` will only be available if `refit` is set and all of them will be determined w.r.t this specific scorer.

See `scoring` parameter to know more about multiple metric evaluation.

See `this example <sphx_glr_auto_examples_model_selection_plot_grid_search_refit_callable.py>` for an example of how to use `refit=callable` to balance model complexity and cross-validated score.

Changed in version 0.20. Support for callable added.

cv : *int, cross-validation generator or an iterable, default=None* Determines the cross-validation splitting strategy. Possible inputs for cv are:

- None, to use the default 5-fold cross validation,

- integer, to specify the number of folds in a (Stratified)KFold,
- CV splitter,
- an iterable yielding (train, test) splits as arrays of indices.

For integer/None inputs, if the estimator is a classifier and `y` is either binary or multiclass, `StratifiedKFold` is used. In all other cases, `KFold` is used. These splitters are instantiated with shuffle=False so the splits will be the same across calls.

Refer User Guide for the various cross-validation strategies that can be used here.

Changed in version 0.22. `cv` default value if None changed from 3-fold to 5-fold.

`verbose` **: *int, default = 0*** Controls the verbosity of information printed during fitting, with higher values yielding more detailed logging.

- 0 : no messages are printed;
- >=1 : summary of the total number of fits;
- >=2 : computation time for each fold and parameter candidate;
- >=3 : fold indices and scores;
- >=10 : parameter candidate indices and START messages before each fit.

`pre_dispatch` **: *int, or str, default='2n_jobs'*** Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

- None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs
- An int, giving the exact number of total jobs that are spawned
- A str, giving an expression as a function of n_jobs, as in '2*n_jobs'

`random_state` **: *int, RandomState instance or None, default=None*** Pseudo random number generator state used for random uniform sampling from lists of possible values instead of scipy.stats distributions. Pass an int for reproducible output across multiple function calls. See Glossary.

`error_score` **: *'raise' or numeric, default=np.nan*** Value to assign to the score if an error occurs in estimator fitting. If set to 'raise', the error is raised. If a numeric value is given, FitFailedWarning is raised. This parameter does not affect the refit step, which will always raise the error.

`return_train_score` **: *bool, default=False*** If `False`, the `cv_results_` attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance.

New in version 0.19.

Changed in version 0.21. Default value was changed from `True` to `False`

See also: `GridSearchCV`, `ParameterSampler`

Notes

The parameters selected are those that maximize the score of the held-out data, according to the scoring parameter.

If n_jobs was set to a value higher than one, the data is copied for each parameter setting(and not n_jobs times). This is done for efficiency reasons if individual jobs take very little time, but may raise errors if the dataset is large and not enough memory is available. A workaround in

this case is to set pre_dispatch. Then, the memory is copied only pre_dispatch many times. A reasonable value for pre_dispatch is 2 * n_jobs.

Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.model_selection import RandomizedSearchCV
>>> from scipy.stats import uniform
>>> iris = load_iris()
>>> logistic = LogisticRegression(solver='saga', tol=1e-2, max_iter=200,
...                               random_state=0)
>>> distributions = dict(C=uniform(loc=0, scale=4),
...                      l1_ratio=[0, 1])
>>> clf = RandomizedSearchCV(logistic, distributions, random_state=0)
>>> search = clf.fit(iris.data, iris.target)
>>> search.best_params_
{'C': np.float64(2.195...), 'l1_ratio': 1}
```

Pipelines and preprocessing

sklearn.pipeline.Pipeline

```
sklearn.pipeline.Pipeline(steps, *, transform_input=None, memory=None, verbose=False)
```

A sequence of data transformers with an optional final predictor.

Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.

Intermediate steps of the pipeline must be transformers, that is, they must implement fit and transform methods. The final estimator only needs to implement fit. The transformers in the pipeline can be cached using `memory` argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a '__', as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to 'passthrough' or None.

For an example use case of Pipeline combined with `GridSearchCV`, refer to sphx_glr_auto_examples_compose_plot_compare_reduction.py. The example sphx_glr_auto_examples_compose_plot_digits_pipe.py shows how to grid search on a pipeline using '__' as a separator in the parameter names.

Read more in the User Guide.

New in version 0.5.

Parameters

`steps` : *list of tuples* List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define fit. All non-last steps must also define transform. See Combining Estimators for more details.

`transform_input` : *list of str, default=None* The names of the metadata parameters that should be transformed by the pipeline before passing it to the step consuming it.

This enables transforming some input arguments to `fit` (other than `X`) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined

via metadata routing. For instance, this can be used to pass a validation set through the pipeline.

You can only set this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

New in version 1.6.

`memory` : *str or object with the joblib.Memory interface, default=None* Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute `named_steps` or `steps` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py for an example on how to enable caching.

`verbose` : *bool, default=False* If True, the time elapsed while fitting each step will be printed as it is completed.

See also: `make_pipeline`

Examples

```
>>> from sklearn.svm import SVC
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.datasets import make_classification
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.pipeline import Pipeline
>>> X, y = make_classification(random_state=0)
>>> X_train, X_test, y_train, y_test = train_test_split(X, y,
...                                                     random_state=0)
>>> pipe = Pipeline([('scaler', StandardScaler()), ('svc', SVC())])
>>> # The pipeline can be used as any other estimator
>>> # and avoids leaking the test set into the train set
>>> pipe.fit(X_train, y_train).score(X_test, y_test)
0.88
>>> # An estimator's parameter can be set using '__' syntax
>>> pipe.set_params(svc__C=10).fit(X_train, y_train).score(X_test, y_test)
0.76
```

sklearn.pipeline.make_pipeline

```
sklearn.pipeline.make_pipeline(*steps, memory=None, transform_input=None, verbose=False)
```

Construct a `Pipeline` from the given estimators.

This is a shorthand for the `Pipeline` constructor; it does not require, and does not permit, naming the estimators. Instead, their names will be set to the lowercase of their types automatically.

Parameters

`*steps` : *list of Estimator objects* List of the scikit-learn estimators that are chained together.

`memory` : *str or object with the joblib.Memory interface, default=None* Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use

the attribute `named_steps` or `steps` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming.

`transform_input` : *list of str, default=None* This enables transforming some input arguments to `fit` (other than `X`) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via metadata routing. This can be used to pass a validation set through the pipeline for instance.

You can only set this if metadata routing is enabled, which you can enable using `sklearn.set_config(enable_metadata_routing=True)`.

New in version 1.6.

`verbose` : *bool, default=False* If True, the time elapsed while fitting each step will be printed as it is completed.

Returns

`p` : *Pipeline* Returns a scikit-learn `Pipeline` object.

See also: `Pipeline`

Examples

```
>>> from sklearn.naive_bayes import GaussianNB
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.pipeline import make_pipeline
>>> make_pipeline(StandardScaler(), GaussianNB(priors=None))
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('gaussiannb', GaussianNB())])
```

sklearn.compose.ColumnTransformer

```
sklearn.compose.ColumnTransformer(transformers, *, remainder='drop', sparse_threshold=0.3, n_jobs=None,
↪  transformer_weights=None, verbose=False, verbose_feature_names_out=True)
```

Applies transformers to columns of an array or pandas DataFrame.

This estimator allows different columns or column subsets of the input to be transformed separately and the features generated by each transformer will be concatenated to form a single feature space. This is useful for heterogeneous or columnar data, to combine several feature extraction mechanisms or transformations into a single transformer.

Read more in the User Guide.

New in version 0.20.

Parameters

`transformers` : *list of tuples* List of (name, transformer, columns) tuples specifying the transformer objects to be applied to subsets of the data.

name : str Like in Pipeline and FeatureUnion, this allows the transformer and its parameters to be set using `set_params` and searched in grid search.

transformer : {'drop', 'passthrough'} or estimator Estimator must support fit and transform. Special-cased strings 'drop' and 'passthrough' are accepted as well, to indicate to drop the columns or to pass them through untransformed, respectively.

columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable Indexes the data on its second axis. Integers are interpreted as positional columns, while strings can reference DataFrame columns by name. A scalar string or int should be used where `transformer` expects X to be a 1d array-like (vector), otherwise a 2d

array will be passed to the transformer. A callable is passed the input data X and can return any of the above. To select multiple columns by name or dtype, you can use `make_column_selector`.

`remainder` : ***{'drop', 'passthrough'} or estimator, default='drop'*** By default, only the specified columns in transformers are transformed and combined in the output, and the non-specified columns are dropped. (default of `'drop'`). By specifying `remainder='passthrough'`, all remaining columns that were not specified in transformers, but present in the data passed to fit will be automatically passed through. This subset of columns is concatenated with the output of the transformers. For dataframes, extra columns not seen during fit will be excluded from the output of transform. By setting `remainder` to be an estimator, the remaining non-specified columns will use the `remainder` estimator. The estimator must support fit and transform. Note that using this feature requires that the DataFrame columns input at fit and transform have identical order.

`sparse_threshold` : ***float, default=0.3*** If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use `sparse_threshold=0` to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.

`n_jobs` : ***int, default=None*** Number of jobs to run in parallel. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

`transformer_weights` : ***dict, default=None*** Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.

`verbose` : ***bool, default=False*** If True, the time elapsed while fitting each transformer will be printed as it is completed.

`verbose_feature_names_out` : ***bool, str or Callable[[str, str], str], default=True***

- If True, `ColumnTransformer.get_feature_names_out` will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting verbose_feature_names_out="{transformer_name}__{feature_name}".
- If False, `ColumnTransformer.get_feature_names_out` will not prefix any feature names and will error if feature names are not unique.
- If `Callable[[str, str], str]`, `ColumnTransformer.get_feature_names_out` will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name.
- If `str`, it must be a string ready for formatting. The given string will be formatted using two field names: `transformer_name` and `feature_name`. e.g. `"{feature_name}__{transformer_name}"`. See `str.format` method from the standard library for more info.

New in version 1.0.

Changed in version 1.6. verbose_feature_names_out can be a callable or a string to be formatted.

See also: `make_column_transformer`, `make_column_selector`

Notes

The order of the columns in the transformed feature matrix follows the order of how the columns are specified in the transformers list. Columns of the original feature matrix that are not specified are dropped from the resulting transformed feature matrix, unless specified in the passthrough keyword. Those columns specified with passthrough are added at the right to the output of the transformers.

Examples

```
>>> from sklearn.compose import ColumnTransformer
>>> from sklearn.preprocessing import Normalizer
>>> ct = ColumnTransformer(
...     [("norm1", Normalizer(norm='l1'), [0, 1]),
...      ("norm2", Normalizer(norm='l1'), slice(2, 4))])
>>> X = np.array([[0., 1., 2., 2.],
...               [1., 1., 0., 1.]])
>>> # Normalizer scales each row of X to unit norm. A separate scaling
>>> # is applied for the two first and two last elements of each
>>> # row independently.
>>> ct.fit_transform(X)
array([[0. , 1. , 0.5, 0.5],
       [0.5, 0.5, 0. , 1. ]])
```

`ColumnTransformer` can be configured with a transformer that requires a 1d array by setting the column to a string: ...

sklearn.preprocessing.StandardScaler

```
sklearn.preprocessing.StandardScaler(*, copy=True, with_mean=True, with_std=True)
```

Standardize features by removing the mean and scaling to unit variance.

The standard score of a sample x is calculated as:

```
z = (x - u) / s
```

where u is the mean of the training samples or zero if with_mean=False, and s is the standard deviation of the training samples or one if with_std=False.

Centering and scaling happen independently on each feature by computing the relevant statistics on the samples in the training set. Mean and standard deviation are then stored to be used on later data using `transform`.

Standardization of a dataset is a common requirement for many machine learning estimators: they might behave badly if the individual features do not more or less look like standard normally distributed data (e.g. Gaussian with 0 mean and unit variance).

For instance many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the L1 and L2 regularizers of linear models) assume that all features are centered around 0 and have variance in the same order. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.

StandardScaler is sensitive to outliers, and the features may scale differently from each other in the presence of outliers. For an example visualization, refer to `Compare StandardScaler with other scalers <plot_all_scaling_standard_scaler_section>`.

This scaler can also be applied to sparse CSR or CSC matrices by passing with_mean=False to avoid breaking the sparsity structure of the data.

Read more in the User Guide.

Parameters

`copy` : ***bool, default=True*** If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.

with_mean : *bool, default=True* If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.

with_std : *bool, default=True* If True, scale the data to unit variance (or equivalently, unit standard deviation).

See also: `scale`, `~sklearn.decomposition.PCA`

Notes

NaNs are treated as missing values: disregarded in fit, and maintained in transform.

We use a biased estimator for the standard deviation, equivalent to numpy.std(x, ddof=0). Note that the choice of ddof is unlikely to affect model performance.

Examples

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> print(scaler.fit(data))
StandardScaler()
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

sklearn.preprocessing.MinMaxScaler

```
sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1), *, copy=True, clip=False)
```

Transform features by scaling each feature to a given range.

This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one.

The transformation is given by:

```
X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
X_scaled = X_std * (max - min) + min
```

where min, max = feature_range.

This transformation is often used as an alternative to zero mean, unit variance scaling.

MinMaxScaler doesn't reduce the effect of outliers, but it linearly scales them down into a fixed range, where the largest occurring data point corresponds to the maximum value and the smallest one corresponds to the minimum value. For an example visualization, refer to `Compare MinMaxScaler with other scalers <plot_all_scaling_minmax_scaler_section>`.

Read more in the User Guide.

Parameters

feature_range : *tuple (min, max), default=(0, 1)* Desired range of transformed data.

copy : *bool, default=True* Set to False to perform inplace row normalization and avoid a copy (if the input is already a numpy array).

clip : *bool, default=False* Set to True to clip transformed values of held-out data to provided feature_range. Since this parameter will clip values, inverse_transform may not be able to restore the original data.

> Note
> Setting clip=True does not prevent feature drift (a distribution shift between training and test data). The transformed values are clipped to the feature_range, which helps avoid unintended behavior in models sensitive to out-of-range inputs (e.g. linear models). Use with care, as clipping can distort the distribution of test data.

New in version 0.24.

See also: `minmax_scale`

Notes

NaNs are treated as missing values: disregarded in fit, and maintained in transform.

Examples

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler()
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.   0.  ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0. ]]
```

sklearn.preprocessing.OneHotEncoder

```
sklearn.preprocessing.OneHotEncoder(*, categories='auto', drop=None, sparse_output=True, dtype=<class
↪  'numpy.float64'>, handle_unknown='error', min_frequency=None, max_categories=None,
↪  feature_name_combiner='concat')
```

Encode categorical features as a one-hot numeric array.

The input to this transformer should be an array-like of integers or strings, denoting the values taken on by categorical (discrete) features. The features are encoded using a one-hot (aka 'one-of-K' or 'dummy') encoding scheme. This creates a binary column for each category and returns a sparse matrix or dense array (depending on the `sparse_output` parameter).

By default, the encoder derives the categories based on the unique values in each feature. Alternatively, you can also specify the categories manually.

This encoding is needed for feeding categorical data to many scikit-learn estimators, notably linear models and SVMs with the standard kernels.

Note: a one-hot encoding of y labels should use a LabelBinarizer instead.

Read more in the User Guide. For a comparison of different encoders, refer to: sphx_glr_auto_examples_preprocessing_plot_target_encoder.py.

Parameters

categories : *'auto' or a list of array-like, default='auto'* Categories (unique values) per feature:

- 'auto' : Determine categories automatically from the training data.
- list : `categories[i]` holds the categories expected in the ith column. The passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values.

The used categories can be found in the `categories_` attribute.

New in version 0.20.

drop : *{'first', 'if_binary'} or an array-like of shape (n_features,), default=None* Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model.

However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models.

- None : retain all features (the default).
- 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely.
- 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact.
- array : `drop[i]` is the category in feature `X[:, i]` that should be dropped.

When max_categories or min_frequency is configured to group infrequent categories, the dropping behavior is handled after the grouping.

New in version 0.21. The parameter drop was added in 0.21.

Changed in version 0.23. The option drop='if_binary' was added in 0.23.

Changed in version 1.1. Support for dropping infrequent categories.

sparse_output : *bool, default=True* When `True`, it returns a SciPy sparse matrix/array in "Compressed Sparse Row" (CSR) format.

New in version 1.2. sparse was renamed to sparse_output

dtype : *number type, default=np.float64* Desired dtype of output.

handle_unknown : *{'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error'* Specifies the way unknown categories are handled during `transform`.

- 'error' : Raise an error if an unknown category is present during transform.
- 'ignore' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will be all zeros. In the inverse transform, an unknown category will be denoted as None.
- 'infrequent_if_exist' : When an unknown category is encountered during transform, the resulting one-hot encoded columns for this feature will map to the infrequent category if it exists. The infrequent category will be mapped to the last position in the encoding. During inverse transform, an unknown category will be mapped to the category denoted 'infrequent' if it exists. If the 'infrequent' category does not exist, then `transform` and `inverse_transform` will handle an unknown category as with handle_unknown='ignore'. Infrequent categories exist based on min_frequency and max_categories. Read more in the User Guide.
- 'warn' : When an unknown category is encountered during transform a warning is issued, and the encoding then proceeds as described for handle_unknown="infrequent_if_exist".

Changed in version 1.1. 'infrequent_if_exist' was added to automatically handle unknown categories and infrequent categories.

New in version 1.6. The option "warn" was added in 1.6.

`min_frequency` : *int or float, default=None* Specifies the minimum frequency below which a category will be considered infrequent.

- If int, categories with a smaller cardinality will be considered infrequent.
- If float, categories with a smaller cardinality than min_frequency * n_samples will be considered infrequent.

New in version 1.1. Read more in the User Guide.

`max_categories` : *int, default=None* Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, max_categories includes the category representing the infrequent categories along with the frequent categories. If None, there is no limit to the number of output features.

New in version 1.1. Read more in the User Guide.

`feature_name_combiner` : *"concat" or callable, default="concat"* Callable with signature def callable(input_feature, category) that returns a string. This is used to create feature names to be returned by `get_feature_names_out`.

"concat" concatenates encoded feature name and category with feature + "_" + str(category).E.g. feature X with values 1, 6, 7 create feature names X_1, X_6, X_7.

New in version 1.3.

See also: `OrdinalEncoder`, `TargetEncoder`, `sklearn.feature_extraction.DictVectorizer`, `sklearn.feature_extraction.FeatureHasher`, `LabelBinarizer`, `MultiLabelBinarizer`

Examples

Given a dataset with two features, we let the encoder find the unique values per feature and transform the data to a binary one-hot encoding.

```
>>> from sklearn.preprocessing import OneHotEncoder
```

One can discard categories not seen during 'fit':

```
>>> enc = OneHotEncoder(handle_unknown='ignore')
>>> X = [['Male', 1], ['Female', 3], ['Female', 2]]
>>> enc.fit(X)
OneHotEncoder(handle_unknown='ignore')
>>> enc.categories_
[array(['Female', 'Male'], dtype=object), array([1, 2, 3], dtype=object)]
>>> enc.transform([['Female', 1], ['Male', 4]]).toarray()
array([[1., 0., 1., 0., 0.],
       [0., 1., 0., 0., 0.]])
>>> enc.inverse_transform([[0, 1, 1, 0, 0], [0, 0, 0, 1, 0]])
array([['Male', 1],
       [None, 2]], dtype=object)
>>> enc.get_feature_names_out(['gender', 'group'])
array(['gender_Female', 'gender_Male', 'group_1', 'group_2', 'group_3'], ...)
...
```

sklearn.impute.SimpleImputer

```
sklearn.impute.SimpleImputer(*, missing_values=nan, strategy='mean', fill_value=None, copy=True,
↪  add_indicator=False, keep_empty_features=False)
```

Univariate imputer for completing missing values with simple strategies.

Replace missing values using a descriptive statistic (e.g. mean, median, or most frequent) along each column, or using a constant value.

Read more in the User Guide.

New in version 0.20. SimpleImputer replaces the previous sklearn.preprocessing.Imputer estimator which is now removed.

Parameters

missing_values : *int, float, str, np.nan, None or pandas.NA, default=np.nan* The placeholder for the missing values. All occurrences of missing_values will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, missing_values can be set to either np.nan or pd.NA.

strategy : *str or Callable, default='mean'* The imputation strategy.

- If ”mean”, then replace missing values using the mean along each column. Can only be used with numeric data.
- If ”median”, then replace missing values using the median along each column. Can only be used with numeric data.
- If ”most_frequent”, then replace missing using the most frequent value along each column. Can be used with strings or numeric data. If there is more than one such value, only the smallest is returned.
- If ”constant”, then replace missing values with fill_value. Can be used with strings or numeric data.
- If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column.

New in version 0.20. strategy=”constant” for fixed value imputation.

New in version 1.5. strategy=callable for custom value imputation.

fill_value : *str or numerical value, default=None* When strategy == ”constant”, fill_value is used to replace all occurrences of missing_values. For string or object data types, fill_value must be a string. If None, fill_value will be 0 when imputing numerical data and ”missing_value” for strings or object data types.

copy : *bool, default=True* If True, a copy of X will be created. If False, imputation will be done in-place whenever possible. Note that, in the following cases, a new copy will always be made, even if ‘copy=False‘:

- If X is not an array of floating values;
- If X is encoded as a CSR matrix;
- If add_indicator=True.

add_indicator : *bool, default=False* If True, a `MissingIndicator` transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.

keep_empty_features : *bool, default=False* If True, features that consist exclusively of missing values when fit is called are returned in results when transform is called. The imputed value is always 0 except when strategy=”constant” in which case fill_value will be used instead.

New in version 1.2.

See also: `IterativeImputer`, `KNNImputer`

Notes

Columns which only contained missing values at `fit` are discarded upon `transform` if strategy is not "constant".

In a prediction context, simple imputation usually performs poorly when associated with a weak learner. However, with a powerful learner, it can lead to as good or better performance than complex imputation such as `IterativeImputer` or `KNNImputer`.

Examples

```
>>> from sklearn.impute import SimpleImputer
>>> imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
SimpleImputer()
>>> X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
>>> print(imp_mean.transform(X))
[[ 7.   2.   3. ]
 [ 4.   3.5  6. ]
 [10.   3.5  9. ]]
```

For a more detailed example see sphx_glr_auto_examples_impute_plot_missing_values.py.

Models

sklearn.linear_model.LinearRegression

```
sklearn.linear_model.LinearRegression(*, fit_intercept=True, copy_X=True, tol=1e-06, n_jobs=None,
↪  positive=False)
```

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients w = (w1, ..., wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Parameters

`fit_intercept` : *bool, default=True* Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

`copy_X` : *bool, default=True* If True, X will be copied; else, it may be overwritten.

`tol` : *float, default=1e-6* The precision of the solution (coef_) is determined by tol which specifies the convergence criterion of the underlying solver. tol is set as atol and btol of `scipy.sparse.linalg.lsqr` when fitting on sparse training data. tol is set as cond of `scipy.linalg.lstsq` when fitting on dense training data.

New in version 1.7.

Changed in version 1.9. Now supported on dense data, interpreted as the cond parameter.

`n_jobs` : *int, default=None* The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly n_targets > 1 and secondly X is sparse or if positive is set to True. `None` means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. See Glossary for more details.

positive : *bool, default=False* When set to `True`, forces the coefficients to be positive. This option is only supported for dense arrays.

For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see sphx_glr_auto_examples_linear_model_plot_nnls.py.

New in version 0.24.

See also: `Ridge`, `Lasso`, `ElasticNet`

Notes

From the implementation point of view, this is just plain Ordinary Least Squares (`scipy.linalg.lstsq`) or Non Negative Least Squares (`scipy.optimize.nnls`) wrapped as a predictor object.

Examples

```
>>> from sklearn.linear_model import LinearRegression
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
>>> # y = 1 * x_0 + 2 * x_1 + 3
>>> y = np.dot(X, np.array([1, 2])) + 3
>>> reg = LinearRegression().fit(X, y)
>>> reg.score(X, y)
1.0
>>> reg.coef_
array([1., 2.])
>>> reg.intercept_
np.float64(3.0)
>>> reg.predict(np.array([[3, 5]]))
array([16.])
```

sklearn.linear_model.LogisticRegression

```
sklearn.linear_model.LogisticRegression(penalty='deprecated', *, C=1.0, l1_ratio=0.0, dual=False,
↪  tol=0.0001, fit_intercept=True, intercept_scaling=1, class_weight=None, random_state=None,
↪  solver='lbfgs', max_iter=100, verbose=0, warm_start=False, n_jobs=None)
```

Logistic Regression (aka logit, MaxEnt) classifier.

This class implements regularized logistic regression using a set of available solvers. **Note that regularization is applied by default**. It can handle both dense and sparse input X. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

The solvers 'lbfgs', 'newton-cg', 'newton-cholesky' and 'sag' support only L2 regularization with primal formulation, or no regularization. The 'liblinear' solver supports both L1 and L2 regularization (but not both, i.e. elastic-net), with a dual formulation only for the L2 penalty. The Elastic-Net (combination of L1 and L2) regularization is only supported by the 'saga' solver.

For multiclass problems (whenever n_classes >= 3), all solvers except 'liblinear' optimize the (penalized) multinomial loss. 'liblinear' only handles binary classification but can be extended to handle multiclass by using `OneVsRestClassifier`.

Read more in the User Guide.

Parameters

penalty : *{'l1', 'l2', 'elasticnet', None}, default='l2'* Specify the norm of the penalty:

- 'None': no penalty is added;

- ‘‘l2’‘: add an L2 penalty term and it is the default choice;
- ‘‘l1’‘: add an L1 penalty term;
- ‘‘elasticnet’‘: both L1 and L2 penalty terms are added.

> Warning
> Some penalties may not work with some solvers. See the parameter solver below, to know the compatibility between the penalty and solver.

New in version 0.19. l1 penalty with SAGA solver (allowing 'multinomial' + L1)

Deprecated since version 1.8. penalty was deprecated in version 1.8 and will be removed in 1.10. Use l1_ratio and C instead. l1_ratio=0 for penalty='l2', l1_ratio=1 for penalty='l1', l1_ratio set to any float between 0 and 1 for penalty='elasticnet', and C=np.inf for penalty=None.

C : *float, default=1.0* Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization. C=np.inf results in unpenalized logistic regression. For a visual example on the effect of tuning the C parameter with an L1 penalty, see: sphx_glr_auto_examples_linear_model_plot_logistic_path.py.

l1_ratio : *float, default=0.0* The Elastic-Net mixing parameter, with 0 <= l1_ratio <= 1. Setting l1_ratio=1 gives a pure L1-penalty, setting l1_ratio=0 a pure L2-penalty. Any value between 0 and 1 gives an Elastic-Net penalty of the form l1_ratio * L1 + (1 - l1_ratio) * L2.

> Warning
> Certain values of l1_ratio, i.e. some penalties, may not work with some solvers. See the parameter solver below, to know the compatibility between the penalty and solver.

Changed in version 1.8. Default value changed from None to 0.0.

Deprecated since version 1.8. None is deprecated and will be removed in version 1.10. Always use l1_ratio to specify the penalty type.

dual : *bool, default=False* Dual (constrained) or primal (regularized, see also this equation) formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer dual=False when n_samples > n_features.

tol : *float, default=1e-4* Tolerance for stopping criteria.

fit_intercept : *bool, default=True* Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.

intercept_scaling : *float, default=1* Useful only when the solver liblinear is used and self.fit_intercept is set to True. In this case, x becomes [x, self.intercept_scaling], i.e. a ”synthetic” feature with constant value equal to intercept_scaling is appended to the instance vector. The intercept becomes `intercept_scaling * synthetic_feature_weight`.

> Note
> The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) intercept_scaling has to be increased.

class_weight : *dict or 'balanced', default=None* Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one.

The ”balanced” mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

New in version 0.17. class_weight='balanced'

random_state : *int, RandomState instance, default=None* Only used for solver == 'sag', 'saga' or 'liblinear' to shuffle the data. It has no effect on the other solvers. See Glossary for details.

solver : *{'lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs'* Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- 'lbfgs' is a good default solver because it works reasonably well for a wide class of problems.
- For multiclass problems (n_classes >= 3), all solvers except 'liblinear' minimize the full multinomial loss, 'liblinear' will raise an error.
- 'newton-cholesky' is a good choice for n_samples » n_features * n_classes, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on n_features * n_classes because it explicitly computes the full Hessian matrix.
- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the `OneVsRestClassifier`.

Warning

The choice of the algorithm depends on the penalty chosen (l1_ratio=0 for L2-penalty, l1_ratio=1 for L1-penalty and 0 < l1_ratio < 1 for Elastic-Net) and on (multinomial) multiclass support:

solver	l1_ratio	multinomial multiclass
'lbfgs'	l1_ratio=0	yes
'liblinear'	l1_ratio=1 or l1_ratio=0	no
'newton-cg'	l1_ratio=0	yes
'newton-cholesky'	l1_ratio=0	yes
'sag'	l1_ratio=0	yes
'saga'	0<=l1_ratio<=1	yes

Note
'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from `sklearn.preprocessing`.

See also
Refer to the User Guide for more information regarding `LogisticRegression` and more specifically the summarizing solver/penalty supports.

New in version 0.17. Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18.

New in version 0.19. SAGA solver.

Changed in version 0.22. The default solver changed from 'liblinear' to 'lbfgs' in 0.22.

New in version 1.2. newton-cholesky solver. Multinomial support in version 1.6.

max_iter : *int, default=100* Maximum number of iterations taken for the solvers to converge.

verbose : *int, default=0* For the liblinear and lbfgs solvers set verbose to any positive number for verbosity.

warm_start : *bool, default=False* When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver. See the Glossary.

New in version 0.17. *warm_start* to support *lbfgs*, *newton-cg*, *sag*, *saga* solvers.

n_jobs : *int, default=None* Does not have any effect.

Deprecated since version 1.8. n_jobs is deprecated in version 1.8 and will be removed in 1.10.

See also: `SGDClassifier`, `LogisticRegressionCV`

Notes

For several reasons (floating point arithmetic, random number generators, etc.) the coefficients of a fitted model might differ slightly (among machines, scikit-learn versions, etc.) for the same input data. If that happens and you want to avoid it, you can try with a smaller tol parameter.

Predict output may not match that of standalone liblinear in certain cases. See differences from liblinear in the narrative documentation.

Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.82e-01, 1.82e-02, 1.44e-08],
       [9.72e-01, 2.82e-02, 3.02e-08]])
>>> clf.score(X, y)
0.97
```

For a comparison of the LogisticRegression with other classifiers see: sphx_glr_auto_examples_classification_plot_classification_probability.py.

sklearn.linear_model.Ridge

```
sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True, copy_X=True, max_iter=None, tol=0.0001,
↪  solver='auto', positive=False, random_state=None)
```

Linear least squares with l2 regularization.

Minimizes the objective function:

```
||y - Xw||^2_2 + alpha * ||w||^2_2
```

This model solves a regression model where the loss function is the linear least squares function and regularization is given by the l2-norm. Also known as Ridge Regression or Tikhonov regularization. This estimator has built-in support for multi-variate regression (i.e., when y is a 2d-array of shape (n_samples, n_targets)).

Read more in the User Guide.

Parameters

alpha : *float or array-like of shape (n_targets,), default=1.0* Constant that multiplies the L2 term, controlling regularization strength. alpha must be a non-negative float i.e. in [0, inf).

When alpha = 0, the objective is equivalent to ordinary least squares, solved by the `LinearRegression` object. For numerical reasons, using alpha = 0 with the Ridge object is not advised. Instead, you should use the `LinearRegression` object.

If an array is passed, penalties are assumed to be specific to the targets. Hence they must correspond in number.

See sphx_glr_auto_examples_linear_model_plot_ridge_coeffs.py for an illustration of the effect of alpha on the model coefficients.

`fit_intercept` : *bool, default=True* Whether to fit the intercept for this model. If set to false, no intercept will be used in calculations (i.e. `X` and `y` are expected to be centered).

`copy_X` : *bool, default=True* If True, X will be copied; else, it may be overwritten.

`max_iter` : *int, default=None* Maximum number of iterations for conjugate gradient solver. For 'sparse_cg' and 'lsqr' solvers, the default value is determined by scipy.sparse.linalg. For 'sag' solver, the default value is 1000. For 'lbfgs' solver, the default value is 15000.

`tol` : *float, default=1e-4* The precision of the solution (coef_) is determined by tol which specifies a different convergence criterion for each solver:

- 'svd': tol has no impact.
- 'cholesky': tol has no impact.
- 'sparse_cg': norm of residuals smaller than tol.
- 'lsqr': tol is set as atol and btol of scipy.sparse.linalg.lsqr, which control the norm of the residual vector in terms of the norms of matrix and coefficients.
- 'sag' and 'saga': relative change of coef smaller than tol.
- 'lbfgs': maximum of the absolute (projected) gradient=max smaller than tol.

Changed in version 1.2. Default value changed from 1e-3 to 1e-4 for consistency with other linear models.

`solver` : *{'auto', 'svd', 'cholesky', 'lsqr', 'sparse_cg', 'sag', 'saga', 'lbfgs'}, default='auto'*
Solver to use in the computational routines:

- 'auto' chooses the solver automatically based on the type of data.
- 'svd' uses a Singular Value Decomposition of X to compute the Ridge coefficients. It is the most stable solver, in particular more stable for singular matrices than 'cholesky' at the cost of being slower.
- 'cholesky' uses the standard `scipy.linalg.solve` function to obtain a closed-form solution.
- 'sparse_cg' uses the conjugate gradient solver as found in `scipy.sparse.linalg.cg`. As an iterative algorithm, this solver is more appropriate than 'cholesky' for large-scale data (possibility to set tol and max_iter).
- 'lsqr' uses the dedicated regularized least-squares routine `scipy.sparse.linalg.lsqr`. It is the fastest and uses an iterative procedure.
- 'sag' uses a Stochastic Average Gradient descent, and 'saga' uses its improved, unbiased version named SAGA. Both methods also use an iterative procedure, and are often faster than other solvers when both n_samples and n_features are large. Note that 'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from `sklearn.preprocessing`.
- 'lbfgs' uses L-BFGS-B algorithm implemented in `scipy.optimize.minimize`. It can be used only when positive is True.

All solvers except 'svd' support both dense and sparse data. However, only 'lsqr', 'sag', 'sparse_cg', and 'lbfgs' support sparse input when fit_intercept is True.

New in version 0.17. Stochastic Average Gradient descent solver.

New in version 0.19. SAGA solver.

positive : *bool, default=False* When set to `True`, forces the coefficients to be positive. Only 'lbfgs' solver is supported in this case.

random_state : *int, RandomState instance, default=None* Used when `solver == 'sag'` or 'saga' to shuffle the data. See Glossary for details.

New in version 0.17. random_state to support Stochastic Average Gradient.

See also: `RidgeClassifier`, `RidgeCV`, `~sklearn.kernel_ridge.KernelRidge`

Notes

Regularization improves the conditioning of the problem and reduces the variance of the estimates. Larger values specify stronger regularization. Alpha corresponds to `1 / (2C)` in other linear models such as `LogisticRegression` or `LinearSVC`.

Examples

```
>>> from sklearn.linear_model import Ridge
>>> n_samples, n_features = 10, 5
>>> rng = np.random.RandomState(0)
>>> y = rng.randn(n_samples)
>>> X = rng.randn(n_samples, n_features)
>>> clf = Ridge(alpha=1.0)
>>> clf.fit(X, y)
Ridge()
```

sklearn.ensemble.RandomForestClassifier

```
sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini', max_depth=None,
↪   min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='sqrt',
↪   max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None,
↪   random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None,
↪   monotonic_cst=None)
```

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. Trees in the forest use the best split strategy, i.e. equivalent to passing splitter="best" to the underlying `DecisionTreeClassifier`. The sub-sample size is controlled with the max_samples parameter if bootstrap=True (default), otherwise the whole dataset is used to build each tree.

For a comparison between tree-based ensemble models see the example sphx_glr_auto_examples_ensemble_plot_forest_hist_grad_boosting_comparison.py.

This estimator has native support for missing values (NaNs). During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently. If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

Read more in the User Guide.

Parameters

n_estimators : *int, default=100* The number of trees in the forest.

Changed in version 0.22. The default value of `n_estimators` changed from 10 to 100 in 0.22.

criterion : *{"gini", "entropy", "log_loss"}, default="gini"* The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see tree_mathematical_formulation. Note: This parameter is tree-specific.

max_depth : *int, default=None* The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : *int or float, default=2* The minimum number of samples required to split an internal node:

- If int, then consider min_samples_split as the minimum number.
- If float, then min_samples_split is a fraction and ceil(min_samples_split * n_samples) are the minimum number of samples for each split.

Changed in version 0.18. Added float values for fractions.

min_samples_leaf : *int or float, default=1* The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

- If int, then consider min_samples_leaf as the minimum number.
- If float, then min_samples_leaf is a fraction and ceil(min_samples_leaf * n_samples) are the minimum number of samples for each node.

Changed in version 0.18. Added float values for fractions.

min_weight_fraction_leaf : *float, default=0.0* The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when sample_weight is not provided.

max_features : *{"sqrt", "log2", None}, int or float, default="sqrt"* The number of features to consider when looking for the best split:

- If int, then consider max_features features at each split.
- If float, then max_features is a fraction and max(1, int(max_features * n_features_in_)) features are considered at each split.
- If "sqrt", then max_features=sqrt(n_features).
- If "log2", then max_features=log2(n_features).
- If None, then max_features=n_features.

Changed in version 1.1. The default of max_features changed from "auto" to "sqrt".

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

max_leaf_nodes : *int, default=None* Grow trees with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

min_impurity_decrease : *float, default=0.0* A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following:

```
N_t / N * (impurity - N_t_R / N_t * right_impurity
                    - N_t_L / N_t * left_impurity)
```

where `N` is the total number of samples, `N_t` is the number of samples at the current node, `N_t_L` is the number of samples in the left child, and `N_t_R` is the number of samples in the right child.

N, N_t, N_t_R and N_t_L all refer to the weighted sum, if sample_weight is passed.

New in version 0.19.

bootstrap : *bool, default=True* Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

oob_score : *bool or callable, default=False* Whether to use out-of-bag samples to estimate the generalization score. By default, accuracy_score is used. Provide a callable with signature metric(y_true, y_pred) to use a custom metric. Only available if bootstrap=True.

For an illustration of out-of-bag (OOB) error estimation, see the example sphx_glr_auto_examples_ensemble_plot_ensemble_oob.py.

n_jobs : *int, default=None* The number of jobs to run in parallel. fit, predict, decision_path and apply are all parallelized over the trees. None means 1 unless in a joblib.parallel_backend context. -1 means using all processors. See Glossary <n_jobs> for more details.

random_state : *int, RandomState instance or None, default=None* Controls both the randomness of the bootstrapping of the samples used when building trees (if bootstrap=True) and the sampling of the features to consider when looking for the best split at each node (if max_features < n_features). See Glossary for details.

verbose : *int, default=0* Controls the verbosity when fitting and predicting.

warm_start : *bool, default=False* When set to True, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See Glossary and tree_ensemble_warm_start for details.

class_weight : *{"balanced", "balanced_subsample"}, dict or list of dicts, default=None* Weights associated with classes in the form {class_label: weight}. If not given, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of y.

Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be [{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}] instead of [{1:1}, {2:5}, {3:1}, {4:1}].

The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as n_samples / (n_classes * np.bincount(y))

The "balanced_subsample" mode is the same as "balanced" except that weights are computed based on the bootstrap sample for every tree grown.

For multi-output, the weights of each column of y will be multiplied.

Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

ccp_alpha : *non-negative float, default=0.0* Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ccp_alpha will be chosen. By default, no pruning is performed. See minimal_cost_complexity_pruning for details. See sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py for an example of such pruning.

New in version 0.22.

max_samples : *int or float, default=None* If bootstrap is True, the number of samples to draw from X to train each base estimator.

- If None (default), then draw X.shape[0] samples irrespective of sample_weight.
- If int, then draw max_samples samples.

- If float, then draw max_samples * X.shape[0] unweighted samples or max_samples * sample_weight.sum() weighted samples.

New in version 0.22.

Changed in version 1.9. Float max_samples is relative to sample_weight.sum() instead of X.shape[0] for weighted samples.

monotonic_cst : *array-like of int of shape (n_features,), default=None* **Indicates the monotonicity co**

- 1: monotonically increasing
- 0: no constraint
- -1: monotonically decreasing

If monotonic_cst is None, no constraints are applied.

Monotonicity constraints are not supported for:

- multiclass classifications (i.e. when n_classes > 2),
- multioutput classifications (i.e. when n_outputs_ > 1).

The constraints hold over the probability of the positive class.

Read more in the User Guide.

New in version 1.4.

See also: `sklearn.tree.DecisionTreeClassifier`, `sklearn.ensemble.ExtraTreesClassifier`, `sklearn.ensemble.HistGradientBoostingClassifier`

Notes

The default values for the parameters controlling the size of the trees (e.g. `max_depth`, `min_samples_leaf`, etc.) lead to fully grown and unpruned trees which can potentially be very large on some data sets. To reduce memory consumption, the complexity and size of the trees should be controlled by setting those parameter values.

The features are always randomly permuted at each split. Therefore, the best found split may vary, even with the same training data, `max_features=n_features` and `bootstrap=False`, if the improvement of the criterion is identical for several splits enumerated during the search of the best split. To obtain a deterministic behaviour during fitting, `random_state` has to be fixed.

Examples

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.datasets import make_classification
>>> X, y = make_classification(n_samples=1000, n_features=4,
...                            n_informative=2, n_redundant=0,
...                            random_state=0, shuffle=False)
>>> clf = RandomForestClassifier(max_depth=2, random_state=0)
>>> clf.fit(X, y)
RandomForestClassifier(...)
>>> print(clf.predict([[0, 0, 0, 0]]))
[1]
```

sklearn.ensemble.HistGradientBoostingClassifier

```
sklearn.ensemble.HistGradientBoostingClassifier(loss='log_loss', *, learning_rate=0.1, max_iter=100,
↪  max_leaf_nodes=31, max_depth=None, min_samples_leaf=20, l2_regularization=0.0, max_features=1.0,
↪  max_bins=255, categorical_features='from_dtype', monotonic_cst=None, interaction_cst=None,
↪  warm_start=False, early_stopping='auto', scoring='loss', validation_fraction=0.1,
↪  n_iter_no_change=10, tol=1e-07, verbose=0, random_state=None, class_weight=N ...)
```

Histogram-based Gradient Boosting Classification Tree.

This estimator is much faster than `GradientBoostingClassifier` for big datasets (n_samples >= 10 000).

This estimator has native support for missing values (NaNs). During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently. If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

This implementation is inspired by LightGBM.

Read more in the User Guide.

New in version 0.21.

Parameters

`loss` : ***{'log_loss'}, default='log_loss'*** The loss function to use in the boosting process.

For binary classification problems, 'log_loss' is also known as logistic loss, binomial deviance or binary crossentropy. Internally, the model fits one tree per boosting iteration and uses the logistic sigmoid function (expit) as inverse link function to compute the predicted positive class probability.

For multiclass classification problems, 'log_loss' is also known as multinomial deviance or categorical crossentropy. Internally, the model fits one tree per boosting iteration and per class and uses the softmax function as inverse link function to compute the predicted probabilities of the classes.

`learning_rate` : ***float, default=0.1*** The learning rate, also known as *shrinkage*. This is used as a multiplicative factor for the leaves values. Use `1` for no shrinkage.

`max_iter` : ***int, default=100*** The maximum number of iterations of the boosting process, i.e. the maximum number of trees for binary classification. For multiclass classification, n_classes trees per iteration are built.

`max_leaf_nodes` : ***int or None, default=31*** The maximum number of leaves for each tree. Must be strictly greater than 1. If None, there is no maximum limit.

`max_depth` : ***int or None, default=None*** The maximum depth of each tree. The depth of a tree is the number of edges to go from the root to the deepest leaf. Depth isn't constrained by default.

`min_samples_leaf` : ***int, default=20*** The minimum number of samples per leaf. For small datasets with less than a few hundred samples, it is recommended to lower this value since only very shallow trees would be built.

`l2_regularization` : ***float, default=0*** The L2 regularization parameter penalizing leaves with small hessians. Use `0` for no regularization (default).

`max_features` : ***float, default=1.0*** Proportion of randomly chosen features in each and every node split. This is a form of regularization, smaller values make the trees weaker learners and might prevent overfitting. If interaction constraints from interaction_cst are present, only allowed features are taken into account for the subsampling.

New in version 1.4.

`max_bins` : ***int, default=255*** The maximum number of bins to use for non-missing values. Before training, each feature of the input array X is binned into integer-valued bins, which allows for a much faster training stage. Features with a small number of unique values may use

less than `max_bins` bins. In addition to the `max_bins` bins, one more bin is always reserved for missing values. Must be no larger than 255.

`categorical_features` : *array-like of {bool, int, str} of shape (n_features) or shape (n_categorical_feat*
Indicates the categorical features.

- None : no feature will be considered categorical.
- boolean array-like : boolean mask indicating categorical features.
- integer array-like : integer indices indicating categorical features.
- str array-like: names of categorical features (assuming the training data has feature names).
- ‘”from_dtype”: dataframe columns with dtype ”Categorical” and ”Enum” are considered to be categorical features. The input must be a dataframe that is supported by narwhals (or supports it): narwhals.from_native‘ must work. This is the case, for instance, for pandas and polars DataFrames.

For each categorical feature, there must be at most max_bins unique categories. Negative values for categorical features encoded as numeric dtypes are treated as missing values. All categorical values are converted to floating point numbers. This means that categorical values of 1.0 and 1 are treated as the same category.

Read more in the User Guide.

New in version 0.24.

Changed in version 1.2. Added support for feature names.

Changed in version 1.4. Added ”from_dtype” option.

Changed in version 1.6. The default value changed from None to ”from_dtype”.

`monotonic_cst` : *array-like of int of shape (n_features) or dict, default=None* Monotonic constraint to enforce on each feature are specified using the following integer values:

- 1: monotonic increase
- 0: no constraint
- -1: monotonic decrease

If a dict with str keys, map feature to monotonic constraints by name. If an array, the features are mapped to constraints by position. See monotonic_cst_features_names for a usage example.

The constraints are only valid for binary classifications and hold over the probability of the positive class. Read more in the User Guide.

New in version 0.23.

Changed in version 1.2. Accept dict of constraints with feature names as keys.

`interaction_cst` : *{”pairwise”, ”no_interactions”} or sequence of lists/tuples/sets of int, default=No*
Specify interaction constraints, the sets of features which can interact with each other in child node splits.

Each item specifies the set of feature indices that are allowed to interact with each other. If there are more features than specified in these constraints, they are treated as if they were specified as an additional set.

The strings ”pairwise” and ”no_interactions” are shorthands for allowing only pairwise or no interactions, respectively.

For instance, with 5 features in total, interaction_cst=[{0, 1}] is equivalent to interaction_cst=[{0, 1}, {2, 3, 4}], and specifies that each branch of a tree will either only split on features 0 and 1 or only split on features 2, 3 and 4.

See this example on how to use interaction_cst.

New in version 1.2.

warm_start : *bool, default=False* When set to `True`, reuse the solution of the previous call to fit and add more estimators to the ensemble. For results to be valid, the estimator should be re-trained on the same data only. See the Glossary.

early_stopping : *'auto' or bool, default='auto'* If 'auto', early stopping is enabled if the sample size is larger than 10000 or if X_val and y_val are passed to fit. If True, early stopping is enabled, otherwise early stopping is disabled.

New in version 0.23.

scoring : *str or callable or None, default='loss'* Scoring method to use for early stopping. Only used if early_stopping is enabled. Options:

- str: see scoring_string_names for options.
- callable: a scorer callable object (e.g., function) with signature `scorer(estimator, X, y)`. See scoring_callable for details.
- 'None': accuracy is used.
- 'loss': early stopping is checked w.r.t the loss value.

validation_fraction : *int or float or None, default=0.1* Proportion (or absolute size) of training data to set aside as validation data for early stopping. If None, early stopping is done on the training data. The value is ignored if either early stopping is not performed, e.g. early_stopping=False, or if X_val and y_val are passed to fit.

n_iter_no_change : *int, default=10* Used to determine when to "early stop". The fitting process is stopped when none of the last `n_iter_no_change` scores are better than the `n_iter_no_change - 1` -th-to-last one, up to some tolerance. Only used if early stopping is performed.

tol : *float, default=1e-7* The absolute tolerance to use when comparing scores. The higher the tolerance, the more likely we are to early stop: higher tolerance means that it will be harder for subsequent iterations to be considered an improvement upon the reference score.

verbose : *int, default=0* The verbosity level. If not zero, print some information about the fitting process. `1` prints only summary info, `2` prints info per iteration.

random_state : *int, RandomState instance or None, default=None* Pseudo-random number generator to control the subsampling in the binning process, and the train/validation data split if early stopping is enabled. Pass an int for reproducible output across multiple function calls. See Glossary.

class_weight : *dict or 'balanced', default=None* Weights associated with classes in the form {class_label: weight}. If not given, all classes are supposed to have weight one. The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as n_samples / (n_classes * np.bincount(y)). Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

New in version 1.2.

See also: `GradientBoostingClassifier`, `sklearn.tree.DecisionTreeClassifier`, `RandomForestClassifier`, `AdaBoostClassifier`

Examples

```
>>> from sklearn.ensemble import HistGradientBoostingClassifier
>>> from sklearn.datasets import load_iris
>>> X, y = load_iris(return_X_y=True)
>>> clf = HistGradientBoostingClassifier().fit(X, y)
>>> clf.score(X, y)
1.0
```

Metrics

sklearn.metrics.accuracy_score

```
sklearn.metrics.accuracy_score(y_true, y_pred, *, normalize=True, sample_weight=None)
```

Accuracy classification score.

In multilabel classification, this function computes subset accuracy: the set of labels predicted for a sample must *exactly* match the corresponding set of labels in y_true.

Read more in the User Guide.

Parameters

y_true : *1d array-like, or label indicator array / sparse matrix* Ground truth (correct) labels. Sparse matrix is only supported when labels are of multilabel type.

y_pred : *1d array-like, or label indicator array / sparse matrix* Predicted labels, as returned by a classifier. Sparse matrix is only supported when labels are of multilabel type.

normalize : *bool, default=True* If `False`, return the number of correctly classified samples. Otherwise, return the fraction of correctly classified samples.

sample_weight : *array-like of shape (n_samples,), default=None* Sample weights.

Returns

score : *float* If `normalize == True`, returns the fraction of correctly classified samples, else returns the number of correctly classified samples.

The best performance is 1.0 with `normalize == True` and the number of samples with `normalize == False`.

See also: `balanced_accuracy_score`, `jaccard_score`, `hamming_loss`, `zero_one_loss`

Examples

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = [0, 2, 1, 3]
>>> y_true = [0, 1, 2, 3]
>>> accuracy_score(y_true, y_pred)
0.5
>>> accuracy_score(y_true, y_pred, normalize=False)
2.0
```

In the multilabel case with binary label indicators:

```
>>> accuracy_score(np.array([[0, 1], [1, 1]]), np.ones((2, 2)))
0.5
```

sklearn.metrics.f1_score

```
sklearn.metrics.f1_score(y_true, y_pred, *, labels=None, pos_label=1, average='binary',
↪  sample_weight=None, zero_division='warn')
```

Compute the F1 score, also known as balanced F-score or F-measure.

The F1 score can be interpreted as a harmonic mean of the precision and recall, where an F1 score reaches its best value at 1 and worst score at 0. The relative contribution of precision and recall to the F1 score are equal. The formula for the F1 score is:

$$\mathrm{F1} = \frac{2 * \mathrm{TP}}{2 * \mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

Where TP is the number of true positives, FN is the number of false negatives, and FP is the number of false positives. F1 is by default calculated as 0.0 when there are no true positives, false negatives, or false positives.

Support beyond binary targets is achieved by treating multiclass and multilabel data as a collection of binary problems, one for each label. For the binary case, setting average='binary' will return F1 score for pos_label. If average is not 'binary', pos_label is ignored and F1 score for both classes are computed, then averaged or both returned (when average=None). Similarly, for multiclass and multilabel targets, F1 score for all labels are either returned or averaged depending on the average parameter. Use labels specify the set of labels to calculate F1 score for.

Read more in the User Guide.

Parameters

`y_true` : ***1d array-like, or label indicator array / sparse matrix*** Ground truth (correct) target values. Sparse matrix is only supported when targets are of multilabel type.

`y_pred` : ***1d array-like, or label indicator array / sparse matrix*** Estimated targets as returned by a classifier. Sparse matrix is only supported when targets are of multilabel type.

`labels` : ***array-like, default=None*** The set of labels to include when average != 'binary', and their order if average is None. Labels present in the data can be excluded, for example in multiclass classification to exclude a ”negative class”. Labels not present in the data can be included and will be ”assigned” 0 samples. For multilabel targets, labels are column indices. By default, all labels in y_true and y_pred are used in sorted order.

Changed in version 0.17. Parameter labels improved for multiclass problem.

`pos_label` : ***int, float, bool or str, default=1*** The class to report if average='binary' and the data is binary, otherwise this parameter is ignored. For multiclass or multilabel targets, set labels=[pos_label] and average != 'binary' to report metrics for one label only.

`average` : ***{'micro', 'macro', 'samples', 'weighted', 'binary'} or None, default='binary'*** This parameter is required for multiclass/multilabel targets. If `None`, the metrics for each class are returned. Otherwise, this determines the type of averaging performed on the data:

`'binary'`: Only report results for the class specified by `pos_label`. This is applicable only if targets (`y_{true,pred}`) are binary.

`'micro'`: Calculate metrics globally by counting the total true positives, false negatives and false positives.

`'macro'`: Calculate metrics for each label, and find their unweighted mean. This does not take label imbalance into account.

`'weighted'`: Calculate metrics for each label, and find their average weighted by support (the number of true instances for each label). This alters 'macro' to account for label imbalance; it can result in an F-score that is not between precision and recall.

`'samples'`: Calculate metrics for each instance, and find their average (only meaningful for multilabel classification where this differs from `accuracy_score`).

`sample_weight` : ***array-like of shape (n_samples,), default=None*** Sample weights.

`zero_division` : ***{”warn”, 0.0, 1.0, np.nan}, default=”warn”*** Sets the value to return when there is a zero division, i.e. when all predictions and labels are negative.

Notes: - If set to "warn", this acts like 0, but a warning is also raised. - If set to np.nan, such values will be excluded from the average.

New in version 1.3. np.nan option was added.

Returns

f1_score : *float or array of float, shape = [n_unique_labels]* F1 score of the positive class in binary classification or weighted average of the F1 scores of each class for the multiclass task.

See also: `fbeta_score`, `precision_recall_fscore_support`, `jaccard_score`, `multilabel_confusion_matrix`

Notes

When `true positive + false positive + false negative == 0` (i.e. a class is completely absent from both `y_true` or `y_pred`), f-score is undefined. In such cases, by default f-score will be set to 0.0, and `UndefinedMetricWarning` will be raised. This behavior can be modified by setting the `zero_division` parameter.

Examples

```
>>> from sklearn.metrics import f1_score
>>> y_true = [0, 1, 2, 0, 1, 2]
>>> y_pred = [0, 2, 1, 0, 0, 1]
>>> f1_score(y_true, y_pred, average='macro')
0.267
>>> f1_score(y_true, y_pred, average='micro')
0.33
>>> f1_score(y_true, y_pred, average='weighted')
0.267
>>> f1_score(y_true, y_pred, average=None)
array([0.8, 0. , 0. ])
```

```
>>> # binary classification
>>> y_true_empty = [0, 0, 0, 0, 0, 0]
>>> y_pred_empty = [0, 0, 0, 0, 0, 0]
>>> f1_score(y_true_empty, y_pred_empty)
0.0...
>>> f1_score(y_true_empty, y_pred_empty, zero_division=1.0)
1.0...
>>> f1_score(y_true_empty, y_pred_empty, zero_division=np.nan)
nan...
...
```

sklearn.metrics.roc_auc_score

```
sklearn.metrics.roc_auc_score(y_true, y_score, *, average='macro', sample_weight=None, max_fpr=None,
↪  multi_class='raise', labels=None)
```

Compute Area Under the Receiver Operating Characteristic Curve (ROC AUC) from prediction scores.

Note: this implementation can be used with binary, multiclass and multilabel classification, but some restrictions apply (see Parameters).

Read more in the User Guide.

Parameters

y_true : ***array-like of shape (n_samples,) or (n_samples, n_classes)*** True labels or label indicator matrix. The binary and multiclass cases expect labels with shape (n_samples,) while the multilabel case expects a multilabel indicator matrix with shape (n_samples, n_classes).

y_score : ***array-like of shape (n_samples,) or (n_samples, n_classes)*** Target scores.

- In the binary case, it corresponds to an array of shape (n_samples,). Both probability estimates and non-thresholded decision values can be provided. The probability estimates correspond to the **probability of the class with the greater label**, i.e. estimator.classes_[1] and thus estimator.predict_proba(X, y)[:, 1]. The decision values corresponds to the output of estimator.decision_function(X, y). See more information in the User guide;
- In the multiclass case, it corresponds to an array of shape (n_samples, n_classes) of probability estimates provided by the predict_proba method. The probability estimates **must** sum to 1 across the possible classes. In addition, the order of the class scores must correspond to the order of `labels`, if provided, or else to the numerical or lexicographical order of the labels in `y_true`. See more information in the User guide;
- In the multilabel case, it corresponds to an array of shape (n_samples, n_classes). Probability estimates are provided by the predict_proba method and the non-thresholded decision values by the decision_function method. The probability estimates correspond to the **probability of the class with the greater label for each output** of the classifier. See more information in the User guide.

average : ***{'micro', 'macro', 'samples', 'weighted'} or None, default='macro'*** If `None`, the scores for each class are returned. Otherwise, this determines the type of averaging performed on the data. Note: multiclass ROC AUC currently only handles the 'macro' and 'weighted' averages. For multiclass targets, average=None is only implemented for multi_class='ovr' and average='micro' is only implemented for multi_class='ovr'.

'micro': Calculate metrics globally by considering each element of the label indicator matrix as a label.

'macro': Calculate metrics for each label, and find their unweighted mean. This does not take label imbalance into account.

'weighted': Calculate metrics for each label, and find their average, weighted by support (the number of true instances for each label).

'samples': Calculate metrics for each instance, and find their average.

Will be ignored when `y_true` is binary.

sample_weight : ***array-like of shape (n_samples,), default=None*** Sample weights.

max_fpr : ***float > 0 and <= 1, default=None*** If not `None`, the standardized partial AUC over the range [0, max_fpr] is returned. For the multiclass case, `max_fpr`, should be either equal to `None` or `1.0` as AUC ROC partial computation currently is not supported for multiclass.

multi_class : ***{'raise', 'ovr', 'ovo'}, default='raise'*** Only used for multiclass targets. Determines the type of configuration to use. The default value raises an error, so either `'ovr'` or `'ovo'` must be passed explicitly.

'ovr': Stands for One-vs-rest. Computes the AUC of each class against the rest. This treats the multiclass case in the same way as the multilabel case. Sensitive to class imbalance even when `average == 'macro'`, because class imbalance affects the composition of each of the 'rest' groupings.

'ovo': Stands for One-vs-one. Computes the average AUC of all possible pairwise combinations of classes. Insensitive to class imbalance when `average == 'macro'`.

labels : ***array-like of shape (n_classes,), default=None*** Only used for multiclass targets. List of labels that index the classes in `y_score`. If `None`, the numerical or lexicographical

order of the labels in `y_true` is used.

Returns

auc : *float* Area Under the Curve score.

See also: `average_precision_score`, `roc_curve`, `RocCurveDisplay.from_estimator`, `RocCurveDisplay.from_predictions`

Notes

The Gini Coefficient is a summary measure of the ranking ability of binary classifiers. It is expressed using the area under of the ROC as follows:

G = 2 * AUC - 1

Where G is the Gini coefficient and AUC is the ROC-AUC score. This normalisation will ensure that random guessing will yield a score of 0 in expectation, and it is upper bounded by 1.

Examples

Binary case:

```
>>> from sklearn.datasets import load_breast_cancer
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.metrics import roc_auc_score
>>> X, y = load_breast_cancer(return_X_y=True)
>>> clf = LogisticRegression(solver="newton-cholesky", random_state=0).fit(X, y)
>>> roc_auc_score(y, clf.predict_proba(X)[:, 1])
0.99
>>> roc_auc_score(y, clf.decision_function(X))
0.99
```

Multiclass case:

```
>>> from sklearn.datasets import load_iris
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(solver="newton-cholesky").fit(X, y)
>>> roc_auc_score(y, clf.predict_proba(X), multi_class='ovr')
0.99
...
```

sklearn.metrics.confusion_matrix

```
sklearn.metrics.confusion_matrix(y_true, y_pred, *, labels=None, sample_weight=None, normalize=None)
```

Compute confusion matrix to evaluate the accuracy of a classification.

By definition a confusion matrix C is such that $C_{i,j}$ is equal to the number of observations known to be in group i and predicted to be in group j.

Thus in binary classification, the count of true negatives is $C_{0,0}$, false negatives is $C_{1,0}$, true positives is $C_{1,1}$ and false positives is $C_{0,1}$.

Read more in the User Guide.

Parameters

y_true : *array-like of shape (n_samples,)* Ground truth (correct) target values.

y_pred : *array-like of shape (n_samples,)* Estimated targets as returned by a classifier.

labels : *array-like of shape (n_classes,), default=None* List of labels to index the matrix. This may be used to reorder or select a subset of labels. If `None` is given, those that appear at least once in `y_true` or `y_pred` are used in sorted order.

sample_weight : *array-like of shape (n_samples,), default=None* Sample weights.

New in version 0.18.

normalize : *{'true', 'pred', 'all'}, default=None* Normalizes confusion matrix over the true (rows), predicted (columns) conditions or all the population. If None, confusion matrix will not be normalized.

Returns

C : *ndarray of shape (n_classes, n_classes)* Confusion matrix whose i-th row and j-th column entry indicates the number of samples with true label being i-th class and predicted label being j-th class.

See also: `ConfusionMatrixDisplay.from_estimator`, `ConfusionMatrixDisplay.from_predictions`, `ConfusionMatrixDisplay`, `confusion_matrix_at_thresholds`

Examples

```
>>> from sklearn.metrics import confusion_matrix
>>> y_true = [2, 0, 2, 2, 0, 1]
>>> y_pred = [0, 0, 2, 2, 0, 2]
>>> confusion_matrix(y_true, y_pred)
array([[2, 0, 0],
       [0, 0, 1],
       [1, 0, 2]])
```

```
>>> y_true = ["cat", "ant", "cat", "cat", "ant", "bird"]
>>> y_pred = ["ant", "ant", "cat", "cat", "ant", "cat"]
>>> confusion_matrix(y_true, y_pred, labels=["ant", "bird", "cat"])
array([[2, 0, 0],
       [0, 0, 1],
       [1, 0, 2]])
```

In the binary case, we can extract true positives, etc. as follows: ...

sklearn.metrics.classification_report

```
sklearn.metrics.classification_report(y_true, y_pred, *, labels=None, target_names=None,
↪  sample_weight=None, digits=2, output_dict=False, zero_division='warn')
```

Build a text report showing the main classification metrics.

Read more in the User Guide.

Parameters

y_true : *1d array-like, or label indicator array / sparse matrix* Ground truth (correct) target values. Sparse matrix is only supported when targets are of multilabel type.

y_pred : *1d array-like, or label indicator array / sparse matrix* Estimated targets as returned by a classifier. Sparse matrix is only supported when targets are of multilabel type.

labels : *array-like of shape (n_labels,), default=None* Optional list of label indices to include in the report.

target_names : *array-like of shape (n_labels,), default=None* Optional display names matching the labels (same order).

sample_weight : *array-like of shape (n_samples,), default=None* Sample weights.

digits : *int, default=2* Number of digits for formatting output floating point values. When `output_dict` is `True`, this will be ignored and the returned values will not be rounded.

output_dict : *bool, default=False* If True, return output as dict.

New in version 0.20.

zero_division : *{"warn", 0.0, 1.0, np.nan}, default="warn"* Sets the value to return when there is a zero division. If set to "warn", this acts as 0, but warnings are also raised.

New in version 1.3. np.nan option was added.

Returns

report : *str or dict* Text summary of the precision, recall, F1 score for each class. Dictionary returned if output_dict is True. Dictionary has the following structure:

```
{'label 1': {'precision':0.5,
             'recall':1.0,
             'f1-score':0.67,
             'support':1},
 'label 2': { ... },
  ...
}
```

The reported averages include macro average (averaging the unweighted mean per label), weighted average (averaging the support-weighted mean per label), and sample average (only for multilabel classification). Micro average (averaging the total true positives, false negatives and false positives) is only shown for multi-label or multi-class with a subset of classes, because it corresponds to accuracy otherwise and would be the same for all metrics. See also `precision_recall_fscore_support` for more details on averages.

Note that in binary classification, recall of the positive class is also known as "sensitivity"; recall of the negative class is "specificity".

See also: `precision_recall_fscore_support`, `confusion_matrix`, `multilabel_confusion_matrix`

Examples

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 2]
>>> y_pred = [0, 0, 2, 2, 1]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
              precision    recall  f1-score   support
<BLANKLINE>
     class 0       0.50      1.00      0.67         1
     class 1       0.00      0.00      0.00         1
     class 2       1.00      0.67      0.80         3
<BLANKLINE>
    accuracy                           0.60         5
   macro avg       0.50      0.56      0.49         5
weighted avg       0.70      0.60      0.61         5
<BLANKLINE>
>>> y_pred = [1, 1, 0]
>>> y_true = [1, 1, 1]
>>> print(classification_report(y_true, y_pred, labels=[1, 2, 3]))
              precision    recall  f1-score   support
<BLANKLINE>
           1       1.00      0.67      0.80         3
           2       0.00      0.00      0.00         0
```

```
          3       0.00      0.00      0.00         0
<BLANKLINE>
...
```

sklearn.metrics.mean_squared_error

```
sklearn.metrics.mean_squared_error(y_true, y_pred, *, sample_weight=None, multioutput='uniform_average')
```

Mean squared error regression loss.

Read more in the User Guide.

Parameters

y_true : *array-like of shape (n_samples,) or (n_samples, n_outputs)* Ground truth (correct) target values.

y_pred : *array-like of shape (n_samples,) or (n_samples, n_outputs)* Estimated target values.

sample_weight : *array-like of shape (n_samples,), default=None* Sample weights.

multioutput : *{'raw_values', 'uniform_average'} or array-like of shape (n_outputs,), default='uniform*
Defines aggregating of multiple output values. Array-like value defines weights used to average errors.

'raw_values' : Returns a full set of errors in case of multioutput input.

'uniform_average' : Errors of all outputs are averaged with uniform weight.

Returns

loss : *float or array of floats* A non-negative floating point value (the best value is 0.0), or an array of floating point values, one for each individual target.

Examples

```
>>> from sklearn.metrics import mean_squared_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> mean_squared_error(y_true, y_pred)
0.375
>>> y_true = [[0.5, 1],[-1, 1],[7, -6]]
>>> y_pred = [[0, 2],[-1, 2],[8, -5]]
>>> mean_squared_error(y_true, y_pred)
0.708...
>>> mean_squared_error(y_true, y_pred, multioutput='raw_values')
array([0.41666667, 1.        ])
>>> mean_squared_error(y_true, y_pred, multioutput=[0.3, 0.7])
0.825...
```

sklearn.metrics.r2_score

```
sklearn.metrics.r2_score(y_true, y_pred, *, sample_weight=None, multioutput='uniform_average',
↪  force_finite=True)
```

R^2 (coefficient of determination) regression score function.

Best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). In the general case when the true y is non-constant, a constant model that always predicts the average y disregarding the input features would get a R^2 score of 0.0.

In the particular case when `y_true` is constant, the R^2 score is not finite: it is either `NaN` (perfect predictions) or `-Inf` (imperfect predictions). To prevent such non-finite numbers to pollute higher-level experiments such as a grid search cross-validation, by default these cases are replaced with 1.0 (perfect predictions) or 0.0 (imperfect predictions) respectively. You can set `force_finite` to `False` to prevent this fix from happening.

Note: when the prediction residuals have zero mean, the R^2 score is identical to the `Explained Variance score`.

Read more in the User Guide.

Parameters

`y_true` : *array-like of shape (n_samples,) or (n_samples, n_outputs)* Ground truth (correct) target values.

`y_pred` : *array-like of shape (n_samples,) or (n_samples, n_outputs)* Estimated target values.

`sample_weight` : *array-like of shape (n_samples,), default=None* Sample weights.

`multioutput` : *{'raw_values', 'uniform_average', 'variance_weighted'}, array-like of shape (n_outputs,* Defines aggregating of multiple output scores. Array-like value defines weights used to average scores. Default is "uniform_average".

'raw_values' : Returns a full set of scores in case of multioutput input.

'uniform_average' : Scores of all outputs are averaged with uniform weight.

'variance_weighted' : Scores of all outputs are averaged, weighted by the variances of each individual output.

Changed in version 0.19. Default value of multioutput is 'uniform_average'.

`force_finite` : *bool, default=True* Flag indicating if `NaN` and `-Inf` scores resulting from constant data should be replaced with real numbers (`1.0` if prediction is perfect, `0.0` otherwise). Default is `True`, a convenient setting for hyperparameters' search procedures (e.g. grid search cross-validation).

New in version 1.1.

Returns

`z` : *float or ndarray of floats* The R^2 score or ndarray of scores if 'multioutput' is 'raw_values'.

Notes

This is not a symmetric function.

Unlike most other scores, R^2 score may be negative (it need not actually be the square of a quantity R).

This metric is not well-defined for single samples and will return a NaN value if n_samples is less than two.

Examples

```
>>> from sklearn.metrics import r2_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> r2_score(y_true, y_pred)
0.948...
```

```
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> r2_score(y_true, y_pred,
...          multioutput='variance_weighted')
0.938...
>>> y_true = [1, 2, 3]
>>> y_pred = [1, 2, 3]
>>> r2_score(y_true, y_pred)
1.0
>>> y_true = [1, 2, 3]
>>> y_pred = [2, 2, 2]
>>> r2_score(y_true, y_pred)
0.0
>>> y_true = [1, 2, 3]
>>> y_pred = [3, 2, 1]
>>> r2_score(y_true, y_pred)
-3.0
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2]
...
```

Inspection

sklearn.inspection.permutation_importance

```
sklearn.inspection.permutation_importance(estimator, X, y, *, scoring=None, n_repeats=5, n_jobs=None,
↪  random_state=None, sample_weight=None, max_samples=1.0)
```

Permutation importance for feature evaluation [BRE].

The estimator is required to be a fitted estimator. X can be the data set used to train the estimator or a hold-out set. The permutation importance of a feature is calculated as follows. First, a baseline metric, defined by scoring, is evaluated on a (potentially different) dataset defined by the X. Next, a feature column from the validation set is permuted and the metric is evaluated again. The permutation importance is defined to be the difference between the baseline metric and metric from permutating the feature column.

Read more in the User Guide.

Parameters

estimator : *object* An estimator that has already been fitted and is compatible with scorer.

X : *ndarray or DataFrame, shape (n_samples, n_features)* Data on which permutation importance will be computed.

y : *array-like or None, shape (n_samples,) or (n_samples, n_classes)* Targets for supervised or None for unsupervised.

scoring : *str, callable, list, tuple, or dict, default=None* Scorer to use. If scoring represents a single score, one can use:

- str: see scoring_string_names for options.
- callable: a scorer callable object (e.g., function) with signature `scorer(estimator, X, y)`. See scoring_callable for details.
- 'None': the estimator's default evaluation criterion is used.

If scoring represents multiple scores, one can use:

- a list or tuple of unique strings;
- a callable returning a dictionary where the keys are the metric names and the values are the metric scores;
- a dictionary with metric names as keys and callables as values.

Passing multiple scores to scoring is more efficient than calling permutation_importance for each of the scores as it reuses predictions to avoid redundant computation.

n_repeats : *int, default=5* Number of times to permute a feature.

n_jobs : *int or None, default=None* Number of jobs to run in parallel. The computation is done by computing permutation score for each columns and parallelized over the columns. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details.

random_state : *int, RandomState instance, default=None* Pseudo-random number generator to control the permutations of each feature. Pass an int to get reproducible results across function calls. See Glossary.

sample_weight : *array-like of shape (n_samples,), default=None* Sample weights used in scoring.

New in version 0.24.

max_samples : *int or float, default=1.0* The number of samples to draw from X to compute feature importance in each repeat (without replacement).

- If int, then draw max_samples samples.
- If float, then draw max_samples * X.shape[0] samples.
- If max_samples is equal to 1.0 or X.shape[0], all samples will be used.

While using this option may provide less accurate importance estimates, it keeps the method tractable when evaluating feature importance on large datasets. In combination with n_repeats, this allows to control the computational speed vs statistical accuracy trade-off of this method.

New in version 1.0.

Returns

result : *"Bunch" or dict of such instances* Dictionary-like object, with the following attributes.

importances_mean : ndarray of shape (n_features,) Mean of feature importance over n_repeats.

importances_std : ndarray of shape (n_features,) Standard deviation over n_repeats.

importances : ndarray of shape (n_features, n_repeats) Raw permutation importance scores.

If there are multiple scoring metrics in the scoring parameter result is a dict with scorer names as keys (e.g. 'roc_auc') and Bunch objects like above as values.

Examples

```
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.inspection import permutation_importance
>>> X = [[1, 9, 9],[1, 9, 9],[1, 9, 9],
...      [0, 9, 9],[0, 9, 9],[0, 9, 9]]
>>> y = [1, 1, 1, 0, 0, 0]
>>> clf = LogisticRegression().fit(X, y)
>>> result = permutation_importance(clf, X, y, n_repeats=10,
...                                 random_state=0)
>>> result.importances_mean
```

```
array([0.4666, 0.       , 0.       ])
>>> result.importances_std
array([0.2211, 0.       , 0.       ])
```